



Reference Manual
for Crosslight Device Simulation Software

Updated July 2008

©1995-2008 Crosslight Software Inc.

Reference Manual for Crosslight Device Simulation Software

Applicable for APSYS, LASTIP and PICS3D packages

Updated July 2008

©1995-2008 Crosslight Software Inc.

All rights reserved

Trademark Notice

LASTIP, PICS3D, APSYS, PROCOM, CSUPREM, SimuCenter, SimuLastip, SimuPics3d, SimuApsys, LayerBuilder, Layer3d, GeoEditor, CrosslightView, CrosslightEdit, and Crosslight Design Studio are trademarks of Crosslight Software Inc.

Contents

1	INTRODUCTION	1-15
1.1	Applicable Simulation Software Packages	1-15
1.2	Input Files and Input Statements	1-15
1.3	Syntax of Input Statements	1-16
1.4	A Short Cut For Setting Input Parameters	1-17
1.5	Material Macros and Mathematic Functions	1-18
1.6	Lifetime and Deep Level Traps	1-29
1.7	Various Types of MQW Models	1-30
1.8	Segments, Planes, Tapers and Z-Bendings	1-33
1.9	Longitudinal Sections in PICS3D	1-37
1.10	Output Data Format	1-37
1.11	Conventions in This Manual	1-39
2	STATEMENTS FOR SIMULATION SOFTWARE	2-41
2.1	2nd_harmonic	2-41
2.2	3d_amplifier_model	2-42
2.3	3drayplot_angle	2-42
2.4	3drayplot_bias	2-44
2.5	3drayplot_phi	2-46
2.6	3drayplot_project	2-46
2.7	3drayplot_surfpower	2-48
2.8	3drayplot_theta	2-48
2.9	3dget_data	2-49
2.10	3d_plane_control	2-50
2.11	3d_solution_method	2-51
2.12	a_bar	2-52
2.13	a_bulk	2-52
2.14	a_well	2-52
2.15	absorption	2-53
2.16	absorption_spec	2-53
2.17	ac_bar	2-53
2.18	ac_bulk	2-54
2.19	ac_light	2-54

2.20	ac_parameters	2-55
2.21	ac_sparse_solver	2-57
2.22	ac_voltage	2-57
2.23	ac_well	2-59
2.24	active_reg	2-60
2.25	active_reg_zdim	2-65
2.26	active_temper	2-66
2.27	add_arnoldmemory	2-67
2.28	add_mainmemory	2-67
2.29	add_thermalmemory	2-68
2.30	add_acmemory	2-68
2.31	adjust_active_reg	2-69
2.32	adjust_current	2-69
2.33	adjust_doping	2-70
2.34	adjust_resistor	2-71
2.35	affinity	2-71
2.36	ai_bar,i=1,2,...	2-72
2.37	ai_bulk,i=1,2,...	2-72
2.38	ai_well,i=1,2,...	2-72
2.39	alpha_n	2-73
2.40	alpha_p	2-73
2.41	alpha_wavel	2-74
2.42	auger_n	2-74
2.43	auger_p	2-74
2.44	b_bar	2-74
2.45	b_well	2-75
2.46	back_index	2-75
2.47	back_reflection	2-75
2.48	band_discont	2-76
2.49	band_discont_right	2-76
2.50	band_distance	2-77
2.51	band_gap	2-79
2.52	band_offset	2-79
2.53	bandgap_narrow	2-79
2.54	bandgap_optical_gen	2-80
2.55	begin_cavity	2-80
2.56	begin_complex	2-81
2.57	begin_zmater	2-82
2.58	bend_extern	2-82
2.59	bend_xy_plane	2-82
2.60	beta_n	2-83
2.61	beta_p	2-84
2.62	bias3d	2-84

2.63	blank_area	2-85
2.64	bottom_contact	2-85
2.65	boundary_smooth	2-86
2.66	boundary_xpoint	2-86
2.67	bpm_grating	2-87
2.68	bpm_multimode	2-88
2.69	bpmintegr_xy	2-89
2.70	bpmplot_xy1d	2-89
2.71	bpmplot_xyz1d	2-90
2.72	bpmplot_xy	2-91
2.73	bpmplot_xz	2-92
2.74	bpmplot_yz	2-92
2.75	bpm	2-93
2.76	bulk_treatment	2-94
2.77	bulk_xfunck (k=1..9)	2-94
2.78	c11_bar	2-95
2.79	c11_well	2-95
2.80	c12_bar	2-95
2.81	c12_well	2-96
2.82	c13_bar	2-96
2.83	c13_bulk	2-96
2.84	c13_well	2-97
2.85	c33_bar	2-97
2.86	c33_bulk	2-97
2.87	c33_well	2-98
2.88	column	2-98
2.89	complex_region	2-99
2.90	cond_bandj_edge (j=2,3)	2-100
2.91	cond_bandj_valley (j=1..3)	2-101
2.92	condj_mass_para (j=1,2)	2-101
2.93	condj_mass_perp (j=1,2)	2-101
2.94	condj_valley_propk (j=1,2;k=1,2)	2-102
2.95	contact	2-102
2.96	convention	2-105
2.97	cplot_xyz	2-105
2.98	cplot_xy	2-106
2.99	current_conc	2-108
2.100	cylindrical	2-109
2.101	dbr_truncate	2-110
2.102	define_alias	2-110
2.103	define_material	2-111
2.104	define_symbol	2-111
2.105	define_vertical_position	2-112

2.106delta_so_bar	2-113
2.107delta_so_well	2-113
2.108deltai_bar,i=1,2,3	2-114
2.109deltai_bulk,i=1,2,3,4	2-114
2.110deltai_well,i=1,2,3	2-114
2.111di_bar,i=1,2,3,4	2-115
2.112di_bulk,i=1,2,...	2-115
2.113di_well,i=1,2,3,4	2-116
2.114dielectric_constant	2-116
2.115direct_eigen	2-116
2.116do_raytrace	2-118
2.117do_raytrace_3d	2-124
2.118doping	2-130
2.119double_mesh	2-136
2.120dox_efield0_pf_elec	2-138
2.121dox_efield0_pf_hole	2-138
2.122dox_el_weight	2-139
2.123dox_exciton_eg	2-139
2.124dox_extern_spectrum	2-139
2.125dox_gaussian_divj	2-140
2.126dox_gaussian_sdj	2-140
2.127dox_hopping_energy	2-140
2.128dox_peak_abs	2-140
2.129dox_vib_quanta	2-140
2.130dox_xp_coupling	2-140
2.131dox2_el_weight	2-140
2.132dox2_extern_spectrum	2-141
2.133dox3_el_weight	2-142
2.134dox3_extern_spectrum	2-142
2.135dox4_el_weight	2-143
2.136dox4_extern_spectrum	2-143
2.137dox5_el_weight	2-144
2.138dox5_extern_spectrum	2-145
2.139 do_multilayer_optics	2-145
2.140edge_curve	2-147
2.141eeim_optic	2-148
2.142eg0_bar	2-150
2.143eg0_bulk	2-150
2.144eg0_well	2-151
2.145eeim_optic	2-151
2.146efield0_pf_elec	2-153
2.147efield0_pf_hole	2-154
2.148elec_carr_loss	2-154

2.149elec_dos_energy	2-155
2.150electron_mass	2-155
2.151electron_mobility	2-156
2.152electron_ref_dens	2-156
2.153electron_sat_vel	2-156
2.154embedded_material	2-157
2.155end_cavity	2-157
2.156end_complex	2-157
2.157end_loop	2-158
2.158end_zmater	2-158
2.159equilibrium	2-159
2.160equiv_tunnel_junc	2-160
2.161export_gain_data	2-161
2.162export_kp_data	2-162
2.163export_kp_para	2-162
2.164 export_opt_property	2-163
2.165export_raytrace	2-164
2.166export_super_struc	2-167
2.167export_wave	2-168
2.168ext_funck (k=1..9)	2-169
2.169external_cir	2-170
2.170farfield	2-172
2.171farfield_couple	2-173
2.172fdtd_model	2-174
2.173fdtd_source	2-175
2.174fourier_power	2-176
2.175full_ionization	2-177
2.176flux_plot	2-178
2.177freq_control	2-178
2.178front_index	2-178
2.179front_reflection	2-178
2.180gain_density	2-179
2.181gain_module	2-180
2.182gain_fitting	2-181
2.183gain_spectrum	2-182
2.184gain_spon	2-183
2.185gain_wavel	2-184
2.186gammak_bar	2-185
2.187gammak_well	2-185
2.188get_active_layer	2-186
2.189get_data	2-187
2.190grating_compos	2-188
2.191grating_model	2-189

2.192grade_active_mater	2-191
2.193group1	2-192
2.194half_mesh	2-192
2.195heat_flow	2-193
2.196heteroj_capture	2-196
2.197hole_carr_loss	2-197
2.198hole_mass	2-198
2.199hole_mobility	2-198
2.200hole_ref_dens	2-199
2.201hole_sat_vel	2-199
2.202impact_baraff	2-199
2.203impact_chynoweth	2-201
2.204impact_model	2-202
2.205import_super_struc	2-203
2.206import_gain_data	2-204
2.207import_kp_data	2-206
2.208import_wave	2-206
2.209include	2-207
2.210independent_mqw	2-207
2.211index_field_dependence	2-208
2.212index_model	2-209
2.213index_spectrum	2-211
2.214index_wavel	2-212
2.215 ind_file_use	2-213
2.216init_sol	2-214
2.217init_wave	2-215
2.218inner_bar_gain	2-219
2.219integer_func	2-219
2.220interface	2-220
2.221integer_func	2-222
2.222internal_xpoint	2-222
2.223intersubband_model	2-224
2.224qdot_material	2-224
2.225jdos_energy	2-224
2.226kane_para_f_bar	2-225
2.227kane_para_f_well	2-225
2.228lateral_mode3d	2-226
2.229lattice_bar	2-226
2.230lattice_base	2-227
2.231lattice_bulk	2-227
2.232lattice_constant	2-227
2.233lattice_well	2-228
2.234lax_mass_bar	2-228

2.235lax_mass_well	2-228
2.236layer	2-229
2.237layer_conf	2-231
2.238layer_mater	2-232
2.239layer_type	2-235
2.240lband_bar	2-237
2.241lband_well	2-237
2.242led_control	2-238
2.243led_eff_distr	2-240
2.244led_farfield	2-241
2.245led_spectrum	2-241
2.246led_simple	2-242
2.247 led_top_coating	2-242
2.248left_contact	2-243
2.249lifetime_model	2-244
2.250lifetime_n	2-244
2.251lifetime_p	2-245
2.252light_power	2-246
2.253light_power_qwip	2-251
2.254light_current	2-254
2.255linear_heat	2-257
2.256load_macro	2-258
2.257load_mesh	2-259
2.258longitudinal	2-260
2.259loop_integer	2-262
2.260loop_real	2-262
2.261lplot_xy	2-263
2.262lplot_xyz	2-265
2.263mass_density	2-266
2.264mass_gamma_bar	2-266
2.265mass_gamma_bulk	2-267
2.266mass_gamma_well	2-267
2.267mass_l_bar	2-267
2.268mass_l_well	2-268
2.269mater_var	2-268
2.270material	2-269
2.271material_3d	2-271
2.272material_par	2-273
2.273max_electron_mob	2-275
2.274max_hole_mod	2-275
2.275mesh_output	2-276
2.276metal	2-276
2.277min_electron_mob	2-277

2.278min_hole_mob	2-277
2.279mmb_gaintable	2-277
2.280mobility_xy	2-279
2.281mode_srch	2-282
2.282modify_bulk_macro	2-286
2.283modify_opt_gen_rate	2-287
2.284modify_plot	2-288
2.285modify_qw	2-289
2.286modify_wurtzite	2-294
2.287monitor_emission	2-295
2.288modu_bias	2-295
2.289more_output	2-296
2.290multimode	2-298
2.291multimode_detect	2-299
2.292multimode_mirror	2-301
2.293new_doping	2-302
2.294newton_par	2-304
2.295newton_freeze_carrier	2-309
2.296nonlocal_transp_region	2-310
2.297norm_field	2-311
2.298ohmic_junction	2-313
2.299oled_control	2-313
2.300optic_coating	2-314
2.301optical_field	2-315
2.302organic_exciton_diff	2-317
2.303output	2-317
2.304outer_section	2-318
2.305oxd_diff_length	2-318
2.306oxd_lifetime	2-319
2.307oxd2_diff_length	2-319
2.308oxd2_lifetime	2-319
2.309ox_dopant_el_transfer	2-320
2.310ox_el_weight	2-321
2.311ox_exciton_eg	2-321
2.312ox_exciton_eff	2-322
2.313ox_extern_spectrum	2-322
2.314ox_gaussian_sdj	2-323
2.315ox_gaussian_divj	2-323
2.316ox_hopping_energy	2-324
2.317ox_life_field_dependence	2-324
2.318ox_peak_abs	2-325
2.319ox_vib_quanta	2-325
2.320ox_xp_coupling	2-326

2.321para_extract	2-326
2.322passive_3d	2-329
2.323passive_carr_loss	2-329
2.324passive_fiber	2-330
2.325pc_led_model	2-331
2.326plot_1d	2-332
2.327plot_ac_curr	2-334
2.328plot_2d	2-335
2.329plot_3d	2-336
2.330plot_3dcolor	2-338
2.331plot_3dmesh	2-339
2.332plot_ac_curr	2-340
2.333plot_ac_laser	2-343
2.334plot_ac_modal_gain	2-344
2.335plot_amp	2-345
2.336plot_bias	2-346
2.337plot_ac_parameters	2-348
2.338plot_data	2-350
2.339plot_distri	2-350
2.340plot_mesh	2-352
2.341plot_modu	2-352
2.342plot_rtgain	2-353
2.343plot_scan	2-355
2.344plot_shd	2-359
2.345plot_spec	2-360
2.346plot_structure	2-361
2.347plot_z	2-362
2.348pml	2-363
2.349point	2-364
2.350polarization	2-365
2.351polygon	2-365
2.352power3d	2-366
2.353previous_layer	2-366
2.354print_active_layer	2-367
2.355print_macro	2-367
2.356put_mesh	2-368
2.357qdot_layer_mater	2-369
2.358qdot_material	2-371
2.359quasi_3d_transient	2-373
2.360qwell_normal	2-374
2.361qw_optics_control	2-375
2.362q_transport	2-376
2.363raw_output	2-379

2.364rcled_dbr	2-380
2.365rcled_mix_lambertian	2-381
2.366rcled_model	2-382
2.367rcled_optic_layer	2-384
2.368rcled_plot_y	2-385
2.369rcled_power_angle	2-386
2.370rcled_refl_coating	2-386
2.371rcled_spectrum	2-387
2.372rcled_spectrum_angle	2-387
2.373rcled_surface_plot_xy	2-388
2.374radiative_boundary	2-389
2.375radiative_recomb	2-389
2.376rayplot_bias	2-390
2.377rayplot_angle	2-390
2.378rayplot_showray	2-390
2.379real_func	2-391
2.380real_index	2-391
2.381real_index_spec	2-391
2.382refine_mesh	2-392
2.383reload_mater	2-393
2.384replace_macrofile	2-393
2.385resistor_region	2-394
2.386restart	2-394
2.387right_contact	2-395
2.388rtgain_phase	2-395
2.389scan3d	2-396
2.390scan	2-399
2.391scan_function	2-407
2.392section	2-409
2.393section_air	2-411
2.394section_location	2-412
2.395section	2-413
2.396self_consistent	2-416
2.397set_active_reg	2-420
2.398set_barrier_width	2-420
2.399set_minority_carrier	2-420
2.400set_polarization	2-421
2.401set_spon_spectrum	2-421
2.402set_wavelength	2-422
2.403set_xydata_for_scan	2-422
2.404set_3dray_internal_interface	2-424
2.405shal_acpt_level	2-425
2.406shal_acpt_level_i	2-425

2.407shal_dnr_level	2-426
2.408shal_dnr_level_i	2-426
2.409show_ray_3d	2-426
2.410shift_affinity	2-427
2.411sor_par	2-428
2.412sparse_eigen_solver	2-428
2.413sp.rate_wavel	2-429
2.414spec_heat	2-430
2.415splot_xyz	2-431
2.416splot_xy	2-433
2.417start_loop	2-434
2.418stop	2-435
2.419strain_bar	2-435
2.420strain_well	2-435
2.421strained_mobility	2-436
2.422suprem_contact	2-437
2.423suprem_property	2-438
2.424symbol_variable	2-439
2.425tau_density	2-441
2.426tau_energy	2-442
2.427tau_model	2-442
2.428tau_temperature	2-443
2.429tax_mass_bar	2-444
2.430tax_mass_well	2-444
2.431temperature	2-445
2.432thermal_interf	2-445
2.433thermal_kappa	2-447
2.434thermal_kappa_xy	2-447
2.435tmass_gamma_bar	2-448
2.436tmass_gamma_bulk	2-448
2.437tmass_gamma_well	2-449
2.438top_contact	2-449
2.439trap_excitation	2-450
2.440trap_level_i	2-451
2.441trap_ncap_i	2-451
2.442trap_pcap_i	2-451
2.443tunneling	2-453
2.444use_bulk_affinity	2-456
2.445use_bulk_bandgap	2-457
2.446use_bulk_property	2-457
2.447use_macrofile	2-457
2.448use_sor	2-458
2.449valj_mass_para (j=1..3)	2-459

2.450	valj_mass_perp (j=1..3)	2-459
2.451	valj_valley_propk (j=1..3;k=1,2)	2-459
2.452	val_bandj_edge (j=2,3)	2-460
2.453	vcsel_eeim	2-460
2.454	vcsel_model	2-461
2.455	vcsel_section	2-462
2.456	vectorial_wave	2-463
2.457	view_dipole	2-464
2.458	view_ganvalence	2-464
2.459	view_kpsubband	2-465
2.460	vplot_xy	2-466
2.461	vplot_xyz	2-467
2.462	wave_boundary	2-468
2.463	waveguide_input	2-470
2.464	wurtzite_offset_model	2-471
2.465	x_position	2-472
2.466	xpoint_size	2-473
2.467	y_position	2-473
2.468	yz_amplifier_model	2-474
2.469	zero_doping	2-474
2.470	zener	2-475
2.471	zincblende_offset_model	2-475
2.472	zsegment_setting	2-476
2.473	z_structure	2-477
A	List of Graphic Devices	A-481
B	Vectorial Variables For Plotting	B-485
C	Scalar Variables For Plotting	C-487
D	Bias Dependent Variables For Plotting	D-493

CHAPTER 1

INTRODUCTION

1.1 Applicable Simulation Software Packages

This reference manual applies to the following simulation programs:

- **APSYS**, a general purpose 2D simulation program for material and layer design of semiconductor devices.
- **LASTIP**, a two-dimensional simulation program for laser diodes.
- **PICS3D**, a 3-dimensional simulation program for waveguided active/passive devices.
- **GEOMETRY**, a general purpose mesh generator based on fully connected polygons.
- **LAYER**, a user friendly device geometry input program useful for LASTIP, PICS3D and APSYS.
- **SETUPLAYER**, a setup utility program for setting up layer structures.
- **SETUPAPSYS**, a setup utility program for setting up input files for APSYS.

1.2 Input Files and Input Statements

The simulation engine, or the core simulator, may be regarded as a numerical crunching program that is controlled by a set of commands written in ASCII input files. Whether you plan to become an expert of using the software, or simply to become a casual user by clicking on the graphic user interface program, it is a good idea to take a look into various components of the simulation package.

The installation directory is by default "c:

crosslig" under which a number of important ASCII files are located. The files with extension .tab are the one containing a listing of statements or commands recognized

by the simulator. The ones with extension .mac are the material libraries written using a collection of statements (or macros).

To run a simulation program on a PC, one may open a DOS prompt and type the following

```
Simulator.exe inputfile [enter]
```

where the first word is the executable of the corresponding software package (such as Lastip.exe) and *inputfile* is an ASCII text file containing lines of statements or commands recognized by the simulation program. The statements in *inputfile* control what task the application program is going to perform and what type of output is to be produced. For details of how to install the program on your computer and how to run the software, please refer to the General Manual for Crosslight Device Simulation Software. This reference manual is mainly used to explain the input statements in the input file.

1.3 Syntax of Input Statements

An input file contains a collection of statements recognized by the application. The statements have been invented to allow the user to interact with the program. A statement starts with a keyword followed by parameters specified with “=”. The general format of a statement is

```
(statement, statement keyword or statement name) parameter1=value1 &&
    parameter2=value2 parameter3=value3 &&
    parameter4=value4 ...
```

Please note the use of the words “statement”, “parameter” and “value” here. For example, the following line is used to specify a point in 2D space:

```
point label=b5 xy=( 1.5, 1.66 )
```

where “point” is the keyword and “label” and “xy” are names of the parameters. “b5” and 1.5 and 1.66 are the corresponding values of the parameters. The example is used to label a point located at coordinate (1.5, 1.66) to be “b5”. Note that the order of the parameters within a statement is not important; the following statement is equivalent to the previous example:

```
point xy=( 1.5, 1.66 ) label=b5
```

There are three types of parameters: real, integer and character. The values specified must match the corresponding types of the parameters. In the above example, “label” is a character type and “xy” is real type with two values (or real $\times$ 2).

If two statements describe the same property of the device, the latter one will override the former. For example,

```
bandgap mater=3 value=2.14
```

```
bandgap mater=3 value=2.25
```

Material number 3 will have bandgap of 2.25 eV.

The length of a line in a statement must not exceed 80 characters. For a longer statement, the continuation symbol && must be used at the end of the line to be continued.

Comments start with “\$”. A comment can start at the beginning of a line or at any midpoint of a statement line. Space, “,”, “(”, “)”, “[”, “]”, “{” and “}” within a statement are invisible to the program command interpreter. For example the following three statements are equivalent:

```
point xy=( 1.5, 1.66 ) label=b5
point xy=[1.5 1.66] label=b5
point xy=1.5 1.66 label=b5
```

the program scans through the statement line and, when it hits “xy”, will expect two real numbers. It will ignore the dummy symbols mentioned above and fetch the two real numbers 1.5 and 1.66. **Warning:** The tab key must not be used in any statements.

For more advanced syntax and physical variable expressed as functions, please see remarks in the header of crosslight.mac located within the installation directory.

1.4 A Short Cut For Setting Input Parameters

For the convenience of our users, we have made all of the usable input parameters completely transparent to the end users. Everytime a simulation program runs, it must read in a file with a table of usable input parameters. This table contains the names of the parameters, the data types and the corresponding default values. These files have extension .tab and are placed under SOFTWARE directory within the installation directory. For example, the main engine uses crosslig/crosslight.tab as table of usable parameters.

The number of control parameters is large for a sophisticated simulation software like Crosslight device simulator. Fortunately, most parameters in the control statements have reasonable default values and the user does not need to alter them. It is difficult to memorize the spelling and use of these parameters. A short cut to take is to copy part of the input statement from crosslig/crosslight.tab into your input file and make changes to the default values.

For example, the statement **more_output** may be used to direct the simulator to generate more output data than its default settings. Its listing in crosslight.tab looks like the following.

```
more_output
    ac_data          char          no
    qw_states        char          no
    ac_light_input    char          no
    elec_conc_subb    intg          0
    hole_conc_subb    intg          0
```

cond_valley_prop	intg	0
val_valley_prop	intg	0

You may copy the above statement and rearrange it in the following usable form:

```
more_output &&
  ac_data=no &&
  qw_states=yes &&
  ac_light_input=no &&
  elec_conc_subb=0 &&
  hole_conc_subb=0 &&
  cond_valley_prop=0 &&
  val_valley_prop=0
```

1.5 Material Macros and Mathematic Functions

Many statements are used to describe the material properties of a semiconductor device, such as bandgap, mobilities, affinities, *etc.* . Putting all of these statements into the input file makes it long and hard to manage. A good alternative is to put the material statements in a separate file, *i.e.*, the macro library, and refer to it in the main input file through a macro. A macro is used to represent a group of material statements. Therefore a call to the macro in the main input file is equivalent to replacing it with many material statements.

For material statements, it is possible to use a user-defined function to specify the material property. For example

```
band_gap variation=function
function(x,temp)
for 0. < x < 0.45
shift=-5.5e-4 * temp**2 /(temp +225) +9.428E-02
1.424 +1.247 *x +shift
for 0.45 < x < 1.
shift=-5.5e-4 * temp**2 /(temp +225) +9.428E-02
1.9 +0.125 *x +0.143 *x *x +shift
end_function
```

The above material statement has an unusual format because of the additional function. If the parameter “variation” is given a value “function”, the PICS3D program expects the next statement is a mathematical function used to define the numerical value of the bandgap. PICS3D interprets all lines between “function(x,temp)” “end_function” as mathematical expressions. The dummy variables (x,temp) are two symbols of input variables. In this example, they are used to represent the Al composition and temperature, respectively.

The following information from the header of the macro file is relevant to material macros.

\$ What Is A Macro

\$

\$ A macro is a collection of input statements (or commands).

\$ A statement starts with a keyword followed by parameters

\$ specified with "=".

\$ A statement for material parameters overrides a

\$ previous statement with the same keyword.

\$ To make changes of the parameters in a macro, just re-issue

\$ the statement with different parameters after the macro call

\$ in the input file. You can also change the parameters in the

\$ macro directly.

\$

\$ For example, if you are unhappy with the bandgap in gaas macro,

\$ you can do the following:

\$ load_macro name=gaas mater=#m

\$ band_gap value=1.425 mater=#m

\$

\$

\$ Two Types of Macros

\$

\$ For device simulation, you may need two types of macros:

\$

\$ 1) Bulk material macros. These are given lower case macro names

\$ such as "algaas". They are referred to by "load_macro" in the

\$ input files. You must specify bulk material macro for

\$ all material layers. They specify general material

\$ parameters such as bandgap, mobility, index, etc.

\$

\$ 2) Active layer macros. These are given mixed case macro names

\$ such as "AlGaAs". These are only needed for active layers in a

\$ laser diode or quantum well layers in other devices. Quantum

\$ subbands and/or optical processes are considered in these layers.

\$ If there are overlaps with 1), the parameters (such as bandgap)

\$ in active layer macros will override those in 1). Active layer

\$ macro with prefix cx- (such as cx-InGaAsP/InP)

\$ are "complex active layers" which may be used to define

\$ active or quantum regions of complex structures.

\$

\$ You must use both "bulk material macro" and "active layer macro"

\$ if you have an active region or a quantum region.

\$

\$

\$ Syntax

```

$
$ Comments start with "$". A line to be continued terminates
$ with "&&". Symbol definition within a function terminates with ";".
$ A statement starts with a keyword followed by parameters
$ specified with "=". Space, ",", "(", ")", "[", "]", "{" and
$ "}" with a statement are ignored by the program. The length
$ of a statement line must not exceed 80 characters. The tab key is
$ not to be used in any statements.
$
$ A math function is declared using variation=function and
$ followed in the next few lines by
$
$ function(var1,var2,var3,...)
$ for logical_expression_1
$ complete_function_definition_1
$ for logical_expression_2
$ complete_function_definition_2
$ for logical_expression_3
$ complete_function_definition_3
$ ...
$ for else
$ complete_function_definition_N
$ end_function
$
$ The logical_expression takes the format
$ expr1<expr2 or expr1<expr2<expr3
$ It also supports one of | (logical OR) or & (logical AND) operator.
$ to use "for else" as a case indicator. The following lines are legal:
$
$ function(x,y,xlam)
$ for 1.24/xlam<bulk_xfunc1
$ complete_function_definition_1
$ for 0.5<y
$ complete_function_definition_2
$ for y>0.1&xlam<0.8
$ complete_function_definition_3
$ for x<0.8|xlam<0.8
$ complete_function_definition_4
$ for else
$ complete_function_definition_5
$ end_function
$
$ Please note that ">" really means "greater than or equal". Similarly

```

```

$ "<" means "less than or equal". For continuous
$ numerical functions, there is no difference between "greater than or equal"
$ and simply "greater than".
$
$ The case indicator above separates the function into branches
$ of complete function definitions (like having different functions).
$
$ Within each branch of complete function definition, a set of
$ new and more convenient logical switches can be used in the following form
$
$ if(expression1) then
$ else if(expression2) then
$ else if(expression3) then
$ ...
$ else
$ endif
$
$ The "then" word is optional, so is the "if" following the "else".
$ The program looks for expressions within the
$ bracket. The following format is equivalent to the above.
$
$ if(expression1)
$ else (expression2)
$ else (expression3)
$ ...
$ else
$ endif
$
$ It is also possible to use an if-else-endif branching within another
$ if-else-endif. The hierarchy may reach 99 and within
$ each hierarchy, there can be 29 branches.
$ For example the following complicated logical branches are
$ supported:
$
$ if(adfa.eq.k) then
$ some_definition_lines
$
$   if(5<x*y)
$   some_definition_lines
$   else if(wavelength>1.9)
$   some_definition_lines
$   endif
$
$

```

```

$ some_definition_lines
$ else if (bulk_xfunc1>2.5) then
$ some_definition_lines
$ endif
$
$
$ Please note that a complete function definition can not
$ exceed 3000 characters. If you need to construct a function
$ with many logical branches of long math expressions,
$ it is recommended that you use the conventional
$ "for" logical branching method.
$
$ The logical expressions in "for" and "if-then-else" must
$ not exceed 80 characters. The math expressions follow the
$ same syntax as for function definition detailed as follows.
$
$ For a function definition, there can be more than one
$ math expressions. The "=" symbol is used to define intermediate
$ variables. Expression without "=" is interpreted as the returned
$ function value and causes the evaluation procedure to exit the function.
$ All expressions should be separated by ";" or "end-of-line".
$ An expression longer than 80 characters will be jointed
$ by the next line regardless of existence of "&&" at the end of the line.
$
$ The mathematical operators +, -, *, /, **
$ follow those of C/FORTRAN [** means pow() or power() in C] languages.
$ All intrinsic mathematic functions appearing in the
$ FORTRAN language, such as sin(), cos(),
$ sqrt(), log(), log10(), exp(), etc., can be used without modification.
$
$
$ Tables
$
$ In addition to analytical functions, parameters can also be specified
$ by numerical tables expressed in the form of an n-dimensional matrix:
$ para(j1,j2,...,jn). The index in each dimension corresponds to an
$ independent variable (such as composition or temperature).
$ The change of such a variable with the index is not necessarily of equal
$ interval but must be in ascending order. The table must be presented in
$ a column-wise manner and the left most index changes the fastest.
$ Blank lines may be inserted anywhere inside the table. For example:
$
$ electron_mass variation=table
$ table(x,y)

```

```

$   x(1)  y(1)  matrix(1,1)
$   x(2)  y(1)  matrix(2,1)
$   x(3)  y(1)  matrix(3,1)
$
$   x(1)  y(2)  matrix(1,2)
$   x(2)  y(2)  matrix(2,2)
$   x(3)  y(2)  matrix(3,2)
$
$ end_table
$
$
$                               Units
$
$ Dimensions are in micron meters.  Band parameters and
$ potentials are in eV or volts.  All others are in MKS units.
$ For example mobility is in  $m^2/(volt*sec)$ , band_gap in eV
$ and dopant concentration in  $1/m^3$ .
$
$                               Symbol Definition
$
$ max_electron_mob      Maximum dopant dependent elec. mobility.
$ min_electron_mob      Minimum dopant dependent elec. mobility.
$ electron_ref_dens      Reference impurity density in
$                        Dopant dependent elec. mobility function
$ alpha_n               exponent in dopant dependent elec. mobility
$                        function.
$ max_hole_mob           Corresponding symbol for holes.
$ min_hole_mob           ...
$ hole_ref_dens          ...
$ alpha_p               ...
$ tau_energy             Energy relaxation time in hydrodynamic model.
$ lifetime_n             Minority n-carrier (electron) life time.
$                        Deep trap specification will override this.
$ lifetime_p             Minority p-carrier (hole) life time.
$                        Deep trap specification will override this.
$ radiative_recomb       Radiative recombination constant.
$ other symbols          Obvious.
$
$
$                               Special Symbols, Temperature, Doping, etc.
$
$ "temper" is reserved as a variable for the lattice temperature
$ when constructing a material macro.  The unit is degrees Kelvin.

```

\$ If you use any other symbol for temperature within a macro,
 \$ it may not work for the thermal modeling option.
 \$ Please note that in a simulation
 \$ involving setting the environmental temperature, it is not
 \$ sufficient to set the temperature parameter in a material
 \$ macro. You must also use the "temperature" statement.
 \$ The temperature parameter in a macro is used to evaluate the
 \$ material properties (such as bandgap) at a particular temperature,
 \$ while the "temperature" statement is used to model the
 \$ carrier statistics at that temperature.
 \$
 \$ Similarly, "doping_n", "doping_p", and "trap_1", "trap_2", ..., "trap_9"
 \$ are reserved variable symbols in a function definition. The units are
 \$ in 1/m**3. "wavelength" is another reserved variable with unit of
 \$ micron meters.
 \$
 \$ # is used together with a symbol to denote a numerical value
 \$ to be supplied by the user. For example, in
 \$
 \$ load_macro name=gaas mater=#m
 \$
 \$ #m is used to mean a numerical value denoting the material
 \$ number in a device. For example, if your device has GaAs bulk
 \$ material as material number 2, then the above statement should be
 \$ written as
 \$
 \$ load_macro name=gaas mater=2
 \$
 \$
 \$ Macro Styles and Function Formats
 \$
 \$ An advanced feature of Crosslight Software macro is its ability
 \$ to define mathematical functions external to the simulation
 \$ program. Such functions are evaluated dynamically at run-time.
 \$
 \$ Currently, there are two kinds of macros with different formats of
 \$ functions: a) the older "fix-argument-style" macros and
 \$ b) the more recent "free-argument-style" macros. The older macros
 \$ are included here for compatibility reason and the newer formats are
 \$ highly recommended. The two formats are explained as follows.
 \$
 \$ a) Fix-Argument-Style Macros
 \$

\$ The rules for function variables/arguments are rather strict.
 \$ If a function is used for a physical quantity (presented as
 \$ a statement), there should be exactly the same number of
 \$ of function variables as that used in the "load_macro" or
 \$ "get_active_layer" statement. For example, if you use
 \$ (x,temper) as the variables in "load_macro", all functions
 \$ within the macro must have these two variables even if
 \$ all of them do not use the two variables. When calling such a macro,
 \$ the following can be used:

```
$
$ load_macro name=algaas.temp var1=0.3 var2=300
$
```

\$ Since all functions of such a style have the same number of
 \$ arguments, the program knows which value supplied belongs to which
 \$ argument in a function. Please note that when calling these kinds
 \$ of macros, the user must supply all values of the arguments even
 \$ if they are reserved symbols such as "temper", "doping_n", etc.

\$ b) Free-Argument-Style Macros

\$ The use of free-style format is more based on the need of variables.
 \$ A function passes and uses as many function arguments as it needs.
 \$ For example, the bandgap can pass and use both the composition
 \$ and temperature as (x,temper). But for the effective mass, only
 \$ the composition is needed and passed as (x). The evaluation of such
 \$ a macro is based on symbol matching, i.e., the user must specify
 \$ both the symbol and the value of the symbol when calling the macro,
 \$ such as in the following call.

```
$
$ load_macro name=algaas.temp var1=0.3 var2=300 &&
$   var_symbol1=x var_symbol2=temper
$
```

\$ The user may skip the specification of reserved internal arguments
 \$ such as "temper", "doping_n", etc., because the program already
 \$ knows their values. For example, the following may be used instead of
 \$ the above.

```
$
$ load_macro name=algaas.temp var1=0.3 var_symbol1=x
$
```

\$ Searching For a Macro

\$ The macro files can contain both the older and newer style macros

\$ with the same macro name. The search of the correct version depends
 \$ on how the macro is called. If a macro does not require
 \$ any user-supplied macro variables, the simulator will use the first
 \$ macro it encounters in the search. For macro requiring user-supplied
 \$ variables (or function arguments), if "var_symbol" is not specified
 \$ in a macro call, older style macro is assumed and user must strictly
 \$ align all variables in a macro call. The user is encouraged
 \$ to convert to the newer style because all future new macros will
 \$ be created and maintained in the newer free-style.

\$
 \$ Please also note that the older style (fix-argument-style) may be
 \$ regarded as a special case of free-style macro with many un-used
 \$ function variables listed in all macro functions. Or in another word
 \$ the older macro can be loaded also with a complete listing of
 \$ "var_symbol". In fact when the program detects a "load_macro" or
 \$ "get_active_layer" without any "var_symbol", it will search all the
 \$ macro functions and supply the necessary "var_symbol". In the case
 \$ of a macro call fits more than one macros in the macro library, the
 \$ simulator will always use the first one it encounters.

\$
 \$ For the free-style macros to be recognized by the graphic user
 \$ interface (GUI) such as LayerBuilder and GeoEditor, it must be labeled
 \$ as [free-style] in the remarks before the macro. Otherwise, it will be
 \$ treated as an older style macro.

\$

\$

\$ Mobility Parameter Specifications

\$

\$ In Crosslight device simulator, the dopant dependent low field mobility
 \$ takes the following form:

\$

\$
$$\mu_{\text{low_field}} = \mu_{\text{min}} + (\mu_{\text{max}} - \mu_{\text{min}}) / (1 + (\text{total_doping} / \text{ref_dens})^{\alpha})$$

\$

\$ The current version of the simulator supports two versions of
 \$ parameter specifications. The older version is to specify the
 \$ following four parameters for electrons:

\$ max_electron_mob, min_electron_mob, electron_ref_dens, and alpha_n.

\$ The program computes the above for each mesh point and average them
 \$ in mid-point between two nodes. Then the mid-point quantities of the
 \$ above is used in the dopant dependent formula at every iteration.

\$

\$ To speed up the computation, a more efficient version of parameter
 \$ specification is to directly evaluate the dopant dependence for

```

$ for statement of "electron_mobility", as in the following example.
$
$ electron_mobility variation=function
$ function(doping_n,doping_p,trap_1)
$ mu_max=0.14;
$ mu_min=0.0055;
$ ref_dens=1.07d23;
$ alpha=0.73;
$ total_doping=doping_n+doping_p+trap_1;
$ mu_min+(mu_max-mu_min)/(1+(total_doping/ref_dens)**alpha)
$ end_function
$
$ Speed-up is achieved because we need to evaluate the formula
$ once only at the time of processing the macro data.
$
$ The two versions of parameters specification are equivalent in theory
$ but may result in some difference if the mesh is rough because of the
$ non-linearity of the dopant dependent formula. Also, difference in
$ number of trap types may also make a difference. The older version
$ of specification automatically takes into account all trap densities
$ while the user need to explicitly specify the number of traps to
$ be used in the dopant dependent formula.
$
$
$ use_macro_file macro1=my_old_macro1 macro2=my_old_macro2
$
$ The simulation software will first search macros in the above
$ statement before searching the default crosslight.tab/more.tab.
$ Please note that that included macros must be placed in the
$ same directory as the input files.
$
$
$      Crosslight Quaternary Parameter Interpolation Scheme
$
$ For quaternary material such as In(1-x)Ga(x)As(y)P(1-y), experimental
$ data for arbitrary (x,y) normally do not exist. However, data are available
$ for ternaries [such as In(1-x)Ga(x)As] and for the lattice matched line:
$ fm(xm,ym)=0. Therefore, any reasonable interpolation scheme must
$ satisfy experimental data for all the ternaries and at the matched line.
$
$ Crosslight macros use the following interpolation scheme:
$
$ Let us take the bandgap as an example:

```

```

$ Given bandgap of matched line (xm,ym): Eg0_match(ym)
$ and measured ternary bandgaps, we interpolate as follows.
$ If (x,y) is within the left side of the matched line (xm,ym):
$
$ Eg0=Eg0_bilin(x,y)+(x/xm)*(Eg0_match(y)-Eg0_bilin(xm,y))
$
$ If (x,y) is within the right side of the matched line (xm,ym):
$
$ Eg0=Eg0_bilin(x,y)+[(1-x)/(1-xm)]*(Eg0_match(y)-Eg0_bilin(xm,y))
$
$ where Eg0_bilin(x,y) is bi-linear interpolation from ternaries as proposed
$ in the paper of Adachi,
$ [Adachi,S. "Material parameters of In(1-x)Ga(x)As(y)P(1-y) and related
$ binaries." J. Appl. Phys. Vol. 53 No. 12, Dec. 1982, pgs 8775-92.]
$
$ The Crosslight formula ensures that the unstrained bandgap
$ agrees with experiment at ternaries and at the matched line (xm,ym).
$ For any other composition, we draw horizontal line across
$ (x,y) and linearly interpolate given the three known points:
$ (0,y), (xm,y), (1,y)
$
$           Supported Intrinsic Math Functions
$
$ Math functions are directly taken from Fortran90 and accuracy of
$ double precision is already used. Please note that the following
$ symbols are reserved for math functions and these should not be used
$ for user-defined variables.
$
$ function-symbol      equivalent double-precision version
$ log                  dlog
$ log10                dlog10
$ exp                  dexp
$ asin                 dasin
$ acos                 dacos
$ atan                 datan
$ sin                  dsin
$ cos                  dcos
$ tan                  dtan
$ sinh                 dsinh
$ cosh                 dcosh
$ tanh                 dtanh
$ erf                  derf
$ erfc                 derfc

```

```

$  sqrt                      dsqrt
$  abs                      dabs
$
$      Band Offsets and Alignments
$
$  For bulk macro, the band alignment is solely determined by the
$  affinity parameter. For active_layer macro, the band alignment
$  is determined by band_offset, band_discont, or band_discont_right.
$  band_discont and band_discont_right override band_offset. Then
$  band alignment settings in active_layer macro override the affinity
$  setting in bulk macro.
$
$  For quantum well with strain in well and/or barrier, it may be confusing
$  to use band_offset since hydrostatic strain changes the bandgap. Such
$  change may be partitioned between the conduction and
$  valence band with a certain ratio.
$
$  The shear strain may split the HH/LH/CH valence bands, making the
$  bandgap multi-valued. Different use of band_offset is controlled by
$  the command zincblende_offset_model or wurtzite_offset_model, for
$  zincblende or wurtzite material, respectively.
$
$  For zincblende material, the default is to use the strained bandgap
$  to calculate the offset and let the hydrostatic term partitioned
$  between conduction and valence bands.
$
$  For wurtzite the simulator assumes the conduction
$  band alignment is controlled
$  by the unstrained bandgaps of the well and barrier and hydrostatic
$  strain only shifts the valence band (no partition). This default
$  takes effect starting from versions later than July 2007.
$

```

1.6 Lifetime and Deep Level Traps

It is well known that non-radiative minority carrier lifetime is related to deep levels traps. It may be confusing how deep traps (or SRH recombination centers) and lifetimes are specified in a semiconductor simulator. A trap center is fully defined by trap density, trap level, charge type (donor or acceptor), and electron/hole capture cross sections. Here is a more detailed description how these quantities are defined within the program:

- 1) The program examines whether or not the "impurity=trap_i" (i=1,2,...) in "doping" statement has been used to define trap centers.
- 2) If "doping" has been used for traps, it already defines the trap density distribution, trap level, and charge type (donor or acceptor). The program then calculates the capture cross sections based on the lifetimes as specified in the "lifetime_n" and "lifetime_p" statements, as follows.

$$\sigma_n = 1/(v_n N_t \tau_n) \quad (1.1)$$

$$\sigma_p = 1/(v_p N_t \tau_p) \quad (1.2)$$

where σ_n is the capture cross section, v_n the electron thermal velocity, N_t the trap density and τ_n the electron life minority time (non-radiative). For holes, the same analogue applies.

- 3) If "impurity=trap_i" (i=1,2,...) in "doping" statement has not been used to define the traps, then the program takes $1.0 \times 10^{16} \text{ 1/m}^3$ as the default trap density, and donor as the default charge type. The default trap level is taken at half the bandgap. The program then calculates the capture cross sections based on the formulas in 2) above.
- 4) Finally, the program checks to see if the user has given any of the following statements, "trap_level_#" "trap_ncap_#", and "trap_pcap_#" to further detail the traps. If any of the above is given by the user, the trap level, capture cross sections defined through 1) to 3) above are overridden by values supplied by these statements.

To summarize, the user can either 1) define lifetimes only, 2) use "impurity=trap_i" (i=1,2,...) in "doping" to define lifetimes or 3) use "impurity=trap_i" (i=1,2,...) in "doping" statement and one or all of "trap_level_#" "trap_ncap_#", and/or "trap_pcap_#".

1.7 Various Types of MQW Models

The definition of multiple quantum well (MQW) types and the related models are somewhat complicated due to historical and compatibility reasons. Therefore we dedicate this session to explain various types of MQW models.

Fig. 1.1 indicates 3 cases of interests. Early versions of Crosslight device simulators assumed all quantum wells in a device are symmetric and well isolated [case (1)] so that quantum mechanical wave functions can be solved analytically (this explains the fast speed of the simulator). A further simplification is that the applied field is relative weak (as in a forward biased laser diode) so that flat band condition can be assume at all times. If a bias is applied, the quantum model still solves itself under flat-band condition but allows the carrier concentration to adjust as if in a

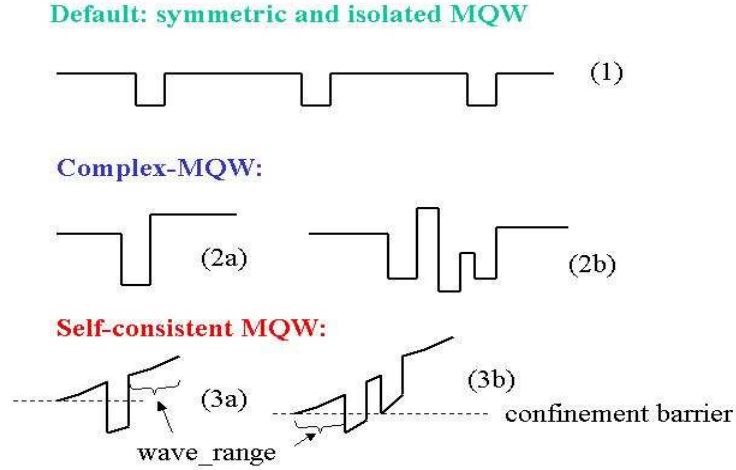


Figure 1.1: Schematic diagram explaining various MQW models

bulk semiconductor except the density of states (DOS) is calculated according to 2D quantum confinement.

As a result of assumptions in case (1), we are able to label each quantum well in an MQW system using the same symbol and the same material number. If you examine the material number in a .mater file, you will find that a single material number is used to identify many wells in a MQW system. This allows the simulator to carry out the quantum model computation once only and saves a large amount of computation time. The active macro for a quantum well in case (1) is usually in the form of XXX/YYYY (such as InGaAs/AlGaAs) requiring the specification of parameters for both the well and the barrier. The question is that if the full device structure itself contains information about the barrier already, why should the parameters for the barrier be entered into again in the XXX/YYYY macro? The answer is that the same XXX/YYYY macro is also used in a mesh-less simulation (a simulation that does not require any mesh) such as in a simulation using the .gain file. In a mesh-less simulation, there may not be any specification of a barrier material at all and one has to rely on parameters in XXX/YYYY. This isolated flat-band model covers most of devices and users have created thousands of input files based on this model, making it difficult to abandon the older scheme of active layer specification. This explains why the legacy of XXX/YYYY macros lives on and likely will remain so for some years to come.

When the material system deviates from the isolated and symmetric situation, we introduce the complex-MQW model as indicated in case (2) where flat-band condition is still assumed. The difficulty in this case is how to inform the simulator where to treat the MQW as an independent quantum region. The use of XXX/YYYY is no longer appropriate since one needs to specify both barriers which may be different

from each other. We have decided to introduce a new type of macro with prefix `cx-`. The whole complex-MQW system can be symbolized as a series of `cx-XXX` layers joined together with each `cx-XXX` describing either the well or the barrier. Therefore, a typical MQW system is labeled by `cx-` macros in the follow form:

$$cx - XXX || cx - YYY || \dots || cx - ZZZ \quad (1.3)$$

There must be odd number of `cx-XXX` macros since we require two layers on the left-most and right-most side to be barriers. A special case is an isolated single well with non-symmetric barrier as indicated in case (2a).

A further upgrade of the above cases (1) and (2) is to relax the flat-band assumption and to apply the quantum mechanical model at all times rigorously, as indicated by case (3) in Fig. 1.1. A relatively simple application of model in case (3) is to relax the flat-band condition in case (1), i.e., isolated, symmetric and non-flat-band situation. This will replace the isolated wells in case (1) by a series wells in case (3a). Even though the materials of the wells are identical, the applied potentials for each well may be totally different. Therefore, the quantum model must be applied separately to all wells as many times as the number of isolated wells. Since each quantum well material number is solved only once, it is necessary we label all the wells by different material numbers even though the macro symbols of `XXX/YYY` are the same. The use of `"independent_mqw"` statement in a `.layer` file will automatically generate different material numbers for different isolated quantum wells, even though the materials are the same.

It is important to realize that the number of confined levels are determined by the confinement barrier which is given by the lower potential on the left-most or the right-most barrier. It is obvious that such a confinement barrier is sensitive to the extend to which the potential profile is selected for quantum treatment. Such extend is given by the parameter `"wave_range"` in the `"self_consistent"` statement and this is indicated in case (3). One has to select `"wave_range"` using a reasonable value. If it is chosen to be too large, there may not be any confined levels. If it is chosen to be too small, the potential variation in the barrier may not be taken into account properly and the accuracy of the levels may be in doubt.

The most sophisticated MQW model is to use both complex-MQW option (i.e., use of `cx-XXX` macros) and self-consistent option (non-flat-band). However one must exercise great caution when using this model if the applied field is strong. For example, in case (3b), due to the position of the confinement barrier, there may only be one confined state located around the first quantum well, whereas in an isolated well model, a confined state will be found in the 2nd well. In reality, a quasi-stable state (with much broadened energy) exists around the 2nd well and may be observed experimentally. Since the quantum mechanical model in the simulator only finds perfectly stable (static model) states, the user should be aware that quasi-stable states may be missed if complex-MQW option with many wells are solved using the self-consistent (non-flat-band) option.

1.8 Segments, Planes, Tapers and Z-Bendings

In Crosslight software, understanding and setting up mesh for a full 3D simulation is very simple. You do not need to be a 3D-meshing expert and there is no need to understand all the technical jargons related to 3D finite-element mesh set up. All we assume is that you already know how to produce a 2D mesh. You just need to learn how to stack up these 2D mesh planes and you will obtain a 3D mesh for your simulation.

Visualizing 3D structure has always been a challenge for both the programmer and the user. At Crosslight, we have created an easy way to set up, visualize and to describe a 3D structure based on well known 2D mesh/structure.

One may argue that such a simple stack-up of 2D mesh can only be regarded as quasi-3D structure and may not be suitable for complicated 3D objects. However, we have invented ways to modify and combine the 2D mesh planes in various ways so that sophisticated 3D structure can be constructed using minimal amount of mesh points. Such variation and combination are called **taper** and **bending** and shall be explained in this section.

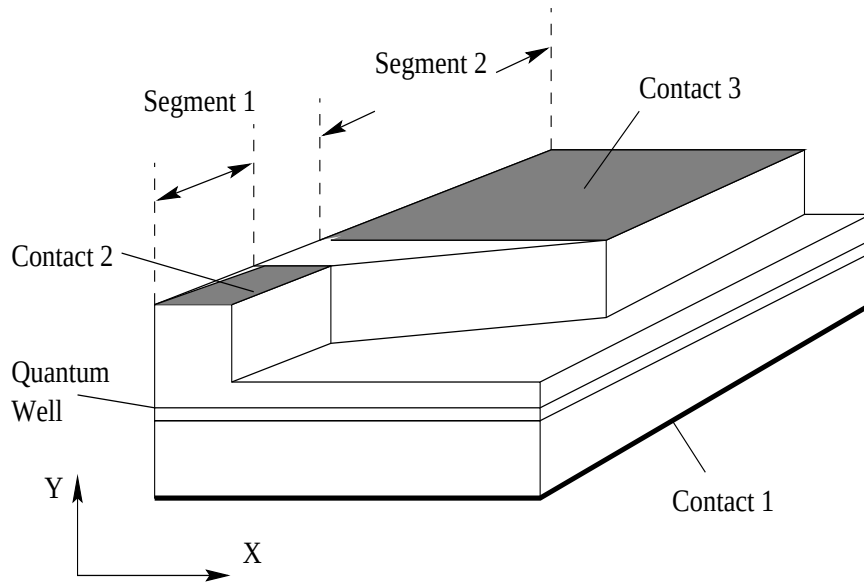


Figure 1.2: An example 3D structure showing multiple segments and a taper wall.

We refer to the schematics in Fig. 1.2. z-segment (or simply segment) is the top level division in z-direction, followed by z-planes and tapers. They have the following properties:

- 2D (x-y) distribution of material, doping, electrode and mesh points must be uniform in the z-direction within a segment.
- Physical variables to be solved (**solvable**s) such as electron/hole concentrations and local potentials are allowed to vary along z-direction within a segment.

- Due the uniformity of material/doping properties, only one mesh file needs to be used for each segment.
- A segment can contain one or more z-planes (or planes simply). All planes within a segment has the same nodal structure if no **bending** operation is applied to these planes.
- Solvables may vary from plane to plane within the same segment.
- There are no planes between two adjacent segments.
- There can be taper walls connecting two adjacent segments. A taper wall is used to force nodes on the two adjacent planes on the same side of the wall to connect to each other.

For further explanation of the taper wall concept, let us consider the example in Fig. 1.2 where a taper wall connects the narrower waveguide in segment 1 to the wider waveguide in segment 2. The last plane within segment 1 (let us label it as plane-1-last) connects to the first plane within segment 2 (let us label it as plane-2-first) via this taper wall. As a result, all nodes on plane-1-last towards the side of $x=0$ of the wall connects to all nodes on plane-2-first on the same side.

The advantage of using taper walls is that even without any mesh planes between two adjacent segments, mesh connectivity may be established wisely to reduce the total number of mesh planes. In the example discussed above, one would need several mesh planes between the two segments in order to obtain a smooth numerical solution within the tapered division (or the gap between the two segments). With the introduction of an x-taper, none is required in the tapered division, resulting in substantial saving of mesh planes or total mesh points.

Taper walls may be used to divide and connect nodes along the x-direction and thus they are referred as x-taper walls. Similarly, one may define y-taper walls.

An important modification to the z-planes are called **bending**. This means shifting a node on an existing plane, situated at an original position of (x_0, y_0, z_0) , to a new position at $(x_0, y_0, z_0 + dz)$. The amount of shift dz is called **bending**. Since it is a function of x_0 and y_0 the operation of dz on a mesh plane looks like physically bending the flat mesh plane into a specific shape (such as spheres, cylinders, angled bends, etc.). In general, it is possible to let the bending dz to vary from plane to plane. Therefore, dz is a function x_0, y_0 and z_0 , written as $dz(x_0, y_0, z_0)$. Crosslight simulator allows the user to use analytical or piece-wise analytical formulas to specify the bending function $dz(x_0, y_0, z_0)$ in any form for any shape. GUI may be used to drag and draw to form such bendings.

One simple bending operation on a single mesh plane may be illustrated by Figs. 1.3 and 1.4. The original flat mesh plane is position at $z=2$. Then we apply a bending of dz in the following form:

$$dz = 0.6 - \text{sqrt}((x - 2.5) * 2/10. + (y - 2.5) * 2/12.)/2. \quad (1.4)$$

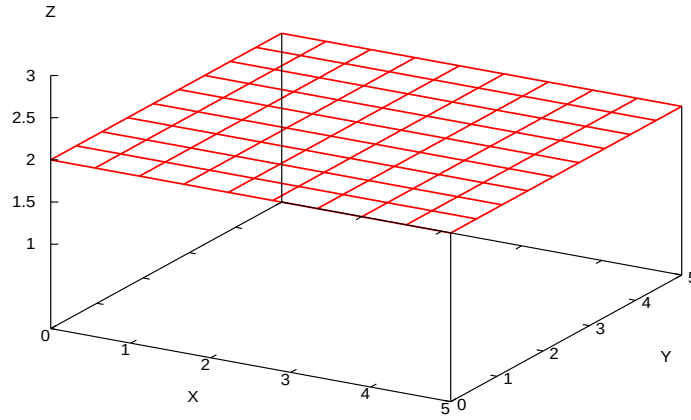


Figure 1.3: An original flat mesh plane before bending operation.

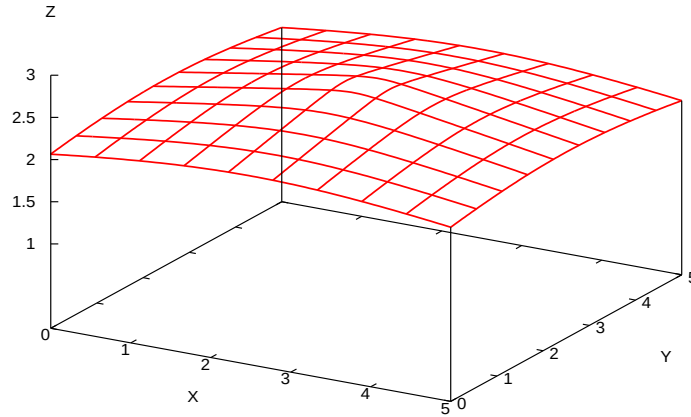


Figure 1.4: The mesh plane after bending operation according to $dz=0.6\text{-}\sqrt{((x-2.5)^2/10.+(y-2.5)^2/12.)/2}$.

The bended mesh plane is shown in Fig. 1.4.

A step by step approach to 3D simulation is to first slice up a realistic device into a number 2D segments/planes. Secondly for each 2D segment, one may perform a 2D simulation to ensure the correct set up of physical parameters within such 2D structure. Finally, one may stack up the 2D mesh segments/planes which have just been well tested, to construct the final full 3D simulation. During this final step,

taper walls and bending operations may be used to capture the full details of the realistic 3D device without the need for heavy meshing. In many realistic cases, only 2 or 3 segments are needed with about 2 to 3 planes within each segments, if taper walls and/or bendings are to be used.

The beauty of this approach is that it can be extended easily to cylindrical coordinate system. A 2D simulation using (r,z) coordinates is usually used to describe a 3D structure with rotation symmetry it is also our starting point. If the rotation symmetry is broken, variation in the phi angle is necessary and we must use all three coordinates (r,z,phi). In such a case, we can make an analogy with rectangular coordinate system of (x,y,z) so that phi-variation in the cylindrical system corresponds to z-variation in the rectangular system. All concepts of segments, planes, tapers and bendings may be borrowed directly from rectangular to cylindrical system. Such an approach as been successfully applied for VCSEL's and cylindrical photo-detectors with non-symmetric contacts and materials.

In a more sophisticated extension of the above approach, Crosslight can combine cylindrical and rectangle coordinate system in the same simulation.

How it works: use "z_structure cylindrical=yes" for the segment you wish to declare as cylindrical. The third coordinate unit of um turns into degrees when the system changes from rectangle to cylindrical in a continuous manner.

Suppose you have rectangle coordinate from 0 to 10 um, followed by a 90-degree cylinder. Then, the value of z (or the third coordinate) goes from 0 to 10 microns for the rectangle part and 10 to 100 (to cover 90 degrees) for the cylindrical one. The following input deck fragment can be used to define such a structure.

```
3d_solution_method 3d_flow=yes
```

```
z_structure    uniform_zseg_from=0.0    uniform_zseg_to=0.0 &&
zplanes=1     zseg_num=1
```

```
z_structure    uniform_zseg_from=1.0    uniform_zseg_to=1.0 &&
zplanes=1     zseg_num=2
```

```
z_structure    uniform_zseg_from=5.0    uniform_zseg_to=5.0 &&
zplanes=1     zseg_num=3
```

```
z_structure    uniform_zseg_from=9.0    uniform_zseg_to=9.0 &&
zplanes=1     zseg_num=4
```

```
z_structure    uniform_zseg_from=10.    uniform_zseg_to=10. &&
zplanes=1     zseg_num=5
```

```
z_structure    uniform_zseg_from=40    uniform_zseg_to=40 &&
zplanes=1     zseg_num=6 cylindrical=yes
```

```
z_structure    uniform_zseg_from=55  uniform_zseg_to=55 &&
zplanes=1  zseg_num=7  cylindrical=yes
```

```
z_structure    uniform_zseg_from=70   uniform_zseg_to=70 &&
zplanes=1  zseg_num=8  cylindrical=yes
```

```
z_structure    uniform_zseg_from=100   uniform_zseg_to=100 &&
zplanes=1  zseg_num=9  cylindrical=yes
```

In summary, Crosslight software allows users to build up sophisticated full 3D simulations starting from simple 2D structures. The full 3D simulation may be regarded as natural extension of the previous 2D simulations. Tapers and bendings can be used to realize fine details in a realistic 3D structure without great sacrifice of computation time.

1.9 Longitudinal Sections in PICS3D

z-segment (or simply segment) is the top level division in z-direction. A multi-section waveguiding structure may consist of more than one section with different electrodes and material properties. When setting up a longitudinal structure, please observe the following division rules.

- A segment can contain one or more sections. A segment can only have one active section but may contain one or more passive sections.
- Each active section is represented by 2D structure in .layer for edge waveguide structure. For VCSEL, both passive and active sections within a segment are represented by one 2D structure in .layer.
- For decoupled solution, each segment [or its active section(s)], there must be separate current scan/injection.

Since current injection is applied to active section/segment only, if you wish to control the physical property of a passive section via current injection, this section must be declared "active". When a section is declared "active", current can be pumped into it using statement **scan3d** and it is also possible to specify any grating structure using statement **grating_model** in .layer or the 2nd part of .sol.

1.10 Output Data Format

This section is to describe the output data format in Crosslight Software which is essential to understand commands related to input/output.

```
[Bias data (scan data):]
0.3 volt, 0.6 Amp.
0.6 volt, 1.2 Amp.
1.0 volt, 2.2 Amp.
...
[poistioin dependent variables such as potential, electron density at 1.0 volt]
```

Figure 1.5: Organization of an output data file (device1.out_0002).

All of the simulation results of are written and stored in files defined by the statement **output**. The output data are divided into bias data (or scan_data) and structural data (or position dependent data or xy_data, which describes the distribution of physical variables over space or frequency domain). Bias data includes quantities such as contact current, contact voltage and laser power. Structural data includes spatial dependent variables such as potential, electron concentration, current distributions and optical gain spectrum.

The simulator prints and stores all bias data but only prints selected structural data because of disk storage limits. For example when the user ramps up the bias from 0 volt to 1 volt, APSYS may have to solve the equations in 4 steps (at 0, 0.3, 0.6 and 1 volt) before the bias reaches 1 volt. In such a case all 4 bias data points (*e.g.*, I-V data) are stored and printed in a single output data file. However the user may only want to store the large amount of structural data at 0 and 1 volt only.

As a convention, the bias data are stored under a file name defined in the **output** statement. All structural data (or xy_data) are stored under the same file name with extension “-##”, where ## ranges form 0001 to 9999. For example, if the output file is given a name “device1.out”, then all the bias and structural data will be stored in “device1.out_0001”, “device1.out_0002”, “device1.out_0003”, *etc.* The assignment of file numbers are controlled by the sequence of the **scan** or **scan3d** commands and the **print_step** parameter.

Every time the simulator prints a set of structural data it also prints the bias data cumulated since the printing of the previous set of structural data. The simulator groups the printed structural data and the cumulated bias data together and assigns a “data set number” to them. The user refers to the output data by its data set number.

For the example above, the output file device1.out_0003 may look like Fig. 1.5.

The bias condition of a particular printed data set can be controlled by the user, and this information is stored in the “message file” with extension “.msg”. For example, if the user has just completed a simulation with input file “gaas1.sol”, then the bias condition for each printed data set can be found in file “gaas1.sol.msg”. The data set numbers are needed when the user wishes to plot the data at a particular bias condition.

Notes on older decoupled solution method of PICS3D: PICS3D’s quasi-3D module is a smart driver which combines a 2D simulation module for the xy-plane of

the laser diode and a 1D simulation module for the z-dimension (longitudinal). This treatment is possible because of a unique feature of the laser diode geometric structure, the longitudinal and lateral or transverse dimensions are relatively decoupled in the sense they can be treated numerically as separate modules. This does not imply that the two modules are physically independent; in fact, almost all parameters of one module are supplied by the other and each can not operate properly without the other.

For clarity, we separate the input files for the main driver (.sol file) and the post-processor into separate modules, one for the 2D simulation module and one for the 1D module in the z-direction.

For efficiency, many edge structures are simulated using the above decoupled method, as controlled by "3d_flow=no" in statement "3d_solution_method". For fully coupled (full 3D) treatment, it is no longer appropriate to use the word 2D module or 2D segment. Instead they should be understood as a z-segment within a full 3D structure whenever they are referred to in this document.

All the simulation results of PICS3D are written in data files with extension .zlog, .zdata, and .out_###. These output data files may be regarded as black boxes to users, where the output data is extracted by specifying the 3D data set number. It is required that all input/output files have the same base file name.

To get the output data at a specified current bias, the user must first determine what data set number to be used. This number is entered into the **get.3ddata** statement in the post-processor input file (.plt). This data set number must correspond to the points in the **scan3d** statement.

At the writing of this document, the PICS3D quasi-3D decoupled solution method is being phased out and replaced by the new coupled round trip gain (RTG) method. The new RTG method uses the same output data structure as described in the first part of this section. The older decoupled method is described for compatibility reason.

1.11 Conventions in This Manual

This manual defines the statements used in the simulation programs. For each statement, all parameters are first tabulated and explained. The first column lists the names of the parameters. The second column is the data type of parameter. It can take the type of character (char), real (real) and integer (intg). The third column is the possible values and defaults in square brackets [].

The program GEOMETRY, LAYER, SETUPPLAYER do not count as independent products because they work together with the products mentioned above.

The default units are dimension in μm or in meters, band parameters and potentials in eV or volts, and all others in MKS units. For example, mobility is in $\text{m}^2/(\text{volt}\cdot\text{sec})$, band_gap in eV, current in A/m, voltage in volts, and dopant concentration in $1/\text{m}^3$. For most parameters, their names, data types, possible values and defaults are tabu-

lated. Material parameters are not tabulated for each individual statement because almost all material parameters have the same parameters. For example, the statement **band_gap** and **affinity** have an identical number of parameters and identical parameter names. Therefore the parameters for all material statements are tabulated under the keyword **material_par**. Note that **material_par** is by itself not a statement but is used to represent a large number of material statements.

The SOFTWARE is designed such that it depends on a number of resource files which list all the input statement keywords and parameter names together with their defaults values. The user can examine these files list as follows

```
{  
crosslig/crosslight.tab  
crosslig/layer.tab  
crosslig/geometry.tab  
}
```

In both the .tab file and in this document, some parameters are given a default value of -9999. This really means that no default has been given. Since -9999 is a rarely used number, the program uses this to recognize an undefined parameter. If you really must use -9999.0 as you input floating number, you need to revise it to be something slightly different, say -9999.0001 so that the program does not think it is undefined.

CHAPTER 2

STATEMENTS FOR SIMULATION SOFTWARE

2.1 2nd_harmonic

parameter	data type	values [defaults]
freq_shd1	real	[0.1e9] (Hz)
freq_shd2	real	[0.1e9] (Hz)
freq_shd3	real	[0.1e9] (Hz)
freq_shd4	real	[0.1e9] (Hz)
freq_shd5	real	[0.1e9] (Hz)
freq_shd_num	intg	[1]

The statement **2nd_harmonic** is used to specify the modulation frequencies at which the second harmonic distortion ratio is to be computed.

- **freq_shd#** is the modulation frequency of channel number #.
- **freq_shd_num** is the number of modulation channels being considered for calculation.

Example(s)

```
2nd_harmonic freq_shd1=1.e7 freq_shd2=1.e8 freq_shd3=2.e8 &&  
freq_shd4=6.e8 freq_shd_num=4
```

2.2 3d_amplifier_model

parameter	data type	values [defaults]
binary_search	char	[yes]
wavelength	real	[1.55] (μm)
input_watt	real	[0.001] (Watt)
init_factor	real	[1]

The statement **3d_amplifier_model** is used to specify the input power and wavelength of the semiconductor optical amplifier (SOA). It is assumed that input light is incident from the left facet and output power is from the right facet.

- **binary_search** indicates whether binary search is used to narrow the solution.
- **wavelength** is the input light wavelength in μm .
- **input_watt** is the input light power in Watt. Please also see **init_factor** parameter below.
- **init_factor** is used to multiply the **input_watt** to obtain the total initial input power. In case the input power needs to be varied, parameters "wg_light_from" and "wg_light_to" of **scan3d** statement can be used to define a factor which is added to this parameter.

Example(s)

```
3d_amplifier_model wavelength=1.3 input_watt=1.e-4
```

2.3 3drayplot_angle

parameter	data type	values [defaults]
data_file	char	void
smooth	char	[yes] no
view_xrot	real	[0.]
view_zrot	real	[0.]
theta_range	realx2	[0. 180.]
phi_range	realx2	[0. 360.]
smooth_width	real	[8.](degree)

3drayplot_angle is used to plot the angular dependence of the emitted power as a function of two angular variables in the form of power(phi,theta), where phi and

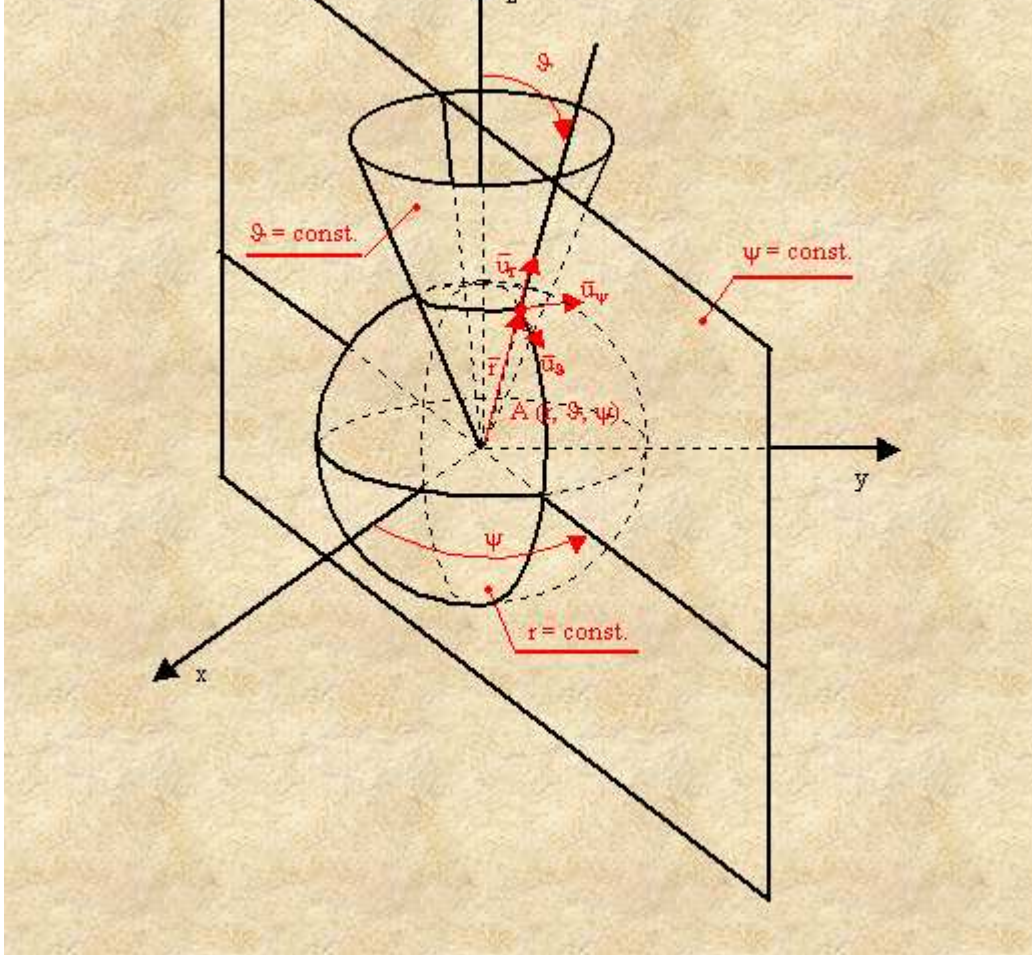


Figure 2.1: The spherical coordinate system used

theta are the two angles in a spherical coordinate system. The two angular variables may be used to construct a 2D axis system and we are able to obtain a 3D surface plot of power(phi,theta).

The definition of the spherical coordinate system is a commonly used one. The angles are in units of degree. Figure 2.1 shows how angles measure.

In order to guarantee results show the expected symmetry when this command is used in a different .plt file as **do_raytrace_3d**, the same **set_3dray_mirror** statement should be reused.

- **data_file** copies the plot results to the specified data file.
- **view_xrot** rotates the plot around the phi-axis.
- **view_zrot** rotates the plot around the power-axis.
- **theta_range** narrows the area plot within theta angle range.
- **phi_range** narrows the area plot within phi angle range.
- **smooth** indicates if data smoothing is performed.

- **smooth_width** is the width of the data broadening/smoothing. A Gaussian function is used for smoothing and this parameter is the standard deviation. The larger the standard deviation, the data curve appears to be more smooth.

Example(s)

```
3drayplot_angle phi_range=(180. 360.)
```

2.4 3drayplot_bias

parameter	data type	values [defaults]
variable	char	[transmitted]
relative	char	[no] yes
bias_variable	char	[current]voltage
data_file	char	[void]
bias_electrode	intg	[1]
data_file	char	

3drayplot_bias is used to plot the bias dependence of the specified variable generated by the 3D ray-tracing program. This statement is used at the post-processing stage after 3D ray-tracing program is run. Please note that for LED application, the results from the ray-tracing program may be different from those from the cubic broad-area device LED model as plotted by the **plot_scan** statement.

In order to scale the results correctly when this command is used in a different .plt file as **do_raytrace_3d**, the same **set_3dray_mirror** statement should be reused.

- **variable** is the physical variable to be plotted. It takes one of the values explained in the following table.
-

variable	definition
total_source	is the total power of the emitting source. It may be used to compute the internal efficiency.
transmitted	is the total transmitted power. It may be used to compute the total extraction efficiency.
semicond_absorb	is the power absorbed in semiconductor materials.
contact_absorb	is the power absorbed in metal contacts.
upper_half	is the power emitted towards the upper half semi-sphere.
lower_half	is the power emitted towards the lower half semi-sphere.
left_half	is the power emitted towards the left half semi-sphere.
right_half	is the power emitted towards the right half semi-sphere.
front_half	is the power emitted towards the front half semi-sphere.
back_half	is the power emitted towards the back half semi-sphere.
upper_side	is the power emitted through the upper side facet(s).
lower_side	is the power emitted through the lower side facet(s).
left_side	is the power emitted through the left side facet(s).
right_side	is the power emitted through the right side facet(s).
front_side	is the power emitted through the front side facet(s).
back_side	is the power emitted through the back side facet(s).

-
-
- **relative** indicates whether relative or absolute values are being plotted.
 - **bias_variable** is the bias variable to appear in the horizontal axis of the plot.
 - **data_file** copies the plot results in the specified data file.
 - **bias_electrode** is the electrode/contact number of parameter **bias_variable**.
 - **data_file** would export the data to a file.

Example(s)

```
3drayplot_bias variable=transmitted bias_variable=current
```

The above statement plots the total transmitted LED power versus the current from electrode number 1.

2.5 3drayplot_phi

parameter	data type	values [defaults]
data_file	char	[void]
smooth	char	[yes] no
polar	char	[yes] no
theta	real	[0.] (degrees)
smooth_width	real	[8.](degree)

3drayplot_phi is used to plot the phi angular dependence of the emitted power. We use phi and theta to denote the two angular variables in a spherical coordinate system.

In order to guarantee results show the expected symmetry when this command is used in a different .plt file as **do_raytrace_3d**, the same **set_3dray_mirror** statement should be reused.

- **theta** is the fixed-theta angle for the plot of power versus phi. It ranges from 0 to 180 degrees.
- **data_file** copies the plot results to the specified data file.
- **polar** determines whether or not the results are shown in polar coordinates.
- **smooth** indicates if data smoothing is performed.
- **smooth_width** is the width of the data broadening/smoothing. A Gaussian function is used for smoothing and this parameter is the standard deviation. The larger the standard deviation, the smoother data curve.

Example(s)

```
3drayplot_phi theta=0
```

2.6 3drayplot_project

parameter	data type	values [defaults]
view_xrot	real	[0.]
view_zrot	real	[0.]
distance	real	[1.]
radius	real	[2.]
position_phi	real	[270.]
position_theta	real	[90.]

The variables of **3drayplot_project** statement

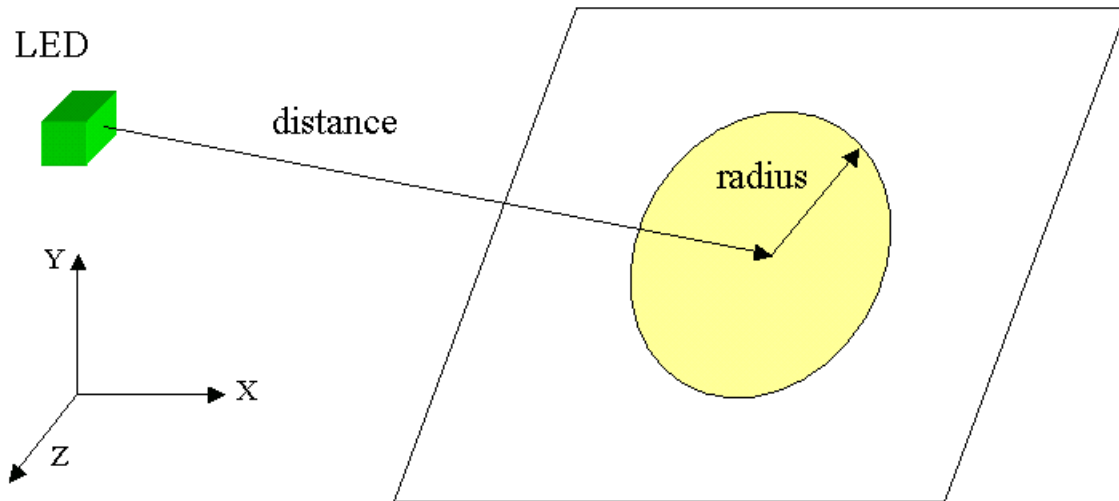


Figure 2.2: The variables **distance** and **radius** of **3drayplot_project** statement

3drayplot_project is used to provide a projection plot with different positions of a screen. The variables **position_phi** and **position_theta** are two angles in the same spherical coordinate system as described in section **3drayplot_angle**. Figure 2.2 shows how the distance from an LED center to the screen center and the radius of circle area of projection are measured.

- **view_xrot** rotates the plot around the phi-axis.
- **view_zrot** rotates the plot around the power-axis.
- **distance** is distance to the screen center and it measures from the center of LED.
- **radius** is radius of the circle area of projection.
- **position_phi** is a screen center position defined by angle phi of the spherical coordinate system with its zero in the center of an LED.
- **position_theta** is a screen center position defined by angle theta of the spherical coordinate system with its zero in the center of an LED.

The angles are in degrees. The distance and radius are in units of meter.

Example(s)

```
3drayplot_project radius=3. position_phi=90.
```

2.7 3drayplot_surfpower

parameter	data type	values [defaults]
data_file	char	[void]
smooth	char	[yes] no
side	char	-x,+x,-y,+y,-z,+z
x_range	realx2	[-9999. , -9999.] (um)
y_range	realx2	[-9999. , -9999.] (um)
smooth_width	real	[1.](um)

3drayplot_surfpower is used to plot the surface emission power in a 3D ray tracing simulation.

- **data_file** copies the plot results to the specified data file.
- **side** determines which side is to be plotted.
- **smooth** indicates if data smoothing is performed.
- **smooth_width** is the width of the data broadening/smoothing. A Gaussian function is used for smoothing and this parameter is the standard deviation. The larger the standard deviation, the smoother the plot.
- **x_range** and **y_range** can be used to zoom in on a particular section of the plot. Note however that the original data is still limited by **surface_power_grid** in **do_raytrace_3d**.

2.8 3drayplot_theta

parameter	data type	values [defaults]
data_file	char	[void]
smooth	char	[yes] no
polar	char	[yes] no
phi	real	[90.] (degrees)
smooth_width	real	[8.](degree)

3drayplot_theta is used to plot the theta angular dependence of the emitted power. We use phi and theta to denote the two angular variables in a spherical coordinate system.

In order to guarantee results show the expected symmetry when this command is used in a different .plt file as **do_raytrace_3d**, the same **set_3dray_mirror** statement should be reused.

- **phi** is the fixed-phi angle for the plot of power versus theta. It ranges from 0 to 360 degrees.
- **data_file** copies the plot results to the specified data file.
- **polar** determines whether or not the results are shown in polar coordinates.
- **smooth** indicates if data smoothing is performed.
- **smooth_width** is the width of the data broadening/smoothing. A Gaussian function is used for smoothing and this parameter is the standard deviation. The larger the standard deviation, the smoother data curve.

Example(s)

```
3drayplot_theta phi=90
```

2.9 3dget_data

parameter	data type	values [defaults]
main_input	char	
sol_inf	char	
3ddata_set	intg	1,2,...

The statement **3dget_data** is used for the post-processor, *i.e.* it is used only after the main solution is complete. It is used to get data from the data output of the main solver for PICS3D.

- **main_input** is the main input file that was used to drive the main simulation. It usually has the file extension .sol .
- **sol_inf** is the main simulation output data file which contains the data for current, carrier distribution, *etc.* It is produced by the input file **main_input**. It should be consistent with the parameter **output** in the main input file (.sol file).

- **3ddata_set** Is used to refer to the 3D spatial output data sets. It should correspond to the **3d_points** in the **scan3d** statement.

Example(s)

```
3dget_data main_input=gaas_yz.sol sol_inf=gaas_yz.out &&
3ddata_set=7
```

2.10 3d_plane_control

parameter	data type	values [defaults]
z_link_same_mater_only	char	[no]
min_cyl_plane_distance	real	[1.e-4](um)
scale_z_coupling	real	[1.]

The statement **3d_plane_control** is similar to the **3d_solution_method** and it is used to define a number of parameters to control the mesh connectivity between z-planes.

- **z_link_same_mater_only** would force mesh connectivity only between the same material.
- **min_cyl_plane_distance** would set a minimum distance between cylindrical planes. At $r=0$ of a cylindrical system, the distance between planes is zero may cause mesh connectivity issues in some cases. This parameter may resolve such problems.
- **scale_z_coupling** would be used to scale the coupling coefficient (in units of areas) between z-planes.

Example:

```
3d_plane_control z_link_same_mater_only=yes
```

would disable mesh connection between different materials on different planes. This avoids undesirable current leakage through different materials.

2.11 3d_solution_method

parameter	data type	values [defaults]
3d_flow	char	[no] yes
z_connect	char	yes [no]
z_connect_method	char	single[multiple]

The statement **3d_solution_method** is used to define a solution method for a 3D structure. It is related to the statement **z_structure**.

- **3d_flow**, if set to “yes”, is used to indicate a full 3D solution allowing current to flow between x-y planes. If set to “no”, only optical wave variation in z-direction is considered while current is forbidden to flow between z-planes (quasi-3D solution). In the new coupled round trip gain (RTG) method for PICS3D when solving active waveguide, this command is also used to mean simultaneous solution of all the z-planes while current flow may or may not be allowed between planes (depending on z_connect).

z_connect is related to the PICS3D-RTG module. The mesh connection may be disconnected to save solution time.

z_connect_method is the numerical method of connecting the mesh points areas between z-planes. A value of “single” connection means any two mesh point areas are connected via clipping of single area. Multiple connection means multiple clipping areas are permitted. The latter are more accurate while the former more stable numerically.

Example:

```
3d_solution_method 3d\_flow=yes
```

would mean full 3D simulation for APSYS.

Example:

```
3d_solution_method 3d\_flow=no
```

In PICS3D the above would mean decoupling all plane interaction except for optical wave propagation.

Example:

```
3d_solution_method 3d\_flow=yes z_connect=no
```

In PICS3D the above would mean solving all planes simultaneously for its dynamic behavior while mesh connection between planes is removed to save CPU time.

2.12 a_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **a_bar** is an active layer macro statement used to define the hydrostatic deformation potential (eV) in the quantum barrier.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.13 a_bulk

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the total hydrostatic deformation potential in eV. This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.14 a_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **a_well** is an active layer macro statement used to define the hydrostatic deformation potential (eV) in the quantum well.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.15 absorption

parameter	data type	values [defaults]
material_par		

This material statement is used to defined the optical absorption or loss (in units of 1/m) in optical wavelength being used in the simulation. It may be used to define the optical loss of specific material region whereas **backg_loss** in statement **init_wave** may be used to define a uniform background loss in regions other than the active region. The use of this statement overrides **backg_loss**.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.16 absorption_spec

parameter	data type	values [defaults]
material_par		

This statement is the wavelength dependent version of the **absorption** statement. As the free-style macro becomes available in recent versions, it has been outdated since the wavelength dependence can easily be incorporated into the **absorption** statement function by adding a variable (xlam).

2.17 ac_bar

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the conduction band part of the hydrostatic deformation potential in eV for the barrier region. The total hydrostatic deformation is given by **a_bar**

This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, "Optical Gain of Strained Wurtzite GaN Quantum Well Lasers", IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.18 ac_bulk

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the conduction part of the hydrostatic deformation potential in eV. This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.19 ac_light

parameter	data type	values [defaults]
output_file	char	[void]
current_distr	char	[no] yes
versus_bias	char	yes [no]
log_freq1	real	[6.]
log_freq2	real	[12.]
freq_point	intg	[20]
scanline	intg	[-9999]
scan_num	intg	[-9999]

ac_light is used to apply a small AC light signal to a photo-sensitive device being simulated. The resulting current responses are saved in a data file so that AC characteristics can be plotted later. This statement is used in .plt files as a post-processing statement after the DC results are obtained. To use this statement, **more_output** should be used in .sol file to instruct the program to save necessary AC data while performing DC simulation.

The plot is usually against AC frequency but can also be against biases if **versus_bias=yes**.

- **output_file** is the output file where AC small signal analysis data for this statement are to be saved.
- **current_distr** is used to indicate if AC current distribution data are to be saved in the output file for later use.
- **versus_bias** indicates if the plot is against bias (or scanned variable).

- **log_freq1** is the starting frequency in log10 scale for the frequency scan.
- **log_freq2** is the ending frequency in log10 scale for the frequency scan.
- **freq_point** is the number of frequency points for the AC scan between **log_freq1** and **log_freq2**
- **scanline** is used only when **versus_bias=yes**. It is the same parameter as used in **plot_scan**. It corresponds to the line of bias statement in the solution file.
- **scan_num** equals **scanline-1**.

example(s)

```
ac_light output_file=testlight.ac log_freq1=6. log_freq2=12. &&
freq_point=20
```

This example applies an AC light signal and performs an AC scan from 1 MHz to 1 THz using 20 data points between. The results of the AC current responses are to be saved in output file testlight.ac.

2.20 ac_parameters

parameter	data type	values [defaults]
output_file	char	[void]
log_scan	char	[yes] no
versus_bias	char	yes [no]
log_freq1	real	[6.]
log_freq2	real	[12.]
scale_lit	real	[2.]
scale_curr	real	[2.]
input_contact	intg	
output_contact	intg	
freq_point	intg	[20]
scanline	intg	[-9999]
scan_num	intg	[-9999]

ac_parameters is used to set up the AC analysis for a 2-port high frequency system for the device being simulated. Once the input and output contacts are defined, AC analysis is performed assuming AC excitation signal at input and output contacts. The resulting current responses are saved in a data file so that AC characteristics (including Y- and S-parameters) can be plotted later. This statement is used in .plt

files as a post-processing statement after the DC results are obtained. To use this statement, **more_output** should be used in .sol file to instruct the program to save necessary AC data while performing DC simulation.

The plot is usually against AC frequency but can also be against biases if **versus_bias=yes**.

- **output_file** is the output file where AC small signal analysis data for this statement are to be saved. The simulator will assign an internal file name if this parameter is not defined.
- **log_scan** indicates whether log10 scale should be used to scan the frequency for the AC analysis.
- **versus_bias** indicates if the plot is against bias (or scanned variable).
- **log_freq1** is the starting frequency in log10 scale for the frequency scan.
- **log_freq2** is the ending frequency in log10 scale for the frequency scan.
- **scale_lit** is applicable for laser simulation. It is the scale of the light power used to plot output light power versus current for the laser simulation (for LASTIP). The default is 2 because LASTIP assumes the device is symmetric and the user only sets up one half of the device for simulation. This factor is multiplied to the total power (from 2 facets) as well as to power emitted from each facet.
- **scale_curr** is applicable for laser simulation. It is the scale of the current used to plot light versus current for the laser simulation. The default is 2 because LASTIP assumes the device is symmetric and the user only sets up one half of the device for simulation.
- **input_contact** is the input contact in a 2-port system. For example, in a common emitter configuration of a BJT, this should be the contact number for the base contact. Please note that in an AC analysis, all contacts are assumed to have zero AC voltage bias unless otherwise specified.
- **output_contact** is the output contact in a 2-port system. For example, in a common emitter configuration of a BJT, this should be the contact number for the collector contact. Please note that in an AC analysis, all contacts are assumed to have zero AC voltage bias unless otherwise specified.
- **freq_point** is the number of frequency points for the AC scan between **log_freq1** and **log_freq2**
- **scanline** is used only when **versus_bias=yes**. It is the same parameter as used in **plot_scan**. It corresponds to the line of bias statement in the solution file.

- **scan_num** equals **scanline**-1.

example(s)

```
ac_parameters log_freq1=6.  log_freq2=12.  input_contact=3 output_contact=2 &&
freq_point=20
```

This example performs AC analysis assuming contact number 3 as input contact, from 1 MHz to 1 THz using 20 data points.

2.21 ac_sparse_solver

parameter	data type	values [defaults]
use_mf	char	[yes],no

sparse_eigen_solver is used to control the sparse solver used in the .plt file for AC small signal analysis.

- **use_mf** is used to indicate whether multi-frontal solver will be used.

Example(s)

```
ac_sparse\_solver use_mf=yes
```

2.22 ac_voltage

parameter	data type	values [defaults]
output_file	char	[void]
log_scan	char	[yes] no
current_distr	char	[no] yes
versus_bias	char	yes [no]
log_freq1	real	[6.]
log_freq2	real	[12.]
scale_lit	real	[2.]
scale_curr	real	[2.]
contact_num	intg	[1]
freq_point	intg	[20]
modu_mode_index	intg	[1]
scanline	intg	[-9999]
scan_num	intg	[-9999]

ac_voltage is used to apply a small AC bias voltage on one of the electrodes. The resulting current responses are saved in a data file so that AC characteristics can be plotted later. This statement is used in .plt files as a post-processing statement after the DC results are obtained. To use this statement, **more_output** should be used in .sol file to instruct the program to save necessary AC data while performing DC simulation.

The plot is usually against AC frequency but can also be against biases if **versus_bias=yes**.

- **output_file** is the output file where AC small signal analysis data for this statement are to be saved. The simulator will assign an internal file name if this parameter is not defined.
- **log_scan** indicates whether log10 scale should be used to scan the frequency for the AC analysis.
- **versus_bias** indicates if the plot is against bias (or scanned variable).
- **current_distr** is used to indicate if AC current distribution data are to be saved in the output file for later use.
- **log_freq1** is the starting frequency in log10 scale for the frequency scan.
- **log_freq2** is the ending frequency in log10 scale for the frequency scan.
- **scale_lit** is applicable for laser simulation. It is the scale of the light power used to plot output light power versus current for the laser simulation (for LASTIP). The default is 2 because LASTIP assumes the device is symmetric and the user only sets up one half of the device for simulation. This factor is multiplied to the total power (from 2 facets) as well as to power emitted from each facet.
- **contact_num** is the used to indicate which contact is to be applied a small AC voltage bias.
- **freq_point** is the number of frequency points for the AC scan between **log_freq1** and **log_freq2**
- **modu_mode_index** applies to laser simulation. It is the lateral mode index for which modulation study is performed.
- **scanline** is used only when **versus_bias=yes**. It is the same parameter as used in **plot_scan**. It corresponds to the line of bias statement in the solution file.

- **scan_num** equals **scanline-1**.

example(s)

```
ac_voltage output_file=pmosB.ac log_freq1=6. log_freq2=12. &&
      contact_num=4 freq_point=20
```

This example applies an AC voltage to contact number 4 and performs an AC scan from 1 MHz to 1 THz using 20 data points between. The results of the AC current responses are to be saved in output file pmosB.

2.23 ac_well

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

It is the conduction band part of the hydrostatic deformation potential in eV for the well region. The total hydrostatic deformation is given by **a_well**

This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, "Optical Gain of Strained Wurtzite GaN Quantum Well Lasers", IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.24 active_reg

parameter	data type	values [defaults]
mode	char	te,tm [te]
broadening	char	[lorentzian] landsberg landsberg_2
axial_approx	char	[yes]no
valence_mixing	char	[no]yes
analytical_recomb	char	[no]yes
override_barrier	char	[yes]no
gain_coulomb	char	[no]yes
exciton	char	[no] yes
coulomb_dim	char	[quasi2d] 2d 3d
coulomb_screen	char	[yes] no
franz_keldysh	char	[no] yes
exch_coef	real	[0.0] (eV/m)
tau_scatt	real	[2.e-13] (sec)
a_scatt	real	[0.0] ($eVm^{3/2}$)
thickness	real	[0.05] (μm)
ncarr_loss	real	[0.0] (m^2)
pcarr_loss	real	[0.0] (m^2)
dip_factor	real	[1.]
k_range	real	[0.06]
tau_bkg	real	[5.e-13](sec)
expo_mix	real	[0.01] (eV)
active_loss	real	[0.] (1/m)
av_over_a	real	[0.33333]
k_range_wurt	real	[0.1]
diel_av	l real	[13.1]
plasma_coeff	real	[4.0]
sommerfeld_fac	real	[1.7]
rmesh_fd	real	[0.2]
mater	intg	
qw_print	intg	[1]
level_srch	intg	[30]
fd_mesh	intg	[80]
qw_plane	intg	[100]
coulomb_int_acc	intg	[6]
embedded_structure	intg	[0] (0/1)

The statement **active_reg** describes the active region of the semiconductor laser for laser related products. It may also be used to define quantum wells for APSYS. This

region may or may not actually produce positive optical gain. Using the statement merely means that for the material region, the most detailed models of optical gain spectrum (if applicable) and quantum wells (if applicable) are evaluated. Therefore one may use this statement to describe a loss region (or quantum well) in order to get an accurate description of the material properties therein.

- **mode** defines whether TE or TM modes are assumed in the gain calculation. The spontaneous emission is not affected by this choice because spontaneous emission is randomly polarized.
- **broadening** defines the type of energy level broadening to be used in the quantum well gain calculation. Bulk optical gain is not affected by this option. The gain spectrum in bulk material is not broadened because the degree of broadening is much smaller than in its quantum well laser counterparts. The following choices are possible:
 - “lorentzian” causes the SOFTWARE to use the ideal Lorentzian function with constant halfwidth Γ over all frequencies.
 - “landsberg” activates a broadening function of the Landsberg type, i.e., a Lorentzian function with an energy and carrier concentration dependent Γ . Note that this model does not include any broadening for optical gain below transparency (or loss). Therefore the gain spectrum contains step-like jumps in the negative gain region.
 - “landsberg_2” combines useful features of “landsberg” and “lorentzian”. The Landsberg type is active mainly in the positive gain region while the ideal Lorentzian type is active in the negative gain region. The parameters **tau_bkg** and **expo_mix** are used for this broadening option only. This can be considered a modified version of the ideal Landsberg model. The original Landsberg model assumes no broadening below transparency and results in sharp edges in the quantum well spectrum. The modified Landsberg model replaces the spectrum below transparency by a Lorentzian broadened gain with a different broadening tau of **tau_bkg** (for tau of background). Above the transparency, the gain function is a mixture of Landsberg broadened gain and Lorentzian broadened gain. Physically the background Lorentzian broadening may be from variation of quantum well thickness and mechanisms other than carrier-carrier scattering.
- **axial_approx** turns on/off the axial approximation in k.p band dispersion calculation.
- **valence_mixing** turns on/off the valence mixing model in the k.p quantum well theory. The 4×4 Luttinger-Kohn Hamiltonian is solved for the quantum well system based on the formulas of S.L. Chuang. For wurtzite structure system, a 6×6 Hamiltonian is used. The details are discussed in the physical model description in the User’s Manual.

- **analytical_recomb** is used to indicate if the analytical recombination term B_{np} is used to model the radiative recombination in the QW active region. For bulk active region, the term B_{np} is always used. For quantum well active region, the default model is to use numerical integral of the spontaneous recombination spectrum to obtain the recombination rate. Setting this parameter to yes will provide a simple alternative model which depends on the parameter of **radiative_recomb** (in macro).
- **override_barrier** is here for historical reason. In older versions, the barrier properties are determined by bulk macro while in newer versions, the bulk parameters (such as bandgap) of strained barrier may be overridden by those in the active layer macro. Setting this parameter to [no] will reproduce the older results.
- **gain_coulomb** indicates whether manybody Coulomb interaction should be taken in account in optical gain model. The shape of the gain and spontaneous emission spectrum may become more symmetric as a result of this model. The magnitude of the gain/PL spectrum may be enhanced. The bandgap renormalization effect is controlled separately by the parameter **exch_coef**.
- **exciton** turns on the exciton model.
- **coulomb_dim** specifies the dimension of the Coulomb potential used for derivation of exciton binding energy. The most appropriate for excitons confined to thin semiconductor layers is "quasi2d" model of Coulomb potential acting in layers with finite thickness such as inversion layers and MQW's. "3d" is a normal Coulomb potential without any spatial confinement; "2d" is a Coulomb potential acting only in 2-dimensional plane of charges (zero thickness), used here only for comparative studies.
- **coulomb_screen** means two options of Coulomb potential model for the purpose of finding binding energy of an exciton with screening ("yes") or without screening ("no").
- **franz_keldysh** turn on/off the Franz-Keldysh effect for bulk active layer.
- **exch_coef** is the bandgap renormalization coefficient due to exchange effects. The formula for this effect is $\Delta E_g = A_x \left(\frac{n+p}{2}\right)^{1/3}$. The parameter is A_x in this expression. A typical value is 1×10^{-10} eV.m.
- **tau_scatt**, **a_scatt** are the parameters used for the intraband scattering lifetime, which causes broadening of the gain spectrum. For the option of **type=lorentzian** the following formulas are used in the broadening of the gain function with a Lorentzian shape function F_l :

$$E_{scat} = \hbar/\tau_{scat} + A_{scat}\sqrt{(n+p)/2}, \quad (2.1)$$

$$g_{qw} = \sum_{i,j} \int g_{ij}(E_x) E_{sact} F_l \left[(E_x - E_{ij}^0) / E_{sact} \right] dE_x - \alpha_{fn} n - \alpha_{fp} p. \quad (2.2)$$

where the symbols have been defined in the User's Manual. For **type=landsberg** the following formula is used for the halfwidth:

$$\Gamma(E_1) = \Gamma_0 \left[1 - 2.229 \frac{E_1}{E_{fn} - E_{fp}} + 1.458 \left(\frac{E_1}{E_{fn} - E_{fp}} \right)^2 - 0.229 \left(\frac{E_1}{E_{fn} - E_{fp}} \right)^3 \right].$$

$$\text{for } E_{fn} - E_{fp} \geq 0 \text{ \& } 0 \leq E_1 \leq E_{fn} - E_{fp} \quad (2.3)$$

where $\Gamma_0 = \hbar / \tau_{scat}$.

- **thickness** defines the thickness of the active quantum well. It is used to compute the subband structures of the quantum well. Note that this parameter should be consistent with the device geometry given in the .geo input file. This parameter is not used if the device is a bulk laser.
- **mode** defines whether TE or TM modes are assumed in the gain calculation. The spontaneous emission is not affected by this choice because spontaneous emission is randomly polarized.
- **ncarr_loss**, **pcarr_loss** are the loss coefficients due to free carrier absorption for n and p carriers, as defined in Eq. (2.2). **pcarr_loss** can also be used to model the intervalence band absorption effects because it is also proportional to the p-carrier densities.
- **dip_factor** is the scaling factor for the dipole moment of the interband optical transition. It can be used to represent the overlap integral of localized and extended states when the optical transition is between these two types of states.
- **k_range** (in units of $2\pi/a_0$, where a_0 is the lattice spacing of the substrate material). It specifies the k-space range within which the in-plane effective mass is fitted from k.p theory for a strained material. In the extreme of large “k_range”, the fitted band averages out the detail of the k.p band and tends to under-estimate the optical gain. In the case of a small “k_range”, the model tends to over-estimate the optical gain.
- **tau_bkg** is the background scattering lifetime. It is used with “broadening = landsberg_2” to broaden the negative part of the gain spectrum.
- **expo_mix** defines the amount of mixing of Landsberg and Lorentzian broadening, used in an exponential factor

$$Gain = Gain_{lorentzian} \exp(-\Delta E_{imref}/expo_mix) \\ + Gain_{landsberg} (1 - \exp(-\Delta E_{imref}/expo_mix))$$

where ΔE_{imref} is the difference between Fermi level splitting and transition bandgaps. The meaning of “`expo_mix`” is that when ΔE_{imref} is greater than “`expo_mix`”, the Lorentzian component is negligible and the Landsberg component takes over. **`expo_mix`** may be of the order of kT to signal the onset of thermal activated carrier-carrier scattering.

- **`active_loss`** is used to add a background loss to the active region to account for loss mechanisms other than interband transition and free carrier absorption.
- **`av_over_a`** is the fraction of hydrastatic potential appearing in the valence band.
- **`k_range_wurt`** is the same as **`k_range`** except that this is for wurtzite material system.
- **`diel_av`** is the average dielectric constant of the well and the barrier for the manybody gain model.
- **`plasma_coeff`** is the plasma coefficient used in the manybody gain model.
- **`sommerfeld_fac`** typically takes a value between unity and two. Often called the Sommerfeld enhancement factor, it characterizes the degree of contribution of continuum state excitons into the exciton absorption. The continuum state excitons give some background in absorption without absorption peak, which is characteristic for bound excitons.
- **`rmesh_fd`** is related to **`fd_mesh`** and is used to control the uniformity of the finite difference mesh. The program uses a mesh ratio (see **`put_mesh`**) to control the uniformity. A ratio of unity means uniform mesh. The actual finite difference mesh ratio is set to $1 + \text{rmesh_fd}$ or $1 - \text{rmesh_fd}$, depending on structural details.
- **`mater`** is the material number assigned by the user to a specific material. This number should be consistent with the corresponding parameter in the **`polygon`** statement of the mesh generator.
- **`qw_print`** is a flag number controlling the amount of information on quantum well models printed to the screen and to the message file with extension “`.msg`”. When the value is “1”, only limited or no amount of information is printed to the screen. When it is “2”, detailed information on the quantum well model such as subband levels and transition matrix elements is printed to a file with extension “`.msg`” or as screen runtime message.

- **level_srch** is a parameter used in the case where the well width is very large (500 Å or greater) and the subband level finder could miss the first couple of quantum levels. To fix this problem, the new parameter **level_srch** may be used to sample the boundary condition function to search for the subband levels. When subband levels are very close to the bottom of the energy bands, a large **level_srch** should be used.
- **fd_mesh** is the finite difference mesh points used in computing the active region MQW subbands.
- **qw_plane** takes 100, 111 or 110. These are crystallographic planes used in the quantum MOS model.
- **coulomb_int_acc** is a selection number on the scale 1-9, which determines the order of Gauss-Legendre quadrature used in calculating integrals of electron-hole Coulomb interaction matrix element. The result is subsequently use in calculating exciton binding energy by Ritz variational method. For example: coulomb_int_acc=1 corresponds to 2nd order quadrature; coulomb_int_acc=6 (default) corresponds to 12th order quadrature; coulomb_int_acc=9 corresponds to 24th order quadrature.
- **embedded_structure** takes types 1 and 2. If it is set to zero, no embedded structures are present for the present layer/region. It is used to define an active material embedded inside a regular active layer. For type 1, it is used to define a corrugation grating in a DFB/DBR waveguide. A corrugation structure is regarded as two different kinds of active material embedded within an active layer material which has a material property equal to the overall average of the grating layer. The two embedded layers are considered active because interband model is used to calculate refractive index change) The 2nd type of embedded structure involves a quantum dot (QDOT) embedded within the wetting layer (regular quantum well). For quantum dot material, material style "cx-" is required. The embedded layers used to be called "mildly active layers" because most of properties of an active layer are included.

Example(s)

```
active_reg  exch_coef=3.e-10  &&
           tau_scatt=1.e-13  thickness=0.010  mater=3
```

2.25 active_reg_zdim

parameter	data type	values [defaults]
zdim	real	0.01(um)

active_reg_zdim is used to work with the LAYER program so that z-dimension information can be defined. Each single .layer file contains information about x-y plane. This statement is used to supply that information.

This statement is usually not needed unless the .layer file is used to describe a quantum well parallel to the x-y plane. To model quantum confinement effect, the program needs to know the thickness of the quantum well.

zdim parameter defines the z-dimension size. It is also the thickness of the quantum well if the .layer file is describing a quantum well parallel to the x-y plane.

Example(s)

```
active_reg_zdim zdim=0.01
```

2.26 active_temper

parameter	data type	values [defaults]
ref_temper	real	[300.] degree
delta_tau_scatt	real	[0.] (sec/K)
delta_ncarr_loss	real	[0.] (m^2/K)
delta_pcarr_loss	real	[0.] (m^2/K)
delta_active_loss	real	[0.] (m^{-1}/K)
mater	intg	[1]

The statement **active_temper** is used to specify temperature dependence in some of the important parameters for the active region of a laser diode. This statement may be used in the .gain or .sol files. For a parameter “apar”, the linear temperature dependence is given by the following formula:

$$apar = apar0 + \Delta apar(T - T_{ref})$$

where *apar0* is the initial value of the parameter at the reference temperature T_{ref} . $\Delta apar$ is the rate of temperature variation.

- **ref_temper** is the reference temperature.
- **delta_tau_scatt** is the rate of temperature change for the intra-band scattering lifetime.
- **delta_ncarr_loss** is the rate of temperature change for the intra-band electron free carrier absorption loss.

- **delta_pcarr_loss** is the rate of temperature change for the intra-band hole free carrier absorption loss.
- **delta_active_loss** is the rate of temperature change for the additional loss mechanism in the active region.
- **mater** is the mater number of the active region. It must match the material number in the **active_reg** statement.

Example(s):

```
active_temper ref_temper=300 delta_tau_scat=-0.01e-13 mater=3
```

2.27 add_arnoldimemory

parameter	data type	values [defaults]
megbyte	real	[0.]

add_arnoldimemory is used to add memory to the direct Arnoldi-eigen sparse solver. This is used only for special cases where the default memory allocation is insufficient.

- **megbyte** is the amount of memory in megabyte to be added.

Example(s)

```
add_arnoldimemory megbyte=5
```

2.28 add_mainmemory

parameter	data type	values [defaults]
megbyte	real	[0.]
sparse_fill	real	[0.] (megabyte)
symbolic	real	[0.] (megabyte)

add_mainmemory is used to add memory to the main solver. This is used only for special cases where the default memory allocation is insufficient. Usually, there will be error messages suggesting the use of this statement.

- **megbyte** is the memory added to main sparse solver at the numerical factorization stage.
- **sparse_fill** is the memory added to the sparse filling space.
- **symbolic** is the memory added to the symbolic factorization stage of the sparse solver.
- **megbyte** is the amount of memory in megabyte to be added.

Example(s)

```
add_mainmemory megbyte=2
```

2.29 add_thermalmemory

parameter	data type	values [defaults]
megbyte	real	[0.]

add_thermalmemory is used to add memory to the thermal solver. This is used only for special cases where the default memory allocation is insufficient.

- **megbyte** is the amount of memory in megabyte to be added.

Example(s)

```
add_thermalmemory megbyte=2
```

2.30 add_acmemory

parameter	data type	values [defaults]
megbyte	real	[0.]

add_acmemory is used to add memory to the ac solver. This is used only for special cases where the default memory allocation is insufficient.

- **megbyte** is the amount of memory in megabyte to be added.

Example(s)

```
add_acmemory megbyte=2
```

2.31 adjust_active_reg

parameter	data type	values [defaults]
valence_mixing	char	[void]
exch_coef	real	
tau_scat	real	
ncarr_loss	real	
pcarr_loss	real	
dip_factor	real	
active_loss	real	
qw_print	intg	

adjust_active_reg is used in the .layer file to adjust parameters in the **active_reg** statement when generating the .mater file. Please see **active_reg** statement for more details.

2.32 adjust_current

parameter	data type	values [defaults]
use_envelop_func	char	[yes] no
range_n	real	[1.e-3] (Vt)
range_p	real	[1.e-3] (Vt)

This statement is used to control some overall current flow characteristics.

One issue is the switch between drift-diffusion (DD) current flow model and thermionic emission (TIE) model. In the extreme of heterojunction with high energy barrier, complete thermionic emission model is used. When the barrier is low, there is a mixture of DD current and TIE current. This statement affects heterojunctions in the whole device.

Another issue is that, when self-consistent model is used to compute the confined carrier states, how we should treat the charge envelop function distribution when it comes to carrier transport in the sense of DD and TIE models. It is rather clear that when solving the static potential using the Poisson's equation, the envelop wave function of the confined carriers should be used to account for the charge distribution. However, question arises when using the same charge distribution in DD/TIE transport model. If we directly use the envelop function for transport (meaning use_envelop_func=yes), that means some part of carriers under the envelop may spill over the barriers and become free carriers, just like other unbound carriers. This contradicts with our assumption that they are "confined". On the other hand,

one may argue that there is good reason to convert the spill-over carriers into free carriers because of quantum tunneling effect. After all, tunneling transport occurs when a wave function penetrates a potential barrier and overlaps with the wave function of a free-carrier state. Such an overlap will certainly happen if an applied field lowers one of the barriers to near the confined energy level. However, the numerical accuracy of such a less rigorous quantum tunneling treatment should be the subject of further studies.

The other approach is to force all confined carriers to be within the well (meaning `use_envelop_func=no`) and let TIE takes care of all the transports. This approach completely ignores any quantum tunneling effects. Another problem is the violation of current continuity as far as carrier dynamics is concerned. One can show that if the Poisson's equation and the current continuity equations take different carrier distributions (i.e., n and p are different), the gradient of the total current (carrier current plus displacement current) will not be zero, thus violating current continuity in the semiconductor device in a transient or AC simulation.

At this point, we find using the same envelop function carrier distribution in all equations yields reasonable results if tunneling model is not activated for the potential barrier involved. On the other hand, if tunneling model is activated, use of the same envelop function carrier distribution for transport definitely produces over estimation of the transport current. Thus we recommend a) if direct tunneling is not activated, one should use “`use_envelop_func=yes`”, or b) if direct tunneling is used to overcome the related barrier, one should use “`use_envelop_func=no`”,

- **`use_envelop_func`** is used to indicate whether the same envelop wave function in the Poisson's equation should be used in current continuity equations. It is used only for self-consistent MQW model.
- **`range_n`** is the electron barrier height (in thermal voltage $V_t = k_b T/q$) below which mixture of DD and TIE takes place.
- **`range_p`** is the hole barrier height (in thermal voltage $V_t = k_b T/q$) below which mixture of DD and TIE takes place.

Example(s)

```
current_mix_range range_n=0.01 range_p=0.01
```

2.33 adjust_doping

parameter	data type	values [defaults]
impurity	char	[void]
pf_model	char	void
level	real	

adjust_doping is used in the `.layer` file to adjust parameters in the **doping** statement when generating the `.doping` file. Please see **doping** statement for more details.

2.34 `adjust_resistor`

parameter	data type	values [defaults]
<code>shift_barrier</code>	real	[0.2](eV)
<code>mater</code>	intg	[1]

The statement **adjust_resistor** is to be used if resistor junction is schottky instead of the default ohmic junction. In the absence of this statement, the program assumes a specific metal/resistor junction are ohmic and the barrier potential of the metal is determined by charge neutrality at local equilibrium. This is similar to the cases of contacts of Ohmic and Schottky types. If this statement is used, the program takes the macro statement **affinity** as the work function of the Schottky resistor plus an adjustable parameter **alter_work_func**.

- **alter_work_func** is used to alter the work function of the metal/resistor.
- **mater** is the material number of the metal/resistor of interests.

Example(s)

```
adjust_resistor  alter_work_func=0.1 mater=4
```

The above statement makes the junction between resistor material number 4 and its adacent semiconductor to be a Schottky junction with band alignment shifted by 0.1 eV.

2.35 `affinity`

parameter	data type	values [defaults]
(see) <code>material_par</code>		

The material statement **affinity** is used to define the electron affinity (in eV) of the semiconductor material being described. It is the energy difference between the vacuum level and the conduction band edge. In a heterojunction device, different

affinities in different materials cause the conduction band discontinuity. For an active quantum well, however, the band offset parameter in the active layer macro used in the quantum well model overrides this parameter.

For resistor/metal material region, this parameter is also used to describe the work function.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.36 ai_bar,i=1,2,...

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the valence band effective mass parameter in the barrier. This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.37 ai_bulk,i=1,2,...

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the effective mass parameter. This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.38 ai_well,i=1,2,...

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the valence band effective mass parameter in the well. This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.39 alpha_n

parameter	data type	values [defaults]
(see) material_par		

The material statement **alpha_n** is used to define a unit-less parameter appearing in the low field electron mobility function of the material. The following generic formula for low field electron mobility is used:

$$\mu_{0n} = \mu_{1n} + \frac{(\mu_{2n} - \mu_{1n})}{1 + \left(\frac{N_D + N_A + \sum_j N_{tj}}{N_{rn}} \right)^{\alpha_n}}, \quad (2.4)$$

For more detail on this formula, please consult the User’s Manual.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.40 alpha_p

parameter	data type	values [defaults]
(see) material_par		

The material statement **alpha_p** is used to define a unit-less parameter appearing in the low field electron mobility function of the material. The following generic formula for low field electron mobility is used:

$$\mu_{0p} = \mu_{1p} + \frac{(\mu_{2p} - \mu_{1p})}{1 + \left(\frac{N_D + N_A + \sum_j N_{tj}}{N_{rp}} \right)^{\alpha_p}}. \quad (2.5)$$

For more detail on this formula, please consult the User's Manual.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.41 alpha_wavel

alpha_wavel is the same as **gain_wavel** except it plots the alpha factor.

2.42 auger_n

parameter	data type	values [defaults]
(see) material_par		

The material statement **auger_n** is used to define the Auger coefficient C_n (in units of m^6/s) as in the formula $R_{aug} = (C_n n + C_p p)(np - n_i^2)$.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.43 auger_p

parameter	data type	values [defaults]
(see) material_par		

The material statement **auger_p** is used to define the Auger coefficient C_p (in units of m^6/s) as in the formula $R_{aug} = (C_n n + C_p p)(np - n_i^2)$.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.44 b_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **b_bar** is an active layer macro statement used to define the shear deformation potential (eV) in the quantum barrier.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.45 b_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **b_well** is an active layer macro statement used to define the shear deformation potential (eV) in the quantum well.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.46 back_index

parameter	data type	values [defaults]
real_index	real	[-9999.]
imag_index	real	[0.]

back_index is used to specify the complex refractive index at the back of a semiconductor device. It serves as the outside index used to compute the Fresnel reflections in **light_power**.

Unlike **front_index**, this command does not assume air or vacuum; instead, the last semiconductor index is assumed to continue so there are no reflections.

This command has no practical effect if **back_reflection** is used.

2.47 back_reflection

back_reflection is used to specify the reflection of light power at the back of the semiconductor device. It is otherwise exactly the same as **front_reflection**.

2.48 band_discont

parameter	data type	values [defaults]
(see) material_par		

The material statement **band_discont** is an active layer macro statement used to define the conduction band discontinuity, in unit of eV, between an active layer and its neighboring non-active layer. It can take positive or negative values. When taking positive values, the active layer conduction band is lower than the passive layer (like a quantum well). Type II quantum well can be defined using a negative value. This statement, if used, overrides the **band_offset** statement. In the absence of the statement of **band_discont_right** this statement intends to define the conduction band discontinuity for both the left and right side of the active layer.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.49 band_discont_right

parameter	data type	values [defaults]
(see) material_par		

The material statement **band_discont_right** works with statement **band_discont** to define the conduction band discontinuity of the right side of the active layer. Other properties are the same as **band_discont**.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.50 band_distance

parameter	data type	values [defaults]
data_file	char	
use_fermi_level	char	[no]
conc_logscale	char	[no]
prop_file	char	[void]
ignore_cband	char [no]	
ignore_vband	char [no]	
plot_qcl_3levels	char [no]	
band_edge_file	char [void]	
conc	real	[1.e23]
pn_ratio	real	[1.]
qw_wave_ht	real	[0.1]
elec_field	real	[0.]
fermi_n	real	[0.]
fermi_p	real	[0.]
band_range	realx2	[-1.5 1.5]
data_point	intg	[90]
cond_valley	intg	[1]
val_valley	intg	[1]
cond_valley_prop	intg	[0]
val_valley_prop	intg	[0]

band_distance plots the band and subband structure versus distance. This statement is only used in the gain preview module or preview session before a full-blown simulation.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **use_fermi_level** indicates to directly set the Fermi levels using **fermi_n** and **fermi_p** below.
- **conc_logscale** used to indicate whether log scale is used in plotting any concentrations.
- **prop_file** is a file to store the property results from this statement. All results will be appended to this file so that multiple simulations can be summarized within this file.
- **ignore_cband** will ignore conduction band so that only the valence band will appear.

- **ignore_vband** will ignore valence band so that only the conduction band will appear.
- **plot_qcl_3levels** lets the program plot only the 3 critical quantum levels related to intersubband transition in quantum cascade laser.
- **band_edge_file** instruct the program to save the band edge profile to this file so that an upper level project can use it later.
- **conc** is the total electron concentration (in $1/m^3$) used to determine the Fermi levels and subband population.
- **pn_ratio** is the ratio of hole over electron concentrations. Note that this ratio can be set to an arbitrary number in the gain preview. In the main solver, this ratio is determined automatically by the simulator according to the local Fermi levels.
- **qw_wave_ht** is the height of the quantum wave amplitude as plotted on the band diagram.
- **elec_field** is a uniform electric field in V/m used to tilt the band diagram.
- **fermi_n** is the electron Fermi level (eV) from the HH valence band edge.
- **fermi_p** is the hole Fermi level (eV) from the HH valence band edge.
- **band_range** is the energy range (in units of eV) in a band diagram plot.
- **data_point** is the number of data points in one curve.
- **cond_valley** directs the program to plot properties related to a particular conduction band valley with this valley index. The valley index means band valleys with labels such as Gamma, L, X, or Delta2, Delta4, etc.
- **val_valley** directs the program to plot properties related to a particular valence band valley with this valley index. The valley index means band valleys with labels such as HH, LH, SO, etc.
- **cond_valley_prop** is kth the conduction band property as defined by statement **condj_valley_propk**. The valley j is defined by the parameter **cond_valley**.
- **val_valley_prop** is kth the conduction band property as defined by statement **valj_valley_propk**. The valley j is defined by the parameter **val_valley**.

Example(s)

```
band_distance  elec_field=-1.e6 cond_valley=1 val_valley=1 &&
  qw_wave_ht=0.05 use_fermi_level=no  &&
  conc=1.e23 pn_ratio=1.e-10 &&
  conc_logscale=no cond_valley_prop=1 prop_file=ssmob.txt &&
  data_point=60 band_range=(-0.5 0.1)
```

2.51 band_gap

parameter	data type	values [defaults]
(see) material_par		

The material statement **band_gap** is used to define the energy band gap (in eV) for the semiconductor material. For an active quantum well, the bandgap parameter used in the quantum well model overrides this parameter.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

Example(s)

```
band_gap  value=1.425 mater=2
```

2.52 band_offset

parameter	data type	values [defaults]
(see) material_par		

The material statement **band_offset** is an active layer macro statement used to define the band offset of the quantum well.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.53 bandgap_narrow

parameter	data type	values [defaults]
model	char	

The statement **bandgap_narrow** is used to include bandgap narrowing effect. Currently, Slotboom model is only supported.

- **model** specifies the model used in bandgap narrowing effect. Currently, “slotb” is only accepted.

Example(s)

```
bandgap_narrow model=slotb
```

2.54 bandgap_optical_gen

parameter	data type	values [defaults]
allowed	char	[no],yes
bandgap_gen_iqe	real	[0.1]

bandgap_optical_gen may be used to allow optical absorption at photon energy smaller than the bandgap to be converted to electron-hole pair generation.

- **allowed** indicates whether to allow photon with energy smaller than bandgap to be converted to electron-hole pairs. By default, this is not allowed.

allowed bandgap_gen_iqe is the internal quantum efficiency (IQE) for the below-bandgap optical generation.

Example(s)

```
bandgap_optical_gen allowed=yes bandgap_gen_iqe=0.01
```

This would allow one percent of below-bandgap absorption be converted to e-h pairs.

2.55 begin_cavity

parameter	data type	values [defaults]
cavity_num	intg	[1]

begin_cavity is used as a starting bracket for the statements belonging to a laser cavity in a multiple laser cavity application.

- **cavity_num** is the cavity number.

Example(s)

```
begin_cavity cavity_number=1
init_wave ...
multimode ...
end_cavity
```

The above `begin_cavity` and `end_cavity` statements brackets two statements used to define the properties of the first laser cavity.

2.56 begin_complex

parameter	data type	values [defaults]
layer_num	intg	[3]
column_num	intg	[1]

The statement **begin_complex** is used to indicate the beginning of a group of layers coupled to each other quantum mechanically. Such a group of layers is called a complex of active layers.

The definition of a complex should be in the following format:

```
{
begin_complex ...
complex_region ...
complex_region ...
complex_region ...
...
end_complex
}
```

There should be odd number of layer for each column due to the historical convention that a complex is used to describe a coupled MQW system in the sequence of barrier-well-barrier...well-barrier.

- **layer_num** is the number of layer within a complex.
- **column_num** is the number of columns within a complex.

Example(s)

```
begin_complex layer_num=3 column_num=1
```

2.57 begin_zmater

parameter	data type	values [defaults]
zseg_num	intg	[1]

The statement **begin_zmater** is used with **end_zmater** to bracket material and doping statements for a z-segment in the following format. zseg_num is the z-segment number.

```
{
begin_zmater zseg_num=#
material statements.
doping statements.
end_zmater
}
```

2.58 bend_extern

parameter	data type	values [defaults]
variation	char	function,table,[constant]
var_symboli(1=1..5)	char	void
vari(i=1..5)	real	
value	real	
func_num	intg	[1]

bend_extern is an external function used by functions of **bend_xy_plane**. Parameter use is identical to those in **bend_xy_plane**, except for "func_num".

- **func_num** is a label to identify different external functions so that this statement can be used more than once.

2.59 bend_xy_plane

parameter	data type	values [defaults]
variation	char	linear_x,linear_y,function,[constant]
var_symboli(1=1..5)	char	void
vari(i=1..5)	real	
value	real	
zseg_num	intg	[1]

bend_xy_plane is used to bend the xy-2D planes in z-direction so that complex shapes in 3D can be realized based on 2D mesh on an x-y plane. This statement causes the z-coordinates of the mesh points on an x-y plane to be shifted (in units of microns) according to parameters in this statement.

- **variation** takes "constant" or "function". If "constant" is set, this statement shift the mesh points by a value set by parameter **value**. If "function" is taken, an analytical function in the same format as material macro functions can be used. The internal variables are x_point, y_point and z_plane, denoting the mesh point coordinates.
- **var_symboli** is an external function variable symbol to be used by the function mentioned above.
- **vari** is an external function variable value to be used by the function mentioned above.
- **value** is the constant value of shift in z-direction if variation=constant.
- **zseg_num** is the z-segment index affected by this statement.

Example(s)

```
$ segment 1: effective from z_plane=0 to 250
bend_xy_plane variation=function zseg_num=1
function(z_plane,x_point,y_point)
y_0=0;
z_0=350;
radius=sqrt(350**2+402**2);
z=z_0-sqrt(radius**2-(y_point-y_0)**2);
zfactor=(250-z_plane)/250.;
(z-z_plane)*zfactor
end_function
```

2.60 beta_n

parameter	data type	values [defaults]
(see) material_par		

The material statement **beta_n** is used to define a unitless parameter appearing in the field dependent electron mobility function of the material. The following generic formula for field-dependent mobility is used:

$$\mu_n = \frac{\mu_{0n}}{(1 + (\mu_{0n}F/v_{sn})^{\beta_n})^{1/\beta_n}}, \quad (2.6)$$

For more detail on this formula, please consult the User's Manual. The statement is actually used only when the parameter **el_vel_model=beta** is selected in the **material** statement.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.61 beta_p

parameter	data type	values [defaults]
(see) material_par		

The material statement **beta_p** is used to define a unitless parameter appearing in the field dependent electron mobility function of the material. The following generic formula for field-dependent mobility is used:

$$\mu_p = \frac{\mu_{0p}}{(1 + (\mu_{0p}F/v_{sp})^{\beta_p})^{1/\beta_p}}. \quad (2.7)$$

For more detail on this formula, please consult the User's Manual. The statement is actually used only when the parameter **hole_vel_model=beta** is selected in the **material** statement.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.62 bias3d

parameter	data type	values [defaults]
step_num	intg	[10]

The statement **bias3d** is used to control the bias of the longitudinal module. The amount of current is already defined by **scan3d** in the previous chapter. This statement is used to set some of the conditions for the longitudinal mode solver.

- **step_num** is the number of bias step taken by the longitudinal solver for each step in the **scan3d** statement.

Example(s)

```
bias3d step_num=20
```

2.63 blank_area

parameter	data type	values [defaults]
point1	realx2	[0. 0.] (μm)
point2	realx2	[0. 0.] (μm)
point3	realx2	[0. 0.] (μm)
point4	realx2	[0. 0.] (μm)
num_points	intg	4

This statement can be used repeatedly to blank out areas in plotting statements in the .plt files. It is especially useful when areas of no-material ("void" material) show non-physical current flows.

Each statement can only blank out one area of quadrilateral or triangle.

- **pointk** **k=1,2,3,4** is the x and y coordinates of point k in counter-clockwise order.
- **num_points** takes either 4 or 3, representing quadrilateral or triangle.

Example(s)

```
blank_area point1=(0. 0.) point2=(1. 0.)  &&
          point3=(1. 0.5) point4=(0. 0.5)
```

2.64 bottom_contact

bottom_contact is the same as **top_contact** except that it is placed at the bottom. Please see statement **top_contact**.

2.65 `boundary_smooth`

parameter	data type	values [defaults]
<code>mesh_inf</code>	char	
<code>mesh_outf</code>	char	
<code>order</code>	char	[yes],no

The mesh generation statement **`boundary_smooth`** is used to make the extension of added mesh lines (by using **`double_mesh`**) from one polygon to the adjacent one smooth. This prevents the accumulation of too many terminated mesh lines extending to the adjacent polygon.

Note that this statement is used only when the user wants to manually refine the mesh in a region of the device geometry. In most cases, this statement is not needed because the mesh may be automatically refined using **`refine_mesh`**.

- **`mesh_inf`** is the mesh input file.
- **`mesh_outf`** is the mesh output file.
- **`order`** specifies the format of the mesh output. If the mesh output is to be used by the solver, “yes” should be chosen, i.e., ordering of the mesh is required to interface with the solver. If the mesh is to be further manipulated after this statement, “no” should be chosen.

Example(s)

```
boundary_smooth mesh_inf=case1.msh1 &&
mesh_outf=case1.msh2 order=yes
```

2.66 `boundary_xpoint`

parameter	data type	values [defaults]
<code>xp_size</code>	real	[0.0001] (um)
<code>boundary</code>	intg	[1]

The statement **`boundary_xpoint`** is used to control the internal extra point at a contact. The mesh spacing at a Schottky contact is important since the simulator treats the contact mesh point as a point at the metal while the next mesh point inside is regarded as the semiconductor. The semiconductor potential (or the Schottky

barrier height) is dependent on the mesh spacing. Thus we need to allocate an extra point near a Schottky contact using this statement whenever we have a Schottky contact. If a contact is defined as Schottky contact in .layer file, this statement will be automatically generated in the .geo file. The generation of this statement may be modified by the parameters **use_schottky_xp** and **intern_xp_size** in the statement of **top_contact** or **bottom_contact**.

- **boundary** is the contact or electrode number.
- **xp_size** is the internal extra point used to define the sharpness of the Schottky contact interface sharpness. The Schottky barrier height is affected by this size. It is recommended that around one angstrom of mesh spacing be used unless convergence problem occurs. In the limit of zero **xp_size**, the internal numerical Schottky barrier height is exactly as defined by the user in the **contact** statement.

2.67 bpm_grating

parameter	data type	values [defaults]
index_varj (j=1,2,...)	realx3	
pointj (j=1,2,...)	realx2	
region_num	intg	[1]
grating_order	intg	[1]
period	intg	[1]
print_points	intg	[11]

bpm_grating is used to specify the optical grating structure to be used in the Beam Propagation Method (BPM). A grating structure is specified by index variation along the z-direction. The variation occurs within a polygon region on the x-y plane. By our convention, the grating structure is defined such that the index variation is zero at $z=0$ and at $z=\text{period}$.

- **index_varj** $j=1,2,\dots$ is the data in the order $(z_j, \Delta n_r, \Delta n_i)$ where z_j is the z-coordinate (must be non-zero), Δn_r is the variation of the real index, Δn_i is the variation of the imaginary index.
- **pointj** is the jth point of the polygon in which the index variation of the grating is to be specified.
- **region_num** is the polygon region number. Since there can be more than one polygon region of index variation, this number is used to identify the regions.
- **grating_order** is the grating order.

- **period** is the total number of periods over which the wave is to be propagated.
- **print_points** is the number of printing point over the path of propagation.

Example(s)

```
bpm_grating index_var1=(0.2053 0 0) &&
  index_var2=(0.2055 0.1 0) &&
  index_var3=(0.2670 0.1 0) &&
  point1=(2 3.121) point2=(7. 3.121) point3=(7. 3.171) point4=(2. 3.171) &&
  period=600 print_points=12 grating_order=2
```

2.68 bpm_multimode

parameter	data type	values [defaults]
initial_amplitude	real	[0.2]
initial_phase	real	[0.0]
mode_index	intg	[2]

bpm_multimode is used to define an initial optical field profile consisting of more than one lateral mode.

- **initial_amplitude** is the initial amplitude of mode profile relative to the fundamental mode.
- **initial_phase** is the initial phase (in unit of Pi) of the mode profile relative to the fundamental mode.
- **mode_index** is the mode index label (integer) of the lateral mode.

Example(s)

The following statements define a profile with 3 modes, with 2nd and 3rd order modes of amplitude being 50 percent and 10 percent of the 1st order mode, respectively.

```
bpm_multimode initial_amplitude=0.5 initial_phase=0 mode_index=2
bpm_multimode initial_amplitude=0.1 initial_phase=0 mode_index=3
```


2.69 bpmintegr_xy

parameter	data type	values [defaults]
outfile	char	[void]
backward	char	[no]
xrange	realx2	
yrange	realx2	
z	real	[0.0]

bpmintegr_xy may be used to integrate the Beam Propagation Method (BPM) wave intensity over an x-y plane.

- **outfile** is the output file into which the result of the numerical integration is written.
- **backward** indicates if forward or backward wave is used in the integration.
- **xrange** is the integration range in graphic x-direction.
- **yrange** is the integration range in graphic y-direction.
- **z** is the position of the x-y plane.

Example(s)

```
bpmintegr_xy z=10. outfile=tmp.data3
```

2.70 bpmplot_xy1d

parameter	data type	values [defaults]
direction	char	[y] (x)
plot_phase	char	[no]
backward	char	[no]
rel_cutpoint	real	[0.5]
xrange	realx2	
yrange	realx2	
z	real	[0.0]

bpmplot_xy1d is used to plot the Beam Propagation Method wave solution on an 1D cut-line on the x-y plane.

- **direction** is used to indicate the direction of the cut-line.
- **contour** indicates if contour plot is required.
- **plot_phase** indicates if phase or intensity is plotted.
- **backward** is used to plot forward or backward wave.
- **rel_cutpoint** is the relative cut point position.
- **xrange** is the plot range in graphic x-direction.
- **yrange** is the plot range in graphic x-direction.
- **z** is the position of the x-y plane.

Example(s)

```
bpmpplot_xy1d direction=y rel_cutpoint=0.5 z=105 plot_phase=no backward=no
```

2.71 bpmpplot_xyz1d

parameter	data type	values [defaults]
plot_phase	char	[no]
backward	char	[no]
rel_x	real	[0.5]
rel_y	real	[0.5]
xrange	realx2	
yrange	realx2	

bpmpplot_xyz1d is used to plot the Beam Propagation Method wave solution on a line along the z-direction.

- **plot_phase** indicates if phase or intensity is plotted.
- **backward** is used to plot forward or backward wave.
- **rel_x** is the relative position of the plotting line in x-direction
- **rel_y** is the relative position of the plotting line in y-direction
- **xrange** is the plot range in graphic x-direction.
- **yrange** is the plot range in graphic x-direction.

Example(s)

```
bpmpplot_xyz1d plot_phase=no backward=no
```

2.72 bpmsplot_xy

parameter	data type	values [defaults]
contour	char	[no]
plot_phase	char	[no]
backward	char	[no]
view_xrot	real	[0.]
view_zrot	real	[0.]
xrange	realx2	
yrange	realx2	
zrange	realx2	
z	real	[0.0]

bpmsplot_xy is used to plot the Beam propagation Method wave solution on an x-y plane.

- **contour** indicates if contour plot is required.
- **plot_phase** indicates if phase or intensity is plotted.
- **backward** is used to plot forward or backward wave.
- **view_xrot** rotates the view angle around the graphic x-axis.
- **view_zrot** rotates the view angle around the graphic z-axis.
- **xrange** is the plot range in graphic x-direction.
- **yrange** is the plot range in graphic y-direction.
- **zrange** is the plot range in graphic z-direction.
- **z** is the position of the x-y plane.

Example(s)

```
bpmsplot_xy z=400 contour=no plot_phase=yes backward=no
```

2.73 `bpmsplot_xz`

parameter	data type	values [defaults]
<code>contour</code>	char	[no]
<code>plot_phase</code>	char	[no]
<code>backward</code>	char	[no]
<code>view_xrot</code>	real	[0.]
<code>view_zrot</code>	real	[0.]
<code>xrange</code>	realx2	
<code>yrange</code>	realx2	
<code>zrange</code>	realx2	
<code>rel_y</code>	real	[0.5]

`bpmsplot_xz` is used to plot the Beam propagation Method wave solution on an x-z plane.

- **contour** indicates if contour plot is required.
- **plot_phase** indicates if phase or intensity is plotted.
- **backward** is used to plot forward or backward wave.
- **view_xrot** rotates the view angle around the graphic x-axis.
- **view_zrot** rotates the view angle around the graphic z-axis.
- **xrange** is the plot range in graphic x-direction.
- **yrange** is the plot range in graphic y-direction.
- **zrange** is the plot range in graphic z-direction.
- **rel_y** is the relative position of the x-z plane.

Example(s)

```
bpmsplot_xz rel_y=0.5 contour=no plot_phase=yes backward=no
```

2.74 `bpmsplot_yz`

`bpmsplot_yz` is similar to `bpmsplot_xz` except `xz` $\rightarrow$ `yz`.

2.75 bpm

parameter	data type	values [defaults]
go_backward	char	[yes]
zstep_size	real	0.2 (lambda)
ref_point	realx2	(0. 0.) (micron)
xsize_power2	intg	6 (2 $\times$ size)
ysize_power2	intg	6 (2 $\times$ size)
smooth_step	intg	0

bpm activates the Beam Propagation Method (BPM) solver to solve the wave equation. The BPM assumes a known wave distribution at an initial x-y plane and solves the wave distribution on the next plane in the z-direction. Repeating this procedure propagates the beam along the whole wave guide. At the present version, the BPM may be used to export the wave solution so that PICS3D solver can import it in a laser simulation. It can also be used to estimate radiation loss and other properties that must be calculated including the longitudinal effects. The starting solution is assumed at z=0 and be provided by 2D wave solution of PICS3D. The wave is then propagated to the right hand side until it is reflected back and make a complete round trip.

- **zero_loss** may be used to force the material loss to be zero so that losses other than material loss be estimated. This may also be used when material loss is of less interests than the wave distribution.
- **go_backward** directs the wave to be propagated backwards.
- **zstep_size** is the step size in unit of wavelength within the semiconductor medium.
- **ref_point** is the reference point where the refractive index is taken as the reference refractive index in the BPM.
- **xsize_power2** is the mesh size in the x-direction in power of 2. The actual size is 2**xsize_power2. Please note that in FFT BPM, the mesh size must be power of 2.
- **ysize_power2** is similar to xsize_power2 but for mesh in y-direction.
- **smooth_step** is used to transfer solution of regular mesh to BPM mesh smoothly. Usually the regular simulation mesh is non-uniform and much different from the uniform BPM mesh. Since the initial solution is provided on the regular mesh, transferring it BPM mesh does not give a smooth solution. Using this parameter (can be set to several hundred) will allow the wave to be propagated

with the uniform mesh (without taper) until it becomes smooth and can be regarded as a real solution on the BPM mesh.

Example(s)

```
bpm zstep_size=0.2 xsize_power2=5 ysize_power2=5 zero_loss=yes
```

2.76 bulk_treatment

parameter	data type	values [defaults]
type	char	[void]n,p

The statement **bulk_treatment** is used to turn on/off the quantum treatment for one type of carrier. Be default, both electron and hole subbands of a quantum well are solved simultaneously. If one wish to achieve faster simulation or easier convergence, one may use this statement to skip the quantum treatment and treat the less important carrier as if it is in a bulk region.

- **type** is the type of carrier to be treated as bulk carrier without quantum models.

Example(s)

```
bulk_treatment type=p
```

The above statement will instruct the program to compute the quantum states of the n-type carriers.

2.77 bulk_xfunck (k=1..9)

parameter	data type	values [defaults]
(see) material_par		

The material statement **bulk_xfunck** (k=1,2,...,9) is a bulk macro statement used to define an external function to be used by other statements in the same macro. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.78 c11_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **c11_bar** is an active layer macro statement used to define the elastic constant C_{11} in the quantum barrier. Since only the ratio is used, any unit is OK for this parameter.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.79 c11_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **c11_well** is an active layer macro statement used to define the elastic constant C_{11} in the quantum well. Since only the ratio is used, any unit is OK for this parameter.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.80 c12_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **c12_bar** is an active layer macro statement used to define the elastic constant C_{12} in the quantum barrier. Since only the ratio is used, any unit is OK for this parameter.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.81 c12_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **c12_well** is an active layer macro statement used to define the elastic constant C_{12} in the quantum well. Since only the ratio is used, any unit is OK for this parameter.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.82 c13_bar

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272. It is the elastic stiffness constant in any unit (since only the ratio is used) for the barrier region. This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.83 c13_bulk

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the elastic stiffness constant in any unit (since only the ratio is used). This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.84 c13_well

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272. It is the elastic stiffness constant in any unit (since only the ratio is used) for the well region. This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.85 c33_bar

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272. It is the elastic stiffness constant in any unit (since only the ratio is used) for the barrier region. This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.86 c33_bulk

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the elastic stiffness constant in any unit (since only the ratio is used). This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.87 c33_well

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272. It is the elastic stiffness constant in any unit (since only the ratio is used) for the well region. This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.88 column

parameter	data type	values [defaults]
indep_xmqw	char	[no] yes
w	real	
r	real	[1.]
column_num	intg	[1]
mesh_num	intg	[6]
xp1	intg	[0]
xp2	intg	[0]
shift_center	real	[0.] (μm)

column is the statement used in the “.layer” geometry file which defines the columns in the cross section of the device, as well as the mesh allocation in the x-direction.

- **indep_xmqw** may be used to force the active layers in different columns to be treated as different (or independent) active regions even if the material parameters are the same. Such treatment may be necessary if the potential distribution in the two columns are substantially different in a self-consistent simulation.
- **column_num** is a number assigned to the column for identification purposes.
- **w** defines the width of the column.
- **mesh_num** sets the number of x-direction mesh lines to be put within this column.

- **r** is the ratio for mesh line distribution (see “ratio” under the statement “put_mesh”).
- **xp1** and **xp2** take zero or unity to indicate if extra points are to be added at the beginning and end of a column, respectively, to account for special boundary conditions. If extra points are specified, **xpoint_size** must be used to define their mesh spacing. In newer versions later than 2003, the simulator will not automatically use these parameters because the extra mesh points are controlled by the statement **internal_xpoint**. They will be used only when the parameter **version2002** is set in statement **internal_xpoint**.
- **shift_center** is used in the case of negative mesh **ratio** (see 2.16). The negative ratio will cause the mesh distribution to vary using the center point of the distance as a symmetric point. This parameter causes the center to be shifted in either direction.

Example(s)

```
column column_num=1 w=1.0 mesh_num=6 r=0.8 xp2=1
```

2.89 complex_region

parameter	data type	values [defaults]
polygon_name	char	[void]
use_xy_range	char	[no] yes
y_start_label	char	[void]
y_end_label	char	[void]
thickness	real	[0.01]
y_start_complex	real	(um)
x_width	real	(um)
x_start_complex	real	(um)
mater	intg	[1]

The statement **complex_region** is used to describe a rectangle region of material within a complex. A complex is a group of layers coupled to each other quantum mechanically.

The definition of a complex should be in the following format:

```
{
begin_complex ...
complex_region ...
```

```

complex_region ...
complex_region ...
...
end_complex
}

```

There should be odd number of layer for each column due to the historical convention that a complex is used to describe a coupled MQW system in the sequence of barrier-well-barrier...well-barrier.

- **polygon_name** is the name of the polygon as defined in .geo file.
- **use_xy_range** indicates whether x-y range is used to define the rectangle region or polygon names are used.
- **y_start_label,y_end_label** are effective only when **use_xy_range=yes** . They are labels used to define the y-range of the rectangle region.
- **thickness** is the thickness of the region in micron meters.
- **y_start_complex** specifies the absolute y-coordinate of the starting point of the rectangle (um).
- **x_width** is the width of the region in microns.
- **x_start_complex** specifies the absolute x-coordinate of the starting point of the rectangle (um).
- **mater** is the material number of the region.

Example(s)

```

complex_region polygon_name=p002 mater=      2  thickness=  0.785000E-01
end_complex

```

2.90 cond_bandj_edge (j=2,3)

parameter	data type	values [defaults]
(see) material_par		

The material statement **cond_bandj_edge** is an active layer macro statement used to define the 2nd or 3rd conduction band valley with respect to the lowest conduction band valley, in unit of eV. It is used for general_cx_strain type of macro, or general complex strain macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.91 cond_bandj_valley (j=1...3)

parameter	data type	values [defaults]
(see) material_par		

The material statement **cond_bandj_valley** is an active layer macro statement used to define the number of conduction band valleys. When used it overrides the parameters of **valley_*** in statement of layer_type. It is used for general_cx_strain type of macro, or general complex strain macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.92 condj_mass_para (j=1,2)

parameter	data type	values [defaults]
(see) material_par		

The material statement **condj_mass_para** is an active layer macro statement used to define the relative conduction band valley mass in a direction parallel to the quantum well, for the jth valley. It is used for general_cx_strain type of macro, or general complex strain macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.93 condj_mass_perp (j=1,2)

parameter	data type	values [defaults]
(see) material_par		

The material statement **condj_mass_perp** is an active layer macro statement used to define the relative conduction band valley mass in a direction perpendicular to the quantum well, for the jth valley. It is used for general_cx_strain type of macro, or general complex strain macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.94 condj_valley_propk (j=1,2;k=1,2)

parameter	data type	values [defaults]
(see) material_par		

The material statement **condj_valley_propk** is an active layer macro statement used to define the valley dependent property (kth property, such as mobility) for the jth conduction band valley. The simulator will automatically compute the property based on population of the subbands concentrations in the valley. It is used for general_cx_strain type of macro, or general complex strain macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.95 contact

parameter	data type	values [defaults]
type	char	[ohmic], schottky
touch_macro	char	
touch_var_symbolk(k=1..0)	char	
barrier	real	(eV)
junc_coefs	realx2	[1. 1.]
elec_temp	real	1.5
hole_temp	real	1.5
lattice_temp	real	300. (Kelvin)
heat_flow	real	100. (W/m)
thermal_cond	real	10. (W/(mK))
extern_temp	real	300.
work_function	real	(eV)
touch_vark(k=1..9)	real	
num	intg	
thermal_type	intg	1
touch_mater	intg	

The statement **contact** defines the properties of a contact or an electrode. For ohmic contact, the simulation program must determine a built-in voltage based on the flat band condition of semiconductor material in contact with the electrode. Therefore, the program will get confused if an ohmic contact is in contact with more than one semiconductor materials. In such a case, a solution is to use the parameters started

with “touch_” to force the electrode to be in touch (or use its properties) with a specific semiconductor.

- **type** defines the type of the contact.
- **touch_macro** is used to force an ohmic contact to take the property of a specific material if more than one semiconductor materials are connected to this electrode. This parameter will not be used if **touch_mater** is defined.
- **touch_var_symbolk(k=1..9)** is used in conjunction with the **touch_macro** parameter to define material whose property is being used by an ohmic contact.
- **touch_var_symbolk(k=1..9)**
- **barrier** is the barrier height in eV which is used only for Schottky contact. It is equal to semiconductor affinity minus the work function of the metal. Since it is related to **work_function**. The user should define either of the above, but not both. This value is used only when **type=schottky** is selected.
- **junc_coefs** are the Schottky junction coefficients of a contact used in the formulas of thermionic emission current model. They scale the emission velocities of electrons and holes, respectively.
- **num** is the contact number. The geometric location of the contact is given in the .geo input file. The contact number is the boundary number there.
- **elec_temp** is the electron temperature at the contact used as the boundary condition for the hot electron model, in units of kT. This parameter is used for the hydrodynamic model (not to be confused with the lattice temperature thermal model).
- **hole_temp** is the hole temperature at the contact used as the boundary condition for the hot hole model, in units of kT. This parameter is used for the hydrodynamic model (not to be confused with the lattice temperature thermal model).
- **lattice_temp** is the lattice temperature at the contact for thermal contact type 1.
- **heat_flow** is the heating power flow going out of the contact for thermal contact type 2. The unit of Watt/m for a 2D section of a cubic coordinate system. The unit is Watt for 3D device or a 2D section of a cylindrical system.
- **thermal_cond** is the thermal conductance connected to the contact for thermal contact type 3. The total heat flow (in Watt) to/from the device is given by $\Sigma(T - T_{ext})$ where Σ is the thermal conductance and T_{ext} is the external

thermal contact. For a realistic 3D device, the unit is Watt/K. For a 2D simulation, the heat flow is in unit of Watt/m and the unit of Σ is Watt/m/K for a 2D section of a cubic coordinate system. That is, one must divide the total thermal conductivity by the device depth (in z-direction). For an ideal uniform device, the thermal conductance of a cubic system can be converted to that of cylindrical system by formula: $\Sigma_{cyl} = \frac{\pi R^2}{2D} \Sigma_{cub}$. R is the radius of the cylindrical device and D is the half-width of the cubic device. If **extern_cir** is used to specify the resistance of this thermal conductor, the self-heating effect will also be taken into account. A temperature difference will appear between the contact and the external heat sink.

- **extern_temp** is the temperature of the heat sink connected to the thermal conductance for thermal contact type 3.
- **work_function** is used only when **type=schottky** is selected. It is the work function of the contact metal. The Schottky barrier height depends on it.
- **touch_var(k=1..9)** is used in conjunction with the **touch_macro** parameter to define material whose property is being used by an ohmic contact.
- **thermal_type** is the boundary condition used for the thermal equations. For **thermal_type=1**, the contact is assumed to have a lattice temperature given by the parameter **lattice_temp**. If **thermal_type=2**, the contact is assumed to have an out going heat power flow given by the parameter **heat_flow**. The lattice temperature of the contact is unknown and is to be determined by the thermal equation solver. If **thermal_type=3**, the contact is assumed to be connected to a thermal conductor with its conductance given by parameter **thermal_cond**. This thermal conductor is connected to a heat sink with a fixed temperature given by parameter **extern_temp**. Please note that if this contact is also declared as having an external circuit with a resistor, then self-heating in this resistor will be taken into account. Again the lattice temperature of the contact is unknown and is to be determined by the thermal equation solver.
- **touch_mater** is use to force an ohmic contact to take the property of a specific material if more than one semiconductor materials are connected to this electrode. It is the material number of the semiconductor in question. This parameter will override the parameter of **touch_mater** .

Example(s)

```
contact num=1 type=ohmic
contact num=2 type=schottky barrier=0.84
```


2.96 convention

parameter	data type	values [defaults]
positive_current_flow	char	[outward],inward

The command **convention** is used to define the positive current flow. By default, the simulator treats current flowing out of the device as being positive.

2.97 cplot_xyz

parameter	data type	values [defaults]
variable	char	(see list)
data_file	char	[vttek]
xy_from	realx2	
xy_to	realx2	
xrange	realx2	
yrange	realx2	
z_min	real	[-1.e9]
z_max	real	[1.e9]
mode_index	intg	[1]
grid_sizes	intgx2	[20, 20]
level	intg	[10]
trap_index	intg	[1]

cplot_xyz is a PICS3D post-processor statement used to plot contour lines on a plane involving all xyz dimensions. The plane is chosen by cutting a line on the xy-plane and extending the line along the z-direction.

- **variable** is the variable to be plotted. Detailed listing is given in App. [C](#).
- **data_file** is the file to which the graphic data is written in ASCII format. Note that plot_device should be set to “data_file” for this parameter to take effect.
- **xy_from** The plane is chosen by cutting a line on the xy-plane and extending the line along the z-direction. This statement is used to define the beginning point of the cut.
- **xy_to** The plane is chosen by cutting a line on the xy-plane and extending the line along the z-direction. This statement is used to define the ending point of the cut.
- **xrange** specifies a range for the x-axis.

- **yrange** specifies a range for the y-axis.
- **z_min** is the minimum value of z-range of the plot.
- **z_max** is the maximum value of z-range of the plot.
- **mode_index** is used to label the mode index for the lateral mode if wave intensity is being plotted.
- **grid_sizes** are the grid sizes of the 2D plot (not to be confused with mesh used in simulation).
- **level** is the number of contour levels used to generate the contour lines.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.

Example(s)

```
cplot_xyz variable=wave_intensity xy_from=(0.5, 0.0) xy_to=(0.5 3.0) &&
grid_sizes=(60, 20)
```

2.98 cplot_xy

parameter	data type	values [defaults]
variable	char	(see list)
data_file	char	[vttek]
mater_boundary	char	[no]yes
point_ll	realx2	(μm)
point_ur	realx2	(μm)
xrange	realx2	
yrange	realx2	
z	realx2	
level	intg	[10]
grid_sizes	intgx2	[20, 20]
mode_index	intg	[1]
trap_index	intg	[1]

cplot_xy is a post-processor statement used to plot two-dimensional contour lines.

- **variable** is the variable to be plotted. It is a scalar variable plotted as contours. The scalar variables are listed in App. C.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **mater_boundary** is used to indicate if material boundary is plotted.
- **point_ll** is the lower-left corner point of the 2D plotting window.
- **point_ur** is the upper-right corner point of the 2D plotting window.
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.
- **z** is the z position of the x-y plane on which the 2D contour is plotted.
- **level** is the number of function division levels to be used in the contour plot.
- **grid_sizes** are the grid sizes of the 2D plot (not to be confused with mesh used in simulation). To generate the vector plot (or flow plot), the vector of the physical quantity must be evaluated at a given **grid_size**.
- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.

Example(s)

cplot_xy variable=potential z=50.

2.99 current_conc

parameter	data type	values [defaults]
data_file	char	[void]
use_macro	char	[no]
fit_outfile	char	[void]
auto_pn_ratio	char	[no]
conc_log_scale	char	[no]
conc_range	realx2	[(1.e23 1.e24)] (m^{-3})
pn_ratio	real	[1.]
av_index	real	[3.3]
auger_n	real	[2.e-42] (m^6/s)
auger_p	real	[2.e-42] (m^6/s)
life_n	real	[1.e-6] (sec.)
life_p	real	[1.e-6] (sec.)
data_point	intg	[30]
well_num	intg	[1]
zseg_num	intg	[1]

The statement **current_conc** is a gain preview statement which allows the user to find the relation between carrier concentration and the recombination current in the quantum well active region. The recombination rate is expressed as

$$R = R_{\text{spont}} + R_{\text{auger}} + n/\tau_n + p/\tau_p$$

where the spontaneous emission spectrum computed from the quantum well model is integrated over all emission wavelengths and the Auger recombination is given by **auger_n** and **auger_p**, defined below. The last two terms are due to traps, surface recombination, thermal leakage, etc.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **use_macro** directs the program to use the material parameters such as Auger coefficients from the material macros instead from parameters in this statement. If this parameter is set to “yes”, parameters of **auger_n**, **auger_p**, **life_n** and **life_p** in this statement will not be used for the model fitting.
- **fit_outfile** if set to a valid file name, is used to direct the model fitting results to a file in the working directory.
- **auto_pn_ratio** is used to indicate if automatic setting of hole/electron density ratio is used in PICS3D simulation. If positive, averaged hole/electron density from the drift-diffusion solver is used. This parameter, if positive, will override **pn_ratio**.

- **conc_range** is the electron concentration range in the well.
- **pn_ratio** is the ratio of hole over electron concentrations. Note that this ratio can be set to an arbitrary number in the gain preview. In the main solver, however, this ratio is determined by the simulator automatically, according to the local Fermi levels.
- **conc_log_scale** would vary the concentration in log scale.
- **av_index** is the estimated average refractive index.
- **auger_n** is the Auger coefficient C_n .
- **auger_p** is the Auger coefficient C_p .
- **life_n** is the minority carrier life time for electrons.
- **life_p** is the minority carrier life time for holes .
- **data_point** is the number of data points to be used in the current versus concentration plot.
- **well_num** is the total number of active quantum wells involved.
- **zseg_num** is the z-segment number.

2.100 cylindrical

parameter	data type	values [defaults]
axis	char	x,[y]

The **cylindrical** statement is used to define a cylindrical coordinate system. It has been implemented so that 3D devices with cylindrical symmetry can be modeled. By default, half of the cylinder is included in the simulation or plotting by GUI programs. The other half can be obtained by symmetry. Therefore, a factor of two should be used when plotting electrical current and/or optical power to/from such a structure.

- **axis** is the axis of symmetry for the cylindrical system. In many devices, this is the y-axis.

2.101 dbr_truncate

parameter	data type	values [defaults]
dir	char	[left] right
kappa_l	real	[5.]
sec_num	intg	[1]

dbr_truncate is used to modify the DBR grating in edge type of lasers so that numerical stability may be achieved. When a DBR grating structure is used as a reflector for a laser diode, the propagating light can only penetrate a finite distance before fully reflected. Such a penetration length is usually in the order of several $1/\text{kappa}$, where kappa is the coupling coefficient. The DBR gratings beyond this penetration length does not contribute to the laser diode operation but may cause numerical stability during Newton iterations. Thus, we artificially truncate the DBR portion beyond the penetration length by setting the kappa there to zero.

- **dir** is the direction towards which the DBR grating is used as a reflector. For example, dir=left means the DBR is used to reflect light coming from the left.
- **kappa_l** is a real number of around five. It is used to measure the penetration length beyond which the DBR is to be truncated. The penetration length is given by $\text{kappa_l}/\text{kappa}$, where kappa is the coupling coefficient.
- **sec_num** is the index of the section where the DBR truncation is applicable.

Example(s):

```
dbr_truncate sec_num=2 kappa_l=5
```

2.102 define_alias

parameter	data type	values [defaults]
alias	char	[void]
name	real	[void]

define_alias is used to define an alias in the input file. **alias** is the alias being used. **name** is the name the alias represents.

Example(s)

```
define_alias alias=Vg name=voltage_2
```

The above alias is used to refer voltage_2 as Vg (gate voltage).

2.103 `define_material`

parameter	data type	values [defaults]
mater	intg	[1]

The statement **`define_material`** is used to signal the existence of a previously defined material in the current material segment.

- **mater** is the number of the material. of interests.

Example(s)

```
define_material mater=8
```

2.104 `define_symbol`

parameter	data type	values [defaults]
symbol	char	[void]
value	real	

`define_symbol` is used to define numerical values with symbols. This may be used for easy understanding of the input file or to represent repeated numerical values so that revision is more convenient.

The program scans each input line and substitute the first appearance of the symbol with the value defined in this statement. A few precautions should be taken when using this statement:

- 1) Avoid using it when the same symbol appears than once within a line;
- 2) The line containing the symbol must have enough line space so that the total length does not exceed 80 character after value substitution;
- 3) Please avoid using symbols that are already defined as parameters in `crosslight.tab`.

For example, the following statement will cause problems:

```
define_symbol symbol=doping value=2.e24
```

Since "doping" is commonly used statement, the program will replace all occurrence of "doping" by "2.e24" and the input file will loss its original meaning. To avoid this problem, try

```
define_symbol symbol=john_smith_doping value=2.e24
```

- **symbol** is the symbol to be used to represent numerical value in the program.
- **value** is the numerical value to be used.

Example(s)

```
define_symbol symbol=my_doping value=1.e24
```

2.105 define_vertical_position

parameter	data type	values [defaults]
label	char	[void]
reference_to	char	[void]
use_y_coord	real	(um)
above_previous	real	(um)
below_previous	real	(um)
cut_at_x	real	(um)
from_top	real	(um)
from_bottom	real	(um)
above_reference	real	(um)
below_reference	real	(um)
between_materials	intgx2	

define_vertical_position is used in .sol file to label a vertical position for later reference. There are three approaches to define a position: 1) Use absolute coordinate; 2) Defined as first occurrence of material interface between two materials when searching from bottom to top; 3) In reference to a previously defined position, i.e., using relative distance; 4) Use relative distance with respect to the top or bottom of the x-y mesh.

- **label** is a position label at a specific y-coordinate.
- **reference_to** is used to define a reference point for approach number 3). It must be a predefined position label.
- **use_y_coord** is the y-coordinate for approach number 1).
- **above_previous** indicates the position is above a label in the preceeding statement for 3).

- **below_previous** indicates the position is below a label in the preceeding statement for 3).
- **cut_at_x** is used to define the x-position when using approach 2).
- **from_top** defines the distance from top of the mesh system in 4).
- **from_bottom** defines the distance from bottom of the mesh system in 4).
- **above_reference** is the distance above the reference point defined by parameter **reference_to**.
- **below_reference** is the distance below the reference point defined by parameter **reference_to**.
- **between_materials** defines the two materials used to identify the interface position in 2).

Example(s)

```
define_vertical_position label=sio2/si &&
    cut_at_x=0.  between_materials=(2 3)
```

This example labels a y-position between materials 2 and 3 and label it as “sio2/si”.

2.106 delta_so_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **delta_so_bar** an active layer macro statement used to define the spin-orbit coupling parameter (eV) in the quantum barrier.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.107 delta_so_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **delta_so_well** an active layer macro statement used to define the spin-orbit coupling parameter (eV) in the quantum well.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.108 **deltai_bar,i=1,2,3**

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the energy parameter [1] for the barrier in eV. This statement is applicable for wurtzite structure active region.

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.109 **deltai_bulk,i=1,2,3,4**

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the energy parameter in eV. This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.110 **deltai_well,i=1,2,3**

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the energy parameter [1] for the well in eV. This statement is applicable for wurtzite structure active region.

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.111 di_bar,i=1,2,3,4

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the shear deformation potential for the barrier region in eV. This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.112 di_bulk,i=1,2,...

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the shear deformation potential in eV. This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.113 di_well,i=1,2,3,4

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the shear deformation potential for the well region in eV. This statement is applicable for wurtzite structure active region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.114 dielectric_constant

parameter	data type	values [defaults]
(see) material_par		

The material statement **dielectric_constant** defines the static dielectric constant which appears in the Poisson’s equation. It does not affect the wave equation which is written for optical frequencies.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.115 direct_eigen

parameter	data type	values [defaults]
select_modes	char	[no] yes
max_index	char	[yes] no
max_element	char	[no] yes
sigma_factor	real	[2.]
select_index	real	
arnoldi_base	intg	[10]

The statement **direct_eigen** is used to activate the Arnoldi direct eigen solver. This is used to find the lateral modes of a waveguide. This statement must be used in conjunction with **multimode**.

- **select_modes** , if positive, is used to select eigen modes based on an effective index value supplied through parameter **select_index**. The eigen modes found will have effective indices below **select_index**. If negative, the search of eigen value will start from effective index estimated from the highest local material index within the solution region.
- **max_index** turns on automatic mode search based on a maximum material index of the device structure.
- **max_element** , if positive, instructs the simulator to find eigen modes based on maximum value of matrix elements.
- **sigma_factor** is used in conjunction with **max_element** to obtain an estimate of the maximum eigen value. This factor is multiplied with the maximum matrix element to provide a starting point for eigen value search.
- **arnoldi_base** is used to control the number of Arnoldi eigen solutions we intend to work with. The total number of eigen vectors is this number plus the number of modes we specify in the **multimode**.

Example(s)

```
direct_eigen sigma_factor=2.
```

2.116 do_raytrace

parameter	data type	values [defaults]
device_kind	char	[led]
beam_model	char	[gaussian]
incident_light_side	char	[left]
emission_model	char	[line]
emit_sector	char	[whole]
set_domed	char	[no]
dome_direction	char	[bottom]
flux_size_auto	char	[yes]
flux_represent	char	[mesh]
precision	real	[0.01]
te_portion	real	[0.5] (unitless)
tm_portion	real	[0.5] (unitless)
flux_area	real	[2.0] (μm)
total_incident_power	real	[100.0] (W/m^2)
beam_deviation	real	[20.0] (degrees)
dome_width	real	[120.0] (μm)
dome_height	real	[10.0] (μm)
dome_radius	real	[125.0] (μm)
dome_opindex	realx2	[1.6,1.e-8]
opindex_incident	realx2	[1.0,0.0]
datafile_range	intgx2	[1 1000]
initial_rays	intg	[32000]
max_secondary	intg	[100000]
number_showray	intg	[30000]
min_flux_points	intg	[80]
set_mirror1_side	char	[void] -x,+x,-y,+y
set_mirror2_side	char	[void] -x,+x,-y,+y
surface_model	char	[void] scale_fresnel_coef,exit_index,import_data,rough_surface,mult
surface_model_side	char	[+y] -x,+x,-y,+y
output_scale_factor	real	[1.0]

do_raytrace is used to set parameters for the ray-tracing module.

- **device_kind** is a type of simulated device (**led** or **detector**).
- **beam_model** is a geometrical profile of an optical incident beam detected by a photodetector. At this time only **gaussian** profile is considered, which user can vary defined standard deviation (see parameter **beam_deviation**).

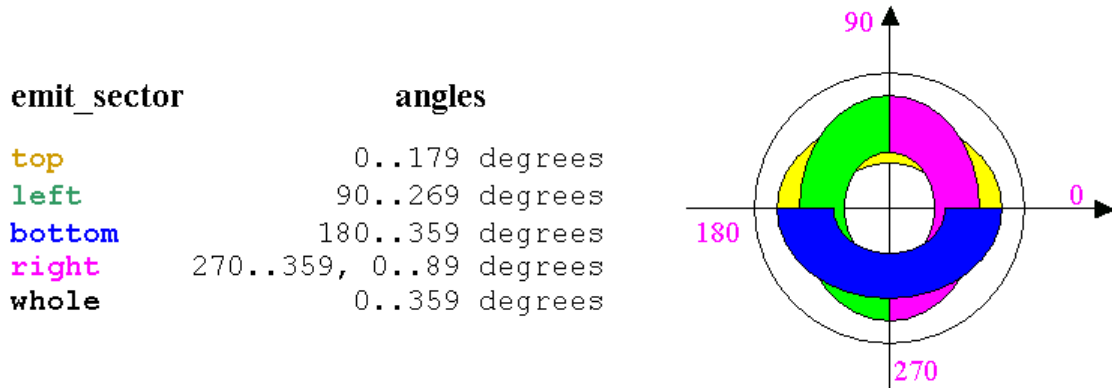


Figure 2.3: An explanation of the "emit_sector" parameter

- **incident_light_side** is a user selected device side illuminated by an incident beam. It is an option for **device_kind=detector**, which could be either **bottom**, **right**, **left**, or **top**.
- **emission_model** is an emission model of an LED active region. It can be chosen as: **one_point**, **line**, **mesh_points**, **region** indicating the light source to be a point, linear or polygon area source type, respectively. Additionally, an option (**mesh_points**) allows for using a type of the emission power source, which consists of light generation mesh-points, obtained in the simulation. The last model usually is the most accurate one, but uses most computational time before reaching realistic solutions.

In case of multi-column structures, if **mesh_points** model is not chosen, Ray-trace program will automatically select a region with a maximum emission power to become a light source. Consequently, an asymmetric Ray-trace simulation result can be produced, if column of that "source region" is not positioned at the center of area of the device cross-section.

- **emit_sector** defines an external to LED sector of light emission, which may include LED encapsulation. This parameter allows a user to select an LED light emission sector. Depending on design and application, it can be chosen out of the following options: **whole**, **bottom**, **right**, **top** or **left**. Figure 2.3
- **set_domed** defines the case whether an LED is encapsulated or not (=yes, or no).
- **dome_direction** defines the working direction of an LED. It can be chosen as: **bottom**, **right**, **top** or **left**. The dome is built around the device if

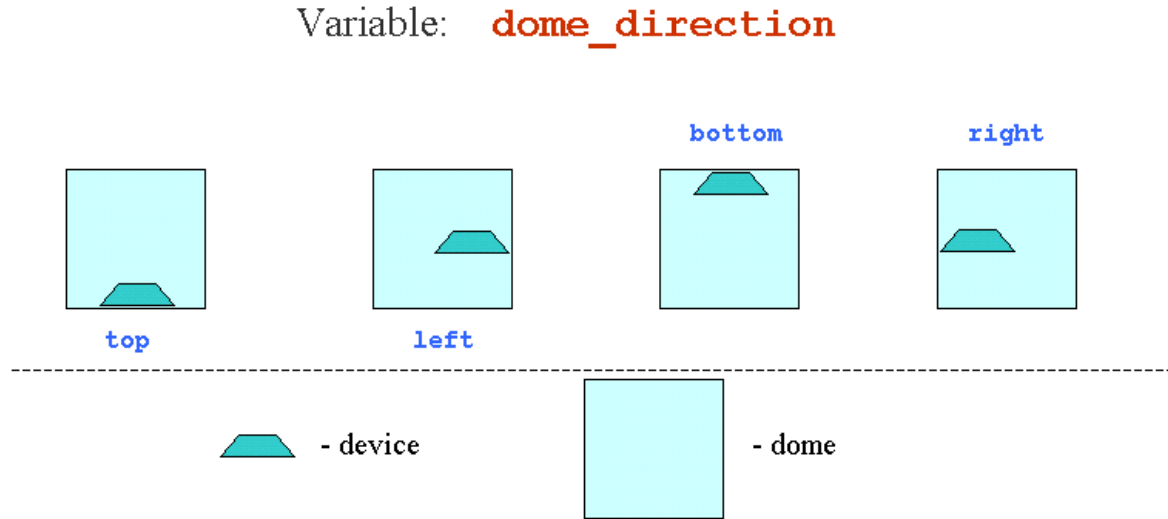


Figure 2.4: An explanation of the "dome_direction" parameter

set_domed=yes. Figure 2.4 illustrates different configurations in positioning a dome around an LED chip.

- **flux_size_auto** is a directive for the program (=yes or no) to generate necessary mesh points for flux density distribution calculations (it is different from APSYS main simulation mesh generation). If option **yes** was chosen, the distance between nodes at a given direction (x or y) is at least as low as defined in parameter **min_flux_points** (see below).
- **flux_represent** is a directive describing what kind of data presentation is chosen for a light energy flux density xy-distribution plot. There are two options: **mesh**, using mesh points from APSYS simulation, with usually not-equal spacing, or **uniform** with equally-spaced points within a device.
- **precision** is the ray-tracing precision of the ray-tracing calculation. The ray-tracing procedure is terminated if the relative power of a ray decays to this value.
- **te_portion**, **tm_portion** are the fractions of TE and TM polarizations of propagating light. For non-polarized light **te_portion** and **tm_portion** are equal.
- **flux_area** is a real-type variable parameter, which is equal to the side length of the square area used to define local light energy flux density distribution. It is measured in microns, and the user can define its value, as long as parameter **flux_size_auto** is set to **no**. The recommended range is 0.01-1000 μm . The run-time of the software depends on this value inversely proportional.

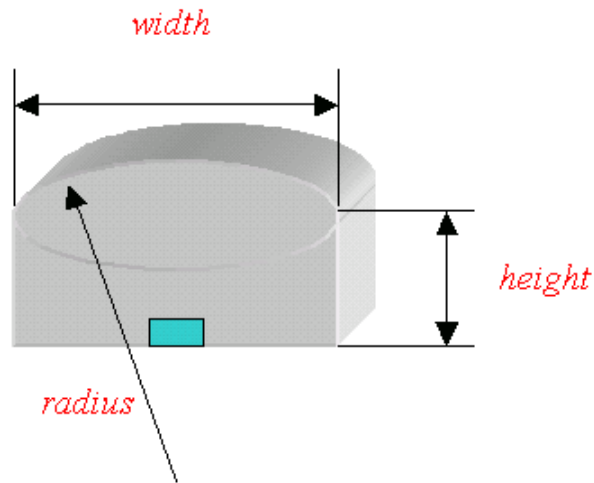
Variables:`dome_width``dome_height``dome_radius`

Figure 2.5: An explanation of the dome geometrical parameters

- **total_incident_power** is a total value of the power of an light beam illuminating photodetector.
- **beam_deviation** is a standard deviation of the incident light beam angle from its main direction. (At present, it can be set only as a normal to the surface of the photodetector). The incident light beam distribution is measured in degree angle, and Gaussian profile is assumed (up to the maximum angle cut-off $\pm 90^\circ$) with a standard deviation set as an input parameter. **beam_deviation** typical range is between 10-40 degrees angle. To create a rigorously parallel beam, one should choose a small enough value of the beam_deviation (less then 1), while a large value (typically larger than 100,000) would produce widely scattered beam of rays.
- **dome_width** is a width-dimension in units of μm of LED encapsulation, only in case **set_domed=yes** (see Figure 2.5). If the width (or height) dimension of a defined dome was set to the value smaller than the LED device dimension, the program will increase its value automatically to the minimum size, allowing for LED chip encapsulation.
- **dome_height** is a height-dimension in units of μm of LED encapsulation, only in case **set_domed=yes** (see Figure 2.5).
- **dome_radius** is a radius-dimension in units of μm of LED encapsulation, only in case **set_domed=yes** (see Figure 2.5). A recommended range for the radius of curvature of the emitted side is from **width**/2 to **width***20, otherwise program will correct automatically its value to a reasonable limit. This range allows to build a wide set of the different dome shapes. Figure 2.6 shows some different shapes of the dome.

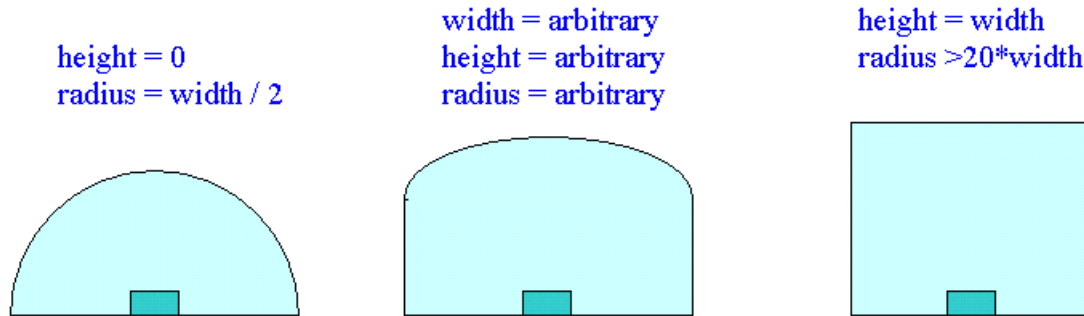


Figure 2.6: Some different shapes of the dome

- **dome_opindex** is the dome refractive index. For dielectric epoxy dome it might equal around [1.6,1.d-8]. This number has to be of complex numbers type.
- **opindex_incident** is the complex-value refractive index of the surrounding detector medium (only valid for **device_kind=detector**).
- **datafile_range** is a file number that starts and ends at certain conditions in APSYS main module, for example, for different voltage biases.
- **initial_rays** denotes quantity number of initial light rays used in Ray-trace simulation. Recommended range **initial_rays** is from 2,000 to 100,000.
- **max_secondary** is an integer value parameter, that defines maximum secondary rays generated during multi-reflection. The ray-tracing procedure will terminate for a certain initial ray if the number of secondary rays exceeds this number. A ray is regarded as secondary if it is generated in a material box outside the box where the initial ray is launched. A value around 100,000 was set as default, for typical device structure simulation. In case of insufficient memory size program may display message with some recommended changes in computational parameters, such as **max_secondary**, precision, increasing absorption, etc.
- **number_showray** It is an integer-type parameter for quantity of all rays that record to "*.sho" file. This file is used by SHOW utility. Value around 30,000 was set as default.
- **min_flux_points** is the minimum number of points in one-dimension (x or y -coordinate) to calculate the light energy flux density distribution. This

number has to be of integer type. Usually, for regular structures it could be set from around 10 to around 400. The run-time of the program increases with the value of this parameter, proportionally.

- **set_mirror1_side** would set a mirror1 boundary.
- **set_mirror2_side** would set a mirror2 boundary.
- **surface_model** is the kind of surface model used for **surface_model_side**.
- **surface_model_side** defines a side to apply a surface model.
- **output_scale_factor** would scale the output power.

Example(s)

```
do_raytrace emission_model=line flux_size_auto=yes &&  
precision=0.01 &&  
    te_portion=0.5  tm_portion=0.5 &&  
        datafile_range=(4,5) initial_rays=16000 &&  
        max_secondary=100000 number_showray=50000
```

The above statement defines the emission model for an LED to be as a line emission source model. The auto-calculation option for adjusting area in light energy flux density distribution calculation was set to be ON. The calculation precision was set to be 1%. The ratio between TE and TM polarization of propagating rays is one (non-polarized light case). The range of device-bias was chosen from 4 to 5. The number of the initial rays was set to 16,000. The number of the secondary rays is 100,000. The number of the rays for "show" viewer was set to 50,000.

2.117 do_raytrace_3d

parameter	data type	values [defaults]
3d_emission_model	char	[plane]
external_package	char	[void]
working_direction	char	[-y]
device_kind	char	[led]
incident_light_side	char	[+y]
surface_model	char	[void]
surface_model_side	char	[+y]
reject_edges	char	[no]
extract_to_file	char	[void]
extract_data	char	[wavelength_absorption]
append_data	char	[no]
dome_bottom_mirror	char	[yes]
precision	real	[0.01](percentage)
te_portion	real	[0.5]
tm_portion	real	[0.5]
plate_thickness	real	[100.] (um)
plate_width	real	[1000.] (um)
dome_cyl_height	real	[500.] (um)
dome_cyl_radius	real	[400.] (um)
dome_sph_radius	real	[400.] (um)
dome_base_thickness	real	[10.] (um)
scale_refl	real	[1.]
scale_tran	real	[1.]
thickness1	real	[0.023]
thickness1	real	[0.034]
package_refr_index	realx2	[1.7,1.e-7]
effective_exit_index	realx2	[1.0,0.0]
refr_index1	realx2	[1.3,0.0]
refr_index2	realx2	[1.7,0.0]
spectrum_loop_range	realx2	[0.4,0.5] (um)
datafile_range	intgx2	[1 1000]
initial_rays	intg	[5000]
max_secondary	intg	[100000]
3d_angle_store	intg	[50]
emit_points_number	intg	[10]
max_primary	intg	[5000]
n_layer	intg	[1]
spectrum_loop_num	intg	[0]
surface_power_grid	intg2	[100,100]

do_raytrace_3d is used to set parameters for and to activate the ray-tracing module for 3D simulation of light emitting diodes (LED).

- **3d_emission_model** is the emission source model of an LED active region. Its values are explained in the following table.

3d_emission_model	definition
one_point	The distribution of the emission source is averaged over space and made equivalent to a single source point with the total power of all source points.
plane	The distribution of the emission source is averaged over space and made equivalent to sources on a single x-z plane. This plane is further divided into source points equally spaced.

- **external_package** indicates the existence of external package. An LED can be simulated with one of the external packages such as side medium, plate or dome. The value of this variable can be chosen as: **void**, **medium**, **plate** or **dome**. Void means no package.
- **working_direction** defines the working direction of all packages. It can be chosen as: **-x**, **+x**, **-y**, **+y**, **-z**, or **+z**. Figure 2.7 shows how this parameter is defined.
- **device_kind** determines what kind of device (led or detector) is being modeled.
- **incident_light_side** determines which side of a detector is illuminated.
- **surface_model** determines which boundary conditions are applied to certain surfaces; this can be used to represent optical coatings. Note that in certain structures, the coating must be modeled inside the device with the `set_3dray_internal_interface` statement.
- **surface_model_side** determines which side of the device is affected by `surface_model`.
- **reject_edges** determines how the mesh points are used internally by the ray tracing program to average material properties. Choosing yes will cause the program to ignore mesh points too close to box edges which can lead to more accurate refractive index near large discontinuities. However, it will cause an error if there are not enough mesh points.

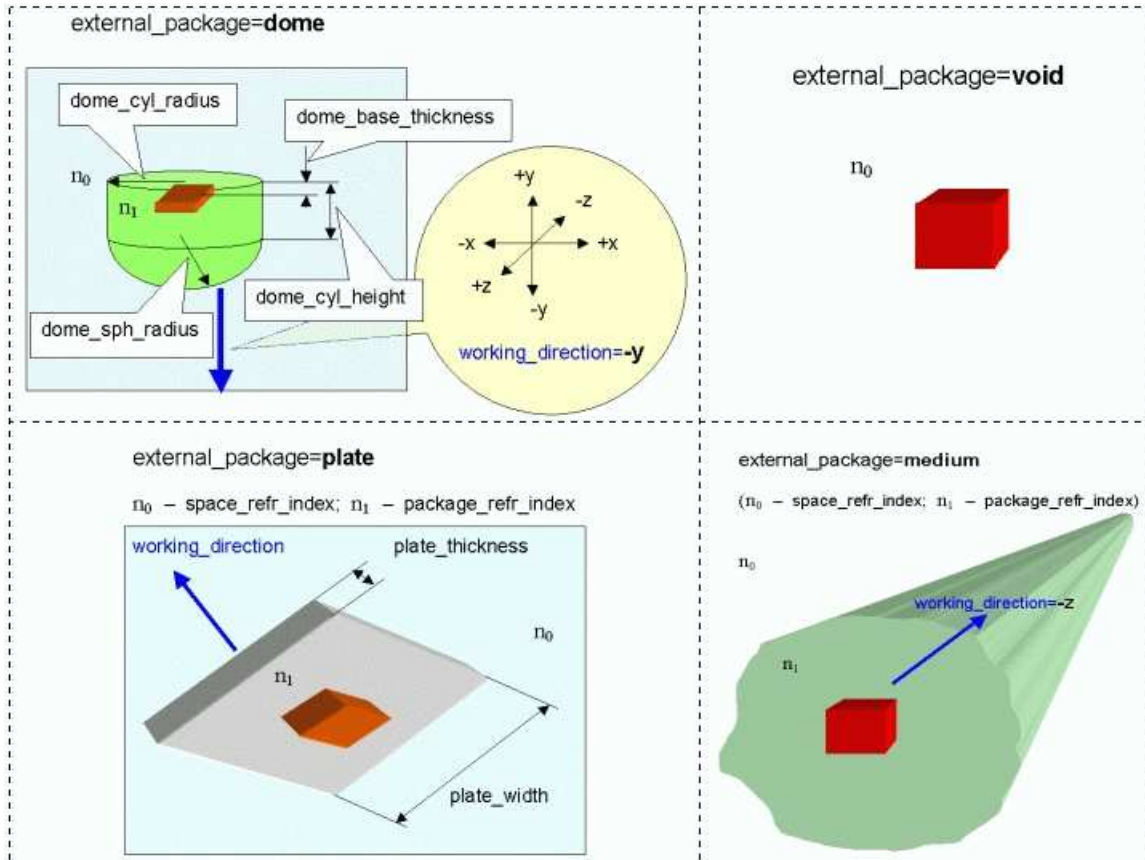


Figure 2.7: An explanation of the packages parameters

- **extract_to_file** specifies which file the ray tracing code will export data to; this will be disabled when set to void. The extracted data will be appended to an existing file or will overwrite it, depending on the value of **append_data**.
- **append_data** would instruct the program to append data to an existing data file instead of creating a new one.
- **dome_bottom_mirror** would set a perfect mirror on the bottom of a dome external package.
- **extract_data** specifies which ray tracing result will be extracted to the file specified by **extract_to_file**.
- **precision** is the ray-tracing precision of the ray-tracing calculation. The ray-tracing procedure is terminated if the relative power of a ray decays to this value.
- **te_portion**, **tm_portion** are the fractions of TE and TM polarizations of propagating light. For non-polarized light **te_portion** and **tm_portion** are equal.
- **plate_thickness**, **plate_width**, **dome_cyl_height**, **dome_cyl_radius**, **dome_sph_radius**, **dome_base_thickness** are geometrical dimensional parameters of the packages. Figure 2.7 illustrates the meaning of all these parameters. They are in units of μm . A given package is built around (or besides) the device. If one or more sizes of a package are set less than device sizes, the program corrects the sizes automatically to their possible minimum values.
- **scale_refl**, **scale_tran** are scaling factors to the Fresnel coefficients which are applied when **surface_model** is set to **scale_fresnel_coef**.
- **thickness1**, **thickness2**, **refr_index1**, **refr_index2**, **n_layer** are the parameters of the **multi_layer** option of **surface_model**.
- **package_refr_index** defines the complex optical refractive index for all packages.
- **effective_exit_index** is the effective index of the coating when **surface_model** is set to **exit_index**.
- **spectrum_loop_range**, **spectrum_loop_num** allow the ray tracing to be done in a wavelength-dependent manner instead of using the peak wavelength. If **spectrum_loop_num**=0, then the default ray tracing with a single peak wavelength will be used. Please note that the loop range and number must be consistent with the parameters defined in **init_wave**. Also, since the multiple ray tracing results modify the structure of the .rto file, **import_rto** and various plotting features are not currently supported when using this feature.

- **datafile_range** is a file number that starts and ends at certain conditions in APSYS main module, for example, for different voltage biases.
- **initial_rays** is the number of initial rays launched from one emission source point. Equal angular distribution is forced upon these initial rays.
- **max_secondary** is an integer value parameter, that defines maximum secondary rays generated during multiple reflections and refractions. The ray-tracing procedure will terminate for a certain initial ray if the number of secondary rays exceeds this number. A ray is regarded as secondary if it is generated in a material box outside the box where the initial ray is launched.
- **3d_angle_store** is the resolution which is used to store the angular power distribution. It is equal to the number of angular divisions in theta angle and is equal to half of the number of divisions in phi angle.
- **emit_points_number** is a total given number of emitting points for a plane emission model. Generated points model an original emitting light power pattern and the numbers of placed points in directions X and Z are calculated on base of this total number.
- **max_primary** is the maximum number of primary rays allowed. If the primary rays exceeds this number, the program will terminate the tracing of the primary rays. A ray is regarded as primary if it is produced by reflection from and within the same material box as the one where the initial ray is launched.
- **surface_power_grid** is the size of the grid used to record the location and power of rays exiting the device. Six such grids are located around the device to form a box. The resulting surface emission map can be plotted with **3drayplot_surfpower**

Example(s)

```
do_raytrace_3d precision=0.01 &&
  datafile_range=(3,4) initial_rays=6000 3d_emission_model=plane &&
  external_package=medium working_direction=-y &&
  package_refr_index=(1.7 0.d0) 3d_angle_store=30
```

The above statement activates the ray-tracing module assuming the source points are located on a single x-z plane. External medium of refractive index 1.7 is used as a substrate material. Therefore the emission power to the bottom direction is the amount into the substrate, not to the outside of the device.

2.118 doping

parameter	data type	values [defaults]
impurity	char	[shal_dopant]
charge_type	char	[donor]
shape	char	[gaussian],polygon
pf_model	char	[no],yes
x_datafile	char	[void]
y_datafile	char	[void]
direction	char	[positive] negative
log_scale	char	[no] yes
xy_datafile	char	[void]
xy_linedata	char	[yes]
datafile_unit	char	[1/m ³],1/cm ³
sheet_density	char	[no] yes
sheet_location	char	[void],bottom,top
max_conc	real	1.e10 (m^{-3})
level	real	[0.01] (eV)
x_prof	realx4	x1,x2,dx1,dx2 (μm)
y_prof	realx	y1,y2,dy1,dy2 (μm)
edge1_prof	realx3	x2,y2,dx2
edge2_prof	realx3	x2,y2,dx2
edge3_prof	realx3	x3,y3,dx3
edge4_prof	realx3	x4,y4,dx4
pf_cont_shift	real	[0.] (eV)
arc_height1	real	[0.](μm)
arc_height2	real	[0.](μm)
arc_height3	real	[0.](μm)
arc_height4	real	[0.](μm)
origin	real	[0.] (μm)
grading_from	real	(fraction)
grading_angle	real	[90.] (degrees)
xy_data_factor	real	[1.]
xy_x_transform	realx2	[1. 0.]
xy_y_transform	realx2	[1. 0.]
pnt_num	intg	[4]
newdoping_index	intg	
xy_skipline	intg	[1]
xy_data_column	intg	[3]
traplevel_model	char	[void]
traplevel_tail_side	char	[conduction]
traplevel_stddev	real	[0.1](eV)
traplevel_tail	real	[0.05](eV)

The statement **doping** is a frequently used statement which describes the doping profile of impurities or impurity concentration distribution. If the **shape** takes a simple “gaussian” form, the profile in the x-direction is uniform from x1 to x2 and falls off on both sides with Gaussian tails of standard deviations of dx1 and dx2, respectively. (see Fig. 2.8)

The profile in the y-direction is similar. Such a profile can be used to define a uniform profile and different types of Gaussian profiles. Repeatedly using this statement in small regions can create many different types of doping profiles. If the **shape** is defined as “polygon”, an arbitrary polygon can be defined within which the doping is uniform. The dopant concentration falls off with a Gaussian tail on the edges of the polygon (see Fig. 2.9).

Note that the parameter **level** is used to describe the energy level of the shallow or deep impurities over the whole device. If the properties of the impurity in each different material need to be defined, the user must override the **level** parameter with one of the statements **shal_dnr_level**, **shal_dnr_level_i**, **shal_acpt_level**, **shal_acpt_level_i** or **trap_level_i**.

Experimental or simulated doping profile in numerical data format may be imported into the simulation SOFTWARE using the following parameters: **origin**, **x_datafile**, **y_datafile**, **direction**, and **log_scale**.

This statement can also be used to pass the following parameters to the doping statement appearing in the .doping file generated by the Layer program: **impurity**, **pf_model**, **x_datafile**, **y_datafile**, **direction**, **log_scale**, **level**, **pf_cont_shift**, and **origin**. In .layer file, this statement must be placed right after the corresponding **layer_mater** so that the layer and column numbers can be identified.

- The parameter **impurity** defines the type of impurity being modeled. It takes the values of “shal_dopant”, “shal_dopanti(k=1..9)”, “trap_i” (i=1,2,3...), and “fix_charge”, for shallow dopant, deep level trap of type i, and fix space charge, respectively. If **impurity=trap_i**, the user should define the statements **trap_ncap_i** and **trap_pcap_i** to specify the capture cross sections of the trap.
- **charge_type** is the charge state of the impurity. It can take one of “donor”, “acceptor”, “positive” or “negative”. “donor” and “acceptor” can be used to specify shallow dopants and deep traps, while “positive” and “negative” can be used on fix space charge.
- **shape** takes either “gaussian” or “polygon”, as explained above. When importing doping data from a data file, please keep it at its default of “gaussian”.
- **pf_model** turns on/off the Poole-Frenkel model for field-induced thermionic emission of partially ionized dopants. Such a model is important when the ionization energy is high or when the operating temperature is low.

- **x_datafile** or **y_datafile** allows the user to import numerical doping data file in the two column format of

```
relative-distance  doping-density
(no blank lines anywhere)
```

for the profile data in the x or y direction. The unit of relative distance is in micron meters and doping-density can be in $1/m^3$ or in $\log_{10}(1/m^3)$ depending on the choice of **log_scale**. The unit of data can also be set by the parameter **datafile_unit**.

- **direction** is used to flip the sign of the coordinates for imported data file **x_datafile**. or **y_datafile**. If negative, the actual doping profile is in a direction opposite to the relative-distance given in the data file.
- **log_scale** is used to indicate if the doping density is in $\log_{10}(1/m^3)$ scale or in $1/m^3$.
- **datafile_unit** sets the unit within the file **x_datafile** or **y_datafile** to be $1/m^3$ or $1/cm^3$.
- **sheet_density** instructs the program that the present command is used to describe a sheet density profile instead of the bulk default. The **max_conc** would be in unit of $1/m^2$ to define a sheet charge over the x-y plane.
- **sheet_location** is used in conjunction with **sheet_density=yes** for a 3D structure. It defines the location of sheet charge either on the top of the present z-segment or the bottom.
- **max_conc** defines the maximum concentration.
- **level** defines the relative energy level of the impurity. For shallow donors and deep level traps, the level is calculated from the conduction band. For shallow acceptors the level is measured from the valence band. For gaussian type of trap model, it is also the maximum trap level position.
- **x_prof** specifies the profile of the dopants in the x-direction. It takes four real numbers: (x1,x2,dx1,dx2) (in μm). The meaning of the parameters are defined in Fig. 2.8. This is used only when “gaussian” is chosen for **shape**.
- **y_prof** specifies the profile of the dopants in the -direction. Its use is similar to **x_prof**.
- **edgei_prof** provides the i th point coordinate of the polygon and Gaussian tail of the i th edge. Note that the i th edge is the line between the i th and $(i+1)$ th point (see Fig. 2.9). It is used only when “polygon” is chosen for **shape**.

- **pf_cont_shift** is the ionization energy level shift at the contact, due to the Poole-Frenkel effect. Once it is specified by this parameter, the ionization energy is fixed during the simulation, while the ionization energy level shift in the bulk material is varied as a function of the local electric field.
- **arc_height1, arc_height2, arc_height3, arc_height4** These parameters are used to bend the edge of the doping profile given in parameters `edge1_prof`, `edge2_prof`, etc. The values of **arc_height** can take positive or negative numbers. The sign of the height is understood as follows: Let the counter-clockwise direction of the vector defined by two adjacent points of the polygon be vector **e** and the vector pointing from the starting point of **e** to a point on the arc be vector **a**. Then a positive height means that the cross product **a** cross **e** must point to positive **z** direction. On the other hand, if the height is negative, the cross product yields a vector in the negative **z** direction. In another word, positive height makes the edge curve outwards while a negative height makes the edge curve inwards. These parameters can be used together with the `edge_curve` statement.
- **origin** is the origin used to calculate the absolute position for the data points of the imported doping profile. The actual coordinates used in device simulation is

$$(\text{origin}) + (\text{sign-of-direction}) * (\text{relative-distance})$$
- **grading_from**, if used, is the relative starting value of linearly graded doping. Its value is relative to the final doping value of **max_conc**. The variation direction of the linear grading is that of **grading_angle**.
- **grading_angle** is the variation direction of the linearly graded doping. The reference direction is along +**x** direction.
- **pnt_num** is the number of points of the polygon. It is used only when “polygon” is chosen for **shape**.
- **newdoping_index**, if set to positive integer, is used to label a doping profile so that the doping concentration of this profile may be varied during a simulation. The variable key word in **scan** statement to control this profile is “new_doping”.
- **xy_datafile, xy_linedata, xy_data_factor, xy_x_transform, xy_y_transform, xy_skipline, xy_data_column**. These parameters with prefix of **xy_** are used to import doping file in ASCII format.
-
- **xy_datafile** is doping data file in a column-wise format: **x y doping-concentration**, where **x** and **y** are in microns and doping-concentration is in $1/m^3$. To convert to a different unit, the parameter **xy_data_factor** may be used.

- - **xy_linedata** indicates whether the doping profile is line-ordered data or randomly distributed. If "yes", the data is ordered in multiple straight x-lines, from left to right. The x-lines are ordered from bottom to top.
 -
 - **xy_data_factor** is multiplied to the doping concentration. It may also be used as a way of converting units. For example, if a the concentration in a data file is in $1/cm^3$, a factor of 1.e6 may be used through this parameter.
 -
 - **xy_x_transform** consists two parameters (say a and b) which can be used to perform a linear transform of the x-coordinate in the doping profile so that the actual coordinate used by the simulator is $a*x+b$.
 -
 - **xy_y_transform** is the same as **xy_x_transform** except it is for y-coordinate.
 -
 - **xy_skipline** is the number of header lines in the doping file that should be skipped.
 -
 - **xy_data_column** is the data column number where the doping-concentration can be found. In some data files, n- and p-doping are stored in different columns and this parameter can be used to specify which type of doping is to be used from the same file.
 - **traplevel_model** may take gaussian or expo_tail to enable gaussian or exponential tail distribution of trap levels when this statement is used to describe trap spatial distribution.
 - **traplevel_tail_side** indicates whether the exponential tail state is relative to the condition band or the valence band.
- traplevel_stddev** is the trap level standard deviation if the trap level model is gaussian.
- traplevel_tail** is the characteristic decay constant of the tail model. The trap density varies with $\exp(-E/L)$ where L is the **traplevel_tail** and E is the energy from either the conduction or valence band.

Example(s)

```
doping impurity=shal_dopant charge_type=donor max_conc=1.e24 &&
  x_prof=(0.0e0, 10.0e0, 0.01e0, 0.01e0 ) &&
  y_prof= 0.e0, 1.4e0, 0.02e0, 0.02e0

doping impurity=trap_1 charge_type=acceptor max_conc=1.e22 &&
  x_prof=(0.0e0, 10.0e0, 0.01e0, 0.01e0 ) &&
  y_prof=(0.e0, 10.e0, 0.02e0, 0.02e0 )

doping impurity=shal_dopant charge_type=donor max_conc=1.e24 &&
  shape=polygon &&
  edge1_prof= (0. 1.1 0.01) &&
  edge2_prof= (0.429 1.1 0.01) &&
  edge3_prof= (0.146 1.5 0.01) &&
  edge4_prof= (0. 1.5 0.01)

doping impurity=shal_dopant charge_type=acceptor max_conc=1.5e23 &&
  level=0.115 pf_model=yes pf_cont_shift=0.115 &&
  x_prof=(0.0e0, 10.0e0, 0.01e0, 0.01e0 ) &&
  y_prof=(0.e0, 1.e0, 0.02e0, 0.02e0 )
```

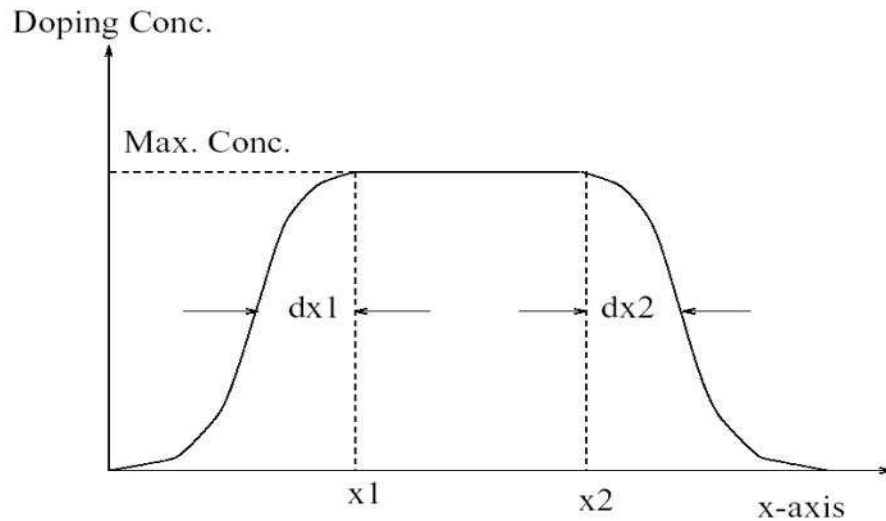


Figure 2.8: Schematics for the Gaussian doping profile in the x-direction.

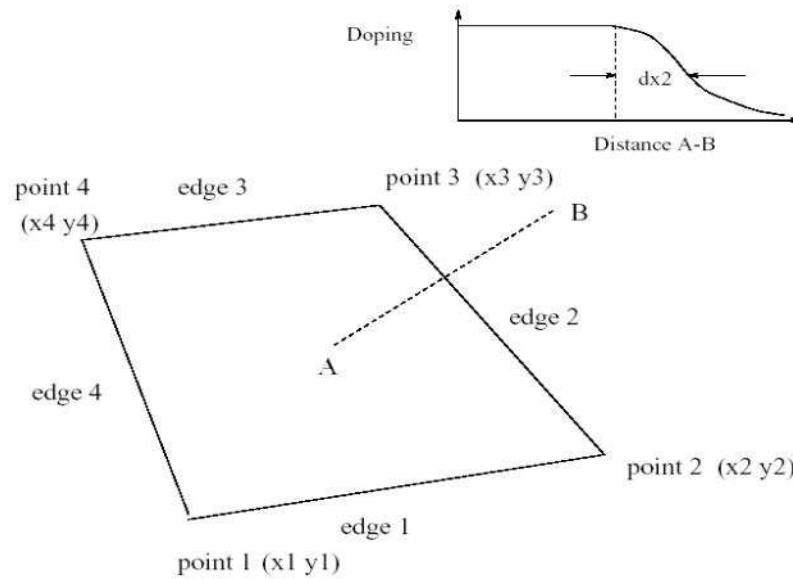


Figure 2.9: Schematics for the Polygon doping profile. Note the labeling of points and corresponding edges.

2.119 double_mesh

parameter	data type	values [defaults]
mesh_inf	char	
mesh_outf	char	
polygon	char	
order	char	[yes],no
direction	intg	1,2
point_ll	realx2	
point_ur	realx2	

The mesh generation statement **double_mesh** provides the basic mechanism to refine a rough mesh manually. It can be used to double the existing mesh density in a given region in the device. This statement is rarely needed because mesh can be automatically refine using the **refine_mesh** statement.

- **mesh_inf** is the mesh input file.
- **mesh_outf** is the mesh output file.
- **polygon** is the name of the polygon to be affected.
- **order** specifies the format of the mesh output. If the mesh output is to be used by the solver, “yes” should be chosen, i.e., ordering of the mesh is required to

interface with the solver. If the mesh is to be further manipulated after this statement, “no” should be chosen.

- **direction** is the direction of the mesh lines to be doubled. Direction 1 corresponds to lines running in the x-direction or the direction parallel to the line defined by the first two points of the polygon.
- **range1** is the relative range in direction 1 (similar to direction x) of the window within which the double mesh will occur. The reference point (or origin of the polygon) is the first point in the **polygon** statement (see Fig. 2.10).
- **range2** is the relative range in direction 2 (similar to direction x) of the window within which the double mesh will occur. The reference point (or origin of the polygon) is the first point in the **polygon** statement (see Fig. 2.10).

Example(s)

```
double_mesh polygon=p1 direction=1 &&
  point_ll=(0.2 0.) point_ur=(1.5 0.9) &&
  mesh_inf=cas1.msh1 mesh_outf=cas1.msh2 order=yes
```

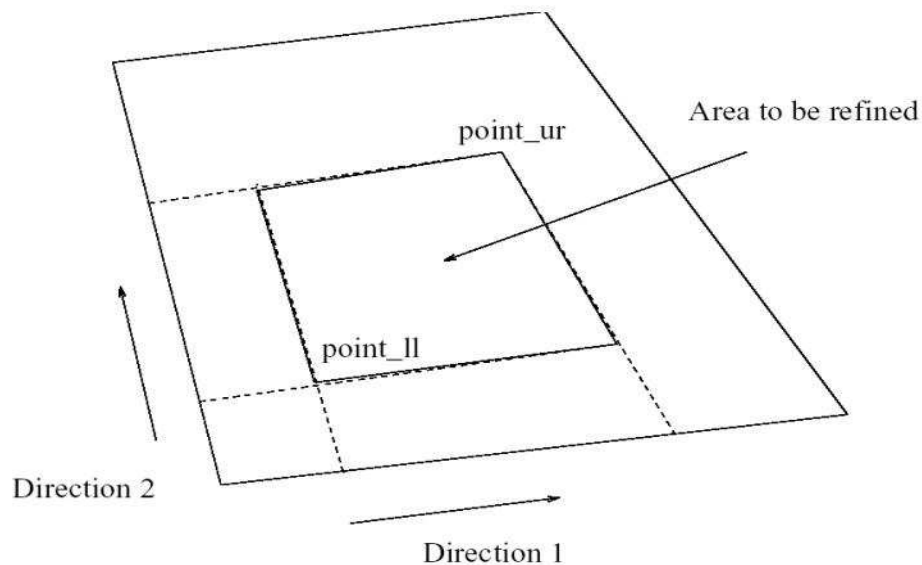


Figure 2.10: Schematics for the region to be manually refined.

2.120 `dox_efield0_pf_elec`

parameter	data type	values [defaults]
(see) <code>material_par</code>		

The material statement **`dox_efield0_pf_elec`** is a constant $E0_{pf}$ (in V/m) used to define the Poole-Frenkel-like E-field dependent electron mobility: $\mu_n = \mu_0 \exp[\sqrt{(E/E0_{pf})}]$. This parameter is used to describe a doped organic (thus the prefix `dox`) material. For more details, please see the following reference:

B. Ryhstaller and S.A. Carter, S. Barth, H. Riel, and W. Riess, "Transient and steady-state behavior of space charges in multilayer organic light-emitting diodes," J. Appl. Phys., 15 April, 2001, Vol. 89, No. 8, pp. 4575-4586.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **`material_par`** in section [2.272](#).

Example(s)

```
dox_efield0_pf_elec value=1.959e8 mater=1
```

2.121 `dox_efield0_pf_hole`

parameter	data type	values [defaults]
(see) <code>material_par</code>		

The material statement **`dox_efield0_pf_hole`** is a constant $E0_{pf}$ (in V/m) used to define the Poole-Frenkel-like E-field dependent hole mobility: $\mu_p = \mu_0 \exp[\sqrt{(E/E0_{pf})}]$. This parameter is used to describe a doped organic (thus the prefix `dox`) material. For more details, please see the following reference:

B. Ryhstaller and S.A. Carter, S. Barth, H. Riel, and W. Riess, "Transient and steady-state behavior of space charges in multilayer organic light-emitting diodes," J. Appl. Phys., 15 April, 2001, Vol. 89, No. 8, pp. 4575-4586.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **`material_par`** in section [2.272](#).

Example(s)

```
dox_efield0_pf_hole value=1.959e8 mater=1
```

2.122 dox_el_weight

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox_el_weight** is used to define the electroluminescent spectrum weight for the dopant material. This is useful when there is dopant in the system so that one has to define the relative contribution from host and the dopant. For EL spectrum model when there is dopant, two approaches are used, depending on whether exciton diffusion equations are solved. If exciton transport is solved by **organic_exciton_diff**, dopant and host spectra at each mesh point are simply mixed by local exciton density, divided by their respective life times, and weighted by this factor. If exciton transport is not considered, their respective bimolecular recombination rates are used along with this factor to determine the relative contributions. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
dox_el_weight value=0.01 mater=1
```

2.123 dox_exciton_eg

This statement is the same as **ox_exciton_eg** except it is for the dopant material.

2.124 dox_extern_spectrum

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox_extern_spectrum** is used to define the electroluminescent (EL) spectrum of the dopant material imported from an external source. This may be based on experimental data or on a different type of model. If table format is used, the program expects a 2-column uniformly spaced data with first column being the wavelength in micron meters. The unit of the EL is arbitrary and the program will normalize it.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
dox_extern_spectrum variation=table
table(wavelength)
  0.488524E+00  0.309522E-01
  0.499601E+00  0.421672E-01
  ...
end_table
```

2.125 dox_gaussian_divj

This statement is the same as **ox_exciton_eg** except it is for the dopant material.

2.126 dox_gaussian_sdj

This statement is the same as **dox_gaussian_sdj** except it is for the dopant material.

2.127 dox_hopping_energy

This statement is the same as **ox_hopping_energy** except it is for the dopant material.

2.128 dox_peak_abs

This statement is the same as **dox_peak_abs** except it is for the dopant material.

2.129 dox_vib_quanta

This statement is the same as **ox_vib_quanta** except it is for the dopant material.

2.130 dox_xp_coupling

This statement is the same as **dox_xp_coupling** except it is for the dopant material.

2.131 dox2_el_weight

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox2_el_weight** is used to define the electroluminescent spectrum weight for the 2nd dopant in multiply doped materials. This is useful to define the relative contribution from host and the dopant.

For EL spectrum model when there is dopant, two approaches are used, depending on whether exciton diffusion equations are solved. If exciton transport is solved by **organic_exciton_diff**, dopant and host spectra at each mesh point are simply mixed by local exciton density, divided by their respective life times, and weighted by this factor. If exciton transport is not considered, their respective bimolecular recombination rates are used along with this factor to determine the relative contributions. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
dox2_el_weight value=0.01 mater=1
```

2.132 dox2_extern_spectrum

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox2_extern_spectrum** is used to define the electroluminescent (EL) spectrum of the 2nd dopant material imported from an external source. This may be based on experimental data or on a different type of model. If table format is used, the program expects a 2-column uniformly spaced data with first column being the wavelength in micron meters. The unit of the EL is arbitrary and the program will normalize it.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
dox2_extern_spectrum variation=table
table(wavelength)
  0.488524E+00  0.309522E-01
  0.499601E+00  0.421672E-01
  ...
```

end_table

2.133 dox3_el_weight

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox3_el_weight** is used to define the electroluminescent spectrum weight for the 3rd dopant in multiply doped materials. This is useful to define the relative contribution from the host and the dopant.

For EL spectrum model when there is dopant, two approaches are used, depending on whether exciton diffusion equations are solved. If exciton transport is solved by **organic_exciton_diff**, dopant and host spectra at each mesh point are simply mixed by local exciton density, divided by their respective life times, and weighted by this factor. If exciton transport is not considered, their respective bimolecular recombination rates are used along with this factor to determine the relative contributions. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
dox3_el_weight value=0.01 mater=1
```

2.134 dox3_extern_spectrum

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox3_extern_spectrum** is used to define the electroluminescent (EL) spectrum of the 3rd dopant material imported from an external source. This may be based on experimental data or on a different type of model. If table format is used, the program expects a 2-column uniformly spaced data with first column being the wavelength in micron meters. The unit of the EL is arbitrary and the program will normalize it.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
dox3_extern_spectrum variation=table
table(wavelength)
  0.488524E+00  0.309522E-01
  0.499601E+00  0.421672E-01
  ...
end_table
```

2.135 dox4_el_weight

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox4_el_weight** is used to define the electroluminescent spectrum weight for the 4th dopant in multiply doped materials. This is useful to define the relative contribution from host and the dopant.

For EL spectrum model when there is dopant, two approaches are used, depending on whether exciton diffusion equations are solved. If exciton transport is solved by **organic_exciton_diff**, dopant and host spectra at each mesh point are simply mixed by local exciton density, divided by their respective life times, and weighted by this factor. If exciton transport is not considered, their respective bimolecular recombination rates are used along with this factor to determine the relative contributions. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

Example(s)

```
dox4_el_weight value=0.01 mater=1
```

2.136 dox4_extern_spectrum

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox4_extern_spectrum** is used to define the electroluminescent (EL) spectrum of the 4th dopant material imported from an external source.

This may be based on experimental data or on a different type of model. If table format is used, the program expects a 2-column uniformly spaced data with first column being the wavelength in micron meters. The unit of the EL is arbitrary and the program will normalize it.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
dox4_extern_spectrum variation=table
table(wavelength)
  0.488524E+00  0.309522E-01
  0.499601E+00  0.421672E-01
  ...
end_table
```

2.137 dox5_el_weight

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox5_el_weight** is used to define the electroluminescent spectrum weight for the 5th dopant in multiply doped materials. This is useful to define the relative contribution from host and the dopant.

For EL spectrum model when there is dopant, two approaches are used, depending on whether exciton diffusion equations are solved. If exciton transport is solved by **organic_exciton_diff**, dopant and host spectra at each mesh point are simply mixed by local exciton density, divided by their respective life times, and weighted by this factor. If exciton transport is not considered, their respective bimolecular recombination rates are used along with this factor to determine the relative contributions.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
dox5_el_weight value=0.01 mater=1
```


2.138 dox5_extern_spectrum

parameter	data type	values [defaults]
(see) material_par		

The material statement **dox5_extern_spectrum** is used to define the electroluminescent (EL) spectrum of the 5th dopant material imported from an external source. This may be based on experimental data or on a different type of model. If table format is used, the program expects a 2-column uniformly spaced data with first column being the wavelength in micron meters. The unit of the EL is arbitrary and the program will normalize it.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
dox5_extern_spectrum variation=table
table(wavelength)
  0.488524E+00  0.309522E-01
  0.499601E+00  0.421672E-01
  ...
end_table
```

2.139 do_multilayer_optics

parameter	data type	values [defaults]
autodef_entry_point	char	[yes], no
autodef_wavelength	char	[yes], no
polarization	char	[te], tm
external_opindex	realx2	[(1.0,0.0)]
entry_point_x	real	[0.01] (um)
entry_point_y	real	[1000.0] (um)
entry_point_z	real	[0.01] (um)
wavelength	real	[1.e-6] (m)
wavel_range_factor	real	[1.7]
datafile_number	intg	[1]
wavel_points_number	intg	[45]

do_multilayer_optics is used to activate the Multilayer Optical Analyzer (MOA) module and to set parameters related to it. It is usually placed in files with extension **.plt**.

- **autodef_entry_point** is a parameter that controls if an entry point (or source point) of light is defined automatically or no. If this parameter is set "yes", an entry point is found as a point with maximum optical power in the device. Otherwise user defines coordinates where light comes from. This is done by setting **autodef_entry_point=no**.

MOA reconstructs two stacks in directions -0Y and +0Y from entry point using structural information so that reflection and transmission are calculated for bottom and top parts (sta

If **autodef_entry_point=no** it is possible to construct also two stacks (if the defined point is within the device) or only one stack, depending on the position of the light

- **autodef_wavelength** is a parameter that defines if a major wavelength of light is calculated automatically by APSYS or defined by user.
- **polarization** is a parameter that controls the kind of light polarization in propagating plane wave, it can be TE or TM (**te** or **tm** parameters, respectively).
- **external_opindex** defines optical refractive index of external medium. In case of air it is set (1.0,0.0)
- **entry_point_x**, **entry_point_y**, **entry_point_z** are coordinates in three dimension that define the point of light entry.
- **wavelength** defines the wavelength of light in case if this parameter is not defined automatically by APSYS, i.e., if **autodef_wavelength** is set to "no".
- **wavel_range_factor** is the range of wavelength change deviating symmetrically from a given wavelength.
- **datafile_number** is used to define file number (with extension **.rta**) which consists of exported data of structural-optical properties of the simulated device. Data are generated for certain bias condition in APSYS main module, for example, for different voltage biases.
- **wavel_points_number** is the number of points for wavelength dependencies. For angle dependence it is 90 points (equals angular degrees) fixed for now.

Figure 2.11: Positions of the bottom and top parts (stacks) of device

Example

```
do_multilayer_optics datafile_number=3 &&
  autodef_entry_point=yes autodef_wavelength=no &&
  wavelength=5.8e-7 &&
  wavel_range_factor=1.35 wavel_points_number=60 &&
  polarization=te
```

The above statement activates the MOA module. RTA file with number three is used. The light source entry point is located inside the device in the center of area with maximum optical power. The light wavelength is defined by user, i.e., it is equal 5.8e-7 micrometers. The range

2.140 edge_curve

parameter	data type	values [defaults]
edge		
shape		[arc]
height		[μm]

The mesh generation statement **edge_curve** bends an edge of a polygon.

- **edge** contains the two points of the edge to be bent.
- **shape** is the shape into which the edge is to be bent.
- **height** is the height of the arc. The sign of the height is understood as follows: Let the counter-clockwise direction of the vector defined by two adjacent points of the polygon be vector *e* and the vector pointing from the starting point of *e* to a point on the arc be vector *a*. Then a positive height means that the cross product $\mathbf{a} \times \mathbf{e}$ must point to positive *z* direction. On the other hand, if the height is negative, the cross product yields a vector in the negative *z* direction. In another word, positive height makes the edge curve outwards while a negative height makes the edge curve inwards.

Example(s)

```
edge_curve edge=(a b) shape=arc height=-0.3
```

2.141 eeim_optic

parameter	data type	values [defaults]
gain_correction	char	[yes],no
sort_imag	char	[yes],no
pure_index_loss	char	[yes],no
export_eemsg	char	[no],yes
modal_base	intg	[3]
xsearch_real	intg	[2000]
xsearch_imag	intg	[40]
muller_maxit	intg	[80]
wi (i=1,2,3...19)	real	(um)
xsearch_range	real	[5.]
muller_vartol	real	[1.e-8]
muller_functol	real	[1.e-5]

The **eeim_optic** statement is used to invoke the enhanced effective index method (EEIM) for the optical mode computation. It can be used to treat device structure with leaky (radiative) modes.

The rectangle solution area is defined by **point_ll** and **point_ur** in the **init_wave** statement. It assumes that the modes in the y-direction is confined. The boundary on the right is such that it can support traveling wave to the right (which may be decaying as in the case of confined modes).

The boundary on the left can be confined as even or odd, or as radiative. The left boundary may be modified using the first number (we shall refer to as **lx1**) in **boundary_type** of the **init_wave** statement. If **lx1** is 1, the left boundary is to set the wave to zero (odd). If **lx1** is 2, the left boundary is to set the wave derivative to zero (even). If **lx1** is 3, the left boundary is to have traveling wave going out of the device (to the left).

To study device with radiation loss to the substrate, we suggest that the device be rotated by 90 degrees so that the substrate is towards the +x direction. The rotated device may be done within the .geo file or by using the GeoEditor.

- **gain_correction** may be used to enable the optical gain in the rate equation to be corrected for radiation loss. With this setting to “no”, The simulator will calculate the modal gain by averaging the local material gain with weight of the wave intensity.
- **sort_imag** is used to decide if EEIM modes are sorted according to the imaginary parts of the optical dielectric constant. If positive, the modes will be sorted according to modal gain with the highest first. If negative, the real part will be sorted with the highest real index first.
- **pure_index_loss**. There are two ways to estimate the modal radiative loss. The first is the pure index method. This is by setting all imaginary material indices to zero throughout the device. The imaginary part of the eigen value of the EEIM is the modal loss. The other method is to estimate the bias dependent radiative loss by comparing the EEIM solution with averaged material gain. The former is a convenient and definite way of characterizing the device. The latter is more realistic but less stable numerically. Please note that if **pure_index_loss=yes**, only the modal radiative loss for pure index structure is printed as run-time message. If **pure_index_loss=no**, both the radiative loss for pure index structure and the bias-dependent radiative loss at different bias will be printed
- **export_eeimmsg** is used to enable the export of EEIM column index data in .sol.msg files.
- **modal_base** is the number of y-modes within each column. In principle, there are infinite number of y-modes is the mode searching range is large enough. In practice, several of these will be sufficient.
- **xsearch_real** is the number of search points for the x-modes in the real part of $\bar{\epsilon}$, the eigen solution of the x-modes.

- **xsearch_imag** is the number of search points for the x-modes in the imaginary part of $\bar{\epsilon}$, the eigen solution of the x-modes.
- **muller_maxit** is the maximum number of the Muller solver that is used to find the complex root as the eigen solution of the lateral modes.
- **wi** ($i=1,2,3,\dots,19$) is the i th column width.
- **xsearch_range** is the search range for the real part of $\bar{\epsilon}$, the eigen solution of the x-modes.
- **muller_vartol** is the variable relative tolerance at convergence of the Muller solver.
- **muller_functol** is the function relative tolerance at convergence of the Muller solver.

```
eeim_optic modal_base=3  &&
w1=2 w2=0.75 w3=1.25 &&
xsearch_real=2000 xsearch_imag=5
```

The above statement uses three y-modes, sets the width for each column. The number search points are also specified.

2.142 eg0_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **eg0_bar** is an active layer macro statement used to define the undrained bandgap (eV) in the quantum barrier.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.143 eg0_bulk

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the unstrained bandgap in eV. This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.144 eg0_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **eg0_well** is an active layer macro statement used to define the unstrained bandgap (eV) in the quantum well.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.145 eeim_optic

parameter	data type	values [defaults]
gain_correction	char	[yes],no
sort_imag	char	[yes],no
pure_index_loss	char	[yes],no
export_eeimmsg	char	[no],yes
modal_base	intg	[3]
xsearch_real	intg	[2000]
xsearch_imag	intg	[40]
muller_maxit	intg	[80]
wi (i=1,2,3...19)	real	(um)
xsearch_range	real	[5.]
muller_vartol	real	[1.e-8]
muller_functol	real	[1.e-5]

The **eeim_optic** statement is used to invoke the enhanced effective index method (EEIM) for the optical mode computation. It can be used to treat device structure with leaky (radiative) modes.

The rectangle solution area is defined by **point_ll** and **point_ur** in the **init_wave** statement. It assumes that the modes in the y-direction is confined. The boundary on the right is such that it can support traveling wave to the right (which may be decaying as in the case of confined modes).

The boundary on the left can be confined as even or odd, or as radiative. The left boundary may be modified using the first number (we shall refer to as lx1) in **boundary_type** of the **init_wave** statement. If lx1 is 1, the left boundary is to set the wave to zero (odd). If lx1 is 2, the left boundary is to set the wave derivative to zero (even). If lx1 is 3, the left boundary is to have traveling wave going out of the device (to the left).

To study device with radiation loss to the substrate, we suggest that the device be rotated by 90 degrees so that the substrate is towards the +x direction. The rotated device may be done within the .geo file or by using the GeoEditor.

- **gain_correction** may be used to enable the optical gain in the rate equation to be corrected for radiation loss. With this setting to “no”, The simulator will calculate the modal gain by averaging the local material gain with weight of the wave intensity.
- **sort_imag** is used to decide if EEIM modes are sorted according to the imaginary parts of the optical dielectric constant. If positive, the modes will be sorted according to modal gain with the highest first. If negative, the real part will be sorted with the highest real index first.
- **pure_index_loss**. There are two ways to estimate the modal radiative loss. The first is the pure index method. This is by setting all imaginary material indices to zero throughout the device. The imaginary part of the eigen value of the EEIM is the modal loss. The other method is to estimate the bias dependent radiative loss by comparing the EEIM solution with averaged material gain. The former is a convenient and definite way of characterizing the device. The latter is more realistic but less stable numerically. Please note that if **pure_index_loss=yes**, only the modal radiative loss for pure index structure is printed as run-time message. If **pure_index_loss=no**, both the radiative loss for pure index structure and the bias-dependent radiative loss at different bias will be printed
- **export_eeimmsg** is used to enable the export of EEIM column index data in .sol.msg files.
- **modal_base** is the number of y-modes within each column. In principle, there are infinite number of y-modes is the mode searching range is large enough. In practice, several of these will be sufficient.

- **xsearch_real** is the number of search points for the x-modes in the real part of $\bar{\epsilon}$, the eigen solution of the x-modes.
- **xsearch_imag** is the number of search points for the x-modes in the imaginary part of $\bar{\epsilon}$, the eigen solution of the x-modes.
- **muller_maxit** is the maximum number of the Muller solver that is used to find the complex root as the eigen solution of the lateral modes.
- **wi** ($i=1,2,3,\dots,19$) is the i th column width.
- **xsearch_range** is the search range for the real part of $\bar{\epsilon}$, the eigen solution of the x-modes.
- **muller_vartol** is the variable relative tolerance at convergence of the Muller solver.
- **muller_functol** is the function relative tolerance at convergence of the Muller solver.

```
eeim_optic modal_base=3  &&
w1=2 w2=0.75 w3=1.25 &&
xsearch_real=2000 xsearch_imag=5
```

The above statement uses three y-modes, sets the with for each column. The number search points are also specified.

2.146 efield0_pf_elec

parameter	data type	values [defaults]
(see) material_par		

The material statement **efield0_pf_elec** is a constant $E_{0_{pf}}$ (in V/m) used to define the Poole-Frenkel-like E-field dependent electron mobility: $\mu_n = \mu_0 \exp[\sqrt{(E/E_{0_{pf}})}]$ For more details, please see the following reference:

B. Ryhstaller and S.A. Carter, S. Barth, H. Riel, and W. Riess, "Transient and steady-state behavior of space charges in multilayer organic light-emitting diodes," J. Appl. Phys., 15 April, 2001, Vol. 89, No. 8, pp. 4575-4586.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
efield0_pf_elec value=1.959e8 mater=1
```

2.147 efield0_pf_hole

parameter	data type	values [defaults]
(see) material_par		

The material statement **efield0_pf_hole** is a constant $E_{0_{pf}}$ (in V/m) used to define the Poole-Frenkel-like E-field dependent hole mobility: $\mu_p = \mu_0 \exp[\sqrt{(E/E_{0_{pf}})}]$ For more details, please see the following reference:

B. Ryhstaller and S.A. Carter, S. Barth, H. Riel, and W. Riess, "Transient and steady-state behavior of space charges in multilayer organic light-emitting diodes," J. Appl. Phys., 15 April, 2001, Vol. 89, No. 8, pp. 4575-4586.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
efield0_pf_hole value=1.959e8 mater=1
```

2.148 elec_carr_loss

parameter	data type	values [defaults]
(see) material_par		

elec_carr_loss: Used to specify the electron part of free carrier absorption in any material. This is to be compared with **ncarr_loss** and **pcarr_loss** in command **active_reg** which only affect the active regions. User may define temperature and composition dependence of free carrier absorption for electron in macro file. The electron part of free carrier loss is expressed as follows:

$$< \text{electron part of free carrier loss} > = < \text{elec_carr_loss} > < n \text{ carrier} >$$

See also section **hole_carr_loss**.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
elec_carr_loss value=2.e-21 mater=1
```

2.149 elec_dos_energy

parameter	data type	values [defaults]
data_file	char	
energy_range	realx2	
data_point	intg	[50]
subband_valley	intg	[1]

elec_dos_energy plots the electron density of states (DOS) versus energy. This statement is only used in the gain preview module.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **energy_range** is the range of energy in eV.
- **data_point** is number of data plots to be plotted.
- **subband_valley** is the index labeling the subband valley. An index of 1 means the Gamma band and 2 the L-band.

Example(s)

```
elec_dos_energy  energy_range=(1 1.7)
```

2.150 electron_mass

parameter	data type	values [defaults]
(see) material_par		

The material statement **electron_mass** defines the electron effective mass relative to the free electron mass.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.151 electron_mobility

parameter	data type	values [defaults]
material_par		

The statement **electron_mobility** is the dopant dependent low field mobility for electrons. It is usually expressed in terms of μ_{2n} , μ_{1n} as in Eqs. (2.4).

In a material macro system, an alternative way to specify the low field mobility is to use the following **max_electron_mob**, **min_electron_mob**, **electron_ref_dens**, and **alpha_n**.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** or **group1** or in section 2.272.

2.152 electron_ref_dens

parameter	data type	values [defaults]
(see) material_par		

The material statement **electron_ref_dens** is the reference density N_{rn} (in m^{-3}) for the electron mobility model of Eq. (2.4).

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.153 electron_sat_vel

parameter	data type	values [defaults]
material_par		

The material statement **electron_sat_vel** is used to define the saturation electron velocity (in m/s). It is used in Eq. (2.6) to define the field-dependent mobility function.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.154 `embedded_material`

parameter	data type	values [defaults]
<code>position_from</code>	real	(um)
<code>position_to</code>	real	(um)
<code>grade_from</code>	real	(um)
<code>grade_to</code>	real	(um)
<code>mater</code>	intg	
<code>grade_var</code>	intg	[0]
<code>grade_num</code>	intg	[5]

`embedded_material` is used to define an embedded material. An embedded material may be regarded as the 2nd material overlaying on the first material in the same region. This works with the **`get_active_layer`** command which loads the macro of the embedded material. This command is used to provide further information related to the macro of the embedded material.

- **`position_from`**, and **`position_to`** are the absolute starting and ending coordinates of the embedded region.
- **`grade_from`**, **`grade_to`** are the starting and ending compositions.
- **`mater`** is the material number which the user must define.

`grade_var` is the material composition variable number with which there is composition grading.

`grade_num` is the number of composition evaluation points in case there is composition grading.

Example(s)

```
embedded_material position_from=0.5 position_to=0.501 mater=6
```

2.155 `end_cavity`

`end_cavity` works with **`begin_cavity`**. Please see the latter for more details. This statement has no parameters.

2.156 `end_complex`

The statement **`end_complex`** is used to end the description of complex coupled quantum well structure.

The definition of a complex should be in the following format:

```
{  
begin_complex ...  
complex_region ...  
complex_region ...  
complex_region ...  
...  
end_complex  
}
```

There should be odd number of layer for each column due to the historical convention that a complex is used to describe a coupled MQW system in the sequence of barrier-well-barrier...well-barrier.

This statement has no parameters.

2.157 **end_loop**

end_loop is to terminate the loop which is initialized by **start_loop**.

2.158 **end_zmater**

The statement **end_zmater** is used with **begin_zmater** to bracket material and doping statements for a z-segment in the following format. **zseg_num** is the z-segment number.

```
{  
begin_zmater zseg_num=#  
material statements.  
doping statements.  
end_zmater  
}
```

2.159 equilibrium

parameter	data type	values [defaults]
print_output	char	[yes]
all_newdoping	char	[no]
outfile_label	char	[void]
bandgap_reduction	real	[0.]
mobility_order	real	[0.]
surf_charge_expo	real	[0.]
newdoping_order	real	[0.]
nstep	intg	[1]
zseg_num	intg	[1]
index_newdoping	intg	
num_newdoping	intg	[1]
n_index_newdoping	intgxn	

- **print_output** is used to decide whether detailed xy-data sets are to be printed.
- **all_newdoping** decides if all doping profile labeled as “newdoping” will be affected by this statement.
- **outfile_label** labels the output file (.out) file so that future scan statements can refer to it.
- **bandgap_reduction** is used to set the overall initial bandgap reduction factor at equilibrium. The bandgap is multiplied by (1-bandgap_reduction).
- **mobility_order** is used to increase or reduce the overall mobility at equilibrium. A factor of $10^{m.o.}$ is multiplied to the mobility where m.o. stands for **mobility_order**.
- **surf_charge_expo** is used to increase or reduce the equilibrium surface charge density as specified by the **surface** statement. A factor of $10^{s.c.e}$ is multiplied to the surface charge density where s.c.e stands for **surf_charge_expo**.
- **newdoping_order** is used to specify an initial doping concentration reduction factor given by $10^{newdoping_order}$. If it is zero (default), no reduction of doping concentration is set initially. The purpose of using this factor is to reduce the doping concentration at equilibrium so that numerical convergence is easily achieved.
- **nstep** is used only when equilibrium is difficult to converge. In such case, several steps may be used.

- **zseg_num** is the segment number.
- **index_newdoping** refers to the single doping profile to be varied in the simulation. This index must corresponds to the new-doping labels attached to a doping statement.
- **num_newdoping** is the number of indices to be specified by **n_index_newdoping**.
- **n_index_newdoping** is the same as **index_newdoping** except it is used to specify indices of more than one doping profile to be affected.

2.160 equiv_tunnel_junc

parameter	data type	values [defaults]
y_start_label	char	[void]
y_end_label	char	[void]
set_n_side	char	[void],up,down
use_physical_model	char	no
xrange	realx2	(-1.e9 1.e9)(um)
yrange	realx2	(um)
equiv_mobility	real	[0.01] ($m^2/V/s$)
tun_mass	real	(relative)
zseg_num	intg	1

equiv_tunnel_junc may be regarded as a simplified form of **zener**. It uses an equivalent mobility model for the exchange of electrons and holes at a tunneling junction. Usually this model is more numerically friendly then **zener**.

- **y_start_label** labels the position for yrange. The label must be predefined using **x_position**, **y_position** or **define_vertical_position** command. They may be predefined in the .layer file relative to a specific layer so that there is no need to find out the absolute coordinates.
- **y_end_label** labels the position for yrange. The label must be predefined using **x_position**, **y_position** or **define_vertical_position** command. They may be predefined in the .layer file relative to a specific layer so that there is no need to find out the absolute coordinates.
- **set_n_side** is not void, defines n-side of the tunnel junction. If it is void, the program will examine the doping profile to decide the n-side. The simulator needs to know this in order to decide the carrier types on both sides to be exchanged.

- **use_physical_model** would use physical model to estimate the equivalent tunneling mobility.
- **xrange** is the x-range of the tunnel junction.
- **yrange** is the y-range of the tunnel junction.
- **equiv_mobility** is the equivalent mobility for the transport of carriers across the junction. This will determine the slope of the I-V curve of the junction.
- **tun_mass** is the Zener interband tunneling mass to be set by the user. If it is not specified, the combined effective mass is used as in the following formula: $2m_em_h/(m_e + m_h)$. This is effective when **use_physical_model=yes**
- **zseg_num** is the z-segment number in a 3D structure.

Example(s)

```
equiv_tunnel_junc y_start_label=y1 y_end_label=y2 equiv_mobility=0.02
$ The labels y1 and y2 are predefined positions.
```

2.161 export_gain_data

parameter	data type	values [defaults]
file	char	[void]
pn_ratio_range	realx2	[1. 1.]
temper_range	realx2	[300. 300.] (K)
pn_ratio_points	intg	[1]
temper_points	intg	[1]

export_gain_data exports the gain/index/PL data into an ASCII file for later use. It is a preview simulation statement (used in a .gain file).

- **file** the data file into which the gain data is to be exported. If not specified, a default file name gain_datafile.txt is to be used.
- **pn_ratio_range** is the range of hole/electron ratio changes for the data.
- **temper_range** is the temperature range in the data.
- **pn_ratio_points** is the number of data points for hole/electron ratio changes.
- **temper_points** the number of temperature points.

Example(s)

```
export_gain_data pn_ratio_range=[0.5 1.5] pn_ratio_points=5 &&
  temper_range=[250 350] temper_points=5
```

2.162 export_kp_data

parameter	data type	values [defaults]
file	char	[void]

export_kp_data exports the k.p subbands and dipole moment data an ASCII file for later use. It is a preview simulation statement (used in a .gain file).

- **file** the data file into which the k.p data is to be exported. If not specified, a default file name kp_datafile.txt is to be used.

Example(s)

```
export_kp_data
```

which will save the k.p data in kp_datafile.txt.

2.163 export_kp_para

parameter	data type	values [defaults]
file	char	[void]

export_kp_data exports the k.p parameters, such as Luttinger numbers as an ASCII file for later use. It is a preview simulation statement (used in a .gain file).

- **file** the data file into which the k.p data is to be exported. If not specified, a default file name kp_parafile.txt is to be used.

Example(s)

```
export_kp_para
```

which will save the k.p parameters in kp_parafile.txt.

2.164 export_opt_property

export_opt_property is used to export data of structural-optical properties of the simulated device for the Multilayer Optical Analyzer (MOA) module. Data are generated for certain bias condition in APSYS main module, for example, for different voltage biases. The files with extension

This statement has no parameters.

Example

```
export_opt_property
```

The above statement in **.sol** file activates APSYS subroutines that generate input data for MOA module.

2.165 export_raytrace

parameter	data type	values [defaults]
every_bias	char	[no]
transp_contact	char	[no]
transp_contact1	char	[void]
transp_contact2	char	[void]
transp_contact3	char	[void]
ray3d_convert	char	[no]
contact_refl	real	[0.99]
contact_refl1	real	[0.99]
contact_refl2	real	[0.99]
contact_refl3	real	[0.99]
contact1_compindex	realx2	[2. 5.]
contact1_thick	real	[0.1] (um)
contact2_compindex	realx2	[2. 5.]
contact2_thick	real	[0.1] (um)
contact3_compindex	realx2	[2. 5.]
contact3_thick	real	[0.1] (um)
dbr1_box1	realx2	(um)
dbr1_box2	realx2	(um)
dbr1_box3	realx2	(um)
dbr2_box1	realx2	(um)
dbr2_box2	realx2	(um)
dbr2_box3	realx2	(um)
dbr1_layer1	real	[0.1] (um)
dbr1_index1	real	[3.5]
dbr1_layer2	real	[0.1] (um)
dbr1_index2	real	[3.5]
dbr2_layer1	real	[0.1] (um)
dbr2_index1	real	[3.5]
dbr2_layer2	real	[0.1] (um)
dbr2_index2	real	[3.5]
contact1_layer2	realx3	
contact2_layer2	realx3	
contact3_layer2	realx3	
contact1_layer3	realx3	
contact2_layer3	realx3	
contact3_layer3	realx3	

export_raytrace is used to export control parameters for the ray-tracing model at

the post-processing stage.

- **every_bias** indicates whether ray-tracing data are exported at every bias point. If "no", the export action will occur only at structure data printing point corresponding to .out and .std files.
- **transp_contact** indicates whether transparent contact is used. If "yes", a model of semi-transparent box with complex refractive index **contactn_compindex** is used. If "no", a model of opaque This parameter applies to all contacts, unless otherwise specified by **transp_contactk**, k=1,2,3.
- **transp_contactk**, k=1,2,3, has the same meaning as **transp_contact** except it is used to specify only the kth contact. This parameter overrides the specification of **transp_contact**.
- **contact_refl** is the power reflectivity of an opaque contact. It is used for the model of opaque mirror only. This parameter applies to all contacts, unless otherwise specified by **contact_reflk**, k=1,2,3.
- **contact_reflk**, k=1,2,3, has the same meaning as **contact_refl** except it is used to specify only the kth contact. This parameter overrides the specification of **contact_refl**.
- **contactn_compindex** (n=1,2,3) is the complex refractive index of a transparent contact. It is used for the transparent contact model only. Table 2.1 presents a list of refractive indexes for some mat
- **contactn_thick** (n=1,2,3) is the thickness of a transparent contact. It is used for the transparent contact model only.
- **dbrj_boxk** (j=1,2; k=1,2,3) is the x-y coordinates of an inside point of the jth DBR stack consisting of kth material box. The point location can be any place within the material box.
- **dbrj_layerl** (j=1,2; l=1,2) is the lth layer within the jth DBR stack.
- **dbrj_indexl** (j=1,2; l=1,2) is the lth real index within the jth DBR stack.
- **ray3d_convert** indicates which ray tracing program should be used (2D or 3D). In 2D simulations, the default is to use the 2D ray tracing program. This program is now considered obsolete and only kept for compatibility purposes; we recommend using the 3D ray tracing program for all simulations. By activating this option in a 2D simulation, the ray tracing program will assume a z-length of 1 μm . The power units will thus always be in Watts and thus may not match the results from the old 2D ray tracing program (which were in W/m). Note that this option will be ignored in simulations with 3D-flow since these always use the 3D ray tracing program.

Substance \	Wavelength [micrometer]								
		0.400	0.500	0.600	0.700	0.800	1.000	4.000	10.000
Aluminum	n	0.400	0.667	1.043	1.550	1.990	1.991	6.100	26.000
	k	4.450	5.573	6.568	7.000	7.050	9.300	30.400	67.300
Copper	n	-	0.880	0.186	0.150	0.170	0.197	1.940	10.520
	k	-	2.420	2.980	4.050	4.840	6.272	23.100	59.290
Germanium	n	2.300	3.400	4.500	5.150	5.270	5.100	4.350	4.300
	k	2.800	2.250	1.700	1.300	0.900	0.450	-	-
Gold	n	-	0.840	0.200	0.131	0.149	0.179	2.040	11.500
	k	-	1.840	2.897	3.842	4.654	6.044	27.900	67.500
Indium	n	-	1.020	1.290	1.390	1.500	1.760	7.600	23.800
	k	-	2.080	2.590	5.750	6.690	7.690	26.100	51.700
Lead	n	-	-	-	1.680	1.510	1.410	6.580	21.000
	k	-	-	-	3.670	4.240	5.400	20.800	37.400
Mercury	n	0.730	1.040	1.390	1.760	2.140	-	-	-
	k	3.010	3.700	4.320	4.830	5.330	-	-	-
Silver	n	0.075	0.050	0.060	0.075	0.090	0.250	2.340	10.800
	k	1.930	2.870	3.750	4.620	5.450	6.810	26.900	60.700

Table 2.1: Complex index of refraction of some metals and semiconductors by wavelength

Example(s)

```

export_raytrace transp_contact=yes &&
contact1_compindex=(2.1 3.) contact1_thick=0.1 &&
dbr1_box1=(5. 1.) &&
dbr1_box2=(30. 1.) &&
dbr1_box3=(55. 1.) &&
dbr2_box1=(30. 4.5) &&
dbr1_layer1 =0.0595 &&
dbr1_index1 = 3.467 &&
dbr1_layer2 =0.0706 &&
dbr1_index2 = 2.92 &&
dbr2_layer1 =0.0706 &&
dbr2_index1 = 2.92 &&
dbr2_layer2 =0.0595 &&
dbr2_index2 = 3.467

```

The above statement defines the contacts of an LED to be transparent. The 1st DBR consists of three boxes which are defined by the .geo file to be three polygons. In .layer files, these may be from three different columns on the same layer. The 2nd DBR only has one material box.

2.166 export_super_struct

parameter	data type	values [defaults]
file	char	[void]
y_start_label	char	[void]
y_end_label	char	[void]
x_start_label	char	[void]
x_end_label	char	[void]

export_super_struct exports data related to a super-structure into an ASCII file for later use. The data contains properties of a structure which is regarded as part of a larger super-structure in a simulation project with larger dimensions. For example, this statement may be used in a .sol file simulating a smaller project of 2 periods of a super-lattice. The data exported may be used in a larger project simulating the whole super-lattice containing tens or hundreds of periods.

- **file** the data file into which the data is to be exported. Usually a file extension .sst is assigned to it.

- **y_start_label** is the starting point label defining the lower range of the super-structure in y-direction. This label must be defined in a .layer file (and appear in a .mater file).
- **y_end_label** is end ending point label defining the upper range of the super-structure in y-direction. This label must be defined in a .layer file (and appear in a .mater file).
- **x_start_label** can be set optionally. It is the starting point label defining the lower range of the super-structure in x-direction. If used, this label must be defined in a .layer file (and appear in a .mater file).
- **x_end_label** can be set optionally. It is the ending point label defining the upper range of the super-structure in x-direction. If used, this label must be defined in a .layer file (and appear in a .mater file).

Example(s)

```
export_super_struct file=al.sst
```

The above statement does not have any labels and thus exports the properties of the whole structure into the al.sst file.

Example(s)

```
export_super_struct file=al.sst y_start_label=y1 y_end_label=y2
```

The above statement exports the properties of a part of the device between y1 to y2. The labels y1 and y2 must be previously defined in a .layer file (or .mater file). To label positions within a layer structure, please see statement **layer_position**.

2.167 export_wave

parameter	data type	values [defaults]
backward	char	[no]
data_file	char	[void]
print_point	intg	

export_wave is used to export the wave solution of Beam Propagation Method to a designated output file.

- **backward** is used to export forward or backward wave.
- **data_file** is the data file name into which the wave data are to be dumped.
- **print_point** is the print point at which the data to be to exported.

Example(s)

```
export_wave backward=no data_file=tmp.wave print_point=12
```

2.168 `ext_funck` (k=1..9)

parameter	data type	values [defaults]
(see) material_par		

The material statement **ext_func*i*** (i=1,2,...,9) is an active layer macro statement used to define an external function to be used by other statements in the same macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.169 external_cir

parameter	data type	values [defaults]
extern_parallel_cir	char	[no]
intern_parallel_cir	char	[no]
z_dim	real	[10]
resistance	real	[0.]
capacitance	real	[0.]
inductance	real	[0.]
extern_parallel_res	real	[0.]
extern_parallel_cap	real	[0.]
extern_parallel_ind	real	[0.]
extern_parallel_voltage	real	[0.]
intern_parallel_res	real	[0.]
intern_parallel_cap	real	[0.]
intern_parallel_ind	real	[0.]
intern_parallel_voltage	real	[0.]
contact	intg	

The statement **external_cir** describes the external circuit attached to a contact. If the external circuit consists of resistor and if external thermal conductor is also specified for the same contact, self-heating effect will be used on the resistor. I.e., the Joule heat generated by the resistor will be taken into account. Please see the schematic in Fig. ?? for the configuration of the three branches of the external circuit attached to a device.

- **extern_parallel_cir** enables the external branch of the parallel circuit.
- **intern_parallel_cir** enables the internal branch of the parallel circuit.
- **z_dim** is the z-dimension (the third dimension) of the device simulated. It is used to multiply/divide to the circuit parameter for 2D simulation or quasi-3D simulation (with current unit of Amp/m). For full 3D simulation (with current unit of Amp), this parameter is not used.
- **resistance** is the resistance of the main branch in Ohm.
- **capacitance** is the capacitance of the main branch in Farad.
inductance is the inductance of the main branch in Henry.
- **extern_parallel_res** is the resistance of the external parallel branch in Ohm.
- **extern_parallel_cap** is the capacitance of the external parallel branch in Farad.

- **extern_parallel_ind** is the inductance of the external parallel branch in Henry.
- **extern_parallel_voltage** is the voltage applied to the external parallel branch.
- **intern_parallel_res** is the resistance of the internal parallel branch in Ohm.
- **intern_parallel_cap** is the capacitance of the internal parallel branch in Farad.
- **intern_parallel_ind** is the inductance of the internal parallel branch in Henry.
- **intern_parallel_voltage** is the voltage applied to the internal parallel branch.
- **contact** is the contact number related to this command.

Example(s)

```
external_cir resistance=50 z_dim=20 contact=2
```

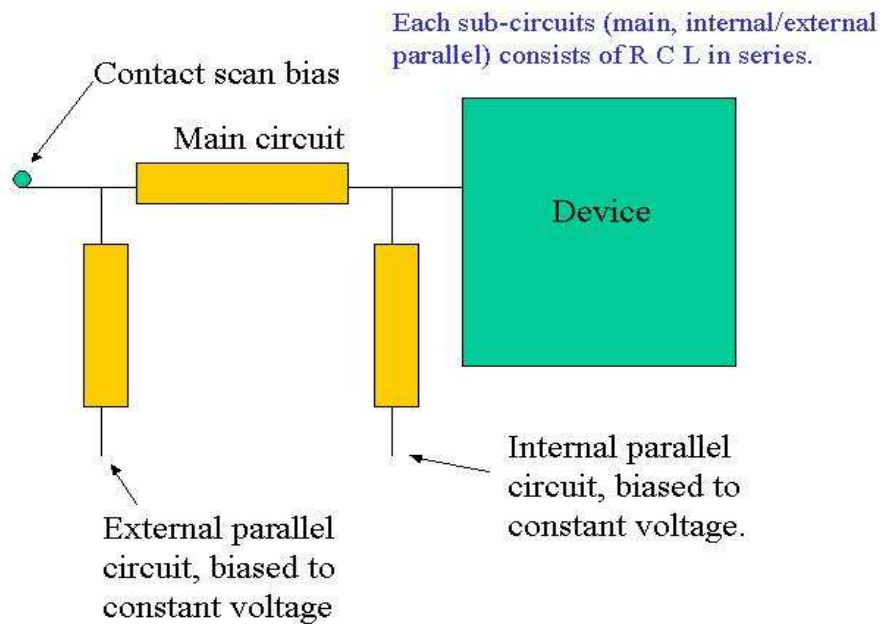


Figure 2.12: Schematics for the `external_cir` statement.

2.170 farfield

parameter	data type	values [defaults]
option	char	[contour], segment_x, segment_y
data_file	char	[void]
symmetric	char	[yes],no
add_all_modes	char	[yes],no
theta_x	real	[10] (degrees)
theta_y	real	[40] (degrees)
2ndmode_phase	real	[0.]
points	intg	[30]
mode_index	intg	[1]
level	intg	[9]
sum_2ndmode	intg	[0]

farfield is a post-processing statement used to compute the farfield distribution.

- **option** specifies the format of the farfield. The choice of “contour” results in the far field shown in contour plots. “segment_x” and “segment_y” give a 1D-segment plot of the far field along the x and y-directions, respectively.
- **data_file** is the output data file.
- **symmetric** specifies whether or not the device being simulated is symmetric. If the choice is “yes”, it is assumed that only the right half is simulated and the far-field is computed with the missing near field on the left half. If the choice is “no”, then the far field is computed using only the near field as it is.

add_all_modes instructs the program to add the intensities of all far-field modes together without consideration of any phase-interference effect. The far field modes are weighted by the respective modal powers.

- **theta_x** is the range of the farfield angle along the x-direction.
- **theta_y** is the range of the farfield angle along the y-direction.
- **2ndmode_phase** is the phase factor of the higher order mode to be added to the farfield of the main mode. This is activated by the **sum_2ndmode** parameter. The summation takes into account phase difference caused by one round trip for each mode. The phase appearing in this parameter is added to the round trip phase difference.
- **points** is the number of points for field evaluation in both directions.

- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used.
- **level** is used in the far field contour plot to specify the number of contour levels. There are **level** number of contour lines drawn between the minimum and maximum values in equal intervals.
- **sum_2ndmode**, if set to non-zero integer, indicates the higher order mode farfield is to be added to the main mode. (see also 2ndmode_phase parameter above).

Example(s)

```
plot_data plot_device=x11
farfield option=contour points=30 &&
  theta_x=15 theta_y=70
```

2.171 farfield_couple

parameter	data type	values [defaults]
symmetric	char	[yes],no
theta	real	[2] (degrees)
2ndmode_phase	real	[0.]
scale_lit	real	[1.]
scale_curr	real	[2.]
points	intg	[50]
mode_index	intg	[1]
sum_2ndmode	intg	[0]
electrode	intg	[1]

farfield_couple used to integrate a certain farfield angle and modify the L-I curve to simulate a detector.

- **symmetric** specifies whether or not the device being simulated is symmetric. If the choice is “yes”, it is assumed that only the right half is simulated and the far-field is computed with the missing near field on the left half. If the choice is “no”, then the far field is computed using only the near field as it is.
- **theta** is the farfield angle of power detection.

- **2ndmode_phase** is the phase factor of the 2nd order mode to be added to the farfield of the main mode.
- **scale_lit** is a scale factor to be multiplied to light power.
- **scale_curr** is a scale factor to be multiplied to current.
- **points** is the number of points in each direction for farfield integration.
- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used.
- **sum_2ndmode** is the mode number of the higher order mode to be added to the main mode for farfield computation. (see also 2ndmode_phase parameter above).
- **electrode** is the electrode upon which the current of the L-I curve is based.

Example(s)

```
farfield_couple theta=5 sum_2ndmode=1
```

2.172 fdtd_model

parameter	data type	values [defaults]
export_var	char	[density],flux_x,flux_y,flux_z,total_flux
export_grid	char	[FEM],ALL,NOBG
export_data	char	[meep_density.dat]
wavel_range	realx2	(um)
PML_thickness	real	(um)
extra_time	real	[0.0] (0.125ps)
buffer_x	realx2	[0.0 0.0] (um)
buffer_y	realx2	[0.0 0.0] (um)
buffer_z	realx2	[0.0 0.0] (um)
nb_wavel	intg	[100]
nb_mesh	intgx3	[20 20 0]
boundary_type	intgx3	

fdtd_model sets parameters for FDTD (finite difference time domain) simulation.

- **export_var** is the variable to export from FDTD calculation to be used by device simulator.

export_grid is the kind of mesh to be exported. Only FEM is used by simulator while other options are used for debugging purpose.

- **export_data** is the file name of FDTD data.
- **wavel_range** is the wavelength range over which FDTD analysis is done.
- **PML_thickness** is the thickness of the PML.
- **extra_time** is the time added to pulse duration in relative unit of $a/C=0.125$ psec. where a is the unit length of the device (μm) and C is the speed of light.
- **buffer_x**, **buffer_y**, **buffer_z** are used to denote empty space added to negative and positive sides of the simulated device in x,y,z directions, respectively.
- **nb_wavel** is the number of wavelength points for data export.
- **nb_mesh** is the grid size of the FDTD simulation. The results would be mapped to the FEM grid of the device. Thus, it is advised that FDTD grid sizes be smaller than the FEM grid sizes of the device being simulated.
- **boundary_type** takes 0 or 1, representing PML (perfectly match layer) or periodic, respectively.

Example(s)

```
fdtd_model export_var = density wavel_range = [0.55,0.65]    &&
      PML_thickness = 1 boundary_type = [1,0,1]    &&
      buffer_y = [2,2] nb_wavel = 10 nb_mesh = [20,150,0]    &&
      extra_time = 0.0
```

2.173 fdtd_source

parameter	data type	values [defaults]
component	char	Ex,Ey,Ez,Hx,Hy,Hz
amplitude	realx2	[1.0 0.0]
center	realx3	(μm)
size	realx3	(μm)

Command **fdtd_source** defines FDTD source terms which may add up coherently. FDTD program decides reasonable pulse width/duration based on spectrum width.

- **component** specifies polarization of incident light pulse. It is advised that for dielectric medium, all of E_x , E_y and E_z be used since complicated device structure requires them all.

- **amplitude** is a complex number specifying amplitude of incident light pulse. It is advised that this is not altered except by FDTD experts.
- **center** is a center position of light source in APSYS coordinate. It may take negative values to mean being placed below the device (for y-direction).
- **size** is the size of light source. User can set various type of light source, e.g. point, linear shape, planar shape and 3D-box source, by changing the size parameter.

Example(s)

```
fdtd_source component=Ez  &&
  center=(5.0 2.0 0.0) size = (10.0 0.0 0.0)
```

The above is a line source of width 10 um with center located at x=5, y=2.

2.174 **fourier_power**

parameter	data type	values [defaults]
data_file	char	[void]
input_var	char	
output_var	char	
facet	char	[void] front/back
log_freq	char	[no]
freq_start	real	[3.] (decade)
freq_end	real	[10.] (decade)
scale_lit	real	[2.]
scale_curr	real	[2.]
scale_horizontal	real	[1.]
scale_vertical	real	[1.]
mode_index	intg	[1]

The statement **fourier_power** is used in the post-processing step to obtain system frequency response function from transient simulation. The program takes the time dependence of input and output variables, makes a Fourier transform and divides to obtain the response function. Suppose these are denoted as $V_{in}(t)$ and $V_{out}(t)$. This statement takes the Fourier transform and computes $20\log_{10}(U_{out}/U_{in})$ where U_{in} and U_{out} are corresponding Fourier transform magnitudes.

- **data_file** is the data file name to which ASCII is exported.

- **input_var** is the input variable and takes the same values as **scan_var** in statement **plot_scan**. These are listed in Appd. D.
- **output_var input_var** is the output variable and takes the same values as **variable** in statement **plot_scan**. These are listed in Appd. D.
- **facet** , if defined, indicates the which facet power is plotted.
- **log_freq** indicates whether log frequency scale is used in the plot.
- **freq_start** is the starting frequency of the Fourier analysis.
- **freq_end** is the ending frequency of the Fourier analysis.
- **scale_lit** scales the laser light output for L-I plot. This is useful when scaling to take into account device symmetry.
- **scale_curr** scales the current in L-I plot in a laser simulation. This is useful when scaling to take into account device symmetry.
- **scale_horizontal** scales the horizontal axis for non-L-I plots.
- **scale_vertical** scales the vertical axis for non-L-I plots.
- **mode_index** is lateral mode index (label).

Example(s)

```
fourier_power input_var=current_1 output_var=laser_power &&
    freq_start=6. freq_end=10.
```

2.175 full_ionization

parameter	data type	values [defaults]
mater	intg	

The statement **full_ionization** may be used to force all shallow dopants of a material to be fully ionized. By default, the incomplete ionization model based on Fermi statistics is used all materials.

- **mater** is the material number representing the material to take the full ionization model. The material numbers can be found in the .mater files.

Example(s)

```
full_ionization mater=3
```

2.176 flux_plot

flux_plot is used to plot the light energy flux density distribution within the ray-tracing device. This statement is used at the post-processing stage after ray-tracing program is run.

This statement has no parameters.

2.177 freq_control

parameter	data type	values [defaults]
freq_start	real	[1.e6] (Hz)
decade	intg	[4]

The statement **freq_control** is used to control the frequency range within which the modal spectrum is computed.

- **freq_start** is the frequency starting point of the range.
- **decade** is the number of decades of the frequency range.

Example(s)

```
freq_control freq_start=1.e7 decade=3
```

2.178 front_index

parameter	data type	values [defaults]
real_index	real	[1.]
imag_index	real	[0.]

front_index is used to specify the complex refractive index in front of a semiconductor device. It serves as the outside index used to compute the Fresnel reflections in **light_power**.

This command has no practical effect if **front_reflection** is used.

2.179 front_reflection

parameter	data type	values [defaults]
spectrum_file	char	void
power_transmission	real	[1.]
reflection_phase	real	[0.] (degrees)

front_reflection is used to specify the reflection of light power at the front of a semiconductor device. It is the very first interface used in the **light_power** calculations so should not be combined with **optic_coating**.

back_reflection is the matching statement for the back of the semiconductor and is the last interface used in the **light_power** calculations. Both statements override the normal Fresnel reflection calculations and so can be used to quickly define an effective multi-layer coating.

- **spectrum_file** reads the power transmission coefficient from an input file. The data must be arranged in columns with the first column being the wavelength in μm and the second the transmission coefficient.
- **power_transmission** is the power transmission coefficient.
- **reflection_phase** is the phase of the field reflection coefficient. The field transmission coefficient is always assumed to be real.

Example(s)

```
front_reflection power_transmission=0.99
```

The above statement is used to model an effective broadband AR coating.

2.180 gain_density

parameter	data type	values [defaults]
data_file	char	
include_data	char	
wavel_range	realx2	(μm)
conc_range	realx2	[1.e23 1.e24] (m^{-3})
pn_ratio	real	[1]
av_index	real	[3.3]
data_point	intg	

gain_density plots the peak material gain versus carrier density in the active region. This statement can be used in the gain preview module for LASTIP and PICS3D. For PICS3D, this statement can also be used in the .sol file to define the gain spectrum of gain type 3. Please also see the statement of **scan3d**.

- **data_file** is the file to which the graphic data is written in ASCII format.

- **include_data** includes data files from other gain calculations for the purpose of comparison.
- **wavel_range** is the wavelength range within which the peak material gain is searched.
- **conc_range** is the electron concentration range in the well.
- **pn_ratio** is the ratio of hole over electron concentrations. Note that this ratio can be set to an arbitrary number in the gain preview. In the main solver, this ratio is determined by the simulator automatically according to the local Fermi levels.
- **av_index** is the estimated average refractive index.
- **data_point** is the number of data points in the curve.

Example(s)

```
gain_density wavel_range=(1.0 1.4) &&
conc_range=(5.e23 5.e24) pn_ratio=1 data_point=20
```

2.181 gain_module

parameter	data type	values [defaults]
use_sheet_density	char	[no]
tilt_imref	char	[no]
apply_e_field	real	[0.]

gain_module is used to modify some conditions in a .gain preview simulation.

- **use_sheet_density** instructs the program to use sheet density in .gain preview simulation.
- **tilt_imref** indicates whether the IMREF is to be tilted in accordance with the applied field.
- **apply_e_field** is the applied electrical field in V/m.

Example(s)

```
gain_module apply_e_field=1.e6
```

2.182 gain_fitting

parameter	data type	values [defaults]
fitting_model	char	[parabolic]
gpeak_model	char	[linear]
w0_model	char	[linear]
n_user_values	realxn	[(-8.e3 1.4e-20 1.32 0.0 -1.7e7 3.2e-17)]
user_values_num	intg	[6]

By default, PICS3D uses a bilinear interpolation of existing data to compute the gain at any carrier density and wavelength. The statement **gain_fitting** allows the use of various models to replace this interpolation. This may help to smooth out the gain function and facilitate convergence.

Currently, four gain fitting models are implemented. The selection is made with the **fitting_model** parameter:

- **parabolic** The default model. The gain is modeled as:

$$g(\lambda, N) = g_0(N) - \gamma(N) * (\lambda - \lambda_0(N))^2 \quad (2.8)$$

γ is always modeled as a linear function of N with the added constraint of being positive. The behavior of g_0 and λ_0 is controlled by the **gpeak_model** and **w0_model** parameters below.

- **splines** A simple bicubic spline interpolation.
- **hybrid** This model is a hybrid between a spectral spline interpolation and a fit on N . The shape of this fit is determined by the **gpeak_model** parameter
- **user_parabola** The same as **parabolic** except that instead of an automatic fit, the model parameters are defined with the **n_user_values** parameter.

In the **parabolic** and **hybrid** models, the carrier dependence of the peak gain is controlled by the **gpeak_model** parameter.

- **linear** The parameter follows a linear regression $y = a + b * N$
- **log** The parameter follows a linear regression on a logarithmic scale $y = a + b * \ln(N)$.

The same options apply to the position of the peak wavelength in the parabolic model **w0_model**.

For user-defined models, the number of model parameters (**user_values_num**) and their value (**n_user_values**) must be defined. Currently, only user-defined parabolas are allowed so the number of model parameters should always be 6. A typical declaration would therefore look something like this:

```
gain_fitting fitting_model=user_parabola &&
             6_user_values=[-8.e3 1.4e-20 1.32 0.0 -1.7e7 3.2e-17] &&
             user_values_num=6
```

The first two values of **n_user_values** are the a and b values used in **gpeak_model**. The result is a material gain value in m^{-1} . The next two are the a and b values for **w0_model**. The result is a peak wavelength in μm . The last two are the a and b values for the linear interpolation of $\gamma(N)$. The result should be adjusted to get a reasonable gain width.

2.183 gain_spectrum

parameter	data type	values [defaults]
data_file	char	[void]
variable	char	[gain], sp.rate, index_change, total_sp.rate
		rtg_spectrum, rtg_spectrum_left, rtg_spectrum_right
scale	real	[1.]
wavelength_range	realx2	(um)

gain_spectrum is a post-processing statement used to plot the modal gain spectrum at different current/voltage bias points corresponding to the printed data set. The data sets associated with this statement are given by **xy_data** in the **get_data** statement. It is also used to plot emission spectrum by a laser/VCSEL in the coupled RTG method.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **variable** can be used to control plot of modal gain, spontaneous emission and index change spectrum at different injection levels.
- **scale** would scale spectrum.
- **wavelength_range** would set the wavelength range for the spectrum.

Example(s)

```
gain_spectrum variable=gain
```

2.184 gain_spon

parameter	data type	values [defaults]
data_file	char	
include_data	char	
wavel_range	realx2	(μm)
conc_range	realx2	[1.e23 1.e24] (m^{-3})
pn_ratio	real	[1]
av_index	real	[3.3]
data_point	intg	

gain_spon plots the peak material gain versus the spontaneous recombination current density in the active region. The recombination current is defined as the spontaneous combination rate, which is the integration of the spontaneous emission spectrum over all wavelengths, times the thickness of the active region. This statement is only used in the gain preview module.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **include_data** includes data files from other gain calculations for the purpose of comparison.
- **wavel_range** is the wavelength range within which the peak material gain is searched.
- **conc_range** is the electron concentration range in the well.
- **pn_ratio** is the ratio of hole over electron concentrations. Note that this ratio can be set to an arbitrary number in the gain preview. In the main solver, this ratio is automatically determined by the simulator according to the local Fermi levels.
- **av_index** is the estimated average refractive index.
- **data_point** is the number of data points in the curve.

Example(s)

```
gain_spon wavel_range=(1.0 1.4) &&
conc_range=(5.e23 5.e24) pn_ratio=1 data_point=20
```

2.185 gain_wavel

parameter	data type	values [defaults]
data_file	char	
include_data	char	
auto_pn_ratio	char	[no]
conc_log_scale	char	[no]
plot_as_abs	char	[no]
wavel_range	realx2	(μm)
conc_range	realx2	(m^{-3})
pn_ratio	real	[1.0]
av_index	real	[3.3]
curve_number	intg	
data_point	intg	
zseg_num	intg	[1]
kp8x8_angle_points	intg	[0]

gain_wavel plots the gain versus wavelength. This statement is only used in the gain preview module.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **include_data** includes data files from other gain calculations for the purpose of comparison.
- **auto_pn_ratio** is used to indicate if automatic setting of hole/electron density ratio is used in PICS3D simulation. If positive, averaged hole/electron density from the drift-diffusion solver is used. This parameter, if positive, will override **pn_ratio**.
- **conc_log_scale** would vary the concentration in log scale.
- **plot_as_abs** instructs the program to plot the absolute value of the gain.
- **wavel_range** is the wavelength range.
- **conc_range** is the electron concentration range in the well.
- **pn_ratio** is the ratio of hole over electron concentrations. Note that this ratio can be set to an arbitrary number in the gain preview. In the main solver, this ratio is determined automatically by the simulator according to the local Fermi levels.
- **av_index** is the estimated average refractive index.

- **curve_number** is the number of curves.
- **data_point** is the number of data points in one curve.
- **zseg_num** is the z-segment number.
- **kp8x8_angle_points** applies to plotting optical gain based on the 8x8 k.p theory. It is the number of angle points of the k-direction along which the subband dispersion is calculated and optical gain evaluated through numerical integral.

Example(s)

```
gain_wavel wavel_range=(1.0 1.4) &&
conc_range=(5.e23 5.e24) curve_number=5 data_point=100
```

2.186 **gammak_bar**

parameter	data type	values [defaults]
(see) material_par		

The material statement **gammak_bar** (k=1,2,3) is an active layer macro statement used to define the Luttinger numbers in the quantum barrier.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.187 **gammak_well**

parameter	data type	values [defaults]
(see) material_par		

The material statement **gammak_well** (k=1,2,3) is an active layer macro statement used to define the Luttinger numbers in the quantum well.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.188 `get_active_layer`

parameter	data type	values [defaults]
name	char	AlGaAs/AlGaAs, etc.
var_name1-9	char	void
var_symbol1-9	char	
var1-9	real	
mater	intg	[1]

The statement **get_active_layer** is used in the solver input file or in the gain preview input file to define the type of active layer.

Note that the material parameters from the active layer macro defined here override those appearing in the bulk material macro.

- **name** specifies the name of the macro to be used. It is usually assigned material names with mix-cased letters to indicate active-layer macro. See the file “crosslight.mac” for the actual macros being used and all other details on each. For example, it may take one of InGaAs/AlGaAs, AlGaAs, InGaAsP/InP, InGaAsP, InGaAsP/GaAs, InGaAlAs/InP, InGaN/InGaN, InGaAlP/GaAs, InGaN/AlGaN, InGaN, etc....
- **var#** define various parameters in the macro called. Refer to the file “crosslight.mac” for details for each macro.
- **mater** is the material number assigned by the user to a specific material. This number should be consistent with the corresponding parameter in the **polygon** statement of the mesh generator.
- The parameter **var_name1-5** is the name(s) of the variable(s) [usually the material composition(s)]. Note that this parameter only gives the name of the variable but does not specify how this parameter varies in space. The material variations can be further defined by the statements of **grade_active_mater** and/or **grade_active_bar**.

Example(s)

```
get_active_layer name=InGaAsP/InP var1=0.467 var2=1. &&
var3=0.202 var4=0.440 mater=3 &&
var_symbol1=xw var_symbol2=yw var_symbol3=xb var_symbol4=yb
```

2.189 `get_data`

parameter	data type	values [defaults]
<code>main_input</code>	char	
<code>sol_inf</code>	char	
<code>xy_data</code>	intgx2	
<code>scan_data</code>	intgx2	

The statement **get_data** is used in the post processor, *i.e.* it is used only after the main solution is complete. It is used to get the output data from the main solver.

- **main_input** is the main input file that was used to drive the main simulation.
- **sol_inf** is the main simulation output data file which contains the current data, carrier distribution data, etc. It is produced by the input file **main_input**.
- **xy_data** specifies the structural data (*i.e.*, data as a function of position x and y) to be used by graphic statements to display position dependent data (*e.g.*, by **plot_1d**). The starting data set and the ending data set numbers are defined by this statement. If one would like to plot data set 1 to 3 on the same plot, one can set **xy_data=(1,3)**.

This parameter is used by any subsequent plot statement that displays structural data such as electron concentration distribution. Note that each data set corresponds to one **print_step** in the **scan** statement. Not all the bias points have the full structural data. Also see the **scan** statement for more detail on bias points and data sets. The bias voltage or current for each data set can be found in the message file with extension “*.sol.msg”.

- **scan_data** defines the beginning and the ending data_set used by any subsequent **plot_scan** statement. For bias data they allow the user to plot the specified variable within this range of data sets.

Example(s)

```
get_data main_input=bulk1d.sol sol_inf=bulk1d.out &&
xy_data=(7,7) scan_data=(1,7)
```

This causes the position dependent data set number 7 to be plotted while the bias data (such as I-V data) ranging from data set 1 to set 7 are displayed.

2.190 grating_compos

parameter	data type	values [defaults]
hi_macro_name	char	[void]
hi_active_macro	char	[void]
lo_macro_name	char	[void]
lo_active_macro	char	[void]
hi_var_symbol1-5	char	[void]
hi_avar_symbol1-5	char	[void]
lo_var_symbol1-5	char	[void]
lo_avar_symbol1-5	char	[void]
d_high	real	[0.1] (um)
d_fall	real	[0.]
d_low	real	[0.1] (um)
d_rise	real	[0.]
hi_var1	real	
hi_var2	real	
hi_var3	real	
hi_var4	real	
hi_var5	real	
hi_avar1	real	
hi_avar2	real	
hi_avar3	real	
hi_avar4	real	
hi_avar5	real	
lo_var1	real	
lo_var2	real	
lo_var3	real	
lo_var4	real	
lo_var5	real	
lo_avar1	real	
lo_avar2	real	
lo_avar3	real	
lo_avar4	real	
lo_avar5	real	
grating_order	intg	[1]
column_num	intg	[1]

grating_compos is used in the .layer file to generate the statement **grating_model** to be used by the .sol file. The main difference between this statement and **grating_model** is that it uses the material composition to define a grating corrugation

instead of directly using refractive indices. Please refer to the **grating_model** statement for most parameters. Since a grating corrugation is treated as consisting of two materials, one with high index and the other low index, most parameters are labeled as “hi_” or “lo_”. The embedded structures are regarded as “mildly active” because we need to consider the variation of index and loss/gain as a function of bias. Thus both active and bulk macros must be used. For more details on embedded active materials, please see statement **active_reg**. For the use of **avar1-5**, please see statement **layer_mater**. For use of variables in bulk and active macros, please see statements **load_macro** and **get_active_layer**.

hi_var_symbol1-5 works with **hi_var1-5**, in the same way as **var_symbol1-5** with **var1-5** in statement **layer_mater**. Please consult statement **layer_mater**. Similar explanation applies to **hi_avar_symbol1-5**, **lo_var_symbol1-5**, and **lo_avar_symbol1-5**.

2.191 grating_model

parameter	data type	values [defaults]
real_index_high	real	
imag_index_high	real	[0.]
real_index_low	real	
imag_index_low	real	[0.]
d_high	real	[0.1] (μm)
d_fall	real	[0.] (μm)
d_low	real	[0.1] (μm)
d_rise	real	[0.] (μm)
grating_xrange	realx2	[0. 5.] (μm)
grating_yrange	realx2	[2. 2.1] (μm)
grating_order	intg	[1]
mode_index	intg	[1]
use_active_mater	intg	[0]

grating_model is used to calculate the grating parameters after the optical field is solved.

This statement can also be used in .layer file to define a certain layer and column as a grating layer. In such a case, there is no need to define the grating ranges because these will be determined by the layer statements. In .layer file, this statement must be placed right after the corresponding **layer_mater** so that the layer and column numbers can be identified.

- **real_index_high** , **real_index_high** , **imag_index_high** , **real_index_low** , **imag_index_low** , **d_high** , **d_fall** , **d_low** , and **d_rise** are used to describe

the variation of effective index along the longitudinal direction (z-direction). They are illustrated in Fig. 2.13.

- **grating_xrange** and **grating_yrange** are used to specify the rectangular cross section on the xy-plane (lateral) of the grating layer.
- **grating_order** is the grating order.
- **mode_index** is the lateral mode index. Mode 1 is the fundamental mode; mode 2 is the 2nd order mode, etc..
- **use_active_mater** is used to decide whether to use user supplied or material macro refractive index value. When **use_active_mater**=0, user has to specify refractive index by the parameters **real_index_high** and **real_index_low** above.

Example:

```
grating_model real_index_high=3.31 &&
  imag_index_high=-3.27 real_index_low=0.1 &&
  imag_index_low=-6.54 d_high=0.18 d_fall=0. &&
  d_low=0.12 d_rise=0. &&
  grating_xrange=(0. 1.5) &&
  grating_yrange=(1.962 2.012) &&
  grating_order=2
```

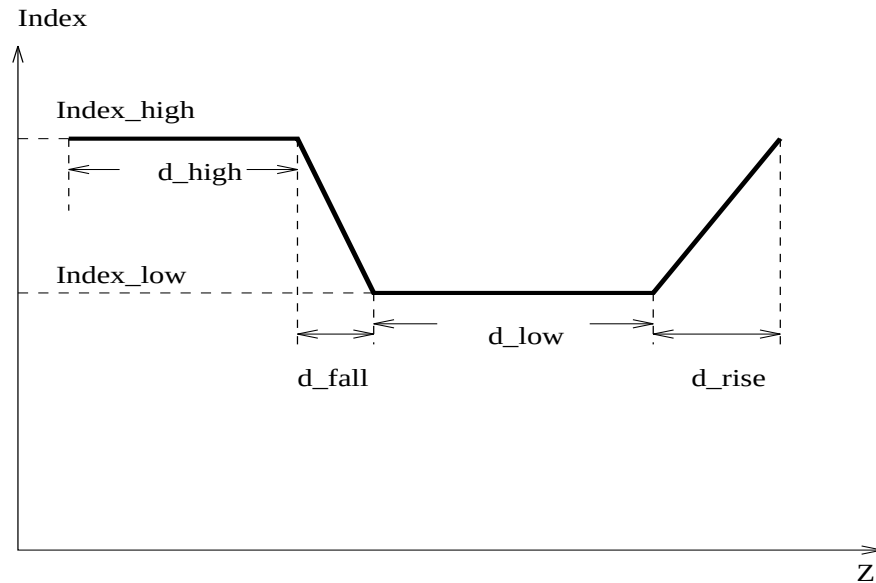


Figure 2.13: Schematics of the index variation in the longitudinal direction for the **grating_model** statement.

2.192 grade_active_mater

parameter	data type	values [defaults]
var_name	char	
variation	char	[linear]
grade_from	real	
grade_to	real	
grade_points	intg [3]	

grade_active_mater is used to indicate the composition grading of an active region and also for the barrier layer of a complex MQW.

- **var_name** is the symbolic name of the composition to be varied. It refers to a variable name already used in a **get_active_layer** statement.
- **variation** defines the behavior of the grading trend. It takes one of linear, function and table as values. For function, a separate analytical (piece-wise) function may be used. For table, numerical values can be used. This is similar to the commands in a material macro.
- **grade_from** is the starting composition for grading (from bottom to top for horizontal layer, and left to right vertically oriented layer).
- **grade_to** is the ending composition for grading (from bottom to top for horizontal layer, and left to right vertically oriented layer).
- **grade_points** is the number of linear grading mesh points. It must be greater than one. Please note that many quantities (such as bandgap) depends on composition in a non-linear manner. So more points may be needed if high accuracy is required. However, the program slows down significantly when grading mesh points are increased.

Examples:

```
grade_active_mater var_name=c1 grade_from=0.1 grade_to=0.2 grade_points=5
```

The above statement grades the composition c1 from 0.1 to 0.2 linearly by default. The following statement defines the variation of composition a2 using an analytical function:

```
get_active_layer name=cx-AlGaAs mater=    2 &&
  var_symbol1=xw &&
  var_name1=a2
grade_active_mater var_name=a2 variation=function
```

```
$ rdist=relative-distance is a reserved internal variable
function(rdist)
0.71+rdist*(0.33-0.71)
end_function
```

2.193 group1

parameter	data type	values [defaults]
variation	char	linear_x,linear_y,function,[constant]
var_namei(i=1..9)	real	
vari(i=1..9)	real	
var_symboli(1=1..9)	char	void
value	real	
n_values	realxn	
mater	intg	[1]
data_num	intg	[0]
var_num	intg	[1]

group1 is not a statement, but a dummy name used to represent a collection of the above parameters in the table. It is used in crosslight.tab to denote commonly used parameters of material property statements, such as **band_gap** or **hole_mass**. For more details of the parameters, please see **material_par**.

2.194 half_mesh

This statement works the same as **double_mesh** except that it reduces the mesh density by one half.

2.195 heat_flow

parameter	data type	values [defaults]
joule	char	[yes]
opt_absorption	char	[yes]
recombination	char	[yes]
thomson	char	[yes]
peltier	char	[yes]
update_mater	char	[no]
j.e_model	char	[no]
linear_source	char	[no]
thm_transient	char	[no]
jtotal_model	char	[no]
original_joule_term	char	[no]
radiation_source	char	[yes]
damping_step	real	[2.e0] (Kelvin)
var_tol	real	[1.e-5]
res_tol	real	[1.e-5]
max_temp_incr	real	[80] (degree)
set_low_temperature	char	
fit_range	real	[300.]
max_iter	intg	[150]
print_flag	intg	[2]
joule_mobil_flag	intg	[1]
mf2_flag	intg	[1] 0
fit_power	intg	[4]

heat_flow is used to control the solver for nonisothermal modeling, or the self-heating model. Various heating sources are controlled by this statement.

For detailed explanation of the physical origins of various heat source terms, the user is referred to the following reference. G.K. Wachutka, "Rigorous thermodynamic treatment of heat generation and conduction in semiconductor modeling", *IEEE Trans.*, **CAD-9**, 1141–1149 (1990).

Please note that all active layer parameters (bandgap, offset, etc.) are uniform throughout the same active layer. If there is a temperature gradient within the same active layer, the temperature dependent active layer parameters (active layer bandgap, etc.) are updated according to the highest temperature within the same active layer. This does not mean the temperature is forced to be uniform within a single active layer. It just means some parameters for the active layer are calculated according to the highest temperature within the single layer. This assumption is reasonable because the hottest region is often the most important region. If you

wish to take into account the temperature variation in all parts of the active region, you should artificially divide it into more than one active regions with different material numbers. This way, the active layer parameters will have different values in different active regions according to their respective local temperatures.

Since many heating source terms rigorously derived from semiconductor theory are difficult to compute accurately when mesh points are few, the user is encouraged to check and ensure the total amount of heating (or integrated heating source as printed at run time) does not exceed the power supplied by the DC bias. If the integrated heating source seem to violate energy conservation, one may wish to switch to the **j.e.model**. If Joule heating is the dominant heating source, this model is likely to yield reasonable results since by definition, it should equal to a simple ohmic resistor heating model.

- **joule** turns on/off the Joule heating term.
- **opt_absorption** turns on / off the Joule heating term due to optical absorption.
- **recombination** turns on/off the recombination heating source.
- **thomson** turns on/off the Thomson heat.
- **peltier** turns on/off the Peltier heat.
- **thm_transient** turns on/off the thermal transient simulation.
- **jttotal_model** The standard Joule heating source term is written as $q|j_n|^2/(\mu_n n) + q|j_p|^2/(\mu_p p)$ which may be unstable in regions of carrier depletion or minority carrier region with strong diffusion because a small amount of n or p may blow up the numerical expression. This flag turns on a more stable alternative $q|\mathbf{j}_n + \mathbf{j}_p|^2/(\mu_n n + \mu_p p)$. One can show that under the assumption of isotropic conductivity and small diffusion assumption, this expression is equivalent to the **j.e.model**.
- **original_joule_term** would instruct the simulator to use the original form of Joule term of the form: $\frac{J_n^2}{q\mu n}$ for electron current, where μ is low field mobility and n is electron concentration. Such a form as disadvantage of being numerical unstable if electron is highly depleted. Setting this parameter to “no” would allow the use of a modified form to avoid such a problem.
- **radiation_source** is a cooling term associated with the loss of energy with emission of photons or it can be a heating term associated with the interband absorption of incident light. It is written as the photon energy times the negative of the recombination rate associated with the photon emission/absorption.
- **damping_step** (in units of degree K) is the step limit used to damp (or reduce) the correction vector in the Newton solver for the temperature distribution.

- **var_tol** (in degree K) is the variable (temperature) tolerance for the thermal equation. If the temperature error is smaller than this tolerance, the thermal equation solver is considered converged.
- **res_tol** is the relative error of the residual of the thermal equation. It is an indication how well the thermal equation is satisfied. If the **res_tol** is zero, the equation is perfectly satisfied.
- **max_temp_incr** is the maximum temperature increase allowed when bias is changed. If the temperature increase exceeds this limit, the program regards the solution a failure and will restart the solution procedure with a smaller bias.
- **fit_range** is used to in the numerical procedure of fitting some of the most time consuming mesh and temperature dependent macro parameters. It specifies the temperature range for fitting a macro function to a polynomial.
- **max_iter** is the maximum number of iteration used in the Newton's method.
- **print_flag** is used to flag the printing of the variable and residual errors during the Newton iteration.
- **update_mater** is used to determine whether or not all of the material parameters are updated (or re-calculated) when a new temperature distribution is obtained by the Newton solver. If this parameter is turned on, all of the parameters are updated at all biases. If "no" is used, only a partial update is performed to save computation time. A partial update involves updating the band gap, the affinity (or the band offset), built-in contact potential, and all of the parameters listed in the "active-layer" macro.
- **j.e_model** is to indicate the use of an alternative expression of Joule heat in the form of $\mathbf{J} \cdot \mathbf{E}$ where $\mathbf{J}$ and $\mathbf{E}$ are the vectorial forms of current density and electrical fields. Since this term does not involve division of small numbers, it is more stable numerically. If non-convergence related to thermal simulation is encountered, one may wish to use this form of Joule heating source. For VCSEL simulation or 3D simulation involving rough mesh points, one may consider this choice. If Joule heating is the dominant heating source, this term is likely to yield reasonable results since by definition, it should equal to a simple ohmic resistor heating model.
- **linear_source** is to indicate the use of a heating source term proportional to the current. Please also see the statement of **linear_heat**.
- **thm_transient** may be used to turn on the thermal transient model.

- **joule_mobil_flag** indicates which mobility model will be used in computing the Joule heating source term. If a value of 1 is used, the field dependent mobility model is used. For a value of 2, the low field mobility is used. The former is more accurate and the latter is compatible with older versions of the thermal model (version 2002 and before).
- **mf2_flag** turns on the multi-frontal linear solver for the thermal equations. Use of this flag should result in better convergence of the thermal equations and improves the speed.
- **fit_power** is the power of the polynomial to represent some of the temperature dependent macros.
- **set_low_temperature** is related to the **fit_range** parameter. These two parameters determine the lower and upper range for fitting the temperature dependence of a parameter with a polynomial.
- **temperature_limit** is an upper limit beyond which the simulator would terminate the present **scan** statement.

Example(s)

```
heat_flow opt_absorption=no damping_step=2.
```

2.196 heteroj_capture

parameter	data type	values [defaults]
all_interface	char	[no]
elec_capture	real	[1.]
hole_capture	real	[1.]
interface_mater	intgx2	

This statement is used to modify the thermal velocity in the thermionic emission model governing the current transport across heterojunction interfaces.

The following current transport model across the heterojunction is used for all abrupt heterojunctions and Schottky metal-semiconductor interfaces.

$$J_{sn} = \gamma_n \bar{v}_n^{therm} (n_s - n_{eq}), \quad (2.9)$$

$$J_{sp} = \gamma_p \bar{v}_p^{therm} (p_s - p_{eq}), \quad (2.10)$$

A quantum well may be regarded as consisting of two heterostructure interfaces as far as current transport is concerned.

From the view point of quantum capture and emission, the thermionic model is equivalent to capture/emission into/out-of the quantum well at thermal velocity which is a very fast process. The actual process may be slower since for a carrier in the barrier to lose energy to be thermalized with the confined states in the well, carrier-phonon/carrier scattering is involved. Phenomenologically, the slower energy relaxation may be represented by a less than unity coefficient multiplied to the thermal velocity. This statement is used to define such coefficients.

- **all_interface** indicates whether or not the thermionic emission model is to be modified for all heterostructure interfaces.
- **elec_capture** is the electron capture coefficient (γ_n above) multiplied to the electron thermal velocity.
- **hole_capture** is the hole capture coefficient (γ_p above) multiplied to the hole thermal velocity.
- **interface_mater** defines the material numbers (or labels) of the two materials consisting the interface.

Example(s)

```
heteroj_capture all_interface=yes
    elec_capture=0.1 hole_capture=0.1
```

This statement reduces the thermal velocities of electrons and holes for all interfaces to ten percent of the thermal velocity.

```
heteroj_capture elec_capture=0.2 hole_capture=0.4 interface_mater=[5 6]
```

The above statement only modifies the thermal velocity for the interface between materials 5 and 6.

2.197 hole_carr_loss

parameter	data type	values [defaults]
(see) material_par		

hole_carr_loss: Used to specify the hole part of free carrier absorption in any materials. This is to be compared with **ncarr_loss** and **pcarr_loss** in command **active_reg** which only affect the active regions. User may define temperature and composition dependence of free carrier absorption for hole in macro file. The hole part of free carrier loss is expressed as follows:

$$< \text{hole part of free carrier loss} > = < \text{hole_carr_loss} > < n \text{ carrier} >$$

See also section **elec_carr_loss**.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
hole_carr_loss value=2.e-21 mater=1
```

2.198 hole_mass

parameter	data type	values [defaults]
(see) material_par		

The material statement **hole_mass** defines the hole effective mass relative to the free electron mass. Note that this mass is the result of average of density of states (DOS) of light hole and heavy hole masses.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.199 hole_mobility

parameter	data type	values [defaults]
material_par		

The statement **hole_mobility** is the dopant dependent low field mobility for holes. It is usually expressed in terms of μ_{2p} , μ_{1p} as in Eqs. (2.5).

In a material macro system, an alternative way to specify the low field mobility is to use the following **max_hole_mob**, **min_hole_mob**, **hole_ref_dens**, and **alpha_n**.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** or **group1** or in section 2.272.

2.200 hole_ref_dens

parameter	data type	values [defaults]
(see) material_par		

The material statement **hole_ref_dens** is the reference density N_{rp} (in m^{-3}) for the hole mobility model of Eq. (2.5).

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.201 hole_sat_vel

parameter	data type	values [defaults]
(see) material_par		

The material statement **hole_sat_vel** is used to define the saturation hole velocity (in m/s). It is used in Eq. (2.7) to define the field-dependent mobility function.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.202 impact_baraff

parameter	data type	values [defaults]
elec_phonon	real	[0.063] (eV)
hole_phonon	real	[0.063] (eV)
elec_lambda0	real	[76] (Ang)
hole_lambda0	real	[58] (Ang)
elec_eifactor	real	[1.5]
hole_eifactor	real	[1.5]
mater	intg	[1]

The statement **impact_baraff** is defined by the following formulas for impact ionization rate:

$$\alpha\lambda = \exp[g(r, x)] \quad (2.11)$$

$$\begin{aligned} g(r, x) = & (11.5r^2 - 1.17r + 3.9 \times 10^{-4})x^2 \\ & (46r^2 - 11.9r + 1.75 \times 10^{-2})x \\ & -757r^2 + 75.5r - 1.92 \end{aligned} \quad (2.12)$$

$$(2.13)$$

where

$$r = \langle E_p \rangle / E_I \quad (2.14)$$

and

$$x = E_I / qF\lambda \quad (2.15)$$

$\langle E_p \rangle$ is the optical phonon energy and λ is the optical phonon scattering mean free path. E_I is the ionization energy for which the following formula was suggested [1]:

$$E_I = f_e E_g \quad (2.16)$$

where f_e is a constant often chosen to be 1.5. The temperature dependence of $\langle E_p \rangle$ and λ are given by

$$\langle E_p \rangle = E_p \tanh\left(\frac{E_p}{2kT}\right) \quad (2.17)$$

$$\lambda = \lambda_0 \tanh\left(\frac{E_p}{2kT}\right) \quad (2.18)$$

- **elec_phonon** , **hole_phonon** are the electron and hole phonon energy (E_p above), respectively.
- **elec_lambda0**, **hole_lambda0** are the zero temperature electron and hole optical phonon mean free paths, respectively.
- **elec_eifactor** , **hole_eifactor** are the f_e factor above for electrons and holes, respectively.
- **mater** is the internal material number.

Example(s) The following example will take the default impact ionization parameters for mater number 4

```
impact_baraff mater=4
```

[1] C.R. Crowell and S.M. Sze , Appl. Phys. Lett., vol. 9, No. 6, pp. 242-244, 1966.

2.203 impact_chynoweth

parameter	data type	values [defaults]
elec_set1	realx3	[7.03e7 1.231e8 1]
fld_range_n1	real	[6.e7]
elec_set2	realx3	[7.03e7 1.231e8 1]
fld_range_n2	real	[9.e7]
elec_set3	realx3	[7.03e7 1.231e8 1]
hole_set1	realx3	[1.582e8 2.036e8 1]
fld_range_p1	real	[4.e7]
hole_set2	realx3	[6.71e7 1.692e8 1]
fld_range_p2	real	[6.e7]
hole_set3	realx3	[6.71e7 1.692e8 1]
elec_setnum	intg	[1]
hole_setnum	intg	[2]
mater	intg	[1]

The statement **impact_baraff** is defined by the following formulas for impact ionization rate:

A commonly used expression was proposed by Chynoweth [1] as follows:

$$\alpha = \alpha_n^\infty \exp[-(F_{cn}/F)] \quad (2.19)$$

This formula was later generalized to the following [2]:

$$\alpha = \alpha_n^\infty \exp[-(F_{cn}/F)^{\kappa_n}] \quad (2.20)$$

Similar expression can be written down for the holes. Usually, the above formulas is valid for a certain field range and we need more than one set of parameters for the whole field range.

If there are N field ranges with N sets of data of $(\alpha_n^\infty, F_{cn}, \kappa_n)$, we will need N-1 fields to divided the field ranges where each set of data is applicable.

Unit of α is 1/m so that its product with current flux density has the unit of recombination/generation.

- **elec_setj** is the data set of $(\alpha_n^\infty, F_{cn}, \kappa_n)$ for the jth field range for electrons.
- **fld_range_nj** is the upper limit of the jth field range and lower limit of (j+1)th field range. When there is only one field range, there is no need to use any field range parameters. This parameter is applicable for electrons.
- **hole_setj** is the same as **elec_setj** except it is for holes.
- **fld_range_pj** is the same as **hole_setj** except it is for holes.

- **elec_setnum** is the number of sets of data (or field ranges) for electrons. If it is one, there is no need to use any field range dividers.
- **hole_setnum** is the same as **elec_setnum** except it is for holes.

Example(s) The following example will take the default impact ionization parameters for mater number 4

```
impact_chynoweth mater=4
```

[1] A.G. Chynoweth, Phys., Rev.. vol. 109, p. 1537, 1958.

[2] S. Selberherr, "Analysis and simulation of semiconductor devices," Springer-Verlag, Wien-New York, 1984.

2.204 impact_model

parameter	data type	values [defaults]
elec_vel_factor	real	[0.7]
hole_vel_factor	real	[0.7]

The statement **impact_model** is used to set the impact ionization carrier velocity model. The theory is as follows. Conventionally, impact ionization (II) rate is calibrated as a function of electrical field while a more accurate description of the II process should use a carrier energy dependent function. That is, it is more accurate to consider II rate as a function of carrier energy (or carrier temperature) because it is the high carrier energy that causes II instead of the high field which is the indirect cause.

A common approach is to convert the carrier energy into an effective field which is then used in the original field dependent II rate formula. The conversion is as follows:

$$w - (3/2)k_B T_l = \tau_{eng} f_{w2f} v_{sat} F \quad (2.21)$$

which states that a hot carrier travels with an average velocity of $f_{w2f} v_{sat}$ for a duration of τ_{eng} under the electrical field F and gained an energy of w above the lattice temperature of T_l . Here k_B is the Boltzman constant and τ_{eng} is the energy relaxation time. f_{w2f} is less than unity meaning that on the average, the hot carrier travels with a fraction of the saturation velocity.

The parameters **elec_vel_factor** and **hole_vel_factor** are used to define f_{w2f} for electrons and holes.

This statement is effective only when the hot carrier model is activated.

Example(s)

```
impact_model elec_vel_factor=0.7
```

2.205 import_super_struc

parameter	data type	values [defaults]
file	char	[void]
import_dir	char	[void]
band_diagram_file	char	[void]
transport_file	char	[void]
mater	intg	[1]

import_super_struc imports data related to a super-structure as saved by a previous **export_super_struc** in a sub-project. The data contains properties of a structure which is regarded as part of a larger super-structure in a simulation project with larger dimensions. For example, this statement may be used in a .sol file simulating a larger of may periods of a super-lattice. The imported data come from a previous sub-project describing the behavior only several periods.

- **file** the data file into which the data are to be imported. Usually a file extension .sst is assigned to it.

import_dir is the directory containing the data file.

band_diagram_file is an auxialary data file containing band structure data information. This file is generated by a .gain file in a preview simulation.

transport_file is an auxialary data file containing carrier transport information. This file is generated by a .gain file in a preview simulation.

mater is the material number for which the data is imported. This material must be of type super_struc_type as defined by a macro.

Example(s)

```
import_super_struc file=al.sst import_dir=subproj band_diagram_file=al.bnd
```

The above statement imports two files from sub-directory subproj.

2.206 `import_gain_data`

parameter	data type	values [defaults]
<code>file</code>	char	[void]
<code>all_active_mater</code>	char	[yes]
<code>active_macro</code>	char	[void]
<code>extrap_energy</code>	char	[yes]
<code>extrap_conc</code>	char	[yes]
<code>extrap_pn_ratio</code>	char	[no]
<code>extrap_temper</code>	char	[yes]
<code>mater_active</code>	intg	
<code>active_and_embedded</code>	intg	

`import_gain_data` imports the gain/index/PL data from an ASCII file generated by **`export_gain_data`** statement.

- **`file`** the data file from which the gain data is to be imported. If not specified, a default file name `gain_datafile.txt` is to be used.
`all_active_mater` allows all active layers to use parameters in this statement, unless a specific material number is defined by **`active_macro`** or **`mater_active`**.
`active_macro` indicates that this statement affects all active layers with the above active macro name.
- **`extrap_energy`** indicates if extrapolation is used when photon energy is outside of the range in imported data.
- **`extrap_conc`** indicates if extrapolation is used when concentration is outside of the range in imported data.
- **`extrap_pn_ratio`** indicates if extrapolation is used when hole/electron ratio is outside of the range in imported data.
- **`extrap_temper`** indicates if extrapolation is used when temperature is outside of the range in imported data.
- **`mater_active`** if used, will only affect one particular material. Otherwise, all active material layer will use the imported data.
- **`active_and_embedded`** is the number of active and embedded (mildly active) regions sharing parameters in this statement. If not specified, all active and embedded regions will use this command.

Example(s)

export_gain_data

will import data from gain.datafile.txt.

Here is the code fragment of the data import:

```

        read(61,'(a)') line80 ! skip a header line
! three 0/1 numbers to indicate gain/index/PL data used.
        read(61,*)  impg(imp)%ixg_use_gain
        *,impg(imp)%ixg_use_index,impg(imp)%ixg_use_spon

        read(61,'(a)') line80 ! skip an info. line
! range of energy (eV) and number of point in energy (uniform)
        read(61,*)  ev_wvl11, ev_wvl22,npnt

        read(61,'(a)') line80 ! skip an info. line
! range of well concent. in 1/m^3 (bulk value) and number of points (uniform)
        read(61,*)  an1,an2,ncur

        read(61,'(a)') line80 ! skip an info. line
! range of hole/elec conc. in well and number of points.
! this can be sent to 1. 1. 1 if you always assum hole=electron
        read(61,*)  impg(imp)%gxi_pn_ratio_range(1)
        *   ,impg(imp)%gxi_pn_ratio_range(2),impg(imp)%num_pn_ratio_points

        read(61,'(a)') line80 ! skip an info. line
! range of temperature and number of points (uniform)
! if only one temperature is available, it is OK.  for example:
! 300. 300. 1   is OK.
        read(61,*)  impg(imp)%gxi_temper_range(1)
        *   ,impg(imp)%gxi_temper_range(2),impg(imp)%num_temper_points

! starting read in the data
        read(61,'(a)') line80 ! skip an info line

        do ix4=1,impg(imp)%num_temper_points ! number of temperature points
        do ix3=1,impg(imp)%num_pn_ratio_points ! number of p/n ratio points

```

```

do ix2=1,ncur ! number of conc. points
do ix1=1,npnt ! number of energy points

!
! 1st column is list of energy (eV),
! 2nd column is gain in 1/m^3
! 3rd column is change of real refractive index with respect to
! equilibrium concentration, or index(n)-index(n0) where
! n0=equilibrium well concentration.
! 3rd column is PL data in 1/m^3/s/eV (also in bulk value)
!
! read(61,'(4e21.12e3)') xdum,(tmpprn(kk),kk=1,numprn)
read(61,*) xdum,(tmpprn(kk),kk=1,numprn)

enddo

! there must be a blank line here so that the file is GNUPLOT friendly.
! for example, you can plot the gain in GNUPLOT using command:
! plot "mygainfile.txt" using 1:2
! for index change:
! plot "mygainfile.txt" using 1:3
! etc.
read(61,'(a)') line80

enddo
enddo
enddo

```

2.207 import_kp_data

import_kp_data is the reverse of **export_kp_data**. For other parameters, please see **import_gain_data**.

2.208 import_wave

parameter	data type	values [defaults]
data_file	char	[void]
segment_num	intg	[1]

import_wave is used to import the wave solution from a designated input file. The first line of the file must be a comment. The 2nd line should have the segment number and number of mesh points on the segment. The rest of the file are complex numbers of wave at all the nodes.

- **data_file** is the data file from which the import data is read in.
- **segment_num** is the segment for which the imported data is to be used.

Example(s)

```
import_wave data_file=tmp.wave segment_num=2
```

2.209 include

parameter	data type	values [defaults]
file	char	[void]

include is used to include another file as part of the present input file. It may also be used in the .layer input files.

- **file** is the name of the file to be included.

Example(s)

```
include file=laser1.doping
```

2.210 independent_mqw

parameter	data type	values [defaults]
no-parameters are needed		

independent_mqw is used in .layer file to assign different material numbers for active layers. Without this statement, active layers with the same material and structure would be assigned the same material number. The main simulator treats

the active layers with the same material number only once. If different material numbers are used for MQW's, the quantum well models will be used differently for different wells.

This statement is equivalent to assigning different active layer material number in .mater file generated from .layer file. It will be most useful when self-consistent quantum-confined carrier density model is to be used.

2.211 index_field_dependence

parameter	data type	values [defaults]
linear_term	real	[0.](m/V)
square_term	real	[0.](m^2/V^2)
mater	intg	[1]

index_field_dependence is used to describe a field dependent refractive index.

- **linear_term** is the linear term in the field dependence. It is related to the Pockels constant.
- **square_term** is the square term in the field dependence. It is related to the Kerr effect.

Example(s)

```
index_field_dependence linear_term=1.e-12 square_term=1.e-19 mater=2
```


2.212 index_model

parameter	data type	values [defaults]
sample_mesh	char	[no]yes
use_l.w.e.factor	char	[no]yes
free-carrier	char	[yes] no
interband	char	[yes] no
linear_active	char	[no] yes
linear_passive	char	[no] yes
free_carr_passive	char	yes [no]
eg_fac1	real	[10.]
eg_fac2	real	[2.]
l.w.e.factor	real	[1.0]
an_active	real	[0.0] (m^3)
bn_active	real	[0.0] (m^3/K)
ap_active	real	[0.0] (m^3)
bp_active	real	[0.0] (m^3/K)
an_passive	real	[0.0] (m^3)
bn_passive	real	[0.0] (m^3/K)
ap_passive	real	[0.0] (m^3)
bp_passive	real	[0.0] (m^3/K)
sample_point	intg	[200]

index_model is used to calculate the index change with respect to equilibrium. If this statement is not used, the simulator will use the average carrier densities in the active region to compute the index change.

There are five independent contributions to index change as follows:

- 1) Use the Kramers-Kronig relation to compute the index change within the active region.

$$\Delta\bar{n}(E) = -\frac{c\hbar}{\pi e} \int_0^\infty \frac{\Delta g(E') - \Delta g(E)}{(E' - E)(E' + E)} dE' \quad (2.22)$$

- 2) Use a line with enhancement factor α_{lw} to compute the index change based on local gain change as follows.

$$\Delta\bar{n}(E) = -\frac{\alpha_{lw}}{2k_0} (g - g_0) \quad (2.23)$$

- 3) Free-carrier/plasma part (active region):

$$\Delta\bar{n}(E) = -\frac{q^2 n}{2m_e \omega^2 \varepsilon_0 \varepsilon} \quad (2.24)$$

- 4) Linear model (active region):

$$\Delta\bar{n}(E) = [a_n^{(a)} + b_n^{(a)}(T - 300)](n - n_0) + [a_p^{(a)} + b_p^{(a)}(T - 300)](p - p_0) \quad (2.25)$$

- 5) Linear model (passive region):

$$\Delta\bar{n}(E) = [a_n^{(p)} + b_n^{(p)}(T - 300)](n - n_0) + [a_p^{(p)} + b_p^{(p)}(T - 300)](p - p_0) \quad (2.26)$$

where n_0 , p_0 and g_0 are electron concentration, hole concentration, and optical gain at equilibrium, respectively.

Please note that many of the models are specified for active regions only. A frequently asked question is that what if index change models available only for active region are also required for non-active regions? The answer is rather simple: just declare the non-active region as an active bulk region. It is possible to model more than one active region for the simulation program.

- **sample_mesh** affects interband part in 1). It is used to indicate whether or not all mesh points are sampled for the purpose of index change calculation. Turning on this option means a large amount of computation time is used for index calculation for all mesh points within the active region. If this option is turned off, only one calculation is performed based on carrier density averaged over the active region.
- **use_l.w.e.factor** indicates whether or not the user-defined line with enhancement factor (given by **l.w.e.factor** parameter) is used to compute the index change in part 2) above.
- **free-carrier** indicates whether index change induced by free carrier in part 3) is included.
free_carr_passive indicates whether index change induced by free carrier in part 3) is included for passive regions in the device.
- **interband** turns on part 1) above.
- **linear_active** turns on part 4) above.
- **linear_passive** turns on part 5) above.
- **eg_fac1** and **eg_fac2** are defined by calculating the index change due to injection current based on the (Kramers-Kronig) K-K relation which involves an integral over an energy range. These upper and lower limits (E_{01} , E_{02}) are defined by

$$E_{01} = \text{MAX}[\text{MIN}(E_g, E) - eg_fac1 E_{scat}, 0]$$

$$E_{02} = eg_fac2E_g$$

$$E_{scat} = \hbar/\tau_{scat}$$

where the E_g is the bandgap and E is the photon energy of interest.

- **l.w.e.factor** is the line width enhancement factor used to relate the change in the real part of the refractive indices to the imaginary part. This parameter is used only when **use_l.w.e.factor=yes**. The formula is given as part 2) above.
- **an_active** is $a_n^{(a)}$ in part 4) above.
- **bn_active** is $b_n^{(a)}$ in part 4) above.
- **ap_active** is $a_p^{(a)}$ in part 4) above.
- **bp_active** is $b_p^{(a)}$ in part 4) above.
- **an_passive** is $a_n^{(p)}$ in part 5) above.
- **bn_passive** is $b_n^{(p)}$ in part 5) above.
- **ap_passive** is $a_p^{(p)}$ in part 5) above.
- **bp_passive** is $b_p^{(p)}$ in part 5) above.
- **sample_point** is the number of sample points in spectrum used to evaluate the integrand in the K-K relation. The size of this parameter directly influences the speed of the integration and therefore the speed of the simulator. Please note that the K-K relation used to compute the index change due to carrier injection into the active region involves an integral over the energy range and is rather time consuming. Turning on this model slows down the simulator substantially.

Example(s)

```
index_model use_l.w.e.factor=yes l.w.e.factor=2
```

2.213 index_spectrum

parameter	data type	values [defaults]
spectrum_file	char	[void]
mater	intg [1]	

The statement is used to specify the material real and imaginary index spectrum to be used to calculate the photo-generation in the device. This statement is to be used with the statement of **light_power**.

- **spectrum_file** is used to specify the file name of the data file describing the index spectrum, or the wavelength dependent real and imaginary refractive indices. The required data format is as follows. The first line must be the number of data points of wavelengths (say for NPNT data points) in this spectrum. The following NPNT lines are the spectrum data points with each data point per line. The first number on each line must be the wavelength in micron meters, while the second and the third numbers must be the corresponding real and imaginary indices, respectively.
- **mater** is the material number of the material being described.

Example(s)

```
index_spectrum spectrum_file=si.data mater=1
```

2.214 index_wavel

index_wavel is the same as **gain_wavel** except it plots the index change.

2.215 ind_file_use

parameter	data type	values [defaults]
file_name1	char	[void]
file_name2	char	[void]
file_name3	char	[void]
file_name4	char	[void]
file_name5	char	[void]
file_name6	char	[void]
file_name7	char	[void]
file_name8	char	[void]
file_name9	char	[void]
mater_num1	intg	[-9999]
mater_num2	intg	[-9999]
mater_num3	intg	[-9999]
mater_num4	intg	[-9999]
mater_num5	intg	[-9999]
mater_num6	intg	[-9999]
mater_num7	intg	[-9999]
mater_num8	intg	[-9999]
mater_num9	intg	[-9999]

ind_file_use is used by the Multilayer Optical Analyzer (MOA) module. It is to overwrite APSYS-generated refractive index data with external refractive index wavelength de

- **file_name1** is a file name where wavelength refractive index dependency is presented.
- **mater_num1** makes the dependency from file with name **file_name1** corresponding to a certain material number. The same is applicable for the rest of the files and mater

Input files for wavelength refractive index dependency consist of three columns: wavelength (in micrometers), real part of refractive index, imaginary part of refractive index. As

0.4500000000000000	1.820000000000000	1.230000000000000E-004
0.5000000000000000	1.770000000000000	1.350000000000000E-004
0.5500000000000000	1.745000000000000	1.330000000000000E-004
0.6000000000000000	1.725000000000000	1.310000000000000E-004

Example

```
ind_file_use &&
  file_name1=tpd.rfi      mater_num1=1 &&
  file_name2=alq3_dcm.rfi mater_num2=2 &&
  file_name3=alq3.rfi     mater_num3=3
```

The above statement allows MOA to use specific user-defined index-wavelength dependencies of structural materials. The file **tpd.rfi** overwrites the index-wavelength dependenc

2.216 init_sol

parameter	data type	values [defaults]
sol_inf	char	
data_set	intg	

The statement **init_sol** initializes the main solver with a solution previously obtained.

- **sol_inf** is the main solution data. It should be set to the previous output data file name. In other words, it should be set to the same parameter as the **sol_outf** in the statement of **output** in the previous simulation.
- **data_set** specifies the data set number in **sol_inf**. Note that each data set corresponds to one **print_step** in the **scan** statement. Not all the bias points have the full structural data. Also see the **scan** statement for more detail on bias points and data_sets. This parameter is used by any subsequent plot statement that displays structural data such as electron concentration distribution.

Example(s)

```
init_sol sol_inf=inp15.out data_set=4
```

2.217 init_wave

parameter	data type	values [defaults]
modal_gain_method char	[wave_avarege]	
point_ll	realx2	(μm)
point_ur	realx2	(μm)
fld_center	realx2	(μm)
init_wavel	real	(μm)
spon_mode	real	[1.e-3]
length	real	[300.0] (μm)
backg_loss	real	[0.0] (1/m)
wavel_range	realx2	(μm) [0.5 1.8]
mirror_ref	real	[0.3]
prn_gain_range	real	[# #] (μm)
gain.sat	real	[0] (m^3)
photon_fac	real	[1.e7]
spon_out	real	[1.e-4]
front_back	realx2	[no-default]
delta_index	real	[0.3]
n_ubdata	realxn	
n_lbdata	realxn	
scale_rtg_var	real	[1e-12]
boundary_type	intgx4	[1 1 1 1]
ubdata_num	intg	[0]
lbdata_num	intg	[0]
prn_gain_num	intg	[50]

The statement **init_wave** initializes several parameters needed to solve the wave equation. Please note that geometric dependent parameters such as **point_ll** and **point_ur** can be overridden by corresponding parameters in the **wave_boundary** statement.

- **modal_gain_method** is used to specify the method used to compute the modal optical gain in laser/amplifier modeling. If “wave_average” is used, the modal gain is calculated as material gain averaged over the lateral model intensity distribution. If “imag_prop_const” is used, the modal gain is directly taken from imaginary part of the propagation constant of wave equation. The former (default) is more stable numerically while the latter is more accurate because the modal gain is by definition proportional to the imaginary part of the propagation constant.
- **point_ll** and **point_ur** are lower-left and upper-right points of the window within which the wave equation is solved (see also Fig. 2.14). This window

must be less than or equal to the full device size defined in the .geo input file. In some cases, this window must be reduced in size because only a limited number of lateral modes are needed. In gain guided devices, the lasing mode is not the highest index mode. Since PICS3D searches the optical mode starting from the highest mode index, there may be many non-lasing modes within the window. In the current version, PICS3D can only solve up to 10 modes. Since the number of non-lasing, high index modes increases with size of the window, the user should carefully adjust the window size for gain guided structures so that the lasing mode is included within the 10 modes.

- **fld_center** is the estimated optical mode center. Note that this information is only used to initialize the wave function and may or may not coincide with the optical field center in the final solution.
- **init_wavel** is the initial (or guess) emission wavelength corresponding to the peak modal gain. LASTIP starts the solution of the wave equation at this value. After finding the wave distribution using this wavelength, LASTIP calculates the modal gain spectrum within “wavel_range”. From this spectrum, LASTIP determines the peak gain wavelength, which will be the new wavelength for the wave solution at the next bias step. Therefore, “init_wavel” is used only once for the first step.
- **spont_mode** is the fraction of spontaneous emission coupled into the lasing mode inside the laser cavity. It appears in the rate equation and determines how spontaneous emission causes the lasing intensity to increase through amplification of spontaneous emission in a gain medium.
- **length** is the cavity length. This is needed to compute the mirror loss or emission loss.
- **backg_loss** is the background loss coefficient. By default, regions outside the active region are assumed to have a constant loss background (which can be determined experimentally). Alternatively one can use the material statement **absorption** to set the material loss in different materials. A non-zero **absorption** setting will override the **backg_loss** here.
- **wavel_range** is the wavelength range within which the peak emission wavelength is searched. PICS3D only solves the wave equation at a single wavelength corresponding to the peak modal gain. If the user has a structure which controls the longitudinal mode and fixes the emission wavelength at a known value, then the upper and lower limit of the wavelength range here should be set to the fixed wavelength. For example, if the DFB laser grating sets the emission wavelength to be $1.55\ \mu m$, one may set **wavel_range**=(1.55, 1.55).

- **mirror_ref** is the mirror reflectivity needed to calculate the mirror loss at a given cavity length. For a DFB laser, the **mirror_ref** should be set to an effective value that will result in an emission loss equivalent to all the grating losses.
- **prn_gain_range** is the wavelength range within which the gain spectral data are computed and printed at each set of structural data.
- **gain_sat** is the gain saturation (or gain suppression) coefficient ϵ given by the following two-level system formula: $g = g_0/(1 + \epsilon S)$, where g and g_0 are the optical gain with and without gain saturation, respectively, and S is the photon density of all possible lateral modes summed together.
- **photon_fac** is the factor used to divide the photon number in the rate equation so that the order of magnitude is the same as for the other equations. If the device is very large, the photon number may be very large in the laser cavity. In such a case, the solver has difficulty finding a converging solution and will show a large error for the rate equation (the 5th equation in the print out message). Increasing this parameter suppresses the error due to the rate equation which has a large photon number associated with it.
- **spont_out** is the fraction of spontaneous emission coupled into the output power. This is dependent on the device geometry and layer structure and how the output power is collected. This parameter is responsible for nonzero output light intensity below the lasing threshold. This term also explains how a light emitting diode (LED) works.
- **front_back** is used to specify the front and the back facet power reflectivity. This will override the **mirror_ref**. It can be used to specify two different mirror reflectivities. If only **mirror_ref** is used, both the front and back facets are assumed to be the same.
- **n_ubdata** and **n_lbdata** are window specifications. By default, the window for the wave equation is specified by "point_ll" and "point_ur" to be a rectangular window. Now it is possible to override the upper and lower borders with the parameters **n_ubdata**, **n_lbdata**, **ubdata_num** and **lbdata_num** where **n_ubdata** (for upper boundary data) is used to specify the upper boundary points (xi, yi) in the format

```
n_ubdata=(x1 y1 x2 y2 x3 y3 ...)
```

where the prefix **n_** should be equal to the number of data values in the bracket (or half of the points in 2D space). The related parameter for this statement, **ubdata_num**, must match the number of data value in **n_ubdata**.

```
init_wave ... &&
```

```
ubdata_num=8 8_ubdata=(0. 3.16 0.773 3.16 1.5 2.16 6. 2.16)
```

is used to describe a non-planar metal boundary for the wave equation. Similarly, **n_lbdata** and **lbdata_num** are used to describe the lower boundary.

- **scale_rtg_var** is used to scale the laser frequency variable in the coupled-RTG method. The laser frequency is converted into a unitless number of the order of unity via this parameter.
- **delta_index** is used to describe a boundary where the wave function decays exponentially by a function:

$$W(x) = W_0 \exp(-k_0 * \text{delta_index} * x)$$

where k_0 is the wave number and **delta_index** is estimated from the effective index (which may be unknown before the simulation):

$$\text{delta_index}^2 = \text{effective_index}^2 - \text{local_index}^2$$

- **boundary_type** is the boundary types on the sides of the window. The boundary type is represented by four integers in the order (lx1, lx2, ly1, ly2). The corresponding boundary is indicated in Fig. 2.14. The integers can only take on values of 1, 2, 3 or 5. Type 1 is the zero intensity boundary condition, *i.e.*, the wave intensity must be zero at the boundary. This type is used when the boundary is far enough away from the optical mode. In the case where the boundary is a symmetric axis (lx1 in Fig. 2.14), a zero intensity boundary condition produces an optical mode of odd symmetry. Type 2 is the symmetrical boundary condition, *i.e.*, the boundary is an axis of symmetry for the wave function. In other words, the wave function at point A is the same as at point B in Fig. 2.14. Using this type on boundary lx1 in Fig. 2.14 results in an optical mode of even symmetry. In a one-dimensional simulation with semiconductor layers parallel to the x-direction, the optical mode has translational symmetry in the x-direction. Therefore, both lx1 and lx2 boundaries should be type 2 while ly1 and ly2 should be type 1. Type 3 is used to represent a truncated wave function that decays exponentially. This type of boundary may be useful in cases where the user wishes to save some mesh points by cutting a large portion of the device area out of the wave solution window. In such case, we may assume the wave decays exponentially at the boundary. This choice works together with the parameter **delta_index**. Please see parameter **delta_index**. Type 5 represents a boundary of perfectly matched layer (PML).
- **prn_gain_num** is the number of wavelength data points within **wavel_range** that are used to compute the optical modal gain spectrum at each data sets [LASTIP].

- **prn.gain_num** is the number of wavelength data points within **wavel_range** that are used to compute the optical modal gain spectrum at each bias points for **scan3d** [PICS3D].

Example(s)

For a 1-D structure:

```
init_wave point_ll=(0., 0.0) point_ur=(1.5, 3.08) &&
  fld_center=(0.5, 1.5) length=300 backg_loss=1000. &&
  boundary_type=(2 2 1 1 ) init_wavel=1.3 mirror_ref=0.32 &&
  wavel_range=(1.2, 1.4)
```

For a 2-D structure with lx1 boundary as the symmetry axis:

```
init_wave point_ll=(0., 0.0) point_ur=(3.5, 3.0) &&
  fld_center=(0.5, 1.5) length=300 backg_loss=1000. &&
  boundary_type=(2 1 1 1 ) init_wavel=1.3 mirror_ref=0.32 &&
  wavel_range=(1.2, 1.4)
```

For a 2-D structure without any symmetry axis:

```
init_wave point_ll=(0., 0.0) point_ur=(7.0, 3.0) &&
  fld_center=(3.5, 1.5) length=300 backg_loss=1000. &&
  boundary_type=(1 1 1 1 ) init_wavel=1.3 mirror_ref=0.32 &&
  wavel_range=(1.2, 1.4)
```

2.218 inner_bar_gain

The statement **inner_bar_gain** is used to distribute optical gain in an inner-barrier layer of a complex MQW system. Usually, optical gain is only calculated in quantum well layers.

This statement has no parameters.

2.219 integer_func

The command is identical to **loop_integer**.

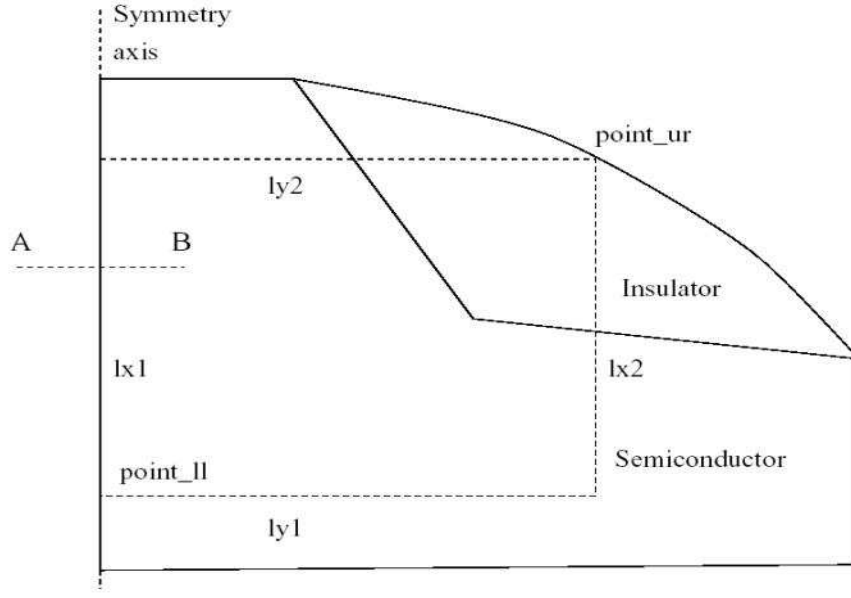


Figure 2.14: Schematics of the boundary condition for the wave equation.

2.220 interface

parameter	data type	values [defaults]
model	char	[recomb], charge, trap
trap_type	char	[donor], acceptor
x_label	char	[void]
y_label	char	[void]
within_x1_label	char	[void]
within_x2_label	char	[void]
within_y1_label	char	[void]
within_y2_label	char	[void]
within_x	realx2	[-1.e5 1.e5] (um)
within_y	realx2	[-1.e5 1.e5] (um)
velocity_n	real	[1.e6] (m/s)
velocity_p	real	[1.e6] (m/s)
fix_charge	real	[1.e14] ($1/m^2$)
trap_density	real	[1.e16] ($1/m^2$)
trap_level	real	[0.5] (eV)
trap_life_n	real	[1.e-7] (s)
trap_life_p	real	[1.e-7] (eV)
x	real	(um)
y	real	(um)
surftrap_num	intg	[1]

interface statement is used to specify semiconductor interface states and the physical models to be used by the simulation software. The interface may be between semiconductor and vacuum or between different types of semiconductors. This statement may be used more than once to specify more than one interface.

- **model** indicates the type of model to be used to describe the physics of the interface. If it takes **recomb**, surface recombination model is used. The simulator will use the parameters of **velocity_n** , **velocity_p** . If it takes **charge**, interface fixed charge is described. The simulator will use the parameters of **fix_charge** . If it takes **trap**, surface traps are described. The simulator will use the parameters of **trap_type** , **trap_density** , **trap_level** , **trap_life_n** , **trap_life_p** , **surftrap_num** .
- **trap_type** takes donor or acceptor. It is the charge type of the deep surface traps.
- **within_x** is used to specify the range in x direction within which the interface is located.
- **within_y** is used to specify the range in y direction within which the interface is located.
- **velocity_n** is the surface recombination velocity for electrons.
- **velocity_p** is the surface recombination velocity for holes.
- **fix_charge** is the fixed charge surface density. It can be negative or positive.
- **trap_density** is the surface trap density.
- **trap_level** is the trap level measured from the conduction band.
- **trap_life_n** is the trap life time of minority electrons.
- **trap_life_p** is the trap life time of minority holes.
- **x** , if specified, all mesh points with such x-coordinate value will be regarded as the interface points.
- **y** , if specified, all mesh points with such y-coordinate value will be regarded as the interface points.
- **surftrap_num** is the surface trap number or identifier. The user may specify different trap number to declared surface states with more than one type of traps.

- **x_label**, **y_label** use pre-defined labels to specify **x** and **y** parameters above, respectively. The labels must be predefined using **x_position**, **y_position** or **define_vertical_position** command. They may be predefined in the .layer file relative to a specific layer so that there is no need to find out the absolute coordinates.
- **within_x1_label**, **within_x2_label**, **within_y1_label**, **within_y2_label** use pre-defined labels to specify **within_x** and **within_y** parameters above, respectively. The labels must be predefined using **x_position**, **y_position** or **define_vertical_position** command. They may be predefined in the .layer file relative to a specific layer so that there is no need to find out the absolute coordinates.

Example(s)

```
$
$ For a surface between 0.5 to 2 um, we define a Fermi level
$ pinning acceptor mid-gap trap
interface model=trap trap_type=acceptor &&
  within_x=(0.5 2.) within_y=(0.4 0.6) &&
  trap_density=1.e16 trap_level=0.8 &&
  trap_life_n=1.e-7 trap_life_p=1.e-7 surftrap_num=1
$
$ We can use a surface recombination model here:
interface model=recomb &&
  within_x=(0.5 2.) within_y=(0.4 0.6) &&
  velocity_n=1.e6 velocity_p=1.e6
$
```

2.221 integer_func

The command is identical to **loop_integer**.

2.222 internal_xpoint

parameter	data type	values [defaults]
all_interfaces	char	yes
version2002	char	no
material_label1	char	void
material_label2	char	void
xp_size	real	[0.0001] (um)
interface_mater	intgx2	

The statement is used to control the internal extra point (see also **put_mesh** statement) which defines the sharpness of the heterojunctions. It may be used in both the .geo and .layer input files. Please note that this statement may appear more than once to define extra points for many interfaces.

- **all_interfaces** flags whether all material interfaces are affected by this statement.
- **version2002** indicate if the mesh file should be generated in older version format. Please also note that the rules for defining the mesh points near the heterojunction are slightly different for newer versions.
- **material_label1** and **material_label2** are used by .layer only. If used, it indicates that this statement is valid for the material interfaces between these two materials. The labels must be previously defined in the **layer_mater** statement in .layer file.
- **xp_size** is the internal extra point used to define the sharpness of the heterojunction. Material properties are different across the two mesh points separated by a distance of this parameter.
- **interface_mater** is used by .geo only. If used, it indicates that this statement is valid for the material interfaces between these two materials. Material numbers must be previously defined in the **polygon** statement in .geo file.

Example 1 for .geo and .layer

The following statement is used to set all heterojunction interface sharpness to be 1 Angstrom.

```
internal_xpoint all_interfaces=yes xp_size=0.0001
```

Example 2 for .geo

The following two statements set all heterojunction interface sharpness to be 1 Angstrom except for the interface between materials 3 and 4 which is set to 100 Angstroms. Materials 3 and 4 should have already been defined by previous statements of **polygon**.

```
internal_xpoint all_interfaces=yes xp_size=0.0001
internal_xpoint all_interfaces=no xp_size=0.01 interface_mater=[3 4]
```

Example 3 for .layer

The following two statements set all heterojunction interface sharpness to be 1 Angstrom except for the interface between materials labeled as “layer_a” and “layer_b” which is set to 100 Angstroms. Materials “layer_a” and “layer_b” should have already been defined by previous statements of **layer_mater** in the .layer file.

```
internal_xpoint all_interfaces=yes xp_size=0.0001
internal_xpoint all_interfaces=no xp_size=0.01 &&
    material_label1=layer_a material_label2=layer_b
```

2.223 intersubband_model

2.224 qdot_material

parameter	data type	values [defaults]
search_subband	char	[no]
ref_energy	real	[-9999.]
tau_broaden	real	[5.e-13]
lower_level	intg	[-9999]

intersubband_model is used to activate the intersubband optical transition model. The simulation software defines a reference energy level and computes optical transitions between levels above and below such a reference.

- search_subband if set to yes, instructs the program to compute all possible transitions to search a the reference energy mentioned above so that maximum optical gain can be obtained.
- ref_energy is the reference energy level mentioned above.
- tau_broaden is used to broaden the optical transition peaks.
- lower_level is highest of lower levels below the reference energy. Either this parameter or the **ref_energy** should be defined, not both.

Example(s)

```
intersubband_model search_subband=yes &&
    tau_broaden=7.e-13
```

The above statement instructs the program to search the ref. energy to obtain maximum optical gain.

2.225 jdos_energy

parameter	data type	values [defaults]
data_file	char	
energy_range	realx2	
data_point	intg	[50]

jdoss_energy plots the joint density of states (JDOS) versus energy. This statement is only used in the gain preview module.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **energy_range** is the range of energy in eV.
- **data_point** is number of data plots to be plotted.

Example(s)

```
jdoss_energy  energy_range=(1 1.2)
```

2.226 kane_para_f_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **kane_para_f_bar** is an active layer macro statement used to define the second order correction due to remote bands to the dipole moments. It affects the value for a quantum barrier. This is the Kane parameter F as in Eq. (2.5) in following Ref:

”Band parameters for III-V compound semiconductors and their alloys,” I. Vurgaftman and J. R. Meyer, L. R. Ram-Mohan, J. Appl. Phys. Vol. 89, No. 11, 1 June 2001, p. 5815

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.227 kane_para_f_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **kane_para_f_well** is an active layer macro statement used to define the second order correction due to remote bands to the dipole moments. It

affects the value for a quantum well. This is the Kane parameter F as in Eq. (2.5) in following Ref:

"Band parameters for III-V compound semiconductors and their alloys," I. Vurgaftman and J. R. Meyer, L. R. Ram-Mohan, J. Appl. Phys. Vol. 89, No. 11, 1 June 2001, p. 5815

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.228 lateral_mode3d

parameter	data type	values [defaults]
sort_modes	char	[yes],no
mode_num	intg	[1]

lateral_mode3d is used to enable the multi-lateral mode capability.

- **sort_modes** if "yes", the lateral mode with highest emission power will always be assigned as mode number one. If "no", the lateral mode order in the 3D model will be the same as that in 2D wave equation solutions.
- **mode_num** is the total number of lateral modes to be simulated.

Example(s)

```
lateral_mode3d sort_modes=no mode_num=2
```

2.229 lattice_bar

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the lattice constant of the barrier in Å. This statement is applicable for wurtzite structure active region.

2.230 lattice_base

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the lattice constant of the whole device using which the strain in each layer is to be computed. This statement is applicable for wurtzite structure bulk layer [1].
 [1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.231 lattice_bulk

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the lattice constant of a bulk layer using which the strain in each layer is to be computed. This statement is applicable for wurtzite structure bulk region [1].
 [1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.232 lattice_constant

parameter	data type	values [defaults]
(see) material_par		

The material statement **lattice_constant** is an active layer macro statement used to define the lattice constant of the base substrate material. The strain is measured against this parameter. The unit is Angstrom.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.233 lattice_well

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the lattice constant of the well in A. This statement is applicable for wurtzite structure active region.

2.234 lax_mass_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **lax_mass_bar** is an active layer macro statement used to define the anisotropic effective mass of the conduction band for silicon quantum MOS.

The band structure model has a ellipsoidal shaped constant-energy surface of cylindrical symmetry around a symmetry axes. For example if z-axis is along the symmetry axes of the ellipsoid, the energy dispersion becomes:

$$E(k_x, k_y, k_z) = E_0 + \frac{\hbar^2}{2m_0} \left(\frac{k_x^2}{lax_mass} + \frac{k_z^2}{lax_mass} + \frac{k_y^2}{lax_mass} \right)$$

lax_mass_bar is the mass along the symmetry axes in the quantum barrier region. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.235 lax_mass_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **lax_mass_well** is an active layer macro statement used to define the anisotropic effective mass of the conduction band for silicon quantum MOS.

The band structure model has a ellipsoidal shaped constant-energy surface of cylindrical symmetry around a symmetry axes. For example if z-axes is along the symmetry axes of the ellipsoid, the energy dispersion becomes:

$$E(k_x, k_y, k_z) = E_0 + \frac{\hbar^2}{2m_0} \left(\frac{k_z^2}{lax_mass} + \frac{k_x^2}{tax_mass} + \frac{k_y^2}{tax_mass} \right)$$

lax_mass_well is the mass along the symmetry axes in the quantum well region. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.236 layer

parameter	data type	values [defaults]
vcsel_type	char	void
d	real	[0.5]
r	real	[1.]
angle	real	[0.]
wj (j=1,2,...,9)	real	
lower_wj	real	
upper_wj	real	
n_dopingj	real	
p_dopingj	real	
gaussian_tail	real	[0.001] (μm)
shift_center	real	[0.] (μm)
grading_fromj	real	
grading_anglej	real	[90.](degree)
n	intg	[5]
mj	intg	[1]

layer defines the layers of material and the mesh allocation associated with each. Please note that this statement can only be used to define a material with either n-type or p-type of doping, not both. To define a material with both types (that is, compensating doping), the statement **layer_mater** should be used.

- **vcsel_type** is used to label the present layer as part of a VCSEL longitudinal section. Please note that this is only a label to be defined by the user. It works together with the same parameter in the **vcsel_section** statement.
- **d** is the thickness of the layer.

- **r** is the ratio which determines mesh spacing characteristics. See “ratio” under the statement “plot_mesh” for more details.
- **angle** is used in conjunction with **upper_w#** and **lower_w#** to specify the shape of the polygon formed by a column and layer. It is defined as the angle between the y-axis (left side of the column) and the edge of the material (left side of the polygon), in degrees.
- **lower_w#** gives the width of the lower edge of a polygon in this layer, where # is the identification number of its column.
- **upper_w#** gives the width of the upper edge of a polygon in this layer, where # is the identification number of its column.
- **n_doping#** defines the n-type doping in units of $1/m^3$ for the present layer. The suffix number is used to indicate the column number of the doping. The result will be to generate the doping profile usable for the .sol input file. The .doping file can be included in the .sol file later.
- **p_doping#** defines the p-type doping in units of $1/m^3$ for the present layer. The suffix number is used to indicate the column number of the doping. The result will be to generate the doping profile usable for the .sol input file. The .doping file can be included in the .sol file later.
- **gaussian_tail** is the Gaussian tail (standard deviation) of the doping profile defined by **n_doping#** and **p_doping#**.
- **shift_center** is used in the case of negative mesh **ratio** (see 2.16). The negative ratio will cause the mesh distribution to vary using the center point of the distance as a symmetric point. This parameter causes the center to be shifted in either direction.
- **grading_fromj** for the jth column, if used, is the relative starting value of linearly graded doping. Its value is relative to the value of the maximum doping concentration. The variation direction of the linear grading is that of **grading_anglej**.
- **grading_anglej** for the jth column, is the variation direction of the linearly graded doping. The reference direction is along +x direction.
- **n** is the number of mesh lines to be placed in this layer.
- **m#** is the material in a polygon formed by the layer and a column, where # is the identification number of the column.

Example(s)

```
layer d=0.29  n=5  r=1.
```

For a layer with non-rectangular polygons in columns 2 and 3,

```
layer d=1.2  n=8  r=1.  &&
  upper_w2=0.8  upper_w3=1.2
```

For a layer with n_doping in column 1 and p_doping in column 2,

```
layer d=1.2  n=6  r=1.  &&
  n_doping1=1.e24  p_doping2=5.e23
```

2.237 layer_conf

parameter	data type	values [defaults]
layer_xrange	realx2	[0. 5.] (μm)
layer_yrange	realx2	[2. 2.1] (μm)
mode_index	intg	[1]

layer_conf is used as a post-processing statement to calculate the optical confinement factor for a rectangular region (not necessarily the active region).

- **layer_xrange** and **layer_yrange** are used to specify the rectangular cross section on the xy-plane (lateral) for which the optical confinement factor is to be calculated.
- **mode_index** is the lateral mode index. Mode 1 is the fundamental mode; mode 2 is the 2nd order mode, etc..

Example:

```
layer_conf layer_xrange=(0. 1.5) layer_yrange(1.7 1.85)
```

2.238 layer_mater

parameter	data type	values [defaults]
macro_name	char	[void]
active_macro	char	[void]
solve_wave	char	[yes] (no)
material_label	char	[void]
var_symbolk(k=1..9)	char	
avar_symbolk(k=1..9)	char	
vari(i=1..9)	real	
grade_from	real	
grade_to	real	
avari(i=1..9)	real	
n_doping	real	
p_doping	real	
agrade_from	real	
agrade_to	real	
bgrade_from	real	
bgrade_to	real	
column_num	intg	[1]
grade_var	intg	[0]
embedded_structure	intg	[0]
n_newdoping_index	intg	
p_newdoping_index	intg	
agrade_var	intg	[0]
bgrade_var	intg	[0]

layer_mater is used to work with the LAYER program so that material definition of uniform or graded material can be used in .layer file. It may be used together with the **layer** statement. **layer_mater** is used to define the material or doping while **layer** is used to define the layer thickness. This statement is used to define a column of a particular layer (a polygon) while the **layer** statement is used to define the layer property.

Use of this statement produces material definition in .mater file which can be included in the .sol file using **include** statement. Without this statement, it would be a pain to define graded material within a complicate layer structure because of the need to know the coordinates of the starting and ending points of the material grading.

Please note that material number **m1**, **m2**,... in **layer** should **NOT** be used if **layer_mater** has been used here to define the material already.

- **macro_name** is the bulk material macro name being used. The detail regarding the macros can be found in file SOFTWARE/bin/crosslight.mac or

in the User's Manual. If **macro_name=void**, the polygon defined by this statement will not appear in the .geo file and the mesh generated. Therefore, **macro_name=void** can be used to define a structure whose layer numbers are different for different columns.

- **active_macro** is the name of the active layer macro. The detail can be found in statement **get_active_layer**. This parameter should be used if the material is active layer or a quantum well.
- **solve_wave** is used to indicate whether or not this layer is included inside of boundary of lateral mode solver as in statement **wave_boundary** or **init_wave**.
- **material_label** is used to label or identify the material defined by this statement. The label may later be used by other statements (such as **internal_xpoint**) to refer to a specific material.
- **var_symbolk**, $k=1, \dots, 9$, are the symbolic variable names used as function arguments in the bulk material macro. If defined by user, they must exactly match the symbols in the macro function definitions. If they are not defined by user, the simulator assumes the macro is an older style macro with the same function arguments appearing in all functions.
- **avar_symbolk**, $k=1, \dots, 9$, are the symbolic variable names used as function arguments in the active layer material macro. If defined by user, they must exactly match the symbols in the macro function definitions. If they are not defined by user, the simulator assumes the macro is an older style macro with the same function arguments appearing in all functions.
- **vari**, ($i=1, 2, \dots, 9$) is the i th variable value used for the bulk material macro defined by **macro_name**. Usually, these are material compositions and/or temperature. The use of these variables can be found in the macro library.
- **grade_from** is the composition of the starting point for the material grading. The starting point is at the lower part of a layer.
- **grade_to** is the composition of the ending point for the material grading. The starting point is at the upper part of a layer.
- **agrade_from** is the composition of the starting point for the active layer material grading. The starting point is at the lower part of a layer.
- **agrade_to** is the composition of the ending point for the active layer material grading. The starting point is at the upper part of a layer.

- **bgrade_from** is the composition of the starting point for the active layer barrier material grading. The starting point is on the edge of the quantum well goes away from the quantum well.
- **bgrade_to** is the composition of the ending point for the active layer barrier material grading. The starting point is on the edge of the quantum well goes away from the quantum well.
- **avari**, ($i=1,2,\dots,9$) is the i th variable value used for the active layer macro defined by **active_macro**. Usually, these are material compositions and/or temperature. The use of these variables can be found in the macro library.
- **n_doping** and **p_doping** are the doping concentration in this layer/column polygon. They may conflict with doping definition in the **layer** statement. So we should define doping in either this statement or in **layer**, not both.
- **column_num** is the column number of the present statement.
- **grade_var** is used to indicate which variable is being graded. If set to zero, no grading is used. If set to i , then **vari** is being linearly graded from **grade_from** to **grade_to**. In such a case, any setting of **vari** is to be ignored.
- **agrade_var** is used to indicate which variable in the active layer macro is being graded. If set to zero, no grading is used. If set to i , then **avari** is being linearly graded from **agrade_from** to **agrade_to**. In such a case, any setting of **avari** is to be ignored.
- **bgrade_var** is used to indicate which variable in the active layer macro is being graded. This is used only when more than one active layer macro compositions need to be graded and **agrade_var** has already been set. If set to zero, no grading is used. If set to i , then **avari** is being linearly graded from **bgrade_from** to **bgrade_to**. In such a case, any setting of **avari** is to be ignored.
- **embedded_structure** takes types 1 and 2. If it is set to zero, no embedded structures are present for the present layer/region. It is used to define an active material embedded inside a regular active layer. For type 1, it is used to define a corrugation grating in a DFB/DBR waveguide. A corrugation structure is regarded as two different kinds of active material embedded within an active layer material which has a material property equal to the overall average of the grating layer. The two embedded layers are considered active because interband model is used to calculate refractive index change) The 2nd type of embedded structure involves a quantum dot (QDOT) embedded within the wetting layer (regular quantum well). For quantum dot material, material style "cx-" is required. The embedded layers used to be called "mildly active layers" because most of properties of an active layer are included.

- **n_newdoping_index** labels the n-doping of this material as “new_doping” so that its concentration can be changed in a simulation.
- **p_newdoping_index** labels the p-doping of this material as “new_doping” so that its concentration can be changed in a simulation.

Example(s)

```
layer_mater macro_name=algaas vari=0.71 column_num=1
layer d=1.35 n=9 r=0.8 n_doping1=1.e24
```

The above two statements are used to define a layer of AlGaAs (Al=0.71) at column 1 with thickness $1.35 \mu\text{m}$ and n-type doping of $1 \times 10^{24} \text{ 1/m}^3$.

```
layer_mater macro_name=algaas grade_var=1 &&
  grade_from=0.71 grade_to=0.33 column_num=1
layer d=0.1462 n=6 r=0.8 xp2=1
```

The above two statements are used to define a graded AlGaAs with Al varying from 0.71 to 0.33 over a thickness of $0.1462 \mu\text{m}$.

2.239 layer_type

parameter	data type	values [defaults]
type	char	(see list)
doped_organic	char	[no]
ox_use_extern_spec	char	[yes]
ox_use_exciton_diff	char	[yes]
dox_triplet	char	[no]
dox($k_L=2$)_triplet	char	[no]
valley_gamma	intg	[1]
valley_l	intg	[4]
valley_hh	intg	[1]
valley_lh	intg	[1]
valley_ch	intg	[1]
mater	intg	[1]
ox_max_phonon	intg	[5]
ox_phonon_cloud	intg	[4]
ox_level_req	intg	[80]
dox_max_phonon	intg	[5]
dox_phonon_cloud	intg	[4]
dox_level_req	intg	[80]

layer_type is used by the active layer macro to denote some general properties of an active layer.

- **type** is the type of active layer and takes one of `bulk`, `unstrained_well`, `strained_well`, `complex`, `strained_complex`, `wurtzite_well` and `wurtzite_complex`. `wurtzite_well` includes strained wells. Complex wells mean coupled wells and / or irregular wells.
- **doped_organic** is to define doped organic material.
- **ox_use_extern_spec** indicate whether external (experimental) EL spectrum is used instead of using Crosslight's internal EL model.
- **ox_use_exciton_diff** indicate whether exciton diffusion model parameters are to be used for the present macro.
- **dox_triplet** is to define a dopant to harvest triplet.
- **doxk(k_i=2)_triplet** is to define the second or higher index of dopant to harvest triplet.
- **valley_gamma** is the number of band valley of the Gamma conduction band.
- **valley_l** is the number of band valley of the L conduction band.
- **valley_hh** is the number of band valley of the HH conduction band.
- **valley_lh** is the number of band valley of the LH conduction band.
- **valley_ch** is the number of band valley of the CH conduction band. **mater** is the internal material number.
- **ox_max_phonon** is the maximum phonon occupation number at a lattice site in the organic crystal.
- **ox_phonon_cloud** is the number of neighboring lattice sites to be considered for the construction of a phonon cloud. A phonon cloud is a set of localized excited lattice vibration states to interact with an exciton state. In the Frenkel exciton theory, a larger phonon cloud tends to give more accurate description of a realistic exciton state.
- **ox_level_req** is the number of energy levels requested when solving the exciton-phonon Hamiltonian.
- **dox_max_phonon** is the maximum phonon occupation number at a dopant lattice site in the organic crystal.

- **dox_phonon_cloud** is similar to **ox_phonon_cloud** except it is for the dopant material.
- **dox_level_req** is similar to **dox_level_req** except it is for the dopant material.

Example(s)

```
layer_type type=strained_well valley_gamma=1 valley_l=4 &&
valley_hh=1 valley_lh=1
```

2.240 lband_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **lband_bar** is an active layer macro statement used to define the L-band energy with respect to the Gamma band (eV) in the quantum barrier.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.241 lband_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **lband_well** is an active layer macro statement used to define the L-band energy with respect to the Gamma band (eV) in the quantum well.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.242 led_control

parameter	data type	values [defaults]
efficiency_model	char	[void] xy, y uniform
import_rto	char	[no] yes
spectrum_all_bias	char	yes [no]
spon_all_points	char	[no]
use_intern_refl	char	[no]
efficiency	real	[1.e-1]
wavelength	real	(μm)
delta_wavel	real	[0.1]
refl_x1	real	[0.3]
refl_x2	real	[0.3]
refl_y1	real	[0.3]
refl_y2	real	[0.3]
group_index	real	[3.8]
led_xrange	realx2	(μm)
led_yrange	realx2	(μm)
spectrum_num	intg	[35]
x_segment	intg	[30]

The statement **led_control** is used to control the light emitting diode modeling with the assumption of simple cubic waveguide structure. Such a simple structure will allow some reasonable estimate of light power emission given the basic material and current injection conditions. Please note that this statement works with the **init_wave** to defined a Fabry-Perot cavity.

For a more realistic power extraction model for devices different than the idea cubic waveguide structure, the 2/3D ray-tracing model is recommended.

- **efficiency_model** is used to compute power extraction efficiency. As we have described in the User's Manual, the power emission in a particular direction depends on the available optical mode density distribution in the directions perpendicular to the direction of emission. Thus, the power emission depends on if the device is designed to confine optical modes in a certain direction. If on confinement of modes are possible for the device, the "uniform" model is recommended.

If this parameter takes "void", the external emission efficiency is not computed and is set to the value of the "efficiency" parameter of this statement.

If efficiency_model=xy, the software assumes a waveguide structure with well confined lateral / transverse modes as if we are treating an edge type of semiconductor laser. For emission in the z-direction, the lateral modes are computed in the same way as for a semiconductor laser (i.e., in combination with **multimode**

statement). For emission in the x-direction we assume 1D mode confinement in y-direction. For emission in the y-direction, no mode confinement is assumed. If `efficiency_model=y`, the software assumes that there is 1D single mode confinement in y-direction for emission in both z- and z-directions. Again, no mode confinement is assume for emission in y-direction.

Finally, if the LED device is not designed to confine modes in any direction, we set `efficiency_model=uniform`. Only plan waves are considered when extraction efficiency is computed for this model.

- **import_rto** is used to indicate whether ray-tracing data file (`.rto_###` file) is to be imported to define light distribution within the device. If positive, the simulator will detect the existence of the `.rto` file first. If the file exists, the program will import the ray-tracing data. If the data file does not exist, the simulator will use the default profile. The `.rto` data file number is automatically determined by the bias in the same way as the `.out` file. This parameter is useful for studying of an LED device when ray-tracing data is generated in a previous run of the simulator. The simulator may be run repeatedly, each time using the previous `.rto` data as input light profiles while new ray-tracing data are produced at the post-processing stage (using `.plt`). The results (also the ray-tracing profile) are expected to become stable after several runs and we may regard them as being final and self-consistent ones.
- **spont_all_points** indicates whether all mesh points are used to evaluate the spontaneous emission spectrum. Computation time is longer if all points are used.

use_intern_refl is used to turn on the internal reflection model for the 1-dimensional (or broad-area) emission calculation. Using this model results in an extraction efficiency lower than realistic values while not using it tends to overestimate the power extraction. Due to advanced power extraction geometrical design for modern LED's, the measured power extraction is closer to the broad-area emission model without internal reflection. Versions of APSYS before to Sept. 2004 used the internal reflection model as default.
- **spectrum_all_bias** indicates whether spontaneous emission spectrum is computed at all biases or only at data set print points.
- **efficiency** is the user-defined external emission efficiency if `efficiency_model=void`. This allows the emission power to be calculated even with the lack of an efficiency model.
- **wavelength** is the estimated peak emission wavelength in microns.
- **delta_wavel** is the wavelength range to cover the whole emission spectrum. The total emission power is the integration of the spectrum over this wavelength range.

- **refl_x1** is the power reflectivity of the LED cavity on the left side in the x-direction.
- **refl_x2** is the power reflectivity of the LED cavity on the right side in the x-direction.
- **refl_y1** is the power reflectivity of the LED cavity on the bottom side in the y-direction.
- **refl_y2** is the power reflectivity of the LED cavity on the top side in the y-direction.
- **group_index** is the group index of the traveling waves of the LED.
- **led_xrange** is the range in the x-direction that we consider to be a Fabry-Perot cavity. If not specified, the program will search the device geometry and decide the range.
- **led_yrange** is the range in the y-direction that we consider to be a Fabry-Perot cavity. If not specified, the program will search the device geometry and decide the range.
- **spectrum_num** is the number of points used to evaluate the spontaneous emission spectrum.
- **x_segment** is the number of segments in the x-direction that the program is to sample the material properties and carrier densities for emission in the x and y directions.

Example(s):

```
led_control wavelength=0.437 efficiency_model=uniform &&
  refl_y1=0.8 refl_y2=0.4 delta_wavel=0.03 &&
  led_xrange=(0 400) group_index=2.9
```

2.243 led_eff_distr

parameter	data type	values [defaults]
data_file	char	
side	char	[top]bottom

led_eff_distr plots the LED external efficiency distribution along the x-direction.

- **data_file** is the file to which the graphic data is written in ASCII format.

- **side** indicates top or bottom side the efficiency is to be plotted.

Example(s)

```
led_eff_distr side=top
```

2.244 led_farfield

parameter	data type	values [defaults]
data_file	char	
side	char	[top]bottom

led_eff_distr plots the relative far field intensity distribution.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **side** indicates top or bottom side the efficiency is to be plotted.

Example(s)

```
led_farfield side=top
```

2.245 led_spectrum

parameter	data type	values [defaults]
data_file	char	
total_amount	char	[yes] no
scale	real	[1.]
scale_wavelength	real	[1.]

led_spectrum is a post-processing statement that plots the LED spontaneous emission spectrum.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **total_amount** indicates whether the total amount (integrated amount) be plotted.
- **scale** is the scale of the vertical axis.
- **scale_wavelength** is the scale of the horizontal axis.

2.246 led_simple

The statement **led_simple** is a simplified version of **led_control**. All its parameters are explained in the latter statement.

This simplified LED parameter setting command defines a number of critical parameters so that some important properties such as internal efficiency and ray-tracing extraction efficiency can be calculated.

Analytical model of 1D power extraction is not included in this statement.

2.247 led_top_coating

parameter	data type	values [defaults]
number_of_layers	intg	[1]
real_index_led	real	[3.5]
real_index_out	real	[1.5]
thicknessj	real	[0.1] (μm)
real_indexj	real	[2.2]
imag_indexj	real	[0.]

led_top_coating specifies an optical coating on the top of LED which is usually applied for better light extraction. The coating options allow to set up **number_of_layers** (up to 9) layers of different thickness and refractive index. It is assumed the LED material has the refractive index **real_index_led** and for the outer medium the refractive index is **real_index_out**. The optical interference effect due to the optical coating can be computed by the simulation program. The layers are numbered from the LED to the outer medium, i.e., the layer with number 1 is the closest to the LED.

Note that use of the **led_top_coating** statement overrides the reflection coefficient **refl_y2** in the **led_control** statement.

- **number_of_layers** is an actual number of layers in the coating. The coating can contain up to 9 different layers.
- **real_index_led** is the refractive index (which is assumed to be real) of the LED material.
- **real_index_out** is the refractive index (which is assumed to be real) of the outer medium.
- **thicknessj** is the thickness of the j -th layer from optical coating.
- **real_indexj** is the real part of the refractive index of the j -th layer in the optical coating.

- **imag_indexj** is the imaginary part of the refractive index of the j -th layer in the optical coating.

Example(s)

```
led_top_coating number_of_layers=2 real_index_led=3.5 real_index_out=1.5 &&
thickness1=0.15 real_index1=2.05 imag_index1=0. thickness2=0.14 &&
real_index2=2.25 imag_index2=0.002
```

2.248 left_contact

parameter	data type	values [defaults]
contact_type	char	[ohmic] schottky
use_schottky_xp	char	[yes] no
from	real	
to	real	
contact_num	intg	
barrier	real	(volt)
work_function	real	(volt)
intern_xp_size	real	[0.0002] (um)

left_contact defines the placement of the left contact in the “.layer” geometry file.

- **contact_type** indicates the type of contact and can take either **ohmic** or **schottky**.
- **use_schottky_xp** is used to control the use of extra point near the contact in case the contact is of Schottky type.
- **to** and **from** set the x-range where the contact lies. Note that the origin is at the upper left-hand corner of each polygon and that units are in μm .
- **contact_num** defines the index number of the contact.
- **barrier** is the barrier height of the Schottky contact.
- **work_function** is the work function of the Schottky contact.
- **intern_xp_size** is the internal extra point size around a Schottky contact if parameter **use_schottky_xp** is set. It will not be used if the contact type is not Schottky.

Example(s)

```
left_contact from=0 to=0.8 contact_num=1
```

2.249 lifetime_model

parameter	data type	values [defaults]
carr_dep_life	char	[no]
current_dep_life	char	[yes]
ref_elec_density	real	[1.e23](1/m**3)
ref_hole_density	real	[1.e23](1/m**3)
ref_elec_current	real	[1.e5](amp/m**2)
ref_hole_current	real	[1.e5](amp/m**2)

lifetime_model is used to modify the lifetime behavior of the carriers using the following formulas. carr_dep_life=yes The lifetime goes like

$$\tau_n = \tau_{n0} / [1 + (n + p) / \tau_{nref}] \quad (2.27)$$

If current_dep_life=yes,

$$\tau_n = \tau_{n0} / [1 + J / \tau_{nref}] \quad (2.28)$$

where n and p are electron and hole densities, respectively. J is the total current density. Similar equations hold for holes.

The meanings of the parameters should be clear when comparing with the above formulas.

Example:

```
lifetime_model current_dep_life=yes ref_elec_current=1.e5 ref_hole_current=1.e5
```

would define a current dependent lifetime for both electrons and holes.

2.250 lifetime_n

parameter	data type	values [defaults]
(see) material_par		

The material statements **lifetime_n** and **lifetime_p** define the minority life time (in seconds) for carriers (**_n** for electrons and **_p** for holes) in the SRH recombination model. The basic mechanism is electron or hole capture by deep level traps. Therefore this statement is related to other trap statements. The relation between lifetime and other trap quantities are written as /vskip 0.1in

$$1/(lifetime) =$$

$$(\text{trap density}) \times (\text{thermal velocity}) \times (\text{capture cross section})$$

If the user does not specify a trap distribution but only gives a minority carrier lifetime, then a uniform distribution of donor mid-gap traps with density of 10^{10} m^{-3} is assumed. The carrier capture cross section is given by the above formula. However, if the user specifies the capture cross section through the material statement of **trap_ncap_i** or **trap_pcap_i**, then the minority carrier lifetime is overridden. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
lifetime_n  value=4.e-9 mater=2
```

2.251 lifetime_p

parameter	data type	values [defaults]
(see) material_par		

The material statements **lifetime_n** and **lifetime_p** define the minority life time (in seconds) for carriers (**_n** for electrons and **_p** for holes) in the SRH recombination model. The basic mechanism is electron or hole capture by deep level traps. Therefore this statement is related to other trap statements. The relation between lifetime and other trap quantities are written as

$$1/(\text{lifetime}) =$$

$$(\text{trap density}) \times (\text{thermal velocity}) \times (\text{capture cross section})$$

If the user does not specify a trap distribution but only gives a minority carrier lifetime, then a uniform distribution of donor mid-gap traps with density of 10^{10} m^{-3} is assumed. The carrier capture cross section is given by the above formula. However, if the user specifies the capture cross section through the material statement of **trap_ncap_i** or **trap_pcap_i**, then the minority carrier lifetime is overridden. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
lifetime_p  value=4.e-9 mater=2
```

2.252 light_power

parameter	data type	values [defaults]
spectrum_file	char	[void]
light_dir	char	[top] left right bottom
waveguide_mode	char	[no] yes
import_rto	char	[no] yes
gen_datafile	char	[void]
gen_datatype	char	[sorted] random
const_index	char	[no]
fresnel_reflections	char	[yes] no
slit_diff	char	[no]
ox_pump_only	char	[no] yes
std_wave	char	[yes] no
spectrum_field_file	char	[void] myfile
optical_generation	real	[0.] (m^{-3})/s
penetration_depth	real	[1.] (μm)
incident_power	real	[0.] (W/m^2) or W
wavelength	real	[1.] (μm)
profile	realx4	[-1.e8 1.e8 1. 1.] (μm)
fraction_mode2	real	[0.]
fraction_mode3	real	[0.]
fraction_mode4	real	[0.]
fraction_mode5	real	[0.]
angle	real	[0.]
fraction_TE	real	[0.5]

The statement **light_power** is used to specify the incident light power in a photo-sensitive device like a solar cell or an optically pumped laser (requires Pumped Laser module in LASTIP and PICS3D).

By default, this statement activates the Advanced Optics Model and the associated thin film model. A more basic model with simple exponential decay can still be used if you do not have this module by explicitly turning off the Fresnel reflections and standing wave effects.

The thin film model assumes plane wave propagation. Depending on the distribution of real/imaginary part of the refractive indices, one or more lines are cut into the device in the direction of the light incidence. The refractive index information along such line(s) is used to calculate the multi-layer transfer matrix.

Note that when using **light_power**, a **scan** statement must be used to ramp up the effective light power in the device since the equilibrium calculations assume no light input. Using

```
scan var=light value_to=1.0
```

will slowly ramp up the relative light input to the value defined in **incident_power** or **spectrum_file**. This can be combined with other scan variables such as time (for transient pumping) or electrode bias.

- **spectrum_file** is used to specify the file name of the data file describing the incident light power spectrum. The required data format is 2-column and is as follows. The first number on each line must be the wavelength in micrometers, while the second number must be the optical power in $W/(m^2\mu m)$. The μm appears in the unit because it is a power spectrum with wavelength in unit of μm . When the power spectrum is used, the simulation program takes into account all wavelengths. If a single wavelength is used instead of a power spectrum, then this parameter should be set to “void”.
- **light_dir** specifies the direction of the incident light.
- **waveguide_mode** is used to indicate whether waveguide mode is used as light source. If so, statements such as **init_wave** and **optical_field** must be used to solve the optical modes in the same input file.
- **import_rto** is used to indicate whether ray-tracing data file (.rto_### file) is to be imported to define the incident light distribution. If positive, the simulator will detect the existence of the .rto file first. If the file exists, the program will import the ray-tracing data. If the data file does not exist, the simulator will use the default transfer matrix. The .rto data file number is automatically determined by the bias in the same way as the .out file. This parameter is useful for studying photo-sensitive devices when ray-tracing data is generated in a previous run of the simulator. The simulator may be run repeatedly, each time using the previous .rto data as input light profiles while new ray-tracing data are produced at the post-processing stage (using .plt). The results (also the ray-tracing profile) are expected to become stable after several runs and we may regard them as being final and self-consistent ones.
- **gen_datafile**, if specified, will provide the external input data of incident optical wave intensity profile. The format of the data is compatible with the public domain program GNUPLOT. For 2D simulation, the optical intensity profile on x-y plane is given in three columns as follows

```
x y optical\_profile
```

with x varying the fastest. Please leave a blank line to separate data lines between different y-coordinates.

For 3D simulation, the data are arranged in the form

x y z optical_profile

with x varying the fastest and y the second fastest. Again, blank lines are used to separate data lines between different y coordinates.

The unit of "optical_profile" here depends on the unit used in **incident_power**. The program multiplies the "optical_profile" in the data file with **incident_power** to provide the actual optical wave intensity (in units of $Watt/m^2$). If it is a problem of 2D waveguide power problem, it is convenient to set **incident_power** to the total waveguide power (in Watt) while the data file provides a normalized intensity distribution (in units of $1/m^2$ so that the profile integrates to unity). For problem of incident light, it may be convenient to let **incident_power** to represent incident power in units of $Watt/m^2$ while "optical_profile" being the distribution of power transmission coefficient. In any kind of arrangements, it is necessary to remember that **incident_power** and the figure in data file must multiplied to obtain light intensity in units of $Watt/m^2$.

Please note that we use the optical profile here to compute the energy density (or photon density) scaled by the group velocity in units of $Watt/m^2$. It includes any standing wave effects and is used to compute the local optical absorption. This is not to be confused with power flux density which does not present standing wave effects.

It is recommended to consistently use wave intensity relative to the incident wave intensity (unitless ratio) to define optical field profile.

- **gen_datatype** If it is set to "sorted", the data order is arranged in a form compatible with GNUPLOT with left most column varies the fastest and with a blank line space separating different drawing line data. For unsorted data, a simple listing without blank space line is required.
- **const_index** is used to force the simulator to average the complex refractive index along the cut line before the transfer matrix is computed. This will automatically suppress all internal Fresnel reflections and force a uniform exponential decay of the light. Since this model is highly unphysical, the user is warned against using this option.
- **fresnel_reflections** may be used to artificially turn off the internal reflections between layers of the device. This forces all material interfaces to have 100% power transmission but unlike **const_index**, the absorption coefficient is not averaged and the result will be a piecewise exponential decay of the light.

Since this is unphysical, the user is warned against turning off this effect. However, it may be necessary to do so if you do not have access to the Advanced Optics Model. It can still provide a reasonable model for devices with a single material such as silicon solar cells.

Fresnel reflections at the front and back facets may also be artificially defined using the **front_reflection** and **back_reflection** statements. However, **optic_coating** statements will be useless unless this effect is turned on.

- **slit_diff** would enable the single slit diffraction model be used to compute the optical intensity profile. The single slit would be defined by the first two numbers of the **profile**.
- **ox_pump_only** forces the optical generation to create bound excitons instead of free carriers. This is the expected behavior in organic semiconductors.
- **std_wave** determines whether the standing wave component of the transfer matrix is used in the optical generation rate: i.e. whether $|F|^2 + |R|^2$ or $|F + R|^2$ is used. Note that for layers much thicker than the optical wavelength, the extra oscillating term is expected to average out. On the other hand, resonant cavity photo-detectors will require on this effect for accurate modeling.

If you do not have the Advanced Optics Model, you will have to turn this effect off. Since it may require a very fine mesh to resolve the standing wave oscillations, you may also want to turn this off if an insufficiently sampled standing wave looks too noisy and you do not want to work with a finer mesh.

Note also that for optically pumped VCSELs in PICS3D, this term refers only to the standing wave of the optical pump: the round-trip gain calculations automatically include this effect for the laser light.

- **spectrum_field_file** is used to specify the optical field intensity profile for different wavelengths when the light source is continuous. The data format is the same as that defined for **gen_datafile** above, except that a header line “spectrum_field_file” followed by a line containing the wavelength (in micron meters) is used to start a field profile for each wavelengths.

As for the units of field profile, it is recommended to consistently use wave intensity relative to the incident wave intensity (unitless ratio). In such a case, the input power spectrum would be in units of $W/(m^2\mu m)$, as for the parameter **spectrum_file** above.

Please note that it is NOT necessary to have the same number of field profiles as in the data points of the input power spectrum. For each data point in the input power spectrum, the program will interpolate between two spectrum field profiles. For example, one may use 20 field profiles corresponding to 20 wavelengths while the solar spectrum may have more spectrum data points.

- **optical_generation** may be used to directly specify the carrier generation rate at the starting point of the light absorption path. This parameter is used only when the single wavelength power **incident_power=0** and the incident light power spectrum **spectrum_file=void**. If this parameter is used, the

actual distribution of the optical generation rate is calculated by this number multiplied by a attenuation factor due to the absorption of the semiconductor material.

- **penetration_depth** is used together with **optical_generation** only. It is used to describe the exponential decay of the incident light along the absorption path.
- **incident_power** is the single wavelength incident light power. It is used when no light power spectrum is specified, i.e., when **spectrum_file=void**. Its unit is normally in $Watt/m^2$. In the special cases of guided wave and external optical power profile data, it should be in $Watt$ as the total waveguide power and the optical field intensity should be in the unit of $1/m^2$ and be normalized to unity at the incident end.
- **wavelength** is the single wavelength of the incident light. It is used when no light power spectrum is specified, i.e., when **spectrum_file=void**.
- **profile** is the cross section profile of the incident light. It is in the format of (x1, x2, dx1, dx2) for light traveling in the $\pm y$ direction, where the light intensity is assumed to be uniform between x1 and x2 and decay in the left and right sides with standard deviations of dx1 and dx2, respectively. Similar explanation applies to light traveling in $\pm x$ direction with a format of (y1, y2, dy1, dy2).
- **fraction_modek** k=2,...,5. This is the relative power fraction of the higher order mode if multiple waveguided lateral modes are used as in **waveguide_mode** above.
- **angle** is the angle of incidence of the light to the surface and the cut lines. To model angled and textured surfaces, users should use the ray tracing module instead.
- **fraction_TE** is the fraction of TE polarized light; the remainder is assumed to be TM. Appropriate boundary conditions are used for the plane waves in the thin film model. The default value of 0.5 was chosen to represent unpolarized light common in most solar cell applications.

Example(s)

```
light_power incident_power=3.29e7 wavelength=0.82 &&
  profile=(0.5, 2.0, 0.01, 0.01)
```

The above statement is used to define an incident light of single wavelength at $0.82 \mu m$ with power of $3.29e7 W/m^2$. The incident light direction is from the top (default) and the cross section profile is uniform from 0.5 to $2 \mu m$ with a Gaussian tail of $0.01 \mu m$ width.

2.253 light_power_qwip

parameter	data type	values [defaults]
abs_spectrum_file	char	[qwip_specfile.txt]
light_dir	char	[top],bottom
uniform_extraction	char	[no],yes
gen_datafile	char	[void]
gen_datatype	char	[sorted],random
incident_power	real	[1.e4](W/m ²)
wavelength	real	[5.] (um)
profile	realx4	[-9000. 9000. 1. 1.] (um)
power_couple	real	[0.6]
absorption_coef	real	[1.e-12](m)
active_layer_depth	real	[0.](um)
average_index	real	[3.2]
surf_elec_dens	real	[2.e15](1/m ²)
qwip_period	real	[0.04](um)
scale_abs_spec	real	[1.](um)

The statement **light_power_qwip** is used to specify the incident light power and other related parameter for a quantum well infrared photodetector (QWIP) .

- **abs_spectrum_file** is the absorption spectrum file to be imported. Its format is 3-column with first column being the wavelength in meters, 2nd column being the absorption in 1/m, and the 3rd column being the absorption divided by the surface electron density.
- **light_dir** is the direction of light.
- **uniform_extraction** is used to indicate whether uniform or global field is used to extract the photo-carriers. One possible explanation for uniform field model is that photo-carriers with higher energies are not well localized and thus tend to transport with an averaged field instead of the inhomogeneous local field.
- **gen_datafile**, if specified, will provide the external input data of incident optical power flux profile. The format of the data is compatible with the public domain program GNUPLOT. For 2D simulation, the optical intensity profile on x-y plane is given in three columns as follows

```
x y optical\_profile
```

For 3D simulation, the data are arrange in the form

x y z optical_profile

The unit of "optical_profile" here depends on the unit used in **incident_power**. The program multiplies the "optical_profile" in the data file with **incident_power** to provide the actual optical power flux density (in units of $Watt/m^2$). If it is a problem of 2D waveguide power problem, it is convenient to set **incident_power** to the total waveguide power (in Watt) while the data file provides a normalized intensity distribution (in units of $1/m^2$ so that the profile integrates to unity). For problem of incident light, it may be convenient to let **incident_power** to represent incident power in units of $Watt/m^2$ while "optical_profile" being the distribution of power transmission coefficient. In any kind of arrangements, it is necessary to remember that **incident_power** and the figure in data file must multiplied to obtain light intensity in units of $Watt/m^2$.

- **gen_datatype** If it is set to "sorted", the data order is arranged in a form compatible with GNUPLOT with left most column varies the fastest and with a blank line space separating different drawing line data. For unsorted data, a simple listing without blank space line is required.
- **incident_power** is the incident power density.
- **wavelength** is the wavelength of the QWIP.
- **profile** is the cross section profile of the incident light. It is in the format of (x1, x2, dx1, dx2) for light traveling in the $\pm y$ direction, where the light intensity is assumed to be uniform between x1 and x2 and decay in the left and right sides with standard deviations of dx1 and dx2, respectively. Similar explanation applies to light traveling in $\pm x$ direction with a format of (y1, y2, dy1, dy2).
- **power_couple** is a power coupling coefficient of the incident light into the device
- **absorption_coef** is the absorption divided by the surface density. It is not used if **abs_spectrum_file** is used.
- **active_layer_depth** is the depth of the MQW from the exposed surface of QWIP. This is used to locate the starting point of the MQW layers so that optical field profile can be established.
- **average_index** is the average index of the QWIP.
- **surf_elec_dens** is the estimated density so that the absorption can be calculated to produce the incident light intensity profile. The absorption is calculated by multiplying this parameter with the **absorption_coef** above.

- `qwip_period` is the QWIP period.
- `scale_abs_spec` is used to scale the abspectrum spectrum.

Example(s)

\$ power-coupling into absorption mode is treated as fitting parameter for now.

```
light_power_qwip abs_spectrum_file=qwip_specfile.txt light_dir=top &&  
    incident_power=1.e4 wavelength=8 surf_elec_dens=1.5e15 &&  
    qwip_period=0.0376 power_couple=0.1 uniform_extraction=yes
```

The surface electron density is used to multiply to the 3rd column in spectrum-file to obtain the absorption which is used to estimate the optical field intensity profile.

2.254 light_current

parameter	data type	values [defaults]
data_file	char	[void]
use_macro	char	no
wavel_range	realx2	
conc_range	realx2	[(1.e23 1.e24)] (m^{-3})
pn_ratio	real	[1.]
conc_log_scale	char	[no]
init_wavel	real	[0.83e0]
length	real	[300.]
mirror_ref	real	[0.3]
clad_bot	realx3	[1. 3.0 500.]
guide_bot	realx3	[.1 3.2 500.]
active_layer	realx3	[.007 3.33 500.]
guide_top	realx3	[.1 3.2 500.]
clad_top	realx3	[1. 3.0 500.]
front_back	realx2	
delta_index	real	[0.3]
av_index	real	[3.3]
auger_n	real	[2.e-42] (m^6/s)
auger_p	real	[2.e-42] (m^6/s)
life_n	real	[1.e-6] (sec.)
life_p	real	[1.e-6] (sec.)
spn_out	real	[1.e-4]
width	real	[5.]
scale_lit	real	2.
scale_curr	real	2.
max_curr	real	0.5 (Amp)
boundary_type	intgx4	[1 1 1 1]
well_num	intg	[1]
data_point	intg	[30]
mesh_point	intg	[80]

The statement **light_current** is a gain preview statement which allows the user to find a quick solution of a simple laser diode and obtain the light-current characteristics. The basic recombination model is expressed as

$$R = R_{\text{spn}} + R_{\text{auger}} + n/\tau_n + p/\tau_p$$

where the spontaneous emission spectrum computed from the quantum well model is integrated over all emission wavelengths and the Auger recombination is given by

auger_n and **auger_p**, defined below. The last two terms are due to traps, surface recombination, thermal leakage, etc.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **use_macro**, if set to yes, will search the current input file for macros used to define the properties of the active layer.
- **wavel_range** is the wavelength range within which the peak emission wave length is searched. The program only solves the wave equation at a single wavelength corresponding to the peak modal gain. If the user has a structure which controls the longitudinal mode and fixes the emission wavelength at a known value, then the upper and lower limit of the wavelength range here should be set to the fixed wavelength.
- **conc_range** is the electron concentration range in the well.
- **pn_ratio** is the ratio of hole over electron concentrations. Note that this ratio can be set to an arbitrary number in the gain preview. In the main solver, however, this ratio is determined by the simulator automatically, according to the local Fermi levels.
- **conc_log_scale** would vary the concentration in log scale.
- **init_wavel** is the initial (or guess) emission wavelength corresponding to the peak modal gain. The program starts the solution of the wave equation at this value. After finding the wave distribution using this wavelength, LASTIP calculates the modal gain spectrum within “wavel_range”. From this spectrum, LASTIP determines the peak gain wavelength, which will be the new wavelength for the wave solution at the next bias step. Therefore, “init_wavel” is used only once for the first step.
- **length** is the cavity length. This is needed to compute the mirror loss or emission loss.
- **mirror_ref** is the mirror reflectivity needed to calculate the mirror loss at a given cavity length. For a DFB laser, the mirror_ref should be set to an effective value that will result in an emission loss equivalent to all the grating losses.
- **clad_bot** defines the thickness, index and material loss of the bottom cladding layer.
- **guide_bot** defines the thickness, index and material loss of the bottom guiding layer.
- **active_layer** defines the thickness, index and material loss of the active layer.

- **guide_top** defines the thickness, index and material loss of the top guiding layer.
- **clad_top** defines the thickness, index and material loss of the top cladding layer.
- **front_back** is used to specify the front and the back facet power reflectivity. This will override the **mirror_ref**. It can be used to specify two different mirror reflectivities. If only **mirror_ref** is used, both the front and back facets are assumed to be the same.
- **delta_index** is used to describe a boundary where the wave function decays exponentially by a function:

$$W(x) = W_0 \exp(-k_0 * \text{delta_index} * x)$$

where k_0 is the wave number and **delta_index** is estimated from the effective index (which may be unknown before the simulation):

$$\text{delta_index}^2 = \text{effective_index}^2 - \text{local_index}^2$$

- **av_index** is the estimated average refractive index.
- **auger_n** is the Auger coefficient C_n .
- **auger_p** is the Auger coefficient C_p .
- **life_n** is the minority carrier life time for electrons.
- **life_p** is the minority carrier life time for holes.
- **spon_out** is the fraction of spontaneous emission coupled into the output power. This is dependent on the device geometry and layer structure and how the output power is collected. This parameter is responsible for nonzero output light intensity below the lasing threshold. This term also explains how a light emitting diode (LED) works.
- **width** is width of cavity.
- **scale_lit** is the scale of the light power used to plot output light power versus current for the laser simulation (for LASTIP). The default is 2 because LASTIP assumes the device is symmetric and the user only sets up one half of the device for simulation. This factor is multiplied to the total power (from 2 facets) as well as to power emitted from each facet.

- **scale_curr** is the scale of the current used to plot light versus current for the laser simulation. The default is 2 because LASTIP assumes the device is symmetric and the user only sets up one half of the device for simulation.
- **max_curr** is the maximum injection current.
- **boundary_type** is the boundary types on the sides of the window. The boundary type is represented by four integers in the order (lx1, lx2, ly1, ly2). The corresponding boundary is indicated in Fig. 2.14. The integers can only take on values of 1, 2, or 3. Type 1 is the zero intensity boundary condition, *i.e.*, the wave intensity must be zero at the boundary. This type is used when the boundary is far enough away from the optical mode. In the case where the boundary is a symmetric axis (lx1 in Fig. 2.14), a zero intensity boundary condition produces an optical mode of odd symmetry. Type 2 is the symmetrical boundary condition, *i.e.*, the boundary is an axis of symmetry for the wave function. In other words, the wave function at point A is the same as at point B in Fig. 2.14. Using this type on boundary lx1 in Fig. 2.14 results in an optical mode of even symmetry. In a one-dimensional simulation with semiconductor layers parallel to the x-direction, the optical mode has translational symmetry in the x-direction. Therefore, both lx1 and lx2 boundaries should be type 2 while ly1 and ly2 should be type 1. Type 3 is used to represent a truncated wave function that decays exponentially. This type of boundary may be useful in cases where the user wishes to save some mesh points by cutting a large portion of the device area out of the wave solution window. In such case, we may assume the wave decays exponentially at the boundary. This choice works together with the parameter **delta_index**. Please see parameter **delta_index**.
- **well_num** is the total number of active quantum wells involved.
- **data_point** is the number of data points to be used in the current versus concentration plot.
- **mesh_point** is the mesh point number used in this quick laser model.

2.255 linear_heat

parameter	data type	values [defaults]
(see) material_par		

The material statement **linear_heat** is used to define a heating source term proportional to the current density. The unit is $Watt/(mA \cdot cm)$. It may also take a negative value to represent cooling effect.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.256 load_macro

parameter	data type	values [defaults]
name	char	
var_namej (j=1...9)	char	
var_symbolj (j=1...9)	char	
varj (j=1...9)	real	
mater	intg	[1]

The statement `load_macro` activates a group of statements often used to describe the bulk material properties of a device. A material macro is a collection of many material statements. The content of the macro is usually stored in the macro library “c:

crosslig

PRODUCT

crosslight.mac” and “c:

crosslig

PRODUCT

more.mac”. When this statement is used in an input file, SOFTWARE replaces it by the corresponding material statements. Therefore, using this statement is equivalent to typing in all the material statements in the input file. To override a material statement within a macro, one needs to re-issue the same statement with new parameters after the macro call.

- The parameter **name** is the name of the macro being activated.
- The parameter **var_name1-9** is the name(s) of the variable(s) [usually the material composition(s)]. Note that this parameter only gives the name of the variable but does not specify how this parameter varies in space. The material variation is further defined in the statement **mater_var**. This parameter corresponds to the function variable appearing within the macro definition.
- The parameter **var1-9** is the value(s) of the variable(s) appearing in the functions within the macro definition. It is usually used to represent material composition.
- The parameter **var_symbol1-9** are the symbolic variable names used as function arguments in the macro. If defined by user, they must exactly match the

symbols in the macro function definitions. If they are not defined by user, the simulator assumes the macro is an older style macro with the same function arguments appearing in all functions.

- The parameter **mater** is the index of the material being described.

Example(s)

```
load_macro name=ingaas mater=2 var1=0.32 var_symbol1=x
```

This statement activates a material macro named ingaas (for InGaAs) for material number 2 with material variable (i.e. In composition) equal to 0.32. Within the macro the first function variable is the In composition.

```
load_macro name=algaas mater=3 var_name1=Al var_symbol1=x
```

This statement activates a material macro named algaas (for AlGaAs) for material number 3 with material variable (i.e. Al composition) named “Al”. What value this variable takes and how it varies with distance is defined separately in the **mater_var** statement.

```
load_macro name=ingaasp.temp var1=0.950 var2=200 mater=2 &&
var_symbol1=x var_symbol2=temper
```

Here the material macro ingaasp.temp has two function variables (y,temperature), where y is the composition in the formula $In_{1-x}Ga_xAs_yP_{1-y}$. The statement says that the first variable (y) is set to a value of 0.95 and the second variable (temperature) is set to a value of 200 degree K.

2.257 load_mesh

parameter	data type	values [defaults]
mesh_inf	char	
version2002	char	[no]
suprem_import	char	[no]
zseg_num	intg	1

load_mesh loads in a previously generated mesh.

- **mesh_inf** is the data file containing the mesh, which must be the same as the **mesh_outfile** in the mesh generation statement of **mesh_output**.

- **version2002** is used to accept mesh generated by older versions than 2002. The older mesh can also be generated with newer versions of the simulator with special settings of the statement **internal_xpoint** statement in .geo file.
- **suprem_import** indicate if CSuprem mesh is imported.
- **zseg_num** defines the segment in PICS3D simulation. In a 3D simulation, each segment can have different material or different mesh.

Example(s)

```
load_mesh mesh_inf=bulk1d.msh2
```

2.258 longitudinal

parameter	data type	values [defaults]
fphase_input	char	[yes]no
left_phase	real	[0.] (π)
right_phase	real	[0.] (π)
left_f_refl	real	[0.]
right_f_refl	real	[0.]
ref_wavel	real	(m)
ref_density	real	[2.0e24] (m^{-3})
ref_index	real	
ref_pitch	real	(m)
ref_midpoint	real	0.5
add_overlap	real	[100.]

The statement **longitudinal** is used to specify some general information about the longitudinal structure of a laser.

- **fphase_input** is used to determine whether or not the facet phase is taken from the input parameters **left_phase**, **right_phase**. If it is “yes”, the facet phase is given by the above two parameters. If it is “no”, the facet phase is calculated from the length of the laser and equally split between the front (left) and back (right) facets.
- **left_phase** is the corrugation phase at the left facet. It is used only when **fphase_input**=yes.
- **right_phase** is the corrugation phase at the right facet. It is used only when **fphase_input**=yes.

- **left_f_refl** is the facet reflectivity of the left facet (power basis).
- **right_f_refl** is the facet reflectivity of the right facet (power basis).
- **ref_wavel** is the reference wavelength. For grated DFB or DBR lasers, it must be set to the Bragg wavelength of the gratings. Since the grating index is a function of carrier density and the variable index, it must be given under the condition of reference density and reference index. For lasers without any gratings, it is a reference wavelength which must be close to the emission wavelength of the main mode.
- **ref_density** is only used as a number to subtract the density distribution. It is convenient to set it close to the threshold density of the laser.
- **ref_index** is the reference phase index used to compute the Bragg wavelength. The laser designer normally knows this number before they decide on a grating period.
- **ref_pitch** is the reference pitch. Either this parameter or **ref_wavel** must be defined for a structure.
- **ref_midpoint** is the relative (i.e., less than 1) reference point where the reflectivities looking towards left and right directions are computed. The round trip gain is computed based on these reflectivities. This point must be chosen such that the the optical wave is non-zero at such a point. For VCSEL simulation, this reference point is automatically set to the beginning of the section before the active section (ie., the phase matching section) unless "set_ref=yes" in statement **vcSEL_model**.
- **add_overlap** is used to add a small term of $\text{add_overlap} \times 2j/v_g$ to the $\frac{t_{ial}W}{t_{ial}\omega}$ which appears in the expression for $R_{sp,i}$ (the spontaneous emission into the *i*th mode). This term seems to improve the stability of longitudinal mode solver. More detail can be found in the User's manual regarding overlap integrals between traveling waves.

Example(s)

```
longitudinal ref_wavel=1.55d-6 left_f_refl=0. right_f_refl=0.
```

2.259 loop_integer

parameter	data type	values [defaults]
symbol	char	[void]
value_from	intg	[1]
value_to	intg	[2]

loop_integer works together with **start_loop** statement. It is used to define a integer looped variable which depends on the integer variable of **start_loop** statement. In another word, the integer symbol here changes in the same way as the symbol in **start_loop**.

- **symbol** is the symbol of the looped integer variable. **value_from** is the starting value of the looped integer variable. **value_to** is the ending value of the looped integer variable.

Example(s)

```
start\_loop symbol=%i value_from=1 value_to=5 step=2
loop\_integer symbol=%j value_from=2 value_to=6
plot_scan scan_var=voltage_2 variable=total_curr_3 scanline=%i
plot_scan scan_var=voltage_3 variable=total_curr_3 scanline=%j
```

In the above example, symbol %j is a linear function of the loop variable %i. %i would vary as 1,3, and 5 while %j would go as 2,4 and 6. The above is equivalent to

```
plot_scan scan_var=voltage_2 variable=total_curr_3 scanline=1
plot_scan scan_var=voltage_3 variable=total_curr_3 scanline=2
plot_scan scan_var=voltage_2 variable=total_curr_3 scanline=3
plot_scan scan_var=voltage_3 variable=total_curr_3 scanline=4
plot_scan scan_var=voltage_2 variable=total_curr_3 scanline=5
plot_scan scan_var=voltage_3 variable=total_curr_3 scanline=6
```

2.260 loop_real

parameter	data type	values [defaults]
symbol	char	[void]
value_from	real	[1]
value_to	real	[2]

loop_real works together with **start_loop** statement. It is used to define a real looped variable which depends on the integer variable of **start_loop** statement. In another word, it is a linear function of the loop variable.

- **symbol** is the symbol of the looped real variable. **value_from** is the starting value of the looped real variable. **value_to** is the ending value of the looped real variable.

Example(s)

```
start_loop symbol=%i value_from=1 value_to=5
loop_real symbol=%vg value_from=0.2 value_to=1.0
scan var=voltage_2 value_to=%vg
end_loop
```

This means %vg linearly varies with %i. %i would take values of 1,2,3,4 and 5 while %vg would vary as 0.2, 0.4, 0.6, 0.8 and 1.0. The equivalent commands are as follows:

```
scan var=voltage_2 value_to=0.2
scan var=voltage_2 value_to=0.4
scan var=voltage_2 value_to=0.6
scan var=voltage_2 value_to=0.8
scan var=voltage_2 value_to=1.0
```

2.261 lplot_xy

parameter	data type	values [defaults]
data_file	char	[vtttek]
variable	char	(see list)
n_variables	charxn	
integration	char	[no]
xy_from	realx2 (micron)	
xy_to	realx2 (micron)	
xrange	realx2	
yrange	realx2	
z	real	(micron)
integration_start	real	[0.0](um)
integration_length	real	[1.e5](um)
point_num	intg	[300]
var_num	intg	[1]
mode_index	intg	[1]
trap_index	intg	[1]

lplot_xy is a post-processor statement used to plot data along a line (line plot or lplot) on the xy-plane.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **variable** is the physical quantity to be plotted. When the variable of “wave_intensity” is used, the optical confinement factor is also printed with the graphical output. Detailed listing is in App. C.
- **n_variables** is a group of variables to be plotted.
- **integration** indicates whether integration is performed for the physical variable being plotted. If “yes”, the results of the integration will be printed in .plt.msg file.
- **xy_from** is the starting point of the 1D section on the xy-plane.
- **xy_to** is the ending point of the 1D section on the xy-plane.
- **xrange** specifies a range for the abscissa.
- **yrange** specifies a range for the ordinate.
- **z** is the z position of the xy-plane on which the line segment of the plot is taken.
- **integration_start** is the start point of the integration if **integration** is set.
- **integration_length** is the length of the integration if **integration** is set.
- **point_num** is the number of points to be plotted.
- **var_num** is the number of variables to be plotted.
- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.

Example(s)

```
lplot_xy variable=wave_intensity xy_from=(0.5 1.1) &&
      xy_to=(0.5 1.9) z=200.
```


2.262 lplot_xyz

parameter	data type	values [defaults]
data_file	char	[void]
variable	char	[void]
n_variables	charxn	
integration	char	[no]
xy_point	realx2	
z_from	real	
z_to	real	
xrange	realx2	
yrange	realx2	
integration_start	real	[0.0]
integration_length	real	[1.e5] (um)
var_num	intg	[1]
mode_index	intg	[1]
trap_index	intg	[1]

lplot_xyz is a post-processor statement used to plot data along a line (line plot or lplot) on in the z-direction at a fixed xy-coordinate. Its use is similar to **lplot_xy** except the direction is different.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **variable** is the physical quantity to be plotted. When the variable of “wave_intensity” is used, the optical confinement factor is also printed with the graphical output. Detailed listing is in App. C. For a listing of plotable variables, please also see the file plotdata.tab in the installation directory.
- **n_variables** is a group of variables to be plotted.
- **integration** indicates whether integration is performed for the physical variable being plotted. If “yes”, the results of the integration will be printed in .plt.msg file.
- **xy_point** is the fixed point on an xy-plane. The line of plot is formed by extension of this point along the z-direction.
- **z_from** is the starting point of the 1D line plot.
- **z_to** is the ending point of the 1D line plot.
- **xrange** specifies a range for the abscissa.
- **yrange** specifies a range for the ordinate.

- **z** is the z position of the xy-plane on which the line segment of the plot is taken.
- **integration_start** is the start point of the integration if **integration** is set.
- **integration_length** is the length of the integration if **integration** is set.
- **var_num** is the number of variables to be plotted.
- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used. This parameter is used only when plotting variable is set to lateral mode wave intensities.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted. This parameter is used only when trap-related quantities are being plotted.

Example(s)

```
lplot_xyz variable=elec_conc xy_point=(2.5 0.4) &&
z_from=0. z_to=50
```

2.263 mass_density

parameter	data type	values [defaults]
(see) material_par		

The material statement **mass_density** is used to define the density (in kg/m^3) of the semiconductor material.

The use of this parameter and related examples are given under **material_par** in section [2.272](#).

2.264 mass_gamma_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **mass_gamma_bar** is an active layer macro statement used to define the relative effective mass of the Gamma band in the quantum barrier.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.265 mass_gamma_bulk

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the conduction bane effective mass in direction along the c-axis. This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.266 mass_gamma_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **mass_gamma_well** is an active layer macro statement used to define the relative effective mass of the Gamma band in the quantum well.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.267 mass_l_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **mass_l_bar** is an active layer macro statement used to define the relative effective mass of the L-band in the quantum barrier.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.268 mass_l_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **mass_l_well** is an active layer macro statement used to define the relative effective mass of the L-band in the quantum well.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.269 mater_var

parameter	data type	values [defaults]
name	char	
variation	char	linear_x, linear_y
n_values	realxn	
rotation_angle	real	(degrees)
reference_point	realx2	(μm)
data_num	intg	

The statement **mater_var** works with **load_macro** when only the material variable name (usually composition) of the functions within the macro is given. It specifies how the material variable (usually composition) varies with distance.

- **name** is the name of the material variable being specified. It should be the same as the variable name given in the **load_macro** statement.
- **variation** defines how the variable varies with distance. “linear_x” means that the variable varies from point to point as a linear function in the x-direction. This option requires spatial coordinates and material parameter values at more than one position. For example, if the composition A is given

at three points in the x-direction (x1,x2,x3) with values (A1, A2, A3), this statement allows SOFTWARE to linearly interpolate the composition values between the points. The choice of “linear_x” must be used together with the parameter of **data_num** and **n_values**. “linear_y” is the same as “linear_x” except that it is in the y-direction. It may also take “linear_xy” when material composition grading take arbitrary direction which is specified by parameters of **rotation_angle** and **reference_point**.

- **n_values** specifies several numerical values (n values) to provide more details for “variation=linear_x(y)”. This will give n/2 data points over the range within which the statement function varies linearly. The data format is

n_values=(x1,A1,x2,A2,x3,A3, ...)

where x_i and A_i are the x-coordinate and the variable (usually composition) value, respectively, at ith point. For rotated grading region as defined by variation=linear_xy, the coordinates are relative distance from the reference point in **reference_point**. For further details, please see the example of material_par.

- **rotation_angle** is effective when variation=linear_xy. It is the angle from the y-axis.
- **reference_point** is x/y coordinate of the reference point of the rotated grading region. It is effective when variation=linear_xy. In listing of coordinate/composition in n_values above, the coordinate is the distance from this reference point for a rotated grading region.
- **data_num** is the number of data values in the **n_values** parameter (i.e. it is equal to n).

Example(s)

```
mater_var name=In variation=linear_y 4_values=(0. 0.14, 0.3, 0.2) &&
data_num=4
```

2.270 material

parameter	data type	values [defaults]
type	char	[semicond]
el_vel_model	char	constant, 2.piece,[beta],n.gaas
hole_vel_model	char	constant, 2.piece,[beta],n.gaas
band_valleys	realx2	[1 1]
mater	intg	[1]
group_index	real	[3.56]

The statement **material** defines some general material properties for the material. This is used for 2D/3D material description in Crosslight simulators.

- **type** is the type of material. It takes one of *insulator*, *wurtzite*, *resistor*, *organic*, or *super_struct_type*. When *super_struct_type* is used, it usually means dealing with a complicated layer structure with many periodical layers. In such as case, the physical properties are separately simulated in a smaller project and are imported into the main project.
- **mater** is the index of the material being described.
- **el_vel_model** and **hole_vel_model** correspond to the following models for the field-dependent mobility.
 - 1. The simplest mobility model uses constant mobility (i.e. constant model) μ_{0n} or μ_{0p} for electrons or holes, respectively, throughout each material region in the device.
 - 2. Another simplified field dependence mobility model is the two-piece mobility model (i.e. 2.piece model):

$$\begin{aligned}\mu_n &= \mu_{0n}, \\ &\quad \text{for } F < F_{0n}; \\ \mu_n &= v_{sn}/F, \\ &\quad \text{for } F \geq F_{0n};\end{aligned}\tag{2.29}$$

$$v_{sn} = \mu_{0n}F_{0n},\tag{2.30}$$

for the electron mobility. F_{0n} is a threshold field beyond which the electron velocity saturates to a constant. Similar expressions can be defined for holes:

$$\begin{aligned}\mu_p &= \mu_{0p}, \\ &\quad \text{for } F < F_{0p}; \\ \mu_p &= \mu_{0p}F_{0p}/F, \\ &\quad \text{for } F \geq F_{0p};\end{aligned}\tag{2.31}$$

$$v_{sp} = \mu_{0p}F_{0p}.\tag{2.32}$$

- 3. A commonly used mobility model has the following form (the “beta model”) for electrons and holes, respectively:

$$\mu_n = \frac{\mu_{0n}}{(1 + (\mu_{0n}F/v_{sn})^{\beta_n})^{1/\beta_n}},\tag{2.33}$$

$$\mu_p = \frac{\mu_{0p}}{(1 + (\mu_{0p}F/v_{sp})^{\beta_p})^{1/\beta_p}}.\tag{2.34}$$

- 4. Many III-V compound semiconductors [(e.g. GaAs) exhibit negative differential resistance due to transition of carriers into band valleys with lower mobility. Crosslight simulators have implemented the following field dependent model (the “n.gaas model”)

$$\mu_n = \frac{\mu_{0n} + (v_{sn}/F_{0n})(F/F_{0n})^3}{1 + (F/F_{0n})^4}. \quad (2.35)$$

- **band_valleys** is the number of equivalent conduction band valleys and valence band peaks. This is needed to compute the density of states from the effective masses.
- **group_index** is the group index of the 2D cross section being simulated. The group index is used to measure the velocity of the photon energy and it appears in the photon rate equation and in stimulated emission terms.

Example(s)

```
material band_valleys=(6 1) &&
  el_vel_model=beta hole_vel_model=beta
```

This statement is for Si devices. The following is applicable to GaAs based devices:

```
material type=semicond band_valleys=(1 1) &&
  el_vel_model=n.gaas hole_vel_model=beta
```

2.271 material_3d

parameter	data type	values [defaults]
global	char	[yes] no
intern_spon_par	char	[no] yes
dkdn_real	real	[0.] (m^2)
dkdn_imag	real	[0.] (m^2)
dkdn_nref	real	[2.e24] (m^{-3})
dnd_type2	real	-1.e-28 (m^3)
dgdb_type2	real	1.e-22 (m^2)
gdgw_type2	real	0.0
sec_num	intg	[1]

material_3d defines some material parameters used in the longitudinal solver. If this parameter is not used in the input file, the simulator takes the default values of this statement for all sections.

For the passive sections, the optical gain (or loss) is given by the following formula:

$$\begin{aligned} \text{modal_gain} = & -\text{internal_loss} \\ & + \text{dgd_type2} * (\text{N} - \text{nref_type2}) \\ & + \text{dgdw_type2} * (\omega - \omega_{\text{ref}}) \end{aligned}$$

The word **type2** is used following the convention of PICS1D referring to the passive section optical gain. **N** is the carrier density.

Similarly the index change of the passive section is given by:

$$\text{index_change} = \text{dnd_type2} * (\text{N} - \text{index_nref})$$

- **global** indicates if this statement applies to all longitudinal sections. If negative, only the section of **sec_num** takes the material parameters. If positive, it overrides the **sec_num** parameter.
- **intern_spon_par** indicates if "spon_par" is used or not. If not, the program computes spontaneous parameter (n_{sp} in formulas) according to material gain/spontaneous intensities under specific current injection conditions.
- **dndn_type2** is the derivative of real refractive index of the pumped layer in the passive section of a laser. This may be used to tune the frequency of a laser. Also, it will affect the derivative of the propagation constant (k) with respect to carrier density. This appears in the theory section of the PICS3D/PICS1D User's Manual.
- **dgdn_type2** is the derivative of the gain (or the negative loss) with respect to carrier density in the passive section.
- **dgdw_type2** is the derivative of the gain (or the negative loss) with respect to omega ($2\pi \cdot \text{frequency}$) in the passive section.
- **dkdn_real** and **dkdn_imag** describe the derivative of the complex kappa with respect to density ($d\kappa/dN$). These parameters can be used to study the effect of current injection on the coupling coefficient. The expression for kappa is

$$\text{kappa} = \text{kappa_ref} + \text{d_kappa}/\text{dN} * (\text{N} - \text{N_ref})$$

The default is **kappa_ref** (reference kappa) as a constant coupling coefficient.

- **dkdn_nref** is the reference density appearing in the above equation.

- **sec_num** is the longitudinal section number.

Example(s)

```
material_3d global=yes
```

2.272 material_par

parameter	data type	values [defaults]
variation	char	linear_x,linear_y,function,[constant]
var_namej (j=1..9)	char	[void]
varj(j=1..9)	real	
var_symbolj(j=1..9)	char	void
value	real	
n_values	realxn	
mater	intg	[1]
data_num	intg	[0]
var_num	intg	[1]

material_par is not a statement, but a dummy name used to represent any of the material statements. For example, it can be either of the statements **band_gap** or **hole_mass**. Since most of the material statements have the same parameter usage, they are defined here collectively. Usually the material statement itself is some physical quantity which can be given a single numerical value. In many cases, however, the physical quantity varies as a function of spatial coordinates or a function of composition or temperature. The bandgap can vary as a function of position in a graded junction device, for example, and therefore it is necessary to create parameters to define the material statement in more detail.

The parameters described here is also collectively referred to as **group1** parameters which are common to most physical quantities of the simulation program.

- **variation** defines how the physical quantity varies with distance. There are several options for this parameter:

If the parameter takes “constant”, only a single value is needed to describe the physical quantity.

“linear_x” means the physical quantity varies from point to point as a linear function in the x-direction. This option requires spatial coordinates and material parameter values at more than one position. In a case where the bandgap is given at three points in the x-direction (x1,x2,x3) with values (E1,E2,E3), this

statement allows PICS3D to linearly interpolate the bandgap values between the points. “linear_x” must be used together with the parameters **data_num** and **n_values**. “linear_y” is the same as “linear_x” except that it is in the y-direction.

“function” means that a mathematical function follows the statement. The math function must start with the line

```
function(variable1, variable2, ...) and end with the line
end_function
```

The mathematical function must follow a set syntax, which is described in detail in Section 1.5.

- **var_name1-9** is a dummy name of a variable that varies spatially according to the statement of **mater_var**. Usually it is used to describe spatial variation of material compositions.
- The parameter **var1-9** is the value of the variable if a function is defined. In older style macros, The variable must correspond to the variables defined in the math function in exactly the same order as the dummy arguments in function definition. In the more recent macro style (free-style), they can be used to refer to the function variable symbols in parameters **var_symbol1-5**.
- The parameter **var_symbol1-9** is the symbolic variable names used as function arguments in the macro. If defined by user, they must exactly match the symbols in the macro function definitions. If they are not defined by user, the simulator assumes the macro is an older style macro with the same function arguments appearing in all functions.
- **value** is the value of the statement (or the physical quantity) if variation=constant.
- **n_values** specifies several numerical values (n values) to provide more details for “variation=linear_x(y)”. This will give n/2 data points over the range within which the statement function varies linearly. The data format is

$$n_values=(x_1, f_1, x_2, f_2, x_3, f_3, \dots)$$
where x_i and f_i are the x-coordinate and the function value, respectively, at i th point. For the bandgap example below,

$$6_values=(x_1, E_1, x_2, E_2, x_3, E_3) \quad data_num=6$$
- **mater** is the index of the material being described.
- **data_num** is the number of data values in the **n_values** parameter (i.e. it is equal to n).
- **var_num** is the number of function variables if “function” is used for **variation**.

Example(s)

```
band_gap    value=1.425 mater=2

dielectric_constant variation=linear_y data_num=6  &&
  6_values=(0. 10.5, 0.5 11.4, 0.9 12.)

electron_mass variation=function var1=0.3
function(x)
0.4&+0.2&*x
end_function
```

2.273 max_electron_mob

parameter	data type	values [defaults]
material_par		

The statements **max_electron_mob**, **max_hole_mob**, **min_electron_mob** and **min_hole_mob** are the maximum and minimum electron and hole mobilities (in units of $m^2V^{-1}s^{-1}$, respectively, which correspond to μ_{2n} μ_{2p} μ_{1n} μ_{1p} as used in Eqs. (2.4) and (2.5).

The parameters for these statements are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.274 max_hole_mod

parameter	data type	values [defaults]
material_par		

The statements **max_electron_mob**, **max_hole_mob**, **min_electron_mob** and **min_hole_mob** are the maximum and minimum electron and hole mobilities (in units of $m^2V^{-1}s^{-1}$, respectively, which correspond to μ_{2n} μ_{2p} μ_{1n} μ_{1p} as used in Eqs. (2.4) and (2.5).

The parameters for these statements are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.275 mesh_output

parameter	data type	values [defaults]
mesh_outf	char	
order	char	[yes],no

The mesh generation statement **mesh_output** defines the mesh output data files. This statement is used after the placement of mesh lines on the edges of the polygons.

- **mesh_outf** is the mesh output file.
- **order** specifies the format of the mesh output. If the mesh output is to be used by the solver, “yes” should be chosen, i.e., ordering of the mesh is required to interface with the solver. If the mesh is to be further manipulated after this statement, a “no” value should be chosen.

Example(s)

```
mesh_output mesh_outf=bulk2d.msh order=yes
```

2.276 metal

parameter	data type	values [defaults]
mater	intg	[1]
metal_conductivity	real	[0.0] $(ohm * m)^{-1}$

This statement is used to define an enlarge part of the device as a metal in a thermal/heat-transfer environment. This statement force the program to solve only the unipolar current flow equation in the enlarge part without the solution of the semiconductor drift-diffusion equations. A smaller auxiliary program **thermal.exe** is used to solve the external metal part.

- **mater** is the material number this statement refers to.
- **metal_conductivity** is the electrical conductivity the external enlarge part.

2.277 min_electron_mob

parameter	data type	values [defaults]
material_par		

The statements **max_electron_mob**, **max_hole_mob**, **min_electron_mob** and **min_hole_mob** are the maximum and minimum electron and hole mobilities (in units of $m^2V^{-1}s^{-1}$, respectively, which correspond to μ_{2n} μ_{2p} μ_{1n} μ_{1p} as used in Eqs. (2.4) and (2.5).

The parameters for these statements are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.278 min_hole_mob

parameter	data type	values [defaults]
material_par		

The statements **max_electron_mob**, **max_hole_mob**, **min_electron_mob** and **min_hole_mob** are the maximum and minimum electron and hole mobilities (in units of $m^2V^{-1}s^{-1}$, respectively, which correspond to μ_{2n} μ_{2p} μ_{1n} μ_{1p} as used in Eqs. (2.4) and (2.5).

The parameters for these statements are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.279 mmb_gaintable

parameter	data type	values [defaults]
directory	char	[void]
use_gain_spec	char	[yes]
use_index_spec	char	[yes]
use_spon_spec	char	[yes]
active_regnum	intg	[1]

The statement **mmb_gaintable** is used to activate an interface to import the microscopic many-body material data base of Univ. of Arizona (or Non-linear Control Strategy Inc.). The data base consists of quantum well optical gain, refractive index

and spontaneous emission computed at a limited number of temperatures and carrier densities. The assumption in the data base is that there is equal amount of electrons and holes in any quantum wells. This statement simply imports the gain and index data from the mmb-data base and interpolates between data files to obtain a continuous gain spectrum useful for simulation. To make the simulation compatible, we have to average the amount of electrons and holes in the well to come up with a single value of density to compare with the data files in the mmb-data base.

- **directory** is the name of the directory where the data files of mmb-data is stored. This directory must be located in the current directory of the input files of the simulation.
- **use_gain_spec** instructs the simulator to import the gain spectrum data from the mmb-data.
- **use_index_spec** instructs the simulator to import the refractive index spectrum data from the mmb-data.
- **use_index_spec** instructs the simulator to import the spontaneous emission spectrum data from the mmb-data.
- **active_regnum** is the active region number this statement takes effect.

Example(s)

```
mmb_gaintable directory=mmb_data use_gain_spec=yes use_index_spec=yes
```

2.280 mobility_xy

parameter	data type	values [defaults]
dir	char	[y],x
elec_field_model	char	[void],intel1,intel2,lombardi
hole_field_model	char	[void],intel1,intel2,lombardi
use_elec_hv_expo_factor	char	[no]
use_hole_hv_expo_factor	char	[no]
factor_elec	real	[1.]
factor_hole	real	[1.]
elec_intel1_beta	real	[0.5]
elec_intel1_e_crit	real	[4.2e6] (V/m)
elec_intel2_alpha	real	[1.02]
elec_intel2_e_univ	real	[5.71e7] (V/m)
hole_intel1_beta	real	[0.5]
hole_intel1_e_crit	real	[3.e6] (V/m)
hole_intel2_alpha	real	[0.95]
hole_intel2_e_univ	real	[2.57e7] (V/m)
elec_lombardi_b	real	[4.75e7] (cgs unit)
elec_lombardi_alpha	real	[1.74e5] (cgs)
elec_lombardi_beta	real	[0.125]
elec_lombardi_delta	real	[5.82e14] (cgs)
hole_lombardi_b	real	[9.93e7] (cgs)
hole_lombardi_alpha	real	[8.84e5] (cgs)
hole_lombardi_beta	real	[0.0317]
hole_lombardi_delta	real	[2.05e14] (cgs)
elec_hv_critical_field	real	[5.e7] (V/m)
elec_hv_expo_field	real	[1.e7] (V/m)
elec_hv_minfac	real	[0.5]
hole_hv_critical_field	real	[5.e7] (V/m)
hole_hv_expo_field	real	[1.e7] (V/m)
hole_hv_minfac	real	[0.5]
mater	intg	[1]

This statement is created to model anisotropic mobility on the x-y plane. This may be useful for the effective medium approximation of multiple layers (such as DBR mirrors in a VCSEL), or may be useful for vertical field dependent mobility in a MOSFET channel.

For mobility reduction due to transverse field under the MOSFET channel, we have implemented three mobility models as follows:

The models “intel1” and “intel2” are based on pisces-2et manual, (stanford univerisy)

as follows. The intel1 model is defines a factor to reduce the mobility under the channel:

$$r_{perp} = (1 + E_{\perp}/E_{crit})^{-\beta} \quad (2.36)$$

where $E_{\perp}$ is the field perpendicular to the SiO₂/Si interface.

The model intel2 uses the following formula for the reduction factor

$$r_{perp} = [1 + (E_{\perp}/E_{univ})^{\alpha}]^{-1} \quad (2.37)$$

The Lombardi model (ref: C. Lombardi, S. Manzini, A. Saporito, and M. Vanzi, “A physically based mobility model for numerical simulation of non-planar devices,” IEEE CAD, Vol. 7, no. 11, pp. 1164-1171, Nov. 1988.) is to sum the contributions to mobility from different sources as follows:

$$\frac{1}{\mu} = \frac{1}{\mu_{ac}} + \frac{1}{\mu_{srf}} + \frac{1}{\mu_0} \quad (2.38)$$

where μ_0 is the mobility due to longitudinal field/hot-carrier effect. The other terms are as follows.

$$\mu_{ac} = \frac{B}{E_{\perp}} + \frac{\alpha N^{\beta}}{T_L E_{\perp}^{\frac{1}{3}}} \quad (2.39)$$

$$\mu_{srf} = \frac{\delta}{E_{\perp}^2} \quad (2.40)$$

Optionally, an additional mobility reduction factor of the following form is used:

$$\begin{aligned} f_{hv} &= 1; & \text{if } E_{\perp} \leq E_{cr} \\ f_{hv} &= f_{min} + (1 - f_{min}) \exp[-(E_{\perp} - E_{cr})/E_{exp}]; & \text{if } E_{\perp} > E_{cr} \end{aligned} \quad (2.41)$$

This additional reduction is used to describe quasi-saturation effect in high voltage (HV) MOSFET where strong suppression of mobility due to high vertical field (or high V_g) were reported.

In the following description, we only define parameters for electron while those for holes are completely similar.

- **dir** is the direction in which the mobility is modified by a factor.
- **elec_field_model** specifies the model used for electron mobility. If set to void, a constant factor is used to modify the default mobility.
- **factor_elec** is the factor to be multiplied to the electron mobility in one direction.
- **elec_intel1_beta** , **elec_intel1_e_crit** are parameters for the intel1 model described above.

- **elec_intel2_alpha**, **elec_intel2_e_univ** are parameters for the intel2 model described above.
- **elec_lombardi_b**, **elec_lombardi_alpha**, **elec_lombardi_beta**, **elec_lombardi_delta** are parameters for the Lombardi model described above. Please note that to keep the same formulas as in the original paper, we use cgs unit for these parameters here.
- **use_elec_hv_expo_factor** enables the additional mobility reduction factor f_{hv} .
- **elec_hv_critical_field**, **elec_hv_expo_field**, and **elec_hv_minfac** are the E_{cr} , E_{exp} and f_{min} , respectively.
- **mater** is the material number of the material affected by this statement.

Example(s)

```
mobility_xy dir=y factor_elec=0.6 mater=3
```

The above statement decreases the electron mobility in y-direction by a factor of 0.6 in material number 3.

2.281 mode_srch

parameter	data type	values [defaults]
iter_print	char	[yes] no
sort_mode	char	left, right, [total]
adjust_range	char	yes, [no]
threshold_srch	char	[yes], no
srch_sidemode	char	[no], yes
print_rtgain	char	[no], yes
force_yrange	char	[no], yes
global_newton	char	[no], yes
omega_xrange	real	[8.]
omega_yrange	real	[4.0]
srch_start	real	[10.e-3] (Amp)
rtg_start	real	[0.9]
tol_eqn	real	[1.e-3]
tolerance	real	[1.e-5]
omega_ydamp	real	[1.e-8]
power_in	real	[0.e-3] (W)
delta_wavel	real	[0.e-3] (m)
delta_in	real	[0.e-6] (m)
omega_xshift	real	[0.]
step_limit	real	[0.05]
current_scale	real	[1.0]
omega_ymin	real	[1.e3]
wavel_xrange	real	(m)
sample_x	intg	[101]
sample_y	intg	[101]
iter_num	intg	[30]
max_level	intg	[5]

mode_srch is used to define a few important parameters used by PICS1D to search and solve for the longitudinal mode.

- **iter_print** determines whether or not the relative error of the iteration will be printed during the simulation.
- **sort_mode** is used to sort the mode power to decide which is the dominant mode and which is the main side mode. If it takes “left”, it means that the mode power is sorted according to the power from the left facet; if it takes “right”, it means the mode power is sorted according to the power emission from the right. When it takes “total” the sort is based on the total photon

number in the cavity. This parameter may help to ensure that the side mode power from one facet does not look discontinuous, which may occur if the sort is based on power emission other than from that particular facet.

- **adjust_range** turns on/off the option to search for the dominant mode. By default it is on, and the program will attempt to put the dominant mode in the middle of the frequency range.
- **threshold_srch** turns on/off the threshold search before the self-consistent solution is applied. If it is turned off, the program start the self-consistent solution from the starting point of the bias.
- **srch_sidemode** may be used to force the longitudinal mode solver to conduct a longitudinal mode search at all bias points. Normally, the mode search is stopped after the threshold is reached.
- **print_rtgain** is used to force the solver to print the round trip gain spectrum in .info file. The longitudinal mode solver will stop after the printing.
- **force_yrange** is used to force the y-range for mode search on the complex omega plane. If the y-range is not forced, the actual y-range in mode search is determined by a quick search on both ends of x-range. This option may be used to ensure all modes are found within a certain y-range.
- **global_newton** replaces the standard Newton solver with a globally convergent method based on line searches. This prevents overshoots that can occur with the standard Newton method and may prevent restarts in some cases. This, in turn, may also lead to faster simulation times in certain situations. Since this method includes an automatic internal damping of the Newton step, both the **omega_ydamp** and **step_limit** parameters will be ignored when using this method. Note that because of this internal damping, the **tolerance** parameter may need to be set at a lower value than with the standard Newton method.
- **omega_xrange** is a parameter defined in the theory presented in the PICS1D User's Manual. The mode search can be represented by finding the pole of the Wronskian in the ω_x and ω_y plane, where ω_x is the real angular frequency of the longitudinal mode. This parameter is used to define the range of the pole search in the ω_x direction. The units are given by the equivalent Fabry-Perot mode spacing, *i.e.*, $\Delta\omega_x = 2\pi c_0 / (Ln_p)$ where c_0 is the speed of light and n_p is the average phase index of the structure.
- **omega_yrange** is similar to **omega_xrange**, where ω_y is used to represent the amplification of the wave and the amount of photons in a particular longitudinal mode. This parameter is used to define the range of the pole search in the ω_y direction. The units are given by the equivalent Fabry-Perot mode spacing,

i.e., $\Delta\omega_x = 2\pi c_0/(Ln_p)$ where c_0 is the speed of light and n_p is the average phase index of the structure.

- **srch_start** is the starting bias point of the mode search. This bias point must be below the threshold and produce a round trip gain of less than unity. In the case of the optical amplifier model, this is the starting point of the bias current.
- **rtg_start** is the starting point of the round trip gain beyond which the equation solver is activated. Ideally, the round trip gain should be unity under lasing conditions. However, due to the existence of spontaneous emission, the round trip gain can never be unity in practice, although it can be very close. The concept of threshold is also not well defined because the optical power increases continuously from zero. Therefore, the best we can do is to say that if the round trip gain is greater than a certain value, the optical power and spatial hole burning is significant and we should turn on the self-consistent equation solver. This parameter is used to specify this value.
- **tol_eqn** is the tolerance of longitudinal mode solver equations, which consists of RTG in complex omega plane and current injection equations.
- **tolerance** is the tolerance of relative error in all the variables involved in the self-consistent solution.
- **omega_ydamp** is a small number used to damp the fluctuation of photon density as a function of the imaginary part of the complex frequency ω . Since the photon density is inversely proportional to the imaginary part of ω or ω_y , the photon density could go to infinity if ω_y approaches zero. This damping parameter for ω_y limits how small ω_y can be during the iterations and how large the photon density can possibly be. Therefore, this parameter must be much smaller than the minimum ω_y but still be significantly greater than zero. Again, the units are the frequency spacing of the corresponding FP laser.
- **power_in** in Watts, is the initial condition of the input optical power at the right side of the facet. This parameter is used in the optical amplifier model only. The output power emits from the the left side of the facet.
- **delta_wavel** in meters, is the deviation of the input signal wavelength with respect to the reference wavelength given by the **longitudinal** statement. This parameter is only used for the optical amplifier model.
- **omega_xshift** in the same units of **omega_xrange**, controls the omega region within which the modes are to be searched. If the search range is too far away from the dominant mode, this parameter can be used to correct the search range.

- **sample_x** is the number of sample points in the ω_x direction that the solver must take in order to search for the poles of the Wronskian in the complex ω plane.
- **sample_y** is the number of sample points in the ω_y direction the solver must take in order to search for the poles of the Wronskian in the complex ω plane.
- **step_limit** is the step limit used by Newton's method in the self-consistent solution. The solution may diverge due to correction step overshoot. The step limit prevents such an overshoot. The units of this step limit are measured as a fraction of the norm (maximum step) of the correction vector.
- **current_scale** is used to scale the current injection equation.
- **omega_ymin** is the minimum value of imaginary part of the longitudinal mode solution (complex omega). This is used to prevent numerical overflow during iterations.
- **wavel_xrange** (in unit of meter) is the wavelength range within which longitudinal mode is searched. If this is defined, it will override **omega_xrange**.
- **iter_num** is the maximum iteration number used to reach convergence.
- **max_level** is the maximum level of reduced steps if non-convergence is encountered. Each level will reduce the step size by a half.

Example(s)

```
mode_srch omega_xrange=8.  omega_yrange=10.  &&
srch_start=50.e-3  sample_x=101 sample_y=81  &&
std_wave=no tolerance=1.e-6 iter_num=30 adjust_range=yes &&
threshold_srch=yes rtg_start=0.6 step_limit=0.03
```

```
mode_srch omega_xrange=8.  omega_yrange=10.  &&
srch_start=50.e-3  sample_x=101 sample_y=81  &&
std_wave=no tolerance=1.e-8 iter_num=30 adjust_range=yes &&
threshold_srch=yes rtg_start=0.6 global_newton=yes
```

2.282 `modify_bulk_macro`

parameter	data type	values [defaults]
variable	char	[void]
add_to_value	real	[0.0]
scale_value	real	1.0
mater	intg	1

`modify_bulk_macro` provides a convenient way of modifying an existing or previously defined bulk macro variable.

- **variable** takes one of the variables used in the bulk macro. It can any of the following which has been defined as one of the material commands:

band_gap
 affinity
 real_index
 electron_mass
 hole_mass
 dielectric_constant
 electron_mobility
 hole_mobility
 max_electron_mob
 max_hole_mob
 min_electron_mob
 min_hole_mob
 electron_ref_dens
 hole_ref_dens
 alpha_n
 alpha_p
 beta_n
 beta_p
 electron_sat_vel
 hole_sat_vel
 elec_carr_loss
 hole_carr_loss
 radiative_recomb
 auger_n
 auger_p
 tau_energy
 tau_energy_p
 absorption
 efield0_pf_elec

`efield0_pf_hole`
`norm_field`
`resistivity`
`spec_heat`
`mass_density`
`thermal_kappa`
`linear_heat`

- **add_to_value** would add a value to the parameter.
- **scale_value** would scale a value for the parameter.
- **mater** is the material index being affected.

```
modify_bulk_macro variable=affinity add_to_value=0.2
```

The above command would shift the affinity and thus the band alignment by 0.2 eV.

2.283 `modify_opt_gen_rate`

parameter	data type	values [defaults]
factor_file	char	[no] yes

modify_opt_gen_rate offers a way to modify the optical generation rate by a factor for different wavelengths.

- **factor_file** is the data file containing a 2-column data with first column being the wavelength in microns and the 2nd column being the factor to scale the optical generation rate profile.

```
modify_opt_gen_rate factor_file=my_scale_factor
```

2.284 `modify_plot`

parameter	data type	values [defaults]
<code>surface_bottom</code>	char	[yes]
<code>polarization_flag</code>	intg [1] 2	

`modify_plot` is a post-processor statement used to modify plots.

- **`surface_bottom`** affects surface plots to indicate whether a color bottom contour will be plotted to indicate different materials.
- **`polarization_flag`** sets TE/TM polarization if both are involved in a simulation.

Example(s)

```
modify_plot surface_bottom=no
```

The above statement suppresses the plotting of material region at the bottom of a surface plot.

2.285 modify_qw

parameter	data type	values [defaults]
confine_left	char	[no] yes
use_bulkmass	char	[no] yes
kp_pot_sym	char	[yes] no
gain_num_integr	char	[no]
all_active_mater	char	[yes]
active_macro	char	[void]
tail_dos_n	char	[no]
tail_dos_p	char	[no]
kp88_cond_parab	char	[no]
use_av_pmass	char	[yes]
remove_above_line	char	[no]
lower_bound_label	char	[void]
upper_bound_label	char	[void]
complex_start_label	char	[void]
mb_pf_model	char	[no] yes
subhh_levelk(k=1..5)	real	[0.0]
sublh_levelk(k=1..5)	real	[0.0]
subhh_mxfack(k=1..5)	real	[1.0]
sublh_mxfack(k=1..5)	real	[1.0]
left_mesh	real	(um)
right_mesh	real	(um)
more_cond_energy	real	[0.](eV)
more_val_energy	real	[0.](eV)
dip_factor_pump	real	[1.]
tail_energy	real	[0.](eV)
tail_state_scale	real	[1.]
tail_bulk_ref	real	[0.02]eV
mb_damping_factor	real	[1.0]
set_kp8x8_angle	real	
max_kp8x8_angle	real	[90.]
scale_mb_screen_length	real	[1.]
max_mb_screen_length	real	[1.e16] (um)
mb_pf_expo_factor	real	[1.]
mater	intg	
kp_points	intg	[35]
kp_bands	intg	[2]

modify_qw provides a convenient way of modifying an existing or previously defined

quantum well model. One may activate an existing quantum well model implemented in SOFTWARE and then modify one or two parameters to suit one's needs. This statement can be used in both the gain preview module or the main solution module.

- **confine_left** is used to handle the special case of unconfined quantum states. In a self-consistent model of a quantum well, it may happen that the external bias is large such that right side (or the top side if well plane is horizontal) of the barrier is lowered to form another potential well which is made possible by the infinite potential wall at the right-most (or top-most) boundary. Solving the wave equation in such a coupled two-well system may result in many states being confined to the secondary right-well. These states are unrealistic and exist only because we impose an infinite potential wall by forcing the wave function to be zero at the right-most (or top-most) boundary. Thus, we should filter out those unrealistic states while keeping the ones confined in the primary well on the left side (or lower side). The parameter **confine_left** is used to select the confined states in the primary well on the left. It is useful for quantum-MOS model when the gate bias is so large as to lower the oxide barrier potential to a level below that of the left barrier.
- **use_bulkmass** is to instruct the simulator to use bulk macro effective single-band mass when computing unconfined carrier density. We recall that the bulk macro provides effective mass for a single band model. For example, the heavy hole and light hole bands are combined to form an effective single valence band for convenience. On the other hand, the active layer macro for quantum well (or active region) computes the carrier density according to a multi-band model (with distinct HH and LH bands, for example). It is not always possible to reconcile the single-band and multi-band models. For example, if there is a split between the HH and LH levels as a result of strain, the equivalent single band model does not exist. In such a case, the simulator uses the multi-band model to override the single-band one for both confined and unconfined states within the active region. This may become a problem for the unconfined states since the model for the barrier region near a quantum well (in a self-consistent model, both well and barrier belong to the same active region) is different from that for the bulk region adjacent to it. We may see a discontinuity in background bulk (or unconfined) carrier density. Typically, if we plot the total carrier density, we may see the confined carrier density decays nicely to zero whereas there may be a discontinuity in the background bulk density. This behavior may destroy solution convergence causing frustration for users. The parameter **use_bulkmass** may be used to avoid such a situation. Please note that use of this parameter does not affect the discrete quantum states which are the most important in the active region.
- **kp_pot_sym** is used to indicate if symmetric potential is used to calculate the subband structure in the k.p theory.

- **gain_num_integr** is to indicate whether numerical integration is used to obtain the optical gain for the active region. By default, the free carrier optical gain is done by piece-wise analytical formulas. It is switched to numerical integration if many-body gain model is activated. One may choose to use numerical integration for the purpose of fair comparison with many-body gain. Also, in the limit of large **kp_points**, numerical integration produce the most accurate optical gain because direct integration is performed without any analytical fitting approximations.
- **all_active_mater** allows all active layers to use parameters in this statement, unless a specific material number is defined by **active_macro** or **mater**.
- **active_macro** indicates that this statement affects all active layers with the above active macro name.
- **tail_dos_n** assigns the DOS tail (due to inhomogeneous broadening) to the conduction band. This works with the **tail_energy** parameter.
- **tail_dos_p** assigns the DOS tail (due to inhomogeneous broadening) to the valence band. This works with the **tail_energy** parameter.

kp88_cond_parab sets parabolic conduction band for 8x8 valence mixing model. This parameter may be used to study the effect of non-parabolicity due to the conduction band.

use_av_pmass instructs the simulator to use effective mass with average weighted by the envelop wave function for the purpose of computing optical gain. This parameter will has significant effect if the barrier mass and well mass are substantially different.

- **remove_above_line** This parameter should be understood using a band diagram of quantum well with envelop function drawn at the confined energy levels. This parameter instructs the program to remove all quantum states which are centered above a line with connects the left-most and right-most points of the band diagram.
- **lower_bound_label**, if defined, is used by the program to set a lower boundary point in a 1-D potential distribution so that if the wave is localized outside of this lower limit, the state will be eliminated. This position label must be predefined in a .layer or .mater file.
- **upper_bound_label**, if defined, is used by the program to set an upper boundary point in a 1-D potential distribution so that if the wave is localized outside of this upper limit, the state will be eliminated. This position label must be predefined in a .layer or .mater

- **complex_start_label** must be defined if either or both of **lower_bound_label**, or **upper_bound_label** are used. It is a position label for the bottom of the first quantum well of the MQW-complex being studied.
- **mb_pf_model** would turn on the Poole-Frenkel field dependence for the exciton model.
- **subhh_level#** is the amount of modification made to the heavy hole subband level number. "level" is measured downwards from the top of the corresponding bulk valence band.
- **sublh_level#** is the amount of modification made to the light hole subband level number. "level" is measured downwards from the top of the corresponding bulk valence band.
- **subhh_mxfac#** is the factor multiplied to the in-plane heavy hole subband mass number.
- **sublh_mxfac#** is the factor multiplied to the in-plane light hole subband mass number.
- **tail_energy** is the parameter E_{tail} describing an exponential tail of the joint density of states (JDOS) extending into the energy bandgap. The JDOS near the bandgap is written as $JDOS = \rho_r \exp[(E - E_{edge})/E_{tail}]$, where E_{edge} is the energy of the band edge or the quantum subband edge, ρ_r is the reduced density of states at the band edge which is constant for a quantum well. This tail is associated with inhomogeneous broadening.
- **tail_bulk_ref** is for bulk active layer for which the reduced density of states is energy dependent. The exponential tail of the JDOS starts at a value (from the band edge) determined by this parameter.
- **tail_state_scale** may be used to scale the JDOS tail states described in the statement of **tail_energy** above.
- **left_mesh**, if defined, is the distance between the left zero-wave boundary and the left-most quantum well. This is used to define the left side of the range where the wave function is to be solved. By default, three times the quantum well width is used.
- **right_mesh** if defined, is the distance between the right zero-wave boundary and the right-most quantum well. This is used to define the right side of the range where the wave function is to be solved. By default, three times the quantum well width is used.

- **more_cond_energy** is used to extend the search of conduction subband energy levels up into the unbounded states. The densely populated energies in the extended unbound states can be used to treat optical transitions from bound-unbound states. Please note that the distribution of the unbound states depends on the boundary condition of the quantum mechanical wave equation. Only works when finite difference method is used for solving QW states (for non-symmetric QW, complex-MQW or self-consistent model).
- **more_val_energy** same as **more_cond_energy** except for valence band.
- **dip_factor_pump** is the dipole factor affecting only the optical pumping wavelength.
- **kp_points** determines how many points in the k-axis will be included in the k.p valence mixing model. A smaller **kp_points** value results in a faster run-time.
- **kp_bands** is number of band valleys (such as HH, LH, SO) used in the k.p theory for zinc-blende material system. The total order of the k.p matrix is twice such parameter, taking into account spin degeneracy. For example, **kp_bands**=2 means a 4x4 k.p theory taking into account HH and LH; For **kp_bands**=3, a 6x6 k.p theory taking into account HH, LH and SO band valleys; For **kp_bands**=4, an 8x8 k.p theory taking into account, Conduction band, HH, LH and SO bands.
- **mb_damping_factor** is used to control the degree of manybody effect contained in the gain/absorption spectrum. For a value of unity, 100 percent of manybody gain is used and a value of zero reduces the model to free-carrier gain. Please note that this parameter controls the magnitude and shape only. For example, the exciton peaks would be gone if the factor were set to zero. The shift in wavelength, or the bandgap renormalization is controlled by a separate parameter, the **scale_mb_screen_length**.
- **set_kp8x8_angle** is the angle of k-vector with respect to the k_x axis in the 8x8 k.p theory. If this parameter is not specified, the Hamiltonian is averaged over a range from zero to **max_kp8x8_angle** before it is solved.
- **scale_mb_screen_length** is used to scale the screening length in the many-body theory. This parameter is used to control the bandgap renormalization.
- **max_mb_screen_length** is the maximum screening length used in the many-body theory gain model.
- **mb_pf_expo_factor** is a Poole-Frenkel field dependence factor used to account field dependence in the exciton model. This model assumes that external field alters the coulomb potential of the exciton in much the same way as in Poole-Frenkel style ionization. The formula we use is as follows:

$$field - factor = \exp(-qE_{shift}/kT) \quad (2.42)$$

where

$$E_{shift} = \left[\frac{Fq}{\pi\epsilon_0\epsilon} \right]^{(P_f/2)} \quad (2.43)$$

The **mb_pf_expo_factor** is the parameter P_f above.

- **max_kp8x8_angle** is the maximum angle used in averaging the 8x8 k.p Hamiltonian. Please also see parameter **set_kp8x8_angle**.
- **mater** is the material number of the quantum well being modified.

```
modify_qw sublh_level1=-0.01 sublh_mxfac1=1.5
```

To introduce an exponential tail in the joint density of states for optical transition in the active layer, you may use the following

```
modify_qw tail_energy=0.06 tail_states_scale=1
```

2.286 modify_wurtzite

parameter	data type	values [defaults]
fit_mass	char	[no],yes
mass_model	char	[average],small_k,large_k

This statement can be used to modify the effective masses of the valence band of a wurtzite system. For a wurtzite material system, there are several ways to estimate the valence band effective masses.

If this statement is not used, the analytical mass model with mass values averaged over those of small-k and large-k ranges are used.

- **fit_mass** indicates whether parabolic fitted masses are used for the valence band. If not, analytical mass models are used as explained in the **mass_model** parameter.
- **mass_model** indicates which analytical mass model is to be used. For a value of "small_k", the zone center mass is used. **Caution:**, zone center masses can be negative in some material compositions and this may cause numerical trouble. For a value of "large_k", a large k-range is considered for the analytical model. "average" is an average of the above two cases.

Example(s)

```
modify_wurtzite fit_mass=no mass_model=average
```

2.287 monitor_emission

monitor_emission is used to plot the emitting light power in mesh points. It provides viewing the emission process for the emission model "mesh_points". This statement is used at the post-processing stage after ray-tracing program is run. This statement has no parameters.

2.288 modu_bias

parameter	data type	values [defaults]
section1	real	[1.]
section2	real	[1.]
section3	real	[1.]
section4	real	[1.]
section5	real	[1.]
n_sections	realxn	
sec_num	intg	[-1]

modu_bias is used to specify which section of the laser is modulated for the AM and FM responses.

- **section i** ($i=1,2,3,4,5$) gives the relative modulation bias current at section i . For example in a DBR laser, the DBR mirror sections are not modulated and should be set to zero for the **section i** parameter. The modulation bias for each section is relative to the other sections. The absolute value is not used. For example, **section1=1 section2=2** has the same effect as **section1=2 section2=4**. They both mean that section 2 has twice the modulation bias signal as section 1. It is also possible to use negative signs to indicate out of phase modulation of one section relative to the other.
- **n_sections** can be used to define the modulation bias in multiple sections in a single listing. The number n must match the parameter **sec_num**. This listing, if used, will override the parameters **section1**, **section2** etc.. It is used only when **sec_num** is greater than 1.
- **sec_num** should be equal to the number of sections being biased and should only be used if the number of sections is greater than 1. If it is less than or equal to 1, then **n_sections** is not used and the parameters **section1**, **section2**, should be used.

Example(s)

```
modu_bias section1=1 section2=3
```

```
modu_bias sec_num=3 3_sections=(1 3 2)
```

2.289 more_output

parameter	data type	values [defaults]
ac_data	char	[no] yes
qw_states	char	[no] yes
ac_light_input	char	[no] yes
elec_mobility	char	[no] yes
hole_mobility	char	[no] yes
trap_information	char	[no] yes
minimum	char	[no] yes
rcled_information	char	[no] yes
impact_ionization	char	[no] yes
organic_exciton	char	[no] yes
rtgain_scan	char	[no] yes
space_charge	char	[no] yes
elec_conc_subb	intg	[0]
hole_conc_subb	intg	[0]
cond_valley_prop	intg	[0]
val_valley_prop	intg	[0]
sum_space_charge_mater	intg	[0] 1 2..

The statement **more_output** is used to instruct the simulator to print more output data. It will affect all **scan** or **scan3d** statements after it. Thus, you may wish to place this statement in the beginning portion of the .sol file.

- **ac_data** is used to indicate whether AC analysis data are to be printed for later use by a .plt file. The data generated have file extension .ac_### and are required if AC analysis is to be performed as a post-processing step.
- **qw_states** enables the printing of quantum well subband wave functions. May only be used if self-consistent MQW option is activated.
- **ac_light_input** should be used if AC photo-response of a photo-sensitive device is required for post-processing. This parameter is to be used in conjunction with the parameter **ac_data**.

- **elec_mobility** enables the printing of electron mobility distribution averaged over current density.
- **hole_mobility** enables the printing of hole mobility distribution averaged over current density.
- **trap_information** enables the printing of trap information, such as trapped electrons and trap levels.
- **minimum** may be used save disk space by printing the minimum amount of data sufficient to keep the main solver running. However, many quantities may not be plotted.
- **rcled_information** enables the printing of RCLED related information, such as optical cavity standing power distribution.
- **impact_ionization** enables the printing of impact ionization (II) model related information, such as II rate distribution.
- **organic_exciton** enables the printing of organic exciton diffusion profile.
- **rtgain_scan** would enable the data output for longitudinal optical modes related to the coupled-round-trip-gain solution. Such data are usually referred by by variables names prefixed with **rtg_**.
- **space_charge** would enable the data output for space charge distribution.
- **elec_conc_subb** is the number of quantum well subbands for which electron concentrations of all band valleys are to be stored in output data.
- **hole_conc_subb** is the number of quantum well subbands for which hole concentrations of all band valleys are to be stored in output data.
- **cond_valley_prop** (number n) is the number of conduction band valley properties (such as valley effective mass) to be stored in output. This number will cause such valley properties to be stored in output as specified by statement **condk_valley_propj**, k=1,..., number of band valleys; j=1,...,n.
- **val_valley_prop** is the same as the above parameter except it is for valence band valley properties.
- **sum_space_charge_mater** would enable the data output for integrated net space chrage over a material given by this material index. If this parameter is zero, no such data are printed.

Example(s)

```
more_output ac_data=yes qw_states=no
```

2.290 multimode

parameter	data type	values [defaults]
sort_modepeak	char	[void]
sort_modespan	char	[void]
initialize_bpm	char	[no] yes
mode_num	intg	[1]
boundary_type1	intgx4	
boundary_type2	intgx4	
pick_xmode	intg	[0]
pick_ymode	intg	[1]
ymode_num	intg	[1]
polar_num	intg	[1]

The statement **multimode** is used to control the multi-lateral mode solutions. In general, the wave equation has two types of boundary conditions if the device is symmetric and only the right half is simulated. The two types are symmetric and anti-symmetric wave functions with respect to the axis of symmetry (usually located on the left hand side). Please see the definition of boundary conditions in the statement **init_wave**. If the whole device is simulated, only one type of boundary condition is sufficient.

If there are n modes required and there are two types of boundary conditions, the first $n/2$ modes are solutions obtained with the first type of boundary condition and the last $n/2$ modes are those with the second type.

Using the effective index method, the simulator is capable of solving for the optical power of more than one lateral (x-mode) mode. However only one transverse (y-mode) mode is selected.

Please note that by default, the optical modes are numbered according to the modal index, starting from number one for the highest index. Sometimes, it is desirable to order the modes according to their geometries or shapes.

- **sort_modepeak** may take **void**, **+x**, **+y** **-x** or **-y**. If not **void**, it is used to instruct the program to sort the lateral optical modes according to the positions of the mode peak. For example, a parameter value of **+x** will result in modes being sorted such that mode peaks are located increasingly towards the **+x** direction with increasing mode number.
- **sort_modespan** may take **void**, **+x**, **+y** **-x** or **-y**. If not **void**, it is used to instruct the program to sort the lateral optical modes according to the span (or width) of the mode. For example, a parameter value of **+x** will result in modes being sorted such that the mode half width in the x-direction increases with increasing mode number.

- **initialize_bpm** indicates whether a multiple mode solution is used to initialize a BPM calculation.
- **mode_num** is the number of lateral modes to be included in the simulation.
- **boundary_type1** is the first type of wave equation boundary. Its definition is identical to **boundary_type** in the the statement **init_wave**.
- **boundary_type2** is the first type of wave equation boundary. Its definition is identical to **boundary_type** in the the statement **init_wave**.
- If **pick_xmode** is zero, the program will solve all modes from mode number 1 to mode number **mode_num** (specified above) in the x-direction (lateral mode). If greater than zero, the program will solve only for mode number **pick_xmode**. For a device with multi-mode solution, it is recommended that all lateral modes be solved **pick_xmode=0**.
- **pick_ymode** must be a non-zero integer so that one mode in the transverse direction (y-direction) is selected for the simulator. Please note that the optical modes are numbered according to the modal index, starting from number one for the highest index.
- **ymode_num** is the number of y-modes.
- **polar_num** is number of polarizations (TE/TM) involved in the simulation.

Example(s)

```
multimode mode_num=6 boundary_type1=(2 1 1 1)  &&
      boundary_type2=(1 1 1 1)
```

In this example, six lateral modes are required. The device has a symmetric axis located on the left hand side (*i.e.* lx1, please see **init_wave**). The first three modes belong to symmetric (or even parity) modes of the first boundary type while the last three modes correspond to the second type with anti-symmetric (or odd parity) modes.

2.291 multimode_detect

parameter	data type	values [defaults]
detect_ratio	real	[1.]
mode_num	intg	[1]

The statement **multimode_detect** is used to specify the difference in power detection from different lateral modes. Usually, the laser power measurement equipment covers a limited angular range and higher order lateral modes will be detected with less amount of power.

- **detect_ratio** is a factor proportional to the power detected for a certain lateral mode.
- **mode_num** is the lateral mode index being affected.

Example(s)

```
multimode_detect front_back=0.7 mode_num=2
```

In this example, the 2nd order lateral mode detected is 70 percent of the fundamental mode which has a default detection factor of 1.

2.292 multimode_mirror

parameter	data type	values [defaults]
front_back	realx2	
mode_num	intg	[1]

The statement **multimode_mirror** is used to control the multi-lateral mode mirror reflectivity. Without this statement, mirror reflectivities are assumed to be the same as set by statement **init_wave**. It allows setting of different reflectivities for different lateral modes.

- **front_back** is used to define both front and back facet (or left and right) power reflectivity.
- **mode_num** is the lateral mode index being affected.

Example(s)

```
multimode_mirror front_back=(0.2 0.2) mode_num=2
```

In this example, the 2nd order lateral mode is set to experience a mirror reflectivity of 20 percent while the fundamental mode is still defined by statement **init_wave**.

2.293 new_doping

parameter	data type	values [defaults]
impurity	char	
charge_type	char	
shape	char	[gaussian]
pf_model	char	[no]
max_conc	real	
level	real	[0.01]
x_prof	realx4	[0. 1. 0.01 0.01]
y_prof	realx4	[0. 1. 0.01 0.01]
edge1_prof	realx3	[0. 0. 0.01]
edge2_prof	realx3	[1. 0. 0.01]
edge3_prof	realx3	[1. 1. 0.01]
edge4_prof	realx3	[0. 1. 0.01]
pf_cont_shift	real	[0.]
arc_height1	real	[0.]
arc_height2	real	[0.]
arc_height3	real	[0.]
arc_height4	real	[0.]
origin	real	[0.] (μm)
grading_from	real	(fraction)
grading_angle	real	[90.] (degrees)
pnt_num	intg	[4]

The statement **new_doping** identical to **doping** except that the maximum concentration can be modified during simulation by the .sol file.

This statement (without any parameters) can be used in .layer file to change doping into new_doping. In .layer file, this statement must be placed right after the corresponding layer_mater so that the layer and column numbers can be identified.

2.294 newton_par

parameter	data type	values [defaults]
search_gainpeak	char	[full]
extrapolate	char	[yes] no
change_variable	char	[no] yes
limit_elem	char	[no] yes
update_tunneling	char	[yes] no
update_mqw	char	[yes] no
update_lateral_mode	char	[yes] no
negative_stimulate	char	no[yes]
recover_prev_mqw	char	[no] yes
rescale_poisson	char	[yes] no
change_var_type	char	[both] n p
update_rc_extraction	char	[yes]
freeze_carrier	char	[void]electron,hole,both
damping_step	real	[2.e0]
var_tol	real	[1.e-5]
res_tol	real	[1.e-5]
min_lit	real	[0.]
rate_damp	real	[1.e-5]
scale_eqn_n	real	[1]
scale_eqn_p	real	[1]
impact_damp	real	[1.e-9]
var_fac	real	[1.e3]
min_density	real	[10.0]
huge_elem	real	[1.e10]
density_tol	real	[1.e16]
step_decrease	real	
step_increase	real	
max_iter	intg	[50]
opt_iter	intg	[15]
stop_iter	intg	[15]
print_flag	intg	1, [2], 3
loopback	intg	0
converge_test	intg	[2]
gainsearch_num	intg	[30]
iter_flag	intg	[0] 1 2
iter_step	intg	[5]
mf_solver	intg	0 1 [2] 3
mf_md_flag	intg	[1] 0
mf_pivot_update	intg	[64]
mf_cpu	intg	[1]

The statement **newton_par** defines a few parameters for the non-linear iterations in Newton's method. Note that the nonlinear variables are normalized to the order of 1-100.

The device simulation SOFTWARE is based on a large Newton solver. By default, the Fermi levels of electrons and holes are used as the variables of the Newton solver. The user may switch the variables from Fermi levels to carrier densities by setting **change_variable**. For many devices minority carrier Fermi level fluctuation in some areas with low current may cause convergence problems. Use of the carrier densities reduces the effects of those fluctuations and improves the convergence. The following devices may take advantage of this variable change: Buried heterostructure lasers with strong current blocking layers, JFET's and CCD's.

- **search_gainpeak** is used to decide how wavelength of the lateral mode is determined. If it takes "full", the wavelength is obtained from a search of the lateral modal gain peak fully evaluated over all mesh points. If it takes "one_point", the wavelength is obtained from a search of the material gain peak evaluated at a chosen mesh point. If it takes "long_mode", the wavelength is set at the maximum longitudinal mode. For PICS3D, "long_mode" is the default. "One_point" is usually more efficient since the program does not have to run through all mesh points but it is the least stable numerically.
- **extrapolate** turns on/off the solution extrapolation from previous converged Newton solutions. In most simulations, this should be turned set to yes. In low temperature simulation using carrier concentration as variables, the extrapolation should be set to no to avoid numerical overflow.
- **change_variable** is used to change the variables of the device simulator from Fermi levels to carrier densities.
- **limit_elem** is used to set a upper limit to the off-diagonal elements in the Jacobian matrix. If set to yes, the parameter **huge_elem** will be used as the upper limit. When using carrier densities as the variable of solution, the off-diagonal elements may be so large that the computer can not handle it and may give an overflow error (printed as NaN). In such a case, it may help to set this parameter.
- **update_tunneling** can be used to update tunneling model at every bias step.
- **update_mqw** can be used to update quantum well subband calculation model at every bias step.
- **update_lateral_mode** is used to update the scalar/vectorial eigen solver at every bias step. If lateral mode switches cause the main simulator to diverge,

one may wish to switch this off so that a more stable solution (with less self-consistency) may be obtained.

- **negative_stimulate** may be used to allow the stimulated recombination term to become negative. When the recombination term is negative, it should be understood as carrier generation.
- **recover_prev_mqw** is used to modify the behavior of the self-consistent quantum well solver when a bias step in **scan** fails. If it is set to “yes” the program solves and recovers the QW solution for the previous successful step to ensure that the previous convergence condition (including QW solution) be reproduced. This recovery action involves more numerical solutions and takes more computation time. Otherwise the simulator does not recover previous QW solution and simply reduces the bias step in hope for a convergence solution next. In general recovery action increase convergence and accuracy of the simulation but at a cost of more computation time.
- **rescale_poisson** indicates that whether the Poisson’s equation is to be scaled. This may affect converge.
- **change_var_type** is the type of variable change. This may affect the convergence of the solution.
- **update_rc_extraction** would updates the resonant cavity (as in RCLED) extraction coefficient at each bias step. This parameter may be used to control the smoothness of the light output as a function of bias.
- **freeze_carrier** would cause the selected carrier distribution in all materials to be freed (not changing with bias). Please note a related command **newton_freeze_carrier** which would freeze or unfreeze the selected carriers in a particular material.
- **damping_step** is the damping step used to limit the correction vector length.
- **var_tol** is the tolerance or lower limit of the variables beyond which the solution is considered converged.
- **res_tol** is the tolerance or lower limit of the Newton residuals beyond which the solution is considered converged.
- **min_lit** is the minimum normalized light power, used to prevent the solution of light power from accidentally approaching unphysical zero or negative values. It may be useful in transient simulations.
- **rate_damp** is the damping coefficient for the rate equation at steady state. The larger the coefficient, the more stable the solution to the rate equation, but the solution becomes less accurate. For transient simulation, no damping

is used. It is not recommended to use a large **rate_damp** if a steady state **scan** is to be connected to a transient **scan** because the large difference between the damped and undamped rate equations may cause convergence problems for the transient **scan**.

- **scale_eqn_n** or **scale_eqn_p** is the scaling factor for the electron or hole continuity equations.
- **impact_damp** is a small number used to control or damp the impact ionization term. The exponential impact ionization term tends to negative infinity if the local field is zero. This parameter is used to prevent such an infinity.
- **var_fac** is the variable factor used to normalize the carrier densities if **change_variable** is set to yes.
- **ratio_max** is the maximum ratio (or fraction) of change allowed for the carrier density during each Newton iteration. It is effective if **change_variable** is set to yes.
- **huge_elem** is the upper limit imposed on the off-diagonal matrix elements of the Jacobian if
- **density_tol** is a reference value representing a small carrier concentration ($1/m^3$) in the solution when using **change_variable=yes**. The program uses it to decide if the carrier concentration is to be considered small numerically.
- **step_decrease** takes a factor less than unity. It is a bias step reduction factor used when a bias step fails. By default, this factor is not used and the simulation program determines a factor based on the suggested **opt_iter** and the actual number of iterations.
- **step_increase** is a factor greater than unity. It is a bias step increase factor used when a previous bias is successfully converged. By default, this factor is not used and the simulation program determines a factor based on the suggested **opt_iter** and the actual number of iterations.
- **max_iter** is the maximum number of nonlinear iterations. If the required number of iterations is greater than this value, the solution is not found and the program will automatically reduce the bias step and start the nonlinear iteration again.
- **opt_iter** is an estimate of the optimal number of nonlinear iterations. The program adjusts its bias step by comparing the actual iteration number with this optimal number. If the actual number is greater, the program reduces the step. The next bias step is calculated as the present bias step times a ratio of **opt_iter** over present actual iteration number. The purpose is to force

the program to converge within `opt_iter` number of iterations. The method of automatic bias step adjustment is based on an observation that for a given type of device, the simulation program takes a minimum amount of computation time to reach a given bias if the bias steps are chosen such that the average number of non-linear iteration equals to this optimal number.

- **stop_iter** is the upper limit of the number of iterations if the solution error does not get reduced. When this number is reached and the error is still quite large, there is no point in continuing the iteration until **max_iter** is reached. It is wise to terminate the iteration early and start over again with a smaller bias step.
- **print_flag** controls how much information is printed during program execution.
- **loopback** is the number of loopback iteration used to update numerical equations (such as tunneling, optical wave, heat flow) decoupled from the main drift-diffusion solver after the normal updating procedure. Use of this parameter will increase the self-consistency but requires more computation time.
- **converge_test** enables a flexible convergence criteria. It takes integers from 1 to 4. The Newton procedure tries to solve a set of non-linear equations discretized from the related PDE's. It tries to reduce both the equation error (or residual) and the variable error (difference in subsequent iterations) to zero. For a value of "1", the Newton solver is regarded as converged if either the equation error or the variable error is reduced to below tolerance. For a value of "2", the Newton solver is regarded as converged if both the equation error and the variable error are reduced to below tolerance. For a value of "3", the Newton solver is regarded as converged only if the variable error is reduced to below tolerance. For a value of "4", the Newton solver is regarded as converged only if the equation error is reduced to below tolerance.
- **gainsearch_num** is the number of points used in search the lateral modal gain peak in a laser diode simulation. The simulator performs a uniform search of modal gain peak within the wavelength range specified by statement **init_wave**. Then it is followed by the finer search using golden rule. If this number is negative, the finer search will be skipped.
- **iter_step** is the number of GMRES iteration used if CG method is used as set by **mf_solver**.
- **mf_solver** is a linear solver flag related to the multi-frontal sparse matrix solver. It takes the values of 0, 1, 2 and 3. When taking the value of 0, conventional sparse matrix solver is used. When using 1, a combined GMRES iterative method is combined with the multi-frontal LU decomposition. For a flag of 2, multi-frontal LU decomposition is combined with a simple iterative

refinement method which is rather reliable but may be somewhat slower than GMRES. For value of 3, parallel computation technique is used to distribute the computation load of the linear sparse matrix to more than one CPU (see parameter **mf_cpu** below). Please note that the parallel method is helpful only for large mesh size since only the sparse matrix is parallelized.

- **mf_md_flag** takes 0 or 1, indicating different versions of minimum degree reordering method for the multi-frontal sparse solver.
- **mf_pivot_update** is the pivotal matrix size to update the F matrix from the C-matrix. For details, please see Ref: “An unsymmetric-pattern multifrontal method for sparse LU factorization”, T. A. Davis and I. S. Duff, Tech. Report TR-94-038, University of Florida, November, 1994.
- **mf_cpu** is the number of CPU’s when using parallelized computation technique to inverse the sparse matrix. This parameter is active when **mf_solver=3**.

Example(s)

```
newton_par  damping_step=2 var_tol=1.e-5 res_tol=1.e-4  &&
             max_iter=40  opt_iter =12 stop_iter=10  print_flag=2  &&
```

To take the defaults,

```
newton_par  print_flag=3
```

2.295 newton_freeze_carrier

parameter	data type	values [defaults]
freeze_electron_mater	intg	
freeze_hole_mater	intg	
unfreeze_electron_mater	intg	
unfreeze_hole_mater	intg	
freeze_both_mater	intg	

The usage of **newton_freeze_carrier** is similar to **newton_par** and it defines parameters for the non-linear iterations in Newton’s method. It should be used before the **scan** statement it intends to control.

- **freeze_electron_mater** would freeze the electron profile for the material it specifies.

- **freeze_hole_mater** would freeze the hole profile for the material it specifies.
- **unfreeze_electron_mater** would unfreeze the electron profile for the material it specifies.
- **unfreeze_hole_mater** would unfreeze the hole profile for the material it specifies.
- **freeze_both_mater** would freeze both the electron and hole profile for the material it specifies.

Example(s)

```
newton_freeze_carrier freeze_hole_mater=3
```

The above command would freeze the hole distribution for material number 3. This can be set in the middle of a simulation when hole profile is not important.

2.296 nonlocal_transp_region

parameter	data type	values [defaults]
region_label	char	[void]
point1_near	realx2	(um)
point2_near	realx2	(um)
distance1	real	[0.01](um)
distance2	real	[0.01](um)
ref_interface_mater	intgx2	
zseg_num	intg	[1]

nonlocal_transp_region is used to define a region across which non-local transport (such as tunneling) can occur from one side of the region to another, relative to a material interface. This is useful when such a region is not rectangle and when we wish to define a region relative to a irregular material interface. It is preferred that the material interface follows a line that is approximately straight so that the direction of the non-local transport can be certain. Please refer to Fig. 2.15 for details.

- **region_label** would tag the region for future reference.
- **point1_near**, **point2_near** are two reference points used to define exact starting and ending point of the material interface relative to which we wish to define the region. These points do not need to be exactly on the material interface. The program will search for the starting and ending mesh points on the interface that are closest to these two reference points, respectively.

- **distance1** is the perpendicular distance of the region boundary from the reference material interface. It is below the interface if the two reference points are on left and right sides respectively. It may take negative value to mean such a lower region boundary being placed above the interface.
- **distance2** is the perpendicular distance of the region boundary from the reference material interface. It is above the interface if the two reference points are on left and right sides respectively. It may take negative value to mean such an upper region boundary being placed on below the interface.
- **ref_interface_mater** denotes the two materials between which an interface line is defined. Such an interface line is used to define a region for non-local transport.
- **zseg_num** is the z-segment number for 3D simulation.

Example(s)

```
nonlocal_transp_region region_label=%junc1 &&
  point1_near=(0. 11.16) point2_near=(6. 11.16) ref_interface_mater=(3 4) &&
  distance1=0.019 distance2=-0.001
```

The above command would define a region some distance from a reference interface between material 3 and 4.

2.297 norm_field

parameter	data type	values [defaults]
(see) material_par		

The material parameter **norm_field** is added to the list of parameters in the material library. It is used if “n.gaas” is chosen as the type of field-dependent mobility, as the normalizing field (F_{0n}) (see the User’s Manual). It allows the user to specify more accurate electron mobility as a function of field in III-V or II-VI materials, where the Gamma to L and Gamma to X bands cause the velocity vs. field relation to exhibit a negative slope.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

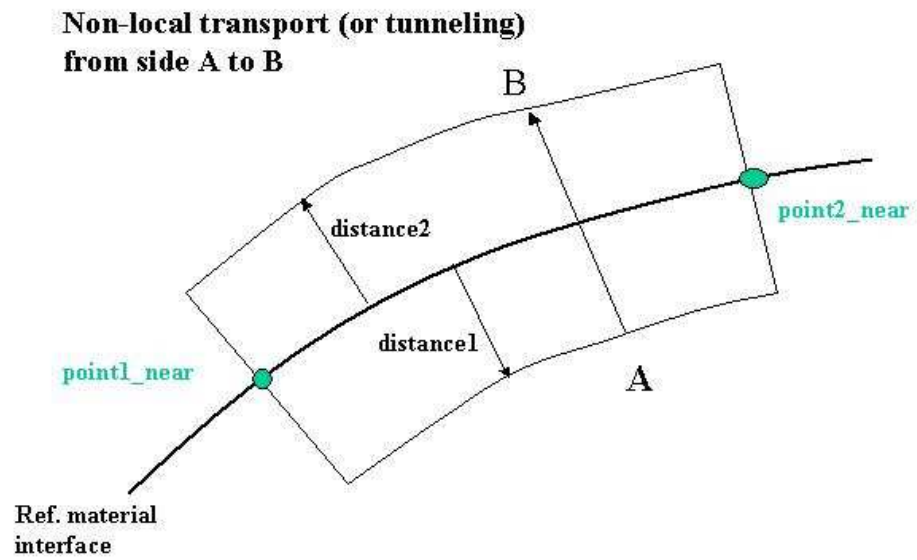


Figure 2.15: Schematic to explain the definition of a non-local transport region.

2.298 ohmic_junction

parameter	data type	values [defaults]
between_mater	intgx2	

The statement **ohmic_junction** may be used to set the current transport model across a heterojunction to be that of a ohmic type, i.e., the current is proportional to the carrier density times the gradient of quasi-Fermi levels. By default, thermionic emission model is used for all heterojunctions. We may call this model a Schottky type junction model because it is capable of rectifying the current by promoting it in the forward direction.

- **between_mater** is used to specify two material numbers representing two materials between which the heterojunction is forced to behave like a ohmic junction. Like in the usual drift-diffusion model, the current is expressed as proportional to the local carrier density times the gradient of the quasi-Fermi levels.

Example(s)

```
ohmic_junction between_mater=[1 2]
```

The above statement forces the heterojunction between materials 1 and 2 to behave like an ohmic junction. The material numbers can be found in the .mater files.

2.299 oled_control

parameter	data type	values [defaults]
wavelength	real	(μm)

The statement **oled_control** activates the organic LED (OLED) model. Its use is similar to statement **led_control**.

- **wavelength** is the estimated peak emission wavelength in microns.

Example(s):

```
oled_control wavelength=0.51
```

2.300 optic_coating

parameter	data type	values [defaults]
spectrum_file	char	[void]
thickness	real	[0.01] (μm)
real_index	real	[3.2] (μm)
imag_index	real	[0.] (μm)
segment_from	real	[-1.e7] (μm)
segment_to	real	[1.e7] (μm)
layer_num	intg	[1]
segment_num	intg	[1]

optic_coating specifies an insulating optical coating used before the incident light enters the semiconductor material. For example, if the light is incident from the top, the coatings are deposited on top of the semiconductor photo-sensitive device. The optical interference effect due to the optical coating can be computed by the simulation program.

Each of the this statement can be used to describe one segment of the coating within one layer. Repeated use of this statement can be used to define complex coating structures with many segment and many layers.

The layers are numbered from the starting point of the light path to the ending point. For example, if the light is incident from the top, then layer number 1 is the very first layer on the top, ie., the light hits layer number 1 first.

There can be different segments within one layer. For light traveling along $\pm y$ direction, the segment numbers from left to right. For light traveling along $\pm x$ direction, the segment numbers from bottom to top.

- **spectrum_file** is the index spectrum data, or the wavelength dependent complex index data file. If **spectrum_file**=void, no data file is specified. This statement may be useful if the incident light has a continuous distribution of light power (see also **light_power** statement) and if the material of the optical coating is wavelength dependent. The required data file format is as follows. It is a 3-column data with the first column being the wavelength in μm . The second and third columns are real and imaginary parts of the complex refractive indices (or the n and k in conventional notations), respectively.
- **thickness** is the thickness of the optical coating.
- **real_index** is the real part of the refractive index of the optical coating. This parameter is used only if **spectrum_file**=void.
- **imag_index** is the imaginary part of the refractive index of the optical coating. This parameter is used only if **spectrum_file**=void.

- **segment_from** is the starting point of this segment.
- **segment_to** is the ending point of this segment.
- **layer_num** is the layer number of this layer.
- **segment_num** is the segment number of this layer.

Example(s)

```
optic_coating  thickness=0.147  real_index=2.05  imag_index=0.
```

2.301 optical_field

parameter	data type	values [defaults]
profile	char	[gaussian], effective_index
y.data	char	[void]
update	char	no
standing_wave	char	no
uniform_stdwave	char	no
x_prof	realx4	x1,x2,dx1,dx2 (μm)
y_prof	realx4	y1,y2,dy1,dy2 (μm)
x_segment	intg	40

optical_field defines the solution of the optical field for the lateral mode of a waveguide. It must be used with the **multimode** statement which defines the number of lateral modes to be solved. If it is used in conjunction with **sor_par**, then the profile specified by this statement is treated as an initial guess of the SOR iterative solver. If the built-in SOR optical field solver is switched off (by setting **max_iter** of the **sor_par** statement to zero), the optical field used by PICS3D/LASTIP is the same as the one initialized by this statement.

- **profile** is the profile of the user-defined optical field. If **profile** takes the simple “gaussian” form, the optical field intensity profile in the x-direction is uniform from x1 to x2 and falls off on both sides with Gaussian tails of standard deviations of dx1 and dx2, respectively. The profile in the y-direction is similar. Such a profile can be used to define a uniform optical mode profile and different types of Gaussian optical field profiles; repeatedly use of this statement in small regions can create many different types of optical field profiles.

If **profile** is set to “effective_index”, the optical field is given by an effective index approximation with the x-direction as the slow varying direction. For the

present version of PICS3D, the built-in optical field solver may encounter non-convergence problems when the number of required modes is greater than 6. In such a case, the effective index method should be used. This means setting the maximum iteration number to zero for the **sor_par** statement and using **profile=effective_index**. Note that the effective index method is accurate for one dimensional (or quasi-1D) problems only and may have difficulties with a general 2D structure.

- **y.data** is the user-supplied data file for the optical field distribution in the y-direction. This allows the optical field in the y-direction to be imported from a data file instead of generated by the self-consistent optical field solver (which does not always work perfectly). For example, in a broad-area laser simulation, one may wish to import the optical field in the y-direction from a separate simulation if the optical field is difficult to obtain because of multi-lateral modes.
- **update** determines whether or not the initial optical mode specified by this statement is calculated or updated at every bias point.
- **x_prof** specifies the profile of the optical field intensity in the x-direction for a Gaussian optical field profile. It is not used if the “effective_index” method is chosen for **profile**.
- **y_prof** specifies the profile of the optical field intensity in the y-direction for a Gaussian optical field profile. It is not used if the “effective_index” method is chosen for **profile**.
- **standing_wave** is used to turn on the analytical model of using standing waves for the lateral modes. This provides the optical field distribution when the device is homogeneous in the lateral direction where in such a case, a standing wave pattern is expected in the lateral direction. This option is most useful in modeling broad area lasers where the lateral dimension is of the order of hundreds of microns.
- **uniform_stdwave** indicates whether a uniform magnitude is used in the x-direction. This parameter is effective when **standing_wave** is used.
- **x_segment** is the number of x-segments used to sample the wave solution the y-direction in the effective index method.

Example(s)

```
optical_field profile=effective_index update=yes
```

2.302 organic_exciton_diff

parameter	data type	values [defaults]
add_oxd_memory	real	[0.] (MB)
heteroj_scrn_factor	real	[0.1]
el_frac_non_active	real	[0.9]

organic_exciton_diff enables organic exciton diffusion model. The exciton diffusion across a heterojunction follows the thermionic emission model: excitons with thermal energy may emit from barrow bandgap to larger bandgap material. Similar to thermionic emission model in electrons, the following flux density is used for exciton (S) flowing from wider bandgap material 2 to material 1:

$$J = V_{th} \{ S_2 - S_1 \exp[-\gamma_s (E_{g2} - E_{g1}) / k_B T] \} \quad (2.44)$$

where V_{th} is thermal velocity of the exciton. γ_s is a screening factor less than unity. Since excitons are charge neutral, they should experience less crystal field potential than charged particles and thus the bandgap difference they experience should be less than electrons or holes. The screening factor is used to denote this effect.

- **add_oxd_memory** is used to add memory to the sparse solver for the diffusion equations.
- **heteroj_scrn_factor** is the heterojunction screening factor above.
- **el_frac_non_active** is the fraction of excitons in non-active (or undoped layers) that are eventually harvested by dopants.

Example(s):

```
organic_exciton_diff el_frac_non_active=0.85
```

The above statement directs 85 percent of excitons in undoped (non-active) layers be collected by dopants to emit light.

2.303 output

parameter	data type	values [defaults]
sol_outf	char	
zseg_num	intg	1

output defines the output data files.

- **sol_outf** is the main output data file.
- **zseg_num** defines the z-segment for PICS3D.

Example(s)

```
output sol_outf=bulk2d.out
```

2.304 outer_section

parameter	data type	values [defaults]
core_radius	real	(um)
sec_num	intg	[1]

outer_section is used to inform the program the termination of VCSEL section with an insulator. It is only used in **.vcsel** files.

- **core_radius** is the radius of core region, which indicates to the simulator that this VCSEL section is terminated by an insulator layer on the outside.
- **sec_num** is a section number.

Example(s)

```
outer_section core_radius=6.0 sec_num=7
```

2.305 oxd_diff_length

parameter	data type	values [defaults]
(see) material_par		

The material statement **oxd_diff_length** is used to define the exciton diffusion length in micron meters for organic material.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

Example(s)

```
oxd_diff_length value=1.e-2 mater=2
```

2.306 oxd_lifetime

parameter	data type	values [defaults]
(see) material_par		

The material statement **oxd_lifetime** is used to define the exciton diffusion life time in seconds for organic material.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
oxd_lifetime value=1.e-6 mater=2
```

2.307 oxd2_diff_length

parameter	data type	values [defaults]
(see) material_par		

The material statement **oxd2_diff_length** is used to define the diffusion length of the 2nd type (triplet) of exciton in micron meters for organic material.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
oxd2_diff_length value=1.e-2 mater=2
```

2.308 oxd2_lifetime

parameter	data type	values [defaults]
(see) material_par		

The material statement **oxd2_lifetime** is used to define the diffusion life time of the 2nd type of exciton (triplet) in seconds for organic material.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
oxd2_lifetime value=1.e-6 mater=2
```

2.309 ox_dopant_el_transfer

parameter	data type	values [defaults]
transfer_from_host	char	[no]
transfer_fraction	real	[0.]
efield_ref	real	[1.e8] (V/m)
mater_from	intg	[1]
mater_to	intg	[1]
dopant_index_from	intg	[1]
dopant_index_to	intg	[1]

ox_dopant_el_transfer is used to enable the transfer of exciton energy from one type of dopant to another type. This reveals itself in the shift of color from shorter to longer wavelengths.

The formula used a factor $factor = 1 - frac - F/F_r$ on the shorter wavelength dopant EL where *frac* is the transfer fraction. *F* is the electric field and F_r is a reference field. Similarly a factor $factor = 1 + frac + F/F_c$ is used for the longer wavelength EL.

- **transfer_from_host** would let the energy be transferred from the host instead of from a dopant.
- **transfer_fraction** is the fraction of exciton transferring from one type of dopant to another.
- **efield_ref** is the reference electrical field above.
- **mater_from** is the material index from which energy transfer originates.
- **mater_to** is the material index to which energy transfer arrives.
- **dopant_index_from** is the dopant index (for multiply dopants) from which energy transfer originates.

- `dopant_index_to` is the dopant index (for multiply dopants) to which energy transfer arrives.

Example(s):

```
ox_dopant_el_transfer mater_from=5 mater_to=4 &&
  transfer_fraction=0.2 efield_ref=-4.e8
```

2.310 ox_el_weight

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_el_weight** is used to define the electroluminescent spectrum weight for the host organic material. This is useful when there is dopant in the system so that one has to define the relative contribution from host and the dopant. For EL spectrum model when there is dopant, two approaches are used, depending on whether exciton diffusion equations are solved. If exciton transport is solved by **organic_exciton_diff**, dopant and host spectra at each mesh point are simply mixed by local exciton density, divided by their respective life times, and weighted by this factor. If exciton transport is not considered, their respective bimolecular recombination rates are used along with this factor to determine the relative contributions. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

Example(s)

```
ox_el_weight value=0.8 mater=1
```

2.311 ox_exciton_eg

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_exciton_eg** is the exciton bandgap of the organic material.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
ox_exciton_eg value=1.9 mater=2
```

2.312 ox_exciton_eff

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_exciton_eff** is the fraction (or efficiency) of exciton related electron-hole recombination (Poole-Frenkel like dependent model) that is converted to spontaneous emission optical power.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
ox_exciton_eff value=0.1 mater=1
```

2.313 ox_extern_spectrum

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_extern_spectrum** is used to define the electroluminescent (EL) spectrum of the host material imported from an external source. This may be based on experimental data or on a different type of model. If table format is used, the program expects a 2-column uniformly spaced data with first column being the wavelength in micron meters. The unit of the EL is arbitrary and the program will normalize it.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```

ox_extern_spectrum variation=table
table(wavelength)
  0.488524E+00  0.309522E-01
  0.499601E+00  0.421672E-01
  ...
end_table

```

2.314 ox_gaussian_sdj

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_gaussian_sdj** is the standard deviation (in unit eV) of the Gaussian function used to broaden the exciton transition energy. If more than one Gaussian function is used in different energy range, this parameter works with parameter **ox_gaussian_div(k=1...)** as follows: For example, the spectrum is broadened with N parameters: **ox_gaussian_sd(j=1...N)**. Then **ox_gaussian_sd(j)** acts on an energy range defined between **ox_gaussian_div(j-1)** and **ox_gaussian_div(j)**. Please note that **ox_gaussian_div(0)** equals zero and **ox_gaussian_div(N)** equals infinity.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

Example(s)

```
ox_gaussian_sd1 value=0.12 mater=2
```

2.315 ox_gaussian_divj

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_gaussian_divj** is the division in energy (in unit eV) for different Gaussian function to be used to broaden the exciton transition energy. If more than one Gaussian function is used in different energy range, this parameter works as follows: For example, the spectrum is broadened with N Gaussian function of parameters: **ox_gaussian_sd(j=1...N)**. Then **ox_gaussian_sd(j)** acts on an energy range defined between **ox_gaussian_div(j-1)** and **ox_gaussian_div(j)**.

Please note that **ox_gaussian_div(0)** equals zero and **ox_gaussian_div(N)** equals infinity.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
ox_gaussian_div1 value=2.21 mater=2
```

2.316 ox_hopping_energy

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_hopping_energy** is the hopping integral (in units of the vibrational quanta) describing the nearest-neighbor interaction of the exciton (as quasiparticle) in a tight-binding model.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
ox_hopping_energy value=0.6 mater=1
```

2.317 ox_life_field_dependence

parameter	data type	values [defaults]
increase	char	[yes]
triplet	char	[yes]
pf_lifetime_expo	real	[0.5]
efield0_pf	real	[2.e8] (V/m)
mater	intg	[1]

ox_life_field_dependence is used to enable the organic exciton diffusion life time field dependence model. As in many other field-dependence (such as mobility) model for organic semiconductor, we assume a Poole-Frenkel type of field dependence here for the life time:

$$\tau_{ox} = \tau_0 \exp[(\text{sign})(F/F_{t0})^\alpha] \quad (2.45)$$

where (sign) takes positive or negative unity.

- **increase** determines whether to choose plus or negative sign for the (sign) above.
- **triplet** indicates whether the model applies to singlet or triplet diffusion.
- **pf_lifetime_expo** is the exponent α in the above formula.
- **efield0_pf** is the threshold field F_{t0} in above formula.
- **mater** is the material number affected by this statement.

Example(s):

```
$ field-dep lifetime needed to tune blue color change
ox_life_field_dependence increase=yes triplet=no &&
  pf_lifetime_expo=0.5 efield0_pf=1.e8 mater=2
```

2.318 ox_peak_abs

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_peak_abs** is the absorption peak in units of 1/m of the organic material.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
ox_peak_abs value=1.e7 mater=2
```

2.319 ox_vib_quanta

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_vib_quanta** is the organic molecular vibrational quanta, or phonon energy unit, in eV.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

Example(s)

```
ox_vib_quanta value=0.16 mater=1
```

2.320 ox_xp_coupling

parameter	data type	values [defaults]
(see) material_par		

The material statement **ox_xp_coupling** is the exciton-phonon (XP) coupling coefficient. It is a dimensionless number with its square equal to the vibrational relaxation energy (Franck-Cordon energy) divided by the vibrational quanta (or molecular phonon quanta).

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

Example(s)

```
ox_xp_coupling value=1.0 mater=1
```

2.321 para_extract

parameter	data type	values [defaults]
file	char	[extract.txt]
append_data	char	[no]
type	char	[intercept]
fit_hori_from	real	[-1.e49]
fit_hori_to	real	[1.e49]
fit_vert_from	real	[-1.e49]
fit_vert_to	real	[1.e49]
vert_intercept	real	
hori_intercept	real	
xy_hori_from	real	[-1.e49]
xy_hori_to	real	[1.e49]
xy_vert_from	real	[-1.e49]
xy_vert_to	real	[1.e49]
xy_product_scale	real	1.
add_vert_intercept	real	[0.]
add_hori_intercept	real	[0.]
scale_fit_slope	real	[1.]
scale_fit_inv_slope	real	[1.]

para_extract is used in .plt file in order to extract data curve characteristics generated by the **plot_scan** statement.

This statement only extracts parameters from the **plot_scan** statement immediately following it. It has no effect on other statements.

- **file** is the file name to which extracted parameters will be written in ASCII text format.
- **append_data** indicates whether the extracted parameters are to be appended to or to override the ASCII data file specified in the **file** parameter.
- **type** is the type of extraction action. It takes one of the following. Please note that x/y here really means horizontal and vertical axis, not spatial coordinates.
 - intercept - intercepts x/y axis or a vertical/horizontal line.
 - fit_line - fits a set of data to a straight line.
 - xy_product_search - searches for an interpolated data point with maximum product of $x*y$.
 - hori_max_search - searches for a data point with maximum x-value.
 - vert_max_search - searches for a data point with maximum y-value.
- **fit_hori_from** is the starting range of the data to be fitted to a straight line. It affects the data in the horizontal axis direction.
- **fit_hori_to** is the ending range of the data to be fitted to a straight line. It affects data in the horizontal axis direction.
- **fit_vert_from** is the starting range of the data to be fitted to a straight line. It affects the data in the vertical axis direction.
- **fit_vert_to** is the ending range of the data to be fitted to a straight line. It affects data in the vertical axis direction.
- **vert_intercept** finds the intercept of a vertical line with the data curve. The horizontal axis value of this vertical line is given by this parameter.
- **hori_intercept** finds the intercept of a horizontal line with the data curve. The vertical axis value of this horizontal line is given by this parameter.
- **xy_hori_from**, **xy_hori_to**, **xy_vert_from**, **xy_vert_to** are used to define a searching range in the vertical and horizontal directions, respectively.
- **xy_product_scale** would scale the maximum $x*y$ product found from the extraction.

- **add_vert_intercept** would add to the vertical line intercept result.
- **add_hori_intercept** would add to the horizontal line intercept result.
- **scale_fit_slope** would scale the fitted slope.
- **scale_fit_inv_slope** would scale the fitted inverse slope.

Example(s):

```
$ Let us extract threshold voltage by taking interception of
$ Log(I_ds)-V_gs curve at -1 (or 0.1 A/m). Output is written in file
$ extract.txt
para_extract file=extract.txt hori_intercept=-1.
plot_scan scan_var=voltage_2 variable=total_curr_3 scanline=3 &&
  merge_next=no scale_2=log
```

Example(s):

```
$ The following statement fits the L-I curve above 1 mW to a straight-line
$ so that threshold current can be found as intercept with horizontal
$ axis. Also, it finds the drive current at 8 mW. Results of parameter
$ extract is written to file extract.txt by default.
```

```
para_extract type=fit_line fit_vert_from=1 &&
  hori_intercept=8.
plot_scan scan_var=current_1 variable=laser_power
```

Example(s):

```
$ The following command finds the maximum power from an I-V curve:
```

```
para_extract type=xy_product_search xy_hori_from=0 xy_vert_from=0
plot_scan scan_var=voltagec_1 variable=current_2
```


2.322 passive_3d

parameter	data type	values [defaults]
index	real	
intern_loss	real	(1/m)
carrier_conc	real	(1/m ³)
sec_num	intg	

passive_3d: In a 3D simulation, the longitudinal sections may be classified as active or passive (non-active) sections. Active sections usually provide the optical gain needed by the laser to operate. The passive sections are used for tuning or other purposes. The **passive_3d** statement is only used for the edge emitting lasers.

- **index** is the refractive index of the passive section.
- **intern_loss** is the internal loss of the passive section.
- **carrier_conc** is for an edge type of waveguide (to be differentiated with VCSEL) containing a passive section within a z-segment (meaning there is at least one active section within the same z-segment), the program sets this value as the solution variable of carrier concentration. This is purely for plotting purpose since the program does not actually solve for the carrier concentration in a passive edge section.
- **sec_num** is the section number (or section index) of the corresponding section.

2.323 passive_carr_loss

parameter	data type	values [defaults]
ncarr_loss	real	[0] m ²
pcarr_loss	real	[0] m ²
mater	intg	[1]

passive_carr_loss: Used to specify the free carrier absorption in regions other than active regions in a laser device. The free carrier loss is expressed as follows:

$$< \text{free carrier loss} > = < \text{ncarr_loss} > < n \text{ carrier} > + < \text{pcarr_loss} > < p \text{ carrier} >$$

- **ncarr_loss** is the coefficient of free carrier absorption for electrons.
 - **pcarr_loss** is the coefficient of free carrier absorption for holes.
- mater** is the material number.

Example(s)

```
passive_carr_loss ncarr_loss=2.e-21 pcarr_loss=1.e-21 mater=1
```

2.324 passive_fiber

parameter	data type	values [defaults]
intern_loss	real	[5.e-5] (1/m)
index_core	real	[1.8]
index_cladding	real	[1.3]
core_radius	real	[8.] (um)
group_index	real	[1.48]
sec_num	intg	[1]
bessel_order	intg	[0]

passive_fiber may be regarded as a special case of **passive_3d**. In addition, fiber scalar modes are computed within the fiber section.

- **intern_loss** is the internal loss of the fiber section.
- **index_core** is the core index of the fiber.
- **index_cladding** is the cladding index of the fiber.
- **core_radius** is the core radius of the fiber.
- **group_index** is the group index of the fiber.
- **sec_num** is the section number of the fiber section.
- **bessel_order** is the Bessel function order used to represent the scalar lateral modes within the fiber section. The fundamental mode order is zero.

Example(s)

```
passive_fiber intern_loss=4.6e-5 sec_num=4 &&
  index_core=1.5 index_cladding=1.496 core_radius=9.
```

2.325 pc_led_model

parameter	data type	values [defaults]
top_emission	char	[yes]
surface_y_label	char	[void]
rectangle	char	[no]
surface_y	real	(um)
grating_height	real	[0.1](um)
grating_dia_ratio	real	[0.2](dia/period)
x_range	realx2	(-9999. 9999.)(um)
height_range	realx2	[0.1 1.](um)
surface_mater	intg	[1]
model_points	intg	[10]

The statement **pc_led_model** is used to define model parameters for photonic crystal LED (PhCLED) model which is treated as a post-processor module.

- **top_emission** indicates whether the LED emits from the top.
- **surface_y_label** is a position label for the material interface of the (air/semiconductor) LED.
- **rectangle** indicates whether the air holes of the PhC is rectangle or hexagonal.
- **surface_y** is the absolute y-coordinate for the material interface of the (air/semiconductor) LED.
- **grating_height** is the height of the air-hole grating.
- **grating_dia_ratio** this ratio of the air-hole diameter over the period (or the real space lattice vector constant) of the photonic crystal.
- **x_range** is range in x over which the current spreading effect is taken into account for the calculation of distribution of extracted power emission.
- **height_range** is the range of air-hole height over which the LED emission properties are to be evaluated.
- **surface_mater** is the semiconductor material number over which the air-holes of the photonic crystal are to be made.
- **model_points** is the number of spacing divisions over the **x_range** for the calculation of spatial variation of LED properties.

Example(s):

```
pc_led_model top_emission=yes surface_y_label=air_hole &&
  grating_height=0.3 grating_dia_ratio=0.5 surface_mater=1 &&
  model_points=20 x_range=(50 200)
```

2.326 plot_1d

parameter	data type	values [defaults]
data_file	char	[vttek]
variable	char	(see list)
n_variables	charxn	
integration	char	[no]
x_from_label	char	[void]
y_from_label	char	[void]
x_to_label	char	[void]
y_to_label	char	[void]
math_oper	char	[void]
from	realx2	
to	realx2	
xrange	realx2	
yrange	realx2	
integration_start	real	[0.0](um)
integration_length	real	[1.e5](um)
qw_wave_ht	real	[0.1] (eV)
var_num	intg	[1]
mode_index	intg	[1]
trap_index	intg	[1]
qw_wave	intg	[0] 1
cond_subband	intg	[1] 2
val_subband	intg	[1] 2 3

plot_1d is a post-processor statement used to plot a one dimensional segment of the structural data generated by the main LASTIP solver.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **variable** is the physical quantity to be plotted. When the variable “wave_intensity” is used, the optical confinement factor is also printed with the graphical output. The definition of physical quantities is clear in most cases. For trapped_elec_conc, it means the trapped electron concentration. Detailed listing is given in App. C.
- **n_variables** is a group of variables to be plotted.

- **from** is the starting point of the 1D section.
- **to** is the ending point of the 1D section.
- **integration** indicates whether integration is performed for the physical variable being plotted. If "yes", the results of the integration will be printed in .plt.msg file.
- **x_from_label**, **y_from_label**, **x_to_label**, and **y_to_label** are used to define two points just like **from** and **to** above. The labels must be predefined using **x_position**, **y_position** or **define_vertical_position** command in the .sol file. They may be generated using the .layer file relative to a specific layer so that there is no need to find out the absolute coordinates.
- **math_oper** if defined, would allow the variable being plotted to be operated by math operator like log10 or power10.
- **xrange** specifies a range for the abscissa.
- **yrange** specifies a range for the ordinate.
- **integration_start** is the start point of the integration if **integration** is set.
- **integration_length** is the length of the integration if **integration** is set.
- **qw_wave_ht** is the height of the quantum wave amplitude as plotted on the band diagram.
- **var_num** is the number of variables to be plotted. **var_num** must be specified if more than one variable is to be plotted.
- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.
- **qw_wave** takes zero or one, is used to indicate whether squared quantum subband wave function amplitude is to be plotted on the band diagram.
- **cond_subband** is used when **qw_wave**=1 to indicate which conduction band is being plotted. Values 1 and 2 are used to represent Gamma and L bands, respectively.
- **val_subband** is used when **qw_wave**=1 to indicate which valence band is being plotted. Values 1, 2 and 3 are used to represent HH, LH and CH bands, respectively.

Example(s)

```
plot_1d variable=elec_conc from=(0.5, 0.) to=(0.5, 1.9)
```

```
plot_1d 2_variables=(elec_conc hole_conc) var_num=2 &&
from=(0.5, 0.) to=(0.5, 1.9)
```

```
plot_1d variable=recomb_aug from=(0.5 0.) to=(0.5, 3.)
```

2.327 plot_ac_curr

parameter	data type	values [defaults]
data_file	char	[void]
variable	char	[void]
imag_part	char	[no]
input_file	char	[void]
from	realx2	
to	realx2	

plot_1d_ac_curr is used to plot the AC current distribution in a 1D slice. It works with the statement of **ac_voltage** assuming a unit AC voltage signal in the input electrode.

- **data_file** is the file to which the graphic data is written in ASCII format.
 - **input_file** is the input file containing the AC analysis results to be plotted.
 - **variable** is the variable being plotted. Its value takes the format “type_curr_d”, where type may be one of “elec”, “hole”, “disp” or “total”, standing for electron, hole, displacement, and total current components. “d” can take “x” or “y” for directions.
 - **imag_part** indicates whether imaginary part of the current is plotted.
- from** is the starting point of the 1D slice.
- to** is the starting point of the 1D slice.

Examples

```
plot_1d_ac_curr variable=elec_curr_y imag_part=no input_file=test.ac &&
from=(0.5 0.) to=(0.5 2.5)
```

The above statement plots 1D distribution of real part of AC current from test.ac file.

2.328 plot_2d

parameter	data type	values [defaults]
variable	char	(see list)
data_file	char	[vttek]
mater_boundary	char	[no]yes
point_ll	realx2	(μm)
point_ur	realx2	(μm)
xrange	realx2	
yrange	realx2	
maxvector_scale	real	[1.]
grid_sizes	intgx2	[20, 20]
level	intg	[10]
mode_index	intg	[1]
trap_index	intg	[1]

plot_2d is a post-processor statement used to plot two-dimensional vector flow diagram (or arrow diagram) or contour plot.

- **variable** is the variable to be plotted. It can be a scalar variable plotted as contours or vectorial variable plotted as arrows representing the relative intensity and direction of the corresponding vector as a function of location. The scalar variables are listed in App. C. The vectorial variables are given in App. B.
- **data_file** is the file to which the graphic data is written in ASCII format.
- **maxvector_scale** is used to scale the maximum length of a 2D vector field. A vector field is plotted by drawing only the vectors of length between the minimum and maximum vector length. Vectors of lengths below the minimum are not plotted. Vectors of lengths above the maximum are plotted with the same vector size. Increasing this parameter generates a plot with many small size vectors. Decreasing this value magnifies the smaller vectors while leaving the large ones with the same size.
- **mater_boundary** is used to indicate if material boundary is plotted.
- **point_ll** is the lower-left corner point of the 2D plotting window.
- **point_ur** is the upper-right corner point of the 2D plotting window.
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.

- **grid_sizes** are the grid sizes of the 2D plot (not to be confused with mesh used in simulation). To generate the vector plot (or flow plot), the vector of the physical quantity must be evaluated at a given **grid_size**.
- **level** is the number of function division levels to be used in the contour plot.
- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.

Example(s)

```
plot_2d variable=total_cur xrange=(0. 1.5) yrange=(0. 3.)
```

2.329 plot_3d

parameter	data type	values [defaults]
variable	char	(see list)
data_file	char	[vttek]
integration	char	[no]yes
point_ll	realx2	(μm)
point_ur	realx2	(μm)
view_xrot	real	[0.]
view_zrot	real	[0.]
xrange	realx2	
yrange	realx2	
zrange	realx2	
integration_xrange	realx2	[-1.e4 1.e4](μm)
integration_yrange	realx2	[-1.e4 1.e4](μm)
grid_sizes	intgx2	[20, 20]
mode_index	intg	[1]
trap_index	intg	[1]

plot_3d is a post-processor statement used to plot two-dimensional scalar variable data in a 3D view.

- **variable** is the variable to be plotted. The variable listing is given in App. C.
- **data_file** is the file to which the graphic data is written in ASCII format.
- **integration** indicates whether integration is performed for the physical variable being plotted. If "yes", the results of the integration will be printed in .plt.msg file.
- **point_ll** is the lower-left corner point of the 2D window.
- **point_ur** is the upper-right corner point of the 2D window.
- **view_xrot** rotates the plot around the x-axis.
- **view_zrot** rotates the plot around the z-axis.
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.
- **zrange** specifies a range for the z-axis.
- **integration_xrange** specifies a x-range for numerical integration. (see also integration parameter above).
- **integration_yrange** specifies a y-range for numerical integration. (see also integration parameter above).
- **grid_sizes** are the grid sizes for the 3D plot.
- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.

Example(s)

```
plot_3d variable=hole_conc point_ll=(0., 0.) &&  
point_ur=(2., 3.)
```

2.330 `plot_3dcolor`

parameter	data type	values [defaults]
<code>data_file</code>	char	[vttek]
<code>connect_planes</code>	char	[yes]
<code>variable</code>	char	(see list)
<code>view_xrot</code>	real	[0.]
<code>view_zrot</code>	real	[0.]
<code>xrange</code>	realx2	
<code>yrange</code>	realx2	
<code>zrange</code>	realx2	
<code>mode_index</code>	intg	[1]
<code>trap_index</code>	intg	[1]

plot_3dcolor is a post-processor statement used to plot 3-dimensional scalar variable data in a 3D color view.

- **variable** is the variable to be plotted. The variable listing is given in App. C.
- **data_file** is the file to which the graphic data is written in ASCII format.
- **connect_planes** connects the color planes.
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.
- **zrange** specifies a range for the z-axis.
- **view_xrot** rotates the plot around the x-axis.
- **view_zrot** rotates the plot around the z-axis.
- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.

Example(s)

```
plot_3dcolor variable=hole_conc
```

2.331 plot_3dmesh

parameter	data type	values [defaults]
data_file	char	[void]
connect_planes	char	[yes]
xrange	realx2	
yrange	realx2	
zrange	realx2	
view_xrot	real	[0.]
view_zrot	real	[0.]

plot_3dmesh is a post-processor statement used to plot 3-dimensional mesh.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **connect_planes** connects the planes with mesh lines.
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.
- **zrange** specifies a range for the z-axis.
- **view_xrot** rotates the plot around the x-axis.
- **view_zrot** rotates the plot around the z-axis.

Example(s)

```
plot_3dmesh  connect_planes=yes
```

2.332 `plot_ac_curr`

parameter	data type	values [defaults]
<code>data_file</code>	char	[void]
<code>input_file</code>	char	[void]
<code>scale_2</code>	char	[linear] log
<code>variable</code>	char	[void]
<code>input_f2</code>	char	[void]
<code>n_variables</code>	charxn	
<code>xrange</code>	realx2	
<code>yrange</code>	realx2	
<code>factor_vertical</code>	real	1
<code>var_num</code>	intg	[1]

`plot_ac_curr` is used to plot the AC current responses and related AC characteristics produced by the AC voltage bias defined in **`ac_voltage`**.

- **`data_file`** is the file to which the graphic data is written in ASCII format.
- **`input_file`** is the input file containing the AC analysis results to be plotted.
- **`scale_2`** is used to control the scale of the AC characteristics being plotted (the ordinate).
- **`factor_vertical`** is a factor scaling the vertical axis of the plot.
- **`variable`** is the variable being plotted. It is defined as follows:

Variable	definition	Unit
n_real_k (k=1,2,...)	Real part of electron AC current at kth electrode in response to one voltage of AC bias.	Amp/m
n_imag_k (k=1,2,...)	Imaginary part of electron AC current at kth electrode in response to one voltage of AC bias.	Amp/m
p_real_k (k=1,2,...)	Real part of hole AC current at kth electrode in response to one voltage of AC bias.	Amp/m
p_imag_k (k=1,2,...)	Imaginary part of hole AC current at kth electrode in response to one voltage of AC bias.	Amp/m
d_real_k (k=1,2,...)	Real part of displacement AC current at kth electrode in response to one voltage of AC bias.	Amp/m
d_imag_k (k=1,2,...)	Imaginary part of displacement AC current at kth electrode in response to one voltage of AC bias.	Amp/m
t_real_k (k=1,2,...)	Real part of total (electron + hole + displacement) AC current at kth electrode in response to one voltage of AC bias.	Amp/m
t_imag_k (k=1,2,...)	Imaginary part of total (electron + hole + displacement) AC current at kth electrode in response to one voltage of AC bias.	Amp/m

Variable	definition	Unit
n_magn_k (k=1,2,...)	Magnitude of electron AC current at kth electrode in response to one voltage of AC bias.	Amp/m
p_magn_k (k=1,2,...)	Magnitude of hole AC current at kth electrode in response to one voltage of AC bias.	Amp/m
d_magn_k (k=1,2,...)	Magnitude of displacement AC current at kth electrode in response to one voltage of AC bias.	Amp/m
t_magn_k (k=1,2,...)	Magnitude of total (electron + hole + displacement) AC current at kth electrode in response to one voltage of AC bias.	Amp/m
ratio_ik/ij (k=1,2,...) (j=1,2,...)	Ratio of magnitudes of AC total current at kth electrode over that at jth electrode in response to one voltage of AC bias.	Unitless
ratio2_ik/ij (k=1,2,...) (j=1,2,...)	Ratio of magnitudes of AC total current at kth electrode in response to one voltage of AC bias applied at one electrode with data from input_file , over that at jth electrode in response to one voltage of AC bias applied at another electrode with data from input_f2 . To generate two AC data input files, you must use ac_voltage twice for AC biases at two different electrodes, respectively.	Unitless
conductance_k (k=1,2,...)	Conductance at kth electrode in response to one voltage of AC bias.	Siemens/m
capacitance_k (k=1,2,...)	Capacitance at kth electrode in response to one voltage of AC bias.	10^{-9} F/m

Please note that for cylindrical coordinate system, the units must be modified. For example, Amp/m becomes Amp.

- **input_f2** is the 2nd AC input data file when ratio2_ik/ij is chosen for **variable**.
- **n_variables** may be used to list more than one variable so that more than one variable may be plotted on the same graph.
- **xrange** specifies a range for the abscissa.
- **yrange** specifies a range for the ordinate.
- **var_num** is the number of variables being plotted and must match the prefix in **n_variables** if used.

Examples

```
plot_ac_curr input_file=pmosB.ac variable=capacitance_3
plot_ac_curr input_file=pmosB.ac variable=conductance_3
```

The above two statements are used to plot capacitance and conductance at electrode 3 in response to the AC voltage defined in **ac_voltage**.

```
plot_ac_curr input_file=pmosB.ac scale_2=log variable=ratio_i3/i4
```

The above statement plots the current ratio at electrode 3 over that at electrode 4 in response to the AC voltage defined in **ac_voltage**.

Suppose we use the following two statements to apply AC voltage to two different electrodes:

```
ac_voltage output_file=pmosB.ac log_freq1=6. log_freq2=12. &&
    contact_num=4 freq_point=20
ac_voltage output_file=pmosB.ac2 log_freq1=6. log_freq2=12. &&
    contact_num=3 freq_point=20
```

Then we can use the following statement to plot the current ratio at the same electrode due to AC voltage biases at two different electrodes:

```
plot_ac_curr input_file=pmosB.ac scale_2=log variable=ratio2_i3/i3 &&
    input_f2=pmosB.ac2
end_pstprc
```

This is just the AC voltage ratio or the AC voltage amplification.

2.333 plot_ac_laser

parameter	data type	values [defaults]
data_file	char	[void]
input_file	char	[void]
facet	char	[void] front, back
xrange	realx2	
yrange	realx2	

plot_ac_laser is used to plot the power response of a laser diode.

- **data_file** is the file to which the graphic data is written in ASCII format.

- **input_file** is the input file containing the AC analysis results to be plotted. If this is not defined, an internal data filename will be assigned.
- **facet**, if specified, is the facet where the power is calculated. If not specified, the total power will be computed.
- **xrange** specifies a range for the abscissa.
- **yrange** specifies a range for the ordinate.

Examples

```
plot_ac_laser facet=front
```

The above statement plots frequency response of front facet power.

2.334 plot_ac_modal_gain

parameter	data type	values [defaults]
data_file	char	[void]
input_file	char	[void]
xrange	realx2	
yrange	realx2	

plot_ac_modal_gain is used to plot the modal gain of a wave guided device. Here modal gain is defined local material gain averaged over optical power distribution of all lateral modes, assuming no interference effects between different modes.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **input_file** is the input file containing the AC analysis results to be plotted. If this is not defined, an internal data filename will be assigned.
- **facet**, if specified, is the facet where the power is calculated. If not specified, the total power will be computed.
- **xrange** specifies a range for the abscissa.
- **yrange** specifies a range for the ordinate.

2.335 plot_amp

parameter	data type	values [defaults]
data_file	char	[void]
scale_1	char	[linear]log
scale_2	char	[linear]log
scan_var	char	[void]
variable	char	[void]
xrange	realx2	
yrange	realx2	

The statement **plot_amp** is used to plot the bias data generated by the simulator for optical amplifier only.

- **data_file** is the data file where the graphic data is to be stored.
- **scale_1** is the type of scale on the abscissa.
- **scale_2** is the type of scale on the ordinate. Note that when the **variable=amp_db** (amplification in dB), the log scale is not used.
- **scan_var** takes one of **bias wavelength** or **input_power**. This variable appears in the abscissa.
- **variable** appears in the ordinate and may take one of **output_power** or **amp_db** which is the amplification of the amplifier in dB.
- **xrange** may be used to force the range on the abscissa.
- **yrange** may be used to force the range on the ordinate.

Example(s)

```
plot_amp scale_1=log scale_2=linear  &&
      scan_var=input_power variable=amp_db
```

2.336 `plot_bias`

parameter	data type	values [defaults]
<code>data_file</code>	char	[void]
<code>variable</code>	char	[void]
<code>n_variables</code>	charxn	
<code>start</code>	real	[0.]
<code>end</code>	real	[1.]
<code>scale_x</code>	real	[2.]
<code>scale_y</code>	real	[1.]
<code>scale_power</code>	real	[2.]
<code>var_num</code>	intg	[1]
<code>curve_math</code>	intg	[0], 1
<code>dataset_start</code>	intg	[1]
<code>dataset_end</code>	intg	[99]

The post-processor statement **plot_bias** is used to plot many important physical variables as a function of bias.

- **data_file** is the data file where the graphic data is to be stored.

- **variable** can be one of following:

delta_freq	Main long. mode frequency deviation from the reference frequency.
power_left	Main long. mode power emission from the left facet.
power_right	Main long. mode power emission from the right facet.
sms_ratio	Side main suppression ratio.
linewidth_spon	Linewidth due to spontaneous emission.
linewidth_carrier	Linewidth due to carrier fluctuation.
linewidth_cross	Linewidth due to cross correlation between carrier and photon density.
linewidth_sidemode	Linewidth due to side long. mode.
linewidth_total	Linewidth from all of the above.
linewidth*power_left	Linewidth power product.
effective_alpha	Effective alpha α_{eff} .
real_freq-#	Real part of the complex frequency of the long. mode number #.
imag_freq-#	Imaginary part of the complex frequency of the long. mode number #.
shd-#	Second harmonic distortion due for channel number #.
power_left_lm_i	Left facet emission power of lateral mode (lm) number i.
power_right_lm_i	Right facet emission power of lateral mode (lm) number i.
power_total_lm_i	Total (left+right) facet emission power of lateral mode (lm) number i.

- **n_variables** takes more than one of the variables tabulated above.
- **start** is the starting bias point in the plot.
- **end** is the relative ending bias point in the plot.
- **scale_x** is used to scale the horizontal axis (i.e., the current). By default, we assume that only one half of the symmetric device is simulated. Thus, a default scale of 2 is needed to recover the current for the whole structure.
- **scale_y** is used to scale the vertical axis of the bias plot for variables other than laser power.
- **scale_power** is used to scale the vertical axis of the light vs. current plot. A default of scale 2 is used to recover the current for the whole structure.

- **var_num** is the number of variables listed in **n_variables**.
- **curve_math** is used to indicate whether or not there is mathematical operation between different curves (*e.g.*, whether or not they should be added together). For a value of 0, no math operation is done between the curves. If it takes a value of 1 all the curves on the same plot is added together to produce an additional curve.
- **dataset_start** and **dataset_end** are used to set the starting and ending data sets used in the plot. If **dataset_end** exceeds the existing number of data sets, the last data set will be used.

Example(s)

```
plot_bias variable=power_left
plot_bias variable=power_right
plot_bias variable=sms_ratio
plot_bias variable=shd_1
```

2.337 plot_ac_parameters

parameter	data type	values [defaults]
data_file	char	[void]
input_file	char	[void]
scale_vertical	char	[linear] log
parameter_type	char	[s] y
smith_chart	char	[yes] no
touchstone_export	char	[void] myfile.s2p
touchstone_freq_units	char	[ghz]hz,khz,mhz,ghz
touchstone_format	char	[ma]ma,db,ri
char_impedance	real	[50] (Ohm)
zdim	real	[10] (um)

plot_ac_parameters is used to plot the AC 2-port parameters produced by the AC analysis defined in **ac_parameters**.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **input_file** is the input file containing the AC analysis results to be plotted. If this is not defined, an internal data filename will be assigned.

- **scale_vertical** is used to control the scale of the AC characteristics being plotted (the ordinate).
- **parameter_type** takes “s” or “y” for S-parameter or Y-parameters, respectively.
- **smith_chart** is applicable only when parameter type is “s”. It is to indicate whether Smith chart is used to plot the S-parameters. If negative, the S-parameters will be plotted against frequencies in conventional way.
- **touchstone_export** instructs the program to export data in Touchstone .s2p format.
- **touchstone_freq_units** specifies the units of the frequency in .s2p format.
- **touchstone_format** takes one of ma,db, or ri corresponding to MA, DB and RI format in .s2p data export.
- **char_impedance** is the characteristic impedance of the high frequency system.
- **zdim** is valid for 2D simulation only. It is the third dimension (normal to the paper) which is needed to compute the absolute impedance of the device being simulated.

Examples

```
plot_ac_parameters parameter_type=y zdim=10
```

The above statement plots Y11, Y21, Y12, and Y22 parameters as functions of frequencies.

```
plot_ac_parameters parameter_type=s zdim=10 smith_chart=no
```

The above statement plots S11, S21, S12, and S22 parameters as functions of frequencies.

```
plot_ac_parameters parameter_type=s zdim=10 smith_chart=yes
```

The above statement plots S11 and S22 in a Smith chart and S21 and S12 in a polar plot.

2.338 plot_data

parameter	data type	values [defaults]
plot_device	char	[vttek]

plot_data defines the graphic output devices.

- **plot_device** is one of devices listed in Table A.1. If this parameter is set to “data_file”, then the graphic output data is written to the files specified by **data_file** in the statement of the corresponding plot.

Example(s)

```
plot_data plot_device=x11
```

This example informs LASTIP that the graphic terminal used is an X11-based window.

```
plot_data plot_device=data_file
plot_1d variable=elec_conc from=(0.5, 0.) &&
to=(0.5, 1.9) data_file=graph.data1
```

The latter example causes **plot_1d** to write the data to a file “graph.data1”.

```
plot_data plot_device=postscript
```

The above produces a postscript output file which should be re-directed to a postscript printer or a postscript file.

2.339 plot_distri

parameter	data type	values [defaults]
data_file	char	[void]
scale_1	char	[linear]log
scale_2	char	[linear]log
variable	char	[void]
xrange	realx2	
yrange	realx2	

plot_distri is used to plot the structural data or the distribution of physical quantities in an optical amplifier. It is similar to **plot_z** except it is used for optical amplifier only.

- **data_file** is the data file where the graphic data is to be stored.
- **scale_1** is the type of scale on the abscissa.
- **scale_2** is the type of scale on the ordinate.
- **variable** is the variable for which the distribution is to be plotted. It takes one of **carrier_conc** or **photon_density**.
- **xrange** may be used to force the range on the abscissa.
- **yrange** may be used to force the range on the ordinate.

2.340 plot_mesh

parameter	data type	values [defaults]
mesh_infile	char	
plot_device	char	[vtttek]
xrange	real	x1,x2 (μm)
yrange	real	y1,y2 (μm)

plot_mesh is used by the mesh generator to plot the generated mesh.

- **mesh_inf** is the input meshfile, not ordered.
- **plot_device** is the plot device defined in Table A.1.
- **xrange** and **yrange** allow the user to define an area of the mesh to zoom in on.

Example(s)

```
plot_mesh mesh_inf=pn.msh plot_device=x11 &&
xrange=(0.15 0.3) yrange=(0.8 1.2)
```

2.341 plot_modu

parameter	data type	values [defaults]
data_file	char	[void]
condition	char	[threshold]
variable	char	[void]
n_variables	charxn	
start	real	
end	real	
var_num	intg	[1]
data_set	intg	[no default]

plot_modu is a post-processor statement used to plot the modulation response the noise spectrum at different bias conditions.

- **data_file** is the data file where the graphic data is to be stored.

- **condition** is dependent on which module of simulation is used. It takes one of “threshold”, “1st_bias” or “2nd_bias” and so on. The meanings of these conditions are explained in **bias** statement.
- **variable** can be set to one of the following.

FMnoise_spon (rad/s-dB)	FM noise due to spontaneous emission.
FMnoise_carrier (rad/s-dB)	FM noise due to carrier fluctuation.
FMnoise_cross (rad/s-dB)	FM noise due to cross correlation between carrier and photon.
FMnoise_sidemode (rad/s-dB)	FM noise due to sidemode.
FMnoise_total (rad/s-dB)	Total FM noise.
AM_magnitude (mW/mA-dB)	AM magnitude.
AM_phase (rad)	AM phase.
FM_magnitude (GHz/mA-dB)	FM magnitude.
FM_phase (rad)	FM phase.

- **start** is the staring point of the spectrum.
- **end** is the ending point of the spectrum.
- **data_set** is equivalent but overrides the **condition** parameter.
- **data_set=0** is equivalent to **condition=threshold**.
- **data_set=1** is equivalent to **condition=1st_bias**.
- **data_set=2** is equivalent to **condition=2nd_bias**.

The latter can only go up to **condition=5th_bias** while the former does not have a limit.

Example(s)

```
plot_modu variable=AM_magnitude condition=1st_bias
```

2.342 plot_rtgain

parameter	data type	values [defaults]
plot_device	char	[void]
rtgain_data	char	[void]
standing_wave	char	[void]
std_wave_range	realx2	

The statement **plot_rtgain** is used as a preprocessing tool to plot the round trip gain and standing wave pattern within a LD or a VCSEL. It takes the data (.rtd file) from PICS3D running a .sol file on a statement **rtgain_phase**.

- **plot_device** is the plot device defined in Table [A.1](#).
- **rtgain_data** is the .rtd file as a result of running **rtgain_phase** statement in .sol file.
- **rtgain_data** is the .rtd file as a result of running
- **standing_wave** is the file (.stw file) containing standing wave in a VCSEL. This will be generated for a VCSEL by the **rtgain_phase**.
- **std_wave_range** specifies the wave range to be plotted.

Example(s)

```
plot_rtgain rtgain_data=jim.rtd standing_wave=jim.stw &&  
plot_device=postscript
```

2.343 plot_scan

parameter	data type	values [defaults]
data_file	char	[vttek]
scale_1	char	log, [linear]
scale_2	char	log, [linear]
scan_var	char	(see table below)
variable	char	(see table below)
n_variables	charxn	
include_data	char	[void]
facet	char	[void] front back
user_xlabel	char	[void]
user_ylabel	char	[void]
user_title	char	[void]
user_label1	char	[void]
user_label2	char	[void]
user_label3	char	[void]
merge_next	char	[no] yes
filter_name	char	[void]
plot_slope	char	[no] yes
set_grid	char	[no] yes
show_points	char	[no] yes
scale_lit	real	2.
scale_curr	real	2.
xrange	realx2	
yrange	realx2	
xy_label1	realx2	[0. 0.]
xy_label2	realx2	[0. 0.]
xy_label3	realx2	[0. 0.]
scale_horizontal	real	[1.]
scale_vertical	real	[1.]
filter_value	real	[0.]
var_num	intg	[1]
inc_curve	intg	[1]
mode_index	intg	[1]
trap_index	intg	[1]
scanline	intg	
scan_num	intg	

plot_scan is a post-processor statement used to plot all the (scanned) bias data points from data_set_1 to data_set_2 in the **get_data** statement.

- **data_file** is the file to which the graphic data is written in ASCII format. Note that **plot_device** should be set to “data_file” for this parameter to take effect.
- **scale_1** is used to indicate linear or log scale on the abscissa.
- **scale_2** is used to indicate linear or log scale on the ordinate.
- **scan_var** is the scanned variable (on the abscissa). For the current from an electrode, the units are A/m, Amperes per unit length in the z-direction or the third dimension of the 2D simulation. In the case of a light versus current plot for LASTIP, the current is the total current from the device in mA. Details are given in the table in Appd. ???. The units of the physical quantities being plotted are labeled in the graphics.
- **variable** is the variable to be plotted (on the ordinate). For the current from an electrode, the units are A/m, Amperes per unit length in the z-direction or the third dimension of the 2D simulation. In the case of a light versus current plot, the light emitted is the total power from the two facets together. “wavelength” and “effective_index” may be plotted as a function of injection current or time (in a transient simulation). Details are given in the table in Appd. ???. The units of the physical quantities being plotted are labeled in the graphics.
- **n_variables** is a group of variables to be plotted.
- **include_data** is the name of the data file containing the data to be added to the plot. It can be used to include experimental data on the same figure. The format of the data in the file is in columns of (x,y) following the number of data points. For example this format can be used to include a data file:

```

3
0.1    0.5
0.2    0.8
0.5    1.2

```

- **facet**, if defined, will force this statement to plot either the front or back facet light out from a laser diode. If not used, the total power from both facets are plotted. Please note that a factor in **scale_lit** is used to multiply to all plotted laser powers to reflect the chosen device simulation area in case of symmetry.
- **user_xlabel** is the user-defined x-label. It must a one-word label less than 68 characters or the blank space must be connected by underscores (_).
- **user_ylabel** is the user-defined y-label. It must a one-word label less than 20 characters or the blank space must be connected by underscores (_).

- **user_title** is the user-defined title appearing on top of the plot. It must be a one-word label less than 20 characters or the blank space must be connected by underscores (`_`).
- **user_label1** is the user-defined label which appears at coordinates given by **xy_label1**. It must be a one-word label less than 20 characters or the blank space must be connected by underscores (`_`).
- **user_label2** is the user-defined label which appears at coordinates given by **xy_label2**. It must be a one-word label less than 20 characters or the blank space must be connected by underscores (`_`).
- **user_label3** is the user-defined label which appears at coordinates given by **xy_label3**. It must be a one-word label less than 20 characters or the blank space must be connected by underscores (`_`).
- **merge_next** is used to instruct the program not to plot this curve but to transfer the data and merge the curve into the next plot statement.
- **filter_name**, if used, is the control variable name (such as `voltage_1`, etc.) that can be used as a filter to select only the data to be plotted while data not selected is filtered. The selection value is given by parameter **filter_value**. This is useful when several parameters are changed in a simulation. For example, suppose we obtain from a simulation several parameters that change with bias, `p1,p2,p3,...`. If we wish to plot parameter `p2` versus `p1` with the condition `p3=value`, `p3` is the filter control variable while the “value” is the **filter_value**.
- **plot_slope** will cause the statement to plot the slope instead of the original quantity.
- **set_grid** sets the grid in the plot.
- **show_points** would let the curve show the data points instead of just connected lines.
- **scale_lit** is the scale of the light power used to plot output light power versus current for the laser simulation (for LASTIP). The default is 2 because LASTIP assumes the device is symmetric and the user only sets up one half of the device for simulation. This factor is multiplied to the total power (from 2 facets) as well as to power emitted from each facet.
- **scale_curr** is the scale of the current used to plot light versus current for the laser simulation. The default is 2 because LASTIP assumes the device is symmetric and the user only sets up one half of the device for simulation.
- **xrange** specifies a range for the abscissa.

- **yrange** specifies a range for the ordinate.
- **xy_label1** are the coordinates for the label in **user_label1**.
- **xy_label2** are the coordinates for the label in **user_label2**.
- **xy_label3** are the coordinates for the label in **user_label3**.
- **var_num** is the number of variables to be plotted.
- **inc_curve** is the number of curves to be included within the data file given by **include_data**.
- **mode_index** is the modal index for optical mode related quantities.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.
- **scale_horizontal** is used to scale the horizontal axis for plots other than light-current curve.
- **scale_vertical** is used to scale the vertical axis for plots other than light-current curve.
- **filter_value** is used when parameter **filter_name** is defined. It is used to select data in a plot. Please see parameter **filter_name**.
- **scanline** is used to denote bias data from a whole **scan** statement. The data from statement **equilibrium** is counted as the scanline=1; those from the next **scan** statement counted as scanline=2, etc. For this parameter to work, the **get_data** command must already include the data sets associated with this parameter.
- **scan_num** is used to denote bias data from a whole **scan** statement. The data from statement **equilibrium** is counted as the scan_num=0; those from the next **scan** statement counted as scanline=1, etc. For this parameter to work, the **get_data** command must already include the data sets associated with this parameter.

Example(s)

```
plot_scan scan_var=voltage_1 variable=current_1
```

This would plot all the I-V data for data collected by the **get_data** command.

```
plot_scan scan_var=voltage_1 variable=current_1 scan_num=3
```

This would plot all the I-V data for data produced by the 3rd **scan**. Please note that for this to work, the **get_data** command must already include all the data sets related to scan_num=3. For example, supposed scan_num=3 corresponds to data set number 4,5,6. Then, the **get_data** command must have scan_data setting to include data sets 4,5 and 6.

2.344 plot_shd

parameter	data type	values [defaults]
data_file	char	[void]
condition	char	[threshold]
variable	char	[void]
n_variables	charxn	
start	real	[-1.e10]
end	real	[1.e10]
var_num	intg	[1]
data_set	intg	

plot_shd is used to plot the second harmonic distortion spectrum.

- **data_file** is the data file where the graphic data is to be stored.
- **condition** is dependent on which module of simulation is used. It takes one of “threshold”, “1st_bias” or “2nd_bias” and so on. The meanings of these conditions are explained in the **bias** statement.
- **variable** is used to specify the second harmonic distortion and may be set to one of “shd_*i*” (*i*=1,2,3,4,5), where *i* represents the channel number. The frequencies of the channels may be specified by the statement **2nd_harmonic** in the 2nd part of .sol file. file.
- **start** is the staring point of the spectrum.
- **end** is the ending point of the spectrum.
- **data_set** is equivalent to but overrides the **condition** parameter.
- **data_set=0** is equivalent to **condition=threshold**;
- **data_set=1** is equivalent to **condition=1st_bias**;
- **data_set=2** is equivalent to **condition=2nd_bias**.

The latter can only go up to **condition=5th_bias** while the former does not have a limit.

Example(s)

```
plot_shd variable=shd_2 condition=1st_bias
```

2.345 plot_spec

parameter	data type	values [defaults]
data_file	char	[void]
condition	char	[threshold]
variable	char	[void]
n_variables	charxn	
start	real	[-1.e10]
end	real	[1.e10]
var_num	intg	[1]
data_set	intg	[no default]

The post-processor statement **plot_spec** is used to plot spectral data generated by the simulation.

- **data_file** is the data file where the graphic data is to be stored.
- **condition** is dependent on which module of simulation is used. It takes one of “threshold”, “1st_bias” or “2nd_bias” and so on. The meaning of these conditions is explained in **bias** statement.
- **variable** can be set to “mode_spectrum”.
- **n_variables** may be used to list more than one variable so that more than one variable may be plotted on the same graph.
- **start** is the starting point of the spectrum.
- **end** is the ending point of the spectrum.
- **data_set** is equivalent but overrides the **condition** parameter.
- **data_set=0** is equivalent to **condition=threshold**.
- **data_set=1** is equivalent to **condition=1st_bias**.
- **data_set=2** is equivalent to **condition=2nd_bias**. The latter can only go up to **condition=5th_bias** while the former does not have a limit.
- **var_num** is the number of variables being plotted and must match the prefix in **n_variables** if used.

Example(s)

```
plot_spec variable=mode_spectrum condition=1st_bias
```

2.346 plot_structure

parameter	data type	values [defaults]
data_file	char	[void]
relative_dist	char	[no]
variable	char	[void]
n_variables	charxn	
start	real	[-1.e20]
end	real	[1.e20]
var_num	intg	[1]

plot_structure is used to plot the structural and geometric data. Since the data are taken directly from data arrays of the program, they are also the data recognized by the program. This statement can be used to check whether the input data are correct.

- **data_file** is the data file where the graphic data is to be stored.
- **relative_dist** is used to decide if relative distance is used to plot the position dependent variables.
- **variable** is the name of the variable to be plotted. It can take oen of the following.

kappa_real	Real part of coupling coefficient
kappa_imag	Imag. part of coupling coefficient
omega_g	Grating phase related to change in pitch (or period), ie., Ω_g in User's manual.
optical_conf	Optical confinement.
thickness	Thickness of active layer(s).
width	Width of active layer(s).
internal_loss	Internal loss.
phase_index	Phase (or effective) index.

- **n_variables** takes more than one of the variables tabulated above.

- **start** is the starting point of the distance.
- **end** is the ending point of the distance.
- **var_num** is the number of variables enlisted in **n_variables**.

2.347 plot_z

parameter	data type	values [defaults]
data_file	char	[void]
condition	char	[threshold]
variable	char	[void]
n_variables	charxn	
relative_dist	char	[yes] no
start	real	[-1.e20]
end	real	[1.e20]
scale_var	real	[1.0]
var_num	intg	[1]
data_set	intg	[no default]

The post-processor statement **plot_z** is used to plot the distribution of z-dependent physical variables in the laser.

- **data_file** is the data file where the graphic data is to be stored.
- **condition** is dependent on which module of simulation is used. It takes one of “threshold”, “1st_bias” or “2nd_bias”, up to “5th_bias”. The meaning of these conditions is explained in **bias** statement.
- **variable** can be one of following:

delta_density	The carrier density minus the reference density.
right_wave	Right going main mode wave intensity.
left_wave	Left going main mode wave intensity.
total_wave	The sum of the above two waves.
delta_beta*l	Deviation of main mode β (wave number) from the reference β_{ref} . “l” is the cavity length. This option is valid for single mode model only.
sm_right_wave	Sidemode (sm) right going wave intensity.
sm_left_wave	Sidemode (sm) left going wave intensity.
sm_total_wave	Sidemode (sm) sum of the above two waves.

- **n_variables** takes more than one of the variables tabulated above.
- **relative_dist** is used to decide if relative distance is used to plot the position dependent variables.
- **start** is the relative starting position of the plot.
- **end** is the relative ending position of the plot.
- **scale_var** can be used to scale the result.
- **var_num** is the number of variables enlisted in **n_variables**.
- **data_set** is equivalent but overrides the **condition** parameter.
- **data_set=0** is equivalent to **condition=threshold**.
- **data_set=1** is equivalent to **condition=1st_bias**.
- **data_set=2** is equivalent to **condition=2nd_bias**. The latter can only go up to **condition=5th_bias** while the former does not have a limit.

Example(s)

```
plot_z variable=right_wave condition=1st_bias
```

Example(s)

```
plot_z var_num=2 2_variables=(right_wave left_wave) &&
condition=1st_bias
```

2.348 pml

parameter	data type	values [defaults]
gain_correction	char	[yes]
pure_index_loss	char	[no]
permittivity_real	real	[1.0]
permittivity_imag	real	[0.0]
pml_length	real	[6.0](um)
pml_mesh	intg	[10]

The statement **pml** is used to activate the perfectly matched layer (PML) boundary model. This is a special type of boundary that absorbs radiative waves from a mode-leaky structure of the lateral optical modes. Effective anisotropic PML is used in the simulator.

- **gain_correction** may be used to enable the optical gain in the rate equation to be corrected for radiation loss. With this setting to “no”, The simulator will calculate the modal gain by averaging the local material gain with weight of the wave intensity.
- **pure_index_loss**. There are two ways to estimate the modal radiative loss. The first is the pure index method. This is by setting all imaginary material indices to zero throughout the device. The imaginary part of the eigen value of the solution is the modal loss due to radiation. The other method is to estimate the bias dependent radiative loss by comparing the PML solution with averaged material gain. The first former is a convenient way of characterizing the device. The latter is more realistic but less stable numerically.
- **permittivity_real** is the real part of permittivity in the absorbing PML material.
- **permittivity_imag** is the imaginary part of permittivity in the absorbing PML material.
- **pml_length** is the length (or thickness) of the PML used for the absorption.
- **pml_mesh** is the number of mesh lines within the PML.

Example(s)

```
pml permittivity_real=1.0 permittivity_imag=0.02 pml_mesh=50 &&
    pure_index_loss=no
```

2.349 point

parameter	data type	values [defaults]
label	char	
xy	real	(μm)

point defines a point in 2D space.

- **label** labels a point.
- **xy** are the coordinates of the point.

Example(s)

```
point label=a xy=(0.4, 0.5)
```

2.350 polarization

parameter	data type	values [defaults]
angle	real	[0.] (degree)

polarization is used to set an arbitrary polarization in laser simulation or for the calculation of optical gain. It overrides the TE and TM mode setting in the **active_reg** statement.

- **angle** is the angle of polarization. The angle is zero for TE mode and 90 degrees for TM mode.

Example(s)

```
polarization angle=45.
```

2.351 polygon

parameter	data type	values [defaults]
name	char	
4_points	realx4	
3_points	realx3	
material	intg	
boundary_i	charx2	
limits_i	realx2	(μm)

polygon is used in the mesh generator to define the geometry of the device.

- **name** is the name of the polygon.

- **4_points** are the labels of the four points used to define a polygon. The points should be arranged in counterclockwise order starting from the reference point, which is equivalent to the (0,0) point of a rectangle.
- **3_points** are the labels for the three points used to define a triangle. The points should be arranged in counterclockwise order starting from the reference point, which is equivalent to the (0,0) point of a rectangle.
- **material** is the material number of the polygon being described.
- **boundary_i** (i=1,2,...) contains the labels of two corner points. These two points define an edge where the ith boundary is to be defined.
- **limits_i** contains the starting and ending points of the ith boundary along the edge defined by **boundary_i**. The reference direction is along the counterclockwise direction.

Example(s)

```

polygon name=rect1 4_points=(a b c d) boundar_1=(a b)
    limits_1=(0. 0.7)

```

2.352 power3d

parameter	data type	values [defaults]
print_coefs	char	no
current_to real		

power3d is an internal statement used by PICS3D for debugging only. User need not use this statement.

2.353 previous_layer

parameter	data type	values [defaults]
file	char	[void]

The statement **previous_layer** is used in a .layer file to inform the program that there is a previous layer file to be used to construct a 3D structure consisting of x-y planes stacked together.

- **file** is the .layer file of the previous layer to be used for 3D structure.

Example(s)

```
previous_layer file=test1.layer
```

2.354 `print_active_layer`

parameter	data type	values [defaults]
mater	intg	[1]

print_active_layer is used to print the active layer material macro in the .gain.msg file. It must be used in the gain preview session.

- **mater** is the material number used to identify the macro used by the **get_active_layer** statement.

Example(s)

```
get_active_layer name=InGaN/AlGaN var1=0.01 var2=0.2 var3=300 mater=3
active_reg type=macro exch_coef=0.e-10 tau_scat=0.5e-11 &&
    thickness=0.005 mater=3 qw_print=2
print_active_layer mater=3
```

2.355 `print_macro`

parameter	data type	values [defaults]
mater	intg	[1]

print_macro is used to print the bulk material macro in the .gain.msg file. It must be used in the gain preview session.

- **mater** is the material number used to identify the macro used by the **load_macro** statement.

Example(s)

```
load_macro name=ingan var1=0.2 mater=1
print_macro mater=1
```

2.356 put_mesh

parameter	data type	values [defaults]
polygon	char	
edge	charx2	
point_from	real	(μm)
point_to	real	(μm)
number	intg	
ratio	real	
extra_point_1	real	(μm)
extra_point_2	real	(μm)
shift_center	real	[0.] (μm)

put_mesh places mesh lines on an edge of a polygon.

- **polygon** is the name of the polygon to be affected.
- **edge** is the edge where mesh lines are to be placed.
- **point_from** is the relative position on the edge where the mesh begins. The zero point of the edge is the point of lower order in the counterclockwise direction.
- **point_to** is the relative position on the edge where the mesh ends. The zero point of the edge is the point of lower order in the counterclockwise direction.
- **number** is the number of mesh lines to be placed.
- **ratio** is the ratio of the adjacent mesh interval. If it is greater than 1, the mesh interval increases with distance. If it is less than 1, the mesh interval decreases with distance. If it is negative, the mesh obeys the absolute value over the first half of the distance and the other half is symmetric to the first half. A schematic diagram illustrating the effect of various ratios is given in Fig. 2.16.
- **extra_point_1** is the relative position of an extra point. It is used when an extra point is needed relative to **point_from**. The statement is used when an abrupt junction needs to be defined. The sharpness of the junction is defined by size of the extra point. Please note that this feature belongs to simulator version 2002 or older. In newer versions, the extra point in mesh generation is controlled by the statement **internal_xpoint**. This statement will be used only when the parameter **version2002** is set.
- **extra_point_2** is the relative position of an extra point. It is used when an extra point is needed relative to **point_to**.

- **shift_center** is used in the case of negative mesh **ratio** (see 2.16). The negative ratio will cause the mesh distribution to vary using the center point of the length as a symmetric point. This parameter causes the center to be shifted in either direction. The distance of the shift is relative to the total length in question. For example, 0.2 means shift 20 percent of the total length.

Example(s)

```
put_mesh polygon=p1 edge=(a b) point_from=0. point_to=1.4
```

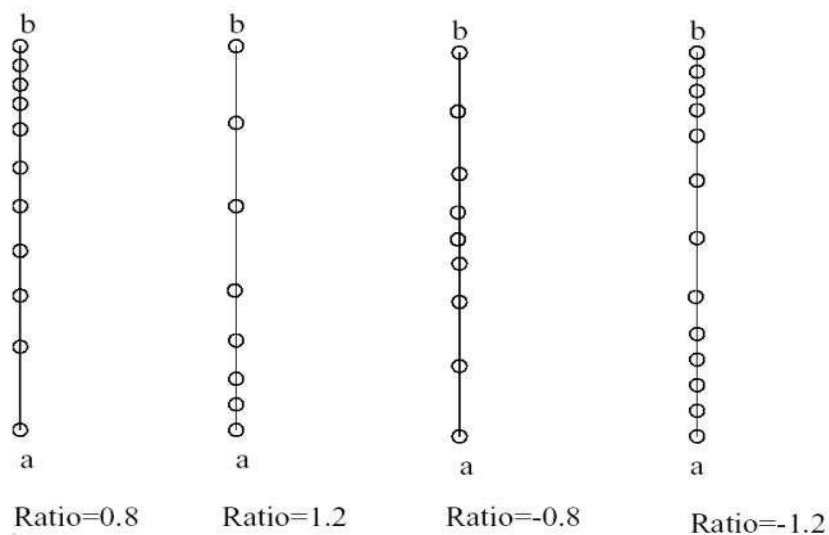


Figure 2.16: Effects of different ratio on putting mesh points on an edge (a b).

2.357 qdot_layer_mater

parameter	data type	values [defaults]
macro_name	char	[void]
active_macro	char	[void]
var_symboli(i=1..9)	char	[void]
avar_symboli(i=1..9)	char	[void]
vari(i=1..9)	real	[-9999.]
avari(i=1..9)	real	[-9999.]
height	real	[-9999.]
column_num	intg	[1]

qdot_layer_mater is used to define a quantum dot (QDOT) embedded within a regular quantum well. The regular quantum well may also be regarded as the wetting layer. This statement must be placed immediately after the **layer_mater** describing the quantum well.

The parameter **height** is the height or thickness of the QDOT. All other parameters are the same as in statement **layer_mater** except a QDOT is described here. Also, the active material macro must be of the complex type with prefix **cx-**.

Example(s)

```
qdot_layer_mater macro_name=ingan active_macro=cx-InGaN &&  
  var_symbol1=x avar_symbol1=xw var1=0.450 avar1=0.450 &&  
  column_num=1 height=0.005
```

2.358 qdot_material

parameter	data type	values [defaults]
import_dir	char	[void]
import_qdot_data	char	[void]
import_dir2-19	char	[void]
import_qdot_data2-19	char	[void]
surface_density	real	[1.e15]
tau_broaden	real	[0.6e-13]
wetting_area_ratio	real	[0.1]
set_cond_level1	real	
set_cond_level2	real	
set_cond_level3	real	
set_hh_level1	real	
set_hh_level2	real	
set_hh_level3	real	
set_lh_level1	real	
set_lh_level2	real	
set_lh_level3	real	
set_ch_level1	real	
set_ch_level2	real	
set_ch_level3	real	
weight	real	[100.]
weight2-19	real	[0.]
tau_broaden2-19	real	[0.6e-13]
dos_reduction_factor	real	1.
mater_active	intg	
coupled_dots	intg	[1]

qdot_material is used in .sol file to define the properties of one type of quantum dot (QDOT) material. A QDOT material must be defined as an embedded (or mildly active) material since it shares the material number with a quantum well active layer, while part of active layer properties are evaluated for it.

Thus both active and bulk macros must be used. For more details on embedded active material, please see statement **active_reg**.

- **import_dir**, and **import_dir2-19** are directories are subprojects with detailed 3D quantum dot simulation input files. If more than one type of QDOTs are involved, different subdirectories may be used to represent different types of QDOT.

- **import_qdot_data**, **import_qdot_data2-19** are data files containing quantum solutions (levels and wave function overlaps). They are usually with file extension of .qdd. If more than one type of QDOTs are involved, different files may be used to represent different types of QDOT.
- **surface_density** is the surface density of the quantum dots associated with each quantum well. If more than one type of QDOTs are involved, this is the total dot density. The distribution of different types of QDOTs are described by parameters of **weight**, and **weight2-19**. The unit of this parameter is $1/m^2$.
- **tau_broaden** , **tau_broaden2-19** are the energy broadening time constant used to broaden the spectrum. Their physical origin includes inhomogeneous broadening (due to size/composition fluctuation) and intraband scattering. Please note that different type of QDOTs may be associated with different time constants.
- **wetting_area_ratio** is a parameter related to area of quantum well over area of quantum dot. This ratio is used to control the intensity of well/wetting layer emission as compared with QDOT emission. Microscopic interpretation of quantum dot size is determined by the span of the wave function.
- **set_cond_level1-3**, if defined, are used to manually set the quantum levels of the conduction subbands.
- **set_hh_level1-3**, if defined, are used to manually set the quantum levels of the heavy hole subbands.
- **set_lh_level1-3**, if defined, are used to manually set the quantum levels of the light hole subbands.
- **set_ch_level1-3**, if defined, are used to manually set the quantum levels of the crystal field hole subbands. This is used for material of wurtzite structure (such as GaN).
- **weight**, **weight2-19** are used to described the relative number of QDOTs of different kinds as simulated in different subprojects imported from different subdirectories. Please note that only the relative figures are important. For example, $\text{weight}=0.9$ and $\text{weight2}=0.1$ are equivalent to $\text{weight}=90$ and $\text{weight2}=10$.
- **dos_reduction_factor** would introduce a factor to reduce the density of states of the quantum dot.
- **mater_active** is the material number of the active layer associated with the QDOTs.

- **coupled_dots** is used to indicate whether a QDOT is quantum mechanically coupled to QDOTs in other wells. If so, this parameter is used to avoid double counting. For example, if two QDOTs in two different wells are coupled to each other, they should be counted only once since there will be a lifting of degeneracy in quantum levels.

Example(s)

```
qdot_material surface_density=5.e15 tau_broaden=0.30e-13 &&
mater_active=3 import_dir=dot import_qdot_data=dot.qdd &&
coupled_dots=1 wetting_area_ratio=0.2 &&
tau_broaden2=0.18e-13 import_dir2=dot1 import_qdot_data2=dot.qdd &&
tau_broaden3=0.10e-13 import_dir3=dot2 import_qdot_data3=dot.qdd &&
tau_broaden4=0.10e-13 import_dir4=dot3 import_qdot_data4=dot.qdd &&
weight=100 weight2=75 weight3=18 weight4=4
```

2.359 quasi_3d_transient

parameter	data type	values [defaults]
method	char	[1],2,3

quasi_3d_transient is used to control the model of large signal analysis. To explain this statement, we need to review the method in quasi-3D simulation in PICS3D: Photon linear density is scanned from S_1 to S_2 to represent linear photon density located at various z-positions. For example, the middle of a FP laser is represented by S_1 and the facet end by S_2 . The electronic equations are symbolically summarized by the following equations after discretization:

$$\begin{aligned}
 G_V(n, p, V) &= 0 \\
 G_n(n, p, V, S) &= \frac{n - n_0}{\Delta t} \\
 G_p(n, p, V, S) &= \frac{p - p_0}{\Delta t}
 \end{aligned}
 \tag{2.46}$$

where n , p , V and S are electron, hole, potential and linear photon density, respectively. For each time step with a photon density ranges from $S_{lo}(t)$ to $S_{hi}(t)$. n , p and V are accurately determined by the 2D drift diffusion solver but the variation of n_0 and p_0 with S is unknown and is subject to assumptions.

We know how n_0 and p_0 vary with S_0 but not S . Since variation of S really represents variation of z-position, we must have a reasonable expression of n_0 and p_0 as a function of S .

In this document, we use subscript 0 to denote a previous time instance, lo and hi to denote lower and higher photon density (or z-position).

For Method 1 (default), we assume all of n , p , n_0 and p_0 scales linearly with S .

For Method 3, we scale n_0 and p_0 to force them to linearly fit to the actual longitudinal solution of $n_0(z)$ in the previous time instance.

Method 2, is similar to method 3 except that the overall Fermi levels/potentials are scaled instead of the density. Since a constant is used to scaled the whole device, this method should work better if the Fermi level scales continuously across the active-passive material interfaces.

2.360 qwell_normal

parameter	data type	values [defaults]
dir	char	[y] x

The **qwell_normal** is used in .sol file to specify the direction normal to the quantum well plane. Usually this statement is not needed unless self-consistent quantum confined carrier density model is used. The simulator needs to compute the density along the normal direction to the quantum well plane.

- **dir** is the direction normal to the quantum well plane.

Example(s):

```
qwell_normal dir=y
```

2.361 qw_optics_control

parameter	data type	values [defaults]
parameterize	char	[yes]no
junction_type	char	[forward] reverse, non-optical
extrap_energy	char	[yes]no
extrap_conc	char	[yes]no
extrap_pn_ratio	char	[no]yes
extrap_temper	char	[yes]no
conc_range	realx2	(m^{-3})
wavel_range	realx2	(μm)
pn_ratio_range	realx2	[0.5 1.5]
temper_range	realx2	(K)
wavel_points	intg	[50]
conc_points	intg	[10]
pn_ratio_points	intg	[3]
temper_points	intg	[8]

qw_optics_control is related to **import_gain_data**. While the latter is explicitly used to import data from a data file, the former is to let the program generate the tabulated gain data internally and then use the interpolated data table to perform the simulation.

- **parameterize** is used to enable or suppress this statement.
- **junction_type**
 - forward sets `conc_range = 5.e23→5.e24`.
 - reverse sets `conc_range = 1.e22→5.e23`.
 - non-optical will suppress optical gain/spont. em. in active layers.
- **extrap_energy** indicates if extrapolation is used when photon energy is outside of the range in imported data.
- **extrap_conc** indicates if extrapolation is used when concentration is outside of the range in imported data.
- **extrap_pn_ratio** indicates if extrapolation is used when hole/electron ratio is outside of the range in imported data.
- **extrap_temper** indicates if extrapolation is used when temperature is outside of the range in imported data.

- **wavel_range** is the wavelength range.
- **conc_range** is the electron concentration range in the well.
- **pn_ratio_range** is the range of ratio of hole over electron concentrations.
- **temper_range** is the temperature range.
- **wavel_points** is the number of points in the **wavel_range**.
- **conc_points** is the number of points in the **conc_range**.
- **pn_ratio_points** is the number of points in the **pn_ratio_range**.
- **temper_points** is the number of points in the **temper_range**. For uniform temperature simulation, only a single temperature is used in gain table evaluation. For heat transport simulation, more than one points must be used. The default for heat transport simulation is eight.

Example(s)

qw_optics_control parameterize=yes

2.362 q_transport

parameter	data type	values [defaults]
n_side_down	char	[yes]
sequential_model	char	[yes]
optic_pumped_escape	char	[no]
optic_pumped_fly	char	[no]
add_all_wells	char	[no]
elec_fly_over	real	[1.]
hole_fly_over	real	[1.]
elec_escape	real	[0.01]
hole_escape	real	[0.01]
drift_away	real	[0.5] (um)

The statement **q_transport** is used to activate a semi-empirical non-equilibrium quantum transport model which improves the assumption of quasi-equilibrium implied in the standard drift-diffusion and thermionic emission models. We limit the

following discussion to electrons since the same argument applies to holes. By default the device simulator assumes that when electrical current flows through a quantum well or dot, the electrons are thermalized with the lattice so that a Fermi distribution with a quasi-Fermi level may be used to describe the electron behaviors. Using Fermi distribution, electron capture into and escape from the well can be described by a thermionic emission model in the same way as a Schottky contact.

However in the limit of thin and deep quantum wells/dots (such as in GaN-based materials), the thermal equilibrium model may fail to describe two effects: direct fly-over and escape over the quantum well (QW) or dot (QDOT). We shall limit the discuss to QW since the same mechanism applies to QDOT if it is sufficiently small in size.

If the QW is thinner than or comparable to the mean free path of the electrons, part of the current may directly fly over the QW without being thermalized. This effect is included in a semi-classical Drude model by computing the mean free path from the mobility and assuming the number of electrons that can directly fly-over the QW vary with a probability of $\exp(-\text{well_width}/\text{mean_free_path})$.

Once within a QW, electrons scatter with phonons (or collide with QW walls in a classical picture) while losing energy to reach the bottom of the QW. The deeper the QW, the more number of collisions the electron needs to reach the QW bottom and thus it will have more chance to gain sufficient energy through electron-electron scattering and quantum tunneling to escape from the QW. The thinner the QW, the easier it is for the electron to escape. The above processes are difficult to model and this version of the simulator uses a parameter to represent the fraction of electrons being able to escape from the quantum well because they fail to reach thermal equilibrium quickly.

- **n_side_down** lets the program to recognized which is the n-doping side. The program assumes that the model operates under forward bias and thus needs to know the direction of flow based on the position of the n-side.
- **sequential_model** indicates the quantum well transport happens from one well to the next instead of collectively or coherently from all the wells to the drift-away distance.

optic_pumped_escape is used to describe a non-equilibrium process where bound carriers are excited by optical pumping and acquires momentum to escape the quantum well. Specifically, this is used to model optical generation process in QWIP. The details are as follows: optical pumping causes bound carriers to become unbound carriers with density N_{ub} . The N_{ub} are estimated based on the following theory: within one period of the QWIP, optical pumping causes unbound carriers to escape quantum confinement with the thermal velocity v_{th} . It can be shown that

$$N_{ub} = \frac{\text{pumping_rate} \times \text{period}}{v_{th}}$$

Given unbound carrier density, it is easy to estimate the total QWIP photocurrent by drift-diffusion theory. This quantum escape term is proportional to the light intensity since pumping rate is such.

optic_pumped_fly is used to describe a non-equilibrium process where unbound carriers acquire momentum from optical excitation to make a direct fly-over the quantum wells. This is implemented in the same way as usual direct fly-over process except that the transport current is made to be proportional to the optical intensity. Specifically, this is applicable for QWIP device.

add_all_wells may be used when **sequential_model=no** or when quantum fly-over and escape take the collective mode, that is, all quantum transport events take place from individual wells to the collector directly. By default, the program averages the transport current contribution from each well. The present parameter instructs the simulator to add contributions from all wells instead.

- **elec_fly_over** is a parameter to scale the direct fly-over current for electrons.
- **hole_fly_over** is a parameter to scale the direct fly-over current for holes.
- **elec_escape** is a parameter proportional to the fraction of electrons unable to reach the bottom of the well. It is used to scale the direct escape current. This parameter may be interpreted as the fraction of electrons but since it is an empirical scaling factor, there is no requirement that it be less than unity.
- **hole_escape** is the same as above except it is for holes.
- **drift_away** is the distance from the QW at which the fly-over and escape current will be directed towards. At a quasi-equilibrium model such as drift-diffusion theory, carriers directly escaped over the QW will partly flow back to the original QW due to diffusion caused by excess carriers forcefully moved over the QW. Thus, it is desirable to set this distance to be away from the p-n junction so that back diffusion be avoided. Please note that the back diffusion is a result of forcing some non-equilibrium current component upon the drift-diffusion model which is based on local equilibrium (i.e., by using quasi-Fermi levels).

Example(s)

```
q_transport elec_fly_over=1 hole_fly_over=1 &&
elec_escape=0.3 hole_escape=0.3 sequentail_model=no
```

The above statement opens up two current flow paths. One allows the full amount (thus factor 1) of carriers with free paths greater than the QW width to be transported to a drift-away distance. The other path allows 30 percent of the carriers

in the QW to directly flow to the drift-away distance. Please note that when we use the word "allow", it does not mean that all of the carriers involved will actually be transported since they are moved via a resistance equal to the average of the starting and ending points. The actual amount will be determined after competition with the current component captured into the quantum well assuming local thermal equilibrium.

Example(s)

```
q_transport elec_fly_over=1 hole_fly_over=1 &&
  elec_escape=0.3 hole_escape=0.3 sequential_model=yes
```

The above statement allows the non-equilibrium transport from one well to the next. Non-equilibrium carriers in the last well will escape to the drift-away distance.

2.363 raw_output

parameter	data type	values [defaults]
variable	char	
output_file	char	void
mode_index	intg	[1]
trap_index	intg	[1]

The statement raw_output is a post processing command used to print all available 2D data in the following form:

```
total node number
listing of x,y,z,variable
```

- The output is written to the output file defined by **output_file**
- **mode_index** is the mode index or mode number of the wave modes if the quantity being printed is wave function.
- **trap_index** is the trap index or trap number of the deep level traps if the quantity being printed is related to traps (eg, trap concentration).
- **variable** is one of the variables listed in appendix [C](#).

2.364 rcled_dbr

parameter	data type	values [defaults]
x_start_label	char	[void]
x_end_label	char	[void]
y_start_label	char	[void]
y_on_top	char	[no]
y_at_bottom	char	[no]
layer1	real	[0.1]
complex_index1	realx2	[(3.5 1.e-6)]
layer2	real	[0.1]
complex_index2	realx2	[(3.5 1.e-6)]
period_number	intg	

The statement **rcled_dbr** is used to define a DBR structure within an RCLED. The basic set up of this model is that optical layers are parallel to the x-axis and the device emits mainly in +y or -y directions.

- **x_start_label**, **x_end_label** are the position labels for the left and right bounds of the DBR. The labels were previously defined by **x_position** statements. If one or more of the labels are not defined, the program uses the boundary of the device.
- **y_start_label** is used to define the vertical position of the DBR. The bottom of the DBR stack starts from this position. It is a position label previously defined by a **y_position** statement. Please note that **y_start_label**, **y_on_top** and **y_at_bottom** are conflicting parameters and should be set with care so that only one of the three is used.
- **y_on_top** is used to define the vertical position of the DBR if the DBR is on top of the RCLED. Please note that **y_start_label**, **y_on_top** and **y_at_bottom** are conflicting parameters and should be set with care so that only one of the three is used.
- **y_at_bottom** is used to define the vertical position of the DBR if the DBR is at the bottom of the RCLED. Please note that **y_start_label**, **y_on_top** and **y_at_bottom** are conflicting parameters and should be set with care so that only one of the three is used.
- **layer1** and **complex_index1** are the layer thickness and complex refractive index of first layer of the DBR pair.
- **layer2** and **complex_index2** are the layer thickness and complex refractive index of second layer of the DBR pair.

- **period_number** is the number of periods of DBR pairs.

Example(s):

```
$ 5 periods  TiO2/SiO2 DBR at 0.525
rcled_dbr x_start_label=x2 x_end_label=x3 &&
  period_number=5 y_on_top=yes &&
  layer1=0.0597  complex_index1=(2.2 0.0) &&
  layer2=0.0899  complex_index2=(1.46 0.0)
```

The above example puts a DBR stack on top of the device. The DBR is defined between position of x2 and x3.

2.365 rcled_mix_lambertian

parameter	data type	values [defaults]
fraction	char	[0.]

The statement **rcled_mix_lambertian** is used to artificially mix the angle dependent optical power from the RCLED by a background component of Lambertian form. The RCLED model may be applied to a limited region in the whole device while the rest of the LED may contribute a background Lambertian portion. This statement may be used to fit experiment.

- **fraction** is used to split the total power from the RCLED model into two parts with one part of this fraction consisting of Lambertian emission form while the rest coming from the RCLED model.

Example(s):

```
rcled_mix_lambertian fraction=0.2
```

2.366 rcled_model

parameter	data type	values [defaults]
spont_all_points	char	no
optic_range_x1_label	char	void
optic_range_x2_label	char	void
optic_range_y1_label	char	void
optic_range_y2_label	char	void
top_emission	char	yes
update_spec_source	char	no
wavelength	real	
top_exit_index	real	1.
bottom_exit_index	real	1.
total_active_layer	real	0.1
group_index	real	3.8
ref_density	real	1.e23
ref_pn_ratio	real	1.
max_emit_angle	real	60
stack_view_point	real	[9999.]
num_optic_y_grid	intg	1000
num_spec_source	intg	120
num_emit_angle	intg	30

The statement **rcled_model** is used to control parameters related to the resonant cavity light emitting diode (RCLED) model. The basic set up of this model is that optical layers are parallel to the x-axis and the device emits mainly in +y or -y directions.

- **spont_all_points** indicates whether all mesh points are used to evaluate the spontaneous emission spectrum. Computation time is longer if all points are used.
- **optic_range_x1_label**, **optic_range_x2_label**, **optic_range_y1_label**, and **optic_range_y2_label** are four labels referring to four coordinate values defined by two previous **x_position** and two previous **y_position**. These are used to define a rectangle region within which multiple layer optical wave propagation calculation is performed. If one or more of these labels are not defined, the program uses the boundary of the device for optical wave modeling.
- **top_emission** indicate the whether main power emits to the top or bottom.
- **update_spec_source** indicate whether the RC source shape function is to be updated at all biases. The RCLED model relies on an angular and wavelength dependent resonant spectrum shape function (representing the finess of

angular information of the cavity). Such a shape function is scaled with the source dipole spectrum (spontaneous bulk emission spectrum) and output as total emission spectrum. Since the shape depends on the reflectivity of the cavity/DBR, it is usually unnecessary to update the shape function at every bias.

- **wavelength** is the main emission wavelength in microns.
- **top_exit_index** is the refractive index of the top exit medium.
- **bottom_exit_index** is the refractive index of the bottom exit medium.
- **total_active_layer** is an estimate of the total active layer thickness for the purpose of normalizing the optical power. This will affect the plotted power transmission distribution but will not affect the results of the simulation.
- **group_index** is the group index used to calculate the group velocity of light within the RCLED cavity.
- **ref_density** is a reference carrier density, or typical carrier density of the active layer which is used to compute the spontaneous dipole spectrum before a full blown simulation. The resonance cavity property is printed at the beginning of the simulation to a .res file (for resonance spectrum) and a .stw file (for standing wave). These files were computed using the reference density here.
- **ref_pn_ratio** is the ratio of hole density over electron density. This ratio is used together with **ref_density** above.
- **max_emit_angle** sets the maximum angle of emission to be computed in angular power distribution.
- **stack_view_point** relates to **stacl_refl** in command **rcled_spectrum_angle**. It is used to define a view point from which multiple layer optical reflection of the layer stacks below such a point can be plotted.
- **num_optic_y_grid** is the number of optical grid points in the y direction of the optical layer region (as defined in parameters such as **optic_range_x1_label**). The standing wave pattern is computed using this optical grid.
- **num_spec_source** is the number of RCLED emission spectrum points.
- **num_emit_angle** is the number of data points used in angular power distribution.

Example(s):

```
rcled_model wavelength=0.93 total_active_layer=0.024 &&
  optic_range_y1_label=dbri1_start top_emission=no
```

The above example defines an RCLED cavity to be between a y-position labeled as `dbr1_start` and the top of the device. The parts below `dbr1_start` will not be regarded as part of the optical cavity. Emission direction is towards the bottom.

2.367 `rcled_optic_layer`

parameter	data type	values [defaults]
<code>x_start_label</code> char [void]		
<code>x_end_label</code> char [void]		
<code>y_on_top</code> char [no]		
<code>layer1</code> real [0.1] (um)		
<code>complex_index1</code> realx2 [(3.5 1.e-6)]		
<code>layer2</code> real		
<code>complex_index2</code> realx2 [(3.5 1.e-6)]		
<code>layer3</code> real		
<code>complex_index3</code> realx2 [(3.5 1.e-6)]		
<code>layer4</code> real		
<code>complex_index4</code> realx2 [(3.5 1.e-6)]		
<code>layer5</code> real		
<code>complex_index5</code> realx2 [(3.5 1.e-6)]		

The statement **`rcled_optic_layer`** is used to define one or more optical layer within an RCLED. The basic set up of this model is that optical layers are parallel to the x-axis and the device emits mainly in +y or -y directions.

- **`x_start_label`**, **`x_end_label`** are the position labels for the left and right bounds of the layers. The labels were previously defined by **`x_position`** statements. If one or more of the labels are not defined, the program uses the boundary of the device.
- **`y_on_top`** is used to define the vertical position of the layer.
- **`layerk` ($k=1..5$)** and **`complex_indexk` ($k=1..5$)** are the layer thickness and complex refractive index of the kth layer.

Example(s):

```
$ SiO2 coating
rcled_optic_layer x_start_label=x1 x_end_label=x2 &&
    y_on_top=yes layer1=0.2 complex_index1=(1.46 0.)
```


2.368 rcled_plot_y

parameter	data type	values [defaults]
data_file	char	[void]
variable	char	[void],relative_power, real_index, imag_index, spon_rate
add_variable	char	[void],relative_power, real_index, imag_index, spon_rate
distance_range	realx2	[-1.e49 1.e49] (um)
variable_range	realx2	[-1.e49 1.e49]
at_mesh_near_x	real	[0.5](um)
scale_variable	real	[1.]
scale_add_variable	real	[1.]

plot_plot_y is a post-processor statement used to plot quantities related to RCLED as a function of the y-coordinate.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **variable** is the variable to be plotted. It takes one of “relative_power” (power distribution relative to the power source), “real_index” (real refractive index), “imag_index” (imaginary refractive index), or “spon_rate” (spontaneous emission rate).

add_variable adds a second variable to be plotted on the same figure. It takes one of the parameters listed under **variable**.

- **distance_range** is the distance range in y-direction.
- **variable_range** specifies a range for the function variable to be plotted.
- **at_mesh_near_x** specifies the position where a 1D slice is cut for the 1D plot.
- **scale_variable** is used to scale the function variable to be plotted.
- **scale_add_variable** is used to scale the 2nd function variable added by **add_variable** on the same plot.

Example(s)

```
rcled_plot_y variable=relative_power add_variable=real_index &&
  at_mesh_near_x=0.5 scale_variable=0.3 scale_add_variable=1
```

The above statement plots the relative optical power along the y-direction. Added to the same plot is the real part of the refractive index.

2.369 **rcled_power_angle**

This command is similar to **rcled_spectrum_angle** except the optical power has been integrated over the whole spectrum for a particular angle.

2.370 **rcled_refl_coating**

parameter	data type	values [defaults]
x_start_label	char	void
x_end_label	char	void
y_on_top	char	yes
power_refl	real	0.32
refl_phase_in_pi	real	0.
power_loss	real	0.

The statement **rcled_refl_coating** is used to define a reflection optical coating for an RCLED. The basic set up of this model is that optical layers are parallel to the x-axis and the device emits mainly in +y or -y directions.

- **x_start_label**, **x_end_label** are the position labels for the left and right bounds of the layer. The labels were previously defined by **x_position** statements. If one or more of the labels are not defined, the program uses the boundary of the device.
- **y_on_top** is used to define the vertical position of the coating.
- **power_refl** power reflectivity of the optical coating.
- **refl_phase_in_pi** is the phase of the reflectivity in π .
- **power_loss** is the power loss percentage related to the optical coating. It may also be used to define the power loss beyond the optical coating so that the total output power is affected by it.

Example(s):

```
$ For HR from metal, phase should be close to 180 degree to make zero
$ metal boundary:
rcled_refl_coating x_start_label=x1 x_end_label=x4 &&
  y_on_top=no power_refl=0.98 refl_phase_in_pi=1.3 power_loss=0
```

The above statement defines a metal coating with a phase close to 180 degrees so that the wave intensity is nearly zero at the metal.

2.371 rcled_spectrum

parameter	data type	values [defaults]
data_file	char	[void]
top_emission	char	[yes],no
stack_refl	char	[no]
wavelength_range	realx2	
angle	real	[0.] (degree)
broaden	real	(um)

rcled_spectrum is a post-processor statement used to plot the RCLED spectrum at a certain emission angle.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **top_emission** indicates whether emission is towards the top or bottom.
- **stack_refl** would enable the plotting of reflectivity of multiple layer stack viewed from a reference point defined in **stack_view_point** in command **rcled_model**.
- **wavelength_range** is the wavelength range in microns.
- **angle** is the emission angle in degrees.
- **broaden** is the spectrum broadening width in microns. It is the standard deviation of the gaussian broadening function.

Example(s)

```
rcled_spectrum top_emission=yes broaden=0.002
```

2.372 rcled_spectrum_angle

parameter	data type	values [defaults]
data_file	char	void
top_emission	char	yes
stack_refl	char	[no]
wavelength_range	realx2	(-9999 9999)
theta_range	realx2	(0 90)
broaden	real	-9999.
spectrum_range	realx2	(-9999 -9999)
view_xrot	real	0.
view_zrot	real	0.

rcled_spectrum_angle is a post-processor statement used to plot surface plot of a emitting power as a function of wavelength and emission angle for an RCLED.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **top_emission** indicates whether emission is towards the top or bottom.
- **stack_refl** would enable the plotting of reflectivity of multiple layer stack viewed from a reference point defined in **stack_view_point** in command **rcled_model**.
- **wavelength_range** is the wavelength range in microns.
- **theta_range** is the emission angle range in degrees.
- **broaden** is the spectrum broadening width in microns. It is the standard deviation of the gaussian broadening function.
- **spectrum_range** is the spectrum range.
- **view_xrot** rotates the plot around the x-axis (or the wavelength axis).
- **view_zrot** rotates the plot around the z-axis (or the power axis).

Example(s)

```
rcled_spectrum_angle top_emission=yes broaden=0.002
```

2.373 rcled_surface_plot_xy

parameter	data type	values [defaults]
data_file	char	[void]
variable	char	
xrange	realx2	
yrange	realx2	

plot_surface_plot_xy is a post-processor statement used to plot surface plot of a function of x and y for quantities related to RCLED.

- **data_file** is the file to which the graphic data is written in ASCII format.

- **variable** is the variable to be plotted. It takes one of “relative_power” (power distribution relative to the power source), “real_index” (real refractive index), “imag_index” (imaginary refractive index), or “spon_rate” (spontaneous emission rate).
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.

Example(s)

```
rcled_surface_plot_xy variable=relative_power
```

2.374 radiative_boundary

This statement is similar to **thermal_intef**. It is used to describe a radiative boundary. Most of the position dependent parameters are the same as those for **thermal_intef**. **emissivity** emissivity is the emissivity (fraction of black body radiation). **environ_temp** is the environment temperature.

Example(s)

```
radiative_boundary y=0.8 emissivity=0.9
```

2.375 radiative_recomb

parameter	data type	values [defaults]
material_par		

The statement **radiative_recomb** is the radiative (or spontaneous) recombination coefficient in units of m^3/sec . It is usually defined as the coefficient B such that the radiative recombination is given by $B(np - n_i)$ where n_i is the intrinsic carrier density. Note that this coefficient is only used for the non-active region of the laser. For active regions, the spontaneous emission rate is computed from first principles, analogous to the optical gain.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.376 rayplot_bias

parameter	data type	values [defaults]
variable	char	
data_file	char	

rayplot_bias is used to plot the bias dependence of specified variables generated by the ray-tracing program. This statement is used at the post-processing stage after ray-tracing program is run. Please note that for LED application, the results from the ray-tracing program may be different from those from the cubic broad-area device LED model as plotted by the **plot_scan** statement.

- **variable** is the physical variable to be plotted. It takes "total_power" (total emitted power of the LED from all emission angles outside of the device) or "extraction_eff" (power extraction efficiency).
- **data_file** would export the data to a file.

2.377 rayplot_angle

parameter	data type	values [defaults]
smooth	char	[yes] no
data_file	char	[void]
smooth_width	real	[8.](degree)

rayplot_angle is used to plot the angular dependence of the emission power from the LED. This statement is used at the post-processing stage after ray-tracing program is run.

- **smooth** indicates if data smoothing is performed.
- **data_file** copies the plot results to the specified data file.
- **smooth_width** is the width of the data broadening/smoothing. A Gaussian function is used for smoothing and this parameter is the standard deviation. The larger the standard deviation, the smoother the data curve.

2.378 rayplot_showray

rayplot_showray is used to plot the travel path of rays during the ray-tracing procedure. The program show.exe is used to plot the rays within a device. This statement is used at the post-processing stage after ray-tracing program is run. This statement has no parameters.

2.379 `real_func`

The command **real_func** is identical to **loop_real**.

2.380 `real_index`

parameter	data type	values [defaults]
material_par		

The material statement **real_index** is the real refractive index at the appropriate optical frequency. Please do not confuse this with the dielectric constant of Poisson's equation at DC or low frequencies.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.381 `real_index_spec`

parameter	data type	values [defaults]
material_par		

This statement is the wavelength dependent version of the **real_index** statement. As the free-style macro becomes available in recent versions, it has been outdated since the wavelength dependence can easily be incorporated into the **real_index** statement function by adding a variable (xlam).

2.382 refine_mesh

parameter	data type	values [defaults]
mesh_outf	char	
variable	char	(see list)
method_line_base	char	[yes]
var_diff	real	[3.0]
xrange1	real	-1.E10 (μm)
xrange2	real	1.E10
yrange1	real	-1.E10
yrange2	real	1.E10
min_density	real	10. (m^{-3})
zseg_num	intg	[1]
max_double_times	intg	[5]

refine_mesh refines an existing mesh adaptively according to a specified variable distribution. It should be used in the main solution module (or the .sol files).

- **mesh_outf** is the mesh output file. This parameter should be specified if the mesh data is used by the post-processor after refinement.
- **variable** is the position dependent variable according to which the mesh is refined. The following variables are accepted: donor_conc, acceptor_conc, 2.dopants(default), elec_conc, hole_conc, potential (volt) x_distance (um), y_distance (um), x_plus_y_distance (um), x_minus_y_distance (um) .
- **var_diff** is the criterion for mesh refinement. When the variable difference between two adjacent nodes is greater than this criterion, additional nodes are added between these two nodes to reduce the variable difference. For concentrations, the unit is decades. For potential, the unit is Volt.
- **method_line_base** indicate whether mesh line extension method is used to do mesh-refinement. For CSuprem imported mesh, only triangle based method is available.
- **xrange1** and **xrange2** are the lower and upper limits, in the x-direction, of the region within which the statement is effective.
- **yrange1** and **yrange2** are the lower and upper limits, in the y-direction, of the region within which the statement is effective.
- **min_density** is the minimum concentration density below which the refinement will not take effect.

- **zseg_num** is the z-segment number.
- **max_double_times** affects the mesh refinement only when **method_line_base=no**. The method for refining triangles are to double the mesh density several times until the criterion is met. This parameter limits the number of mesh double actions.

Example(s)

```
refine_mesh variable=acceptor_conc &&
  mesh_outf=msas.msh2 var_diff=3.0
```

2.383 reload_mater

parameter	data type	values [defaults]
zseg_num	intg	[1]

reload_mater can be used to force the solver to re-initialize the material data for device. It may be useful when doing mesh refinement and the solver needs to re-initialize.

2.384 replace_macrofile

parameter	data type	values [defaults]
macro1	char	[void]
macro2	char	[void]

The statement **replace_macrofile** may be used to replace the two default macro files containing material parameter models. This is useful when a user would like to use his/her own macro files without modifying the default macros.

- **macro1** is used to replace the default macro file **crosslight.mac**.
- **macro2** is used to replace the default macro file **more.mac**.

Example(s)

```
replace_macrofile macro1=my_macro.mac
```

2.385 resistor_region

parameter	data type	values [defaults]
mater	intg	[1]

resistor_region is used to declare that a certain region of semiconductor material can be treated as if it is a pure resistor.

- **mater** is the material number for the resistor region.

Example(s)

```
resistor_region mater=3
```

2.386 restart

parameter	data type	values [defaults]
data_set	intg	99
zseg_num	intg	[1]

This statement can be placed anywhere in the input file and used to restart a simulation that is interrupted by any cause. To restart a simulation, all you need to do is to add this statement to the original input file and re-run the simulation. The simulator will get restarted from the last printed data output file (.out file).

You may modify any parameters (eg., Newton parameters) in the restarted simulation provided that the **scan** statements for the existing output data sets do not change.

The **restart** statement works as follows. The simulator reads all the .out data files and compare the bias information in the saved output data with the **scan** in the input file. It will ignore the scan statements that has been run and will start at the next bias point.

- **data_set** is the starting data sets for the restarted simulator. If the data set exceeds the number of data sets available, the simulator will start at the data set with highest data set number.
- **zseg_num** is the z-segment number.

Example(s)

```
restart data_set=10
```

The above statement instructs the simulator to restart the simulation at data set number 10. However, If there are only 6 available data sets, the simulator will restart from data set 6. If **restart** with no parameter (with default data_set=99) is used, the simulator will always start from the last data set available.

2.387 right_contact

right_contact is the same as **left_contact** except that it is placed on the right side. Please see statement **left_contact**.

2.388 rtgain_phase

parameter	data type	values [defaults]
export_veeim	char	[no],yes
density	real	[3.e24] (m^{-3})
point_x	real	[0.1] (μm)
zseg_num	intg	[1]

rtgain_phase is used to generate round trip gain spectrum data at a specific carrier density so that the user can judge about the general device design. Use of this statement will generate a data file of extension .rtd (for round trip gain data). For VCSEL, the standing wave information is generated as .stw file.

- **export_veeim** works with the statement **vcSEL_eeim** together to export structure data needed by the VCSEL-EEIM (enhanced effective index method) model. The data will be placed in a file "veeim_data.tmp" which will be used by the VEEIM program for calculation of optical modes. The simulation program stops after printing the data. Please see also statements of **vcSEL_eeim** and **vcSEL_model**.
- **density** is the setting of electron and hole density in the active region assuming equal amount of electrons and holes.
- **point_x** is the x-position for plotting the standing wave pattern in a VCSEL.
- **zseg_num** is the z-segment number.

Example(s)

```
rtgain_phase density=1.5e14
```

2.389 scan3d

parameter	data type	values [defaults]
scan_voltage	char	[no]
voltage_control	char	[no]
precalculated_gain	char	[yes]
current_from	real	(Amp/m)
current_to	real	[20.] (Amp/m)
photon_scan	real	0.5
voltage_from	real	(V)
voltage_to	real	[0.] (V)
delta_time	real	[0.] (V)
wg_light_from	real	
wg_light_to	real	
pump_light_from	real	
pump_light_to	real	
3d_points	intg	2
photon_scan_num	intg	[4]
electrode	intg	[1]
zseg_num	intg	[1]
light_ext	intg	[1]
bias_scan_num	intg	[2]

scan3d is the most important statement for the 3D simulator. It is used to ramp up the current bias of the 3D device. It activates both the 2D (xy-plane) modules and the 1D (z-dimension) modules self-consistently. Once this statement is called, the 2D module is activated to scan various parameters of the different 2D xy-planes. The 2D module is put to sleep after scanning the xy-planes. Then the 1D module is activated to compute the longitudinal modes and optical power distribution. The 1D module is put to sleep and the 2D module is wakened again, and procedure repeats itself until the maximum bias is reached. The user will see the 2D and 1D modules switching on and off.

- **scan_voltage** is used to indicate if voltage bias is varied in a simulation.
- **voltage_control** is used to indicate whether voltage control is used during photon scan. This is not to be confused with **scan_voltage** which is the change of bias between photon scan. For reverse biased devices, it is necessary to use voltage control since the injection current is usually too small to be used as a control handle.
- **precalculated_gain** is to activate the gain model that pre-computes the gain spectrum (to be used by the longitudinal mode solver) as a function of the

carrier density assuming uniform carrier distribution within the active region. This is done only at rare intervals and for a large concentration and wavelength range.

It also assumes equal electrons and holes (charge neutrality in undoped layer) in the active region when computing the optical gain. If you know the electron hole ratio, the charge neutrality can be changed using the **gain_wavel** statement with parameter **pn_ratio**. You may also use **auto_pn_ratio** there to let the program adjust such ratio.

This gain model seems to result in better numerical stability, at the cost of self-consistency. The accuracy can be poor if the amount of electrons and holes in the active region differ by a large amount.

If this parameter is set to “no”, the optical gain/index used in the longitudinal mode solver is identical to the one used in the 2/3D drift-diffusion model. As a result, more accurate power efficiency and threshold may be obtained. For **precalculated_gain=no** to work properly, the **phone_scan** parameter should be set to a value comparable to the photon density of the waveguide.

For users of PICS3D for SOA, **precalculated_gain=yes** is recommended since the optical power may vary in a large range and it is hard to set a proper value for **photon_scan**.

- **current_from** gives the initial value of the 3D bias current. The unit is in Amperes per meter for the xy-plane for edge emitting laser and in Amp for cylindrical coordinate system as used in VCSEL.

This value should be slight below threshold while the next 3D bias point should be above threshold.

- **current_to** gives the final value of the bias current. The unit is the same as in **current_from**.
- **voltage_from** is the initial value of the 3D bias voltage. Use voltage bias if current bias has trouble converging. This value should be slight below threshold while the next 3D bias point should be above threshold.
- **voltage_to** gives the final value of the bias voltage.
- **delta_time** is the time interval corresponding to the whole **scan3d** statement in a transient simulation.
- **wg_light_from** is the starting value of the relative waveguide input power. The absolute power is given by the statement **3d_amplifier_model** or **waveguide_input**

- **wg_light_to** is the final value of the relative waveguide input power. The absolute power is given by the statement **3d_amplifier_model** or **waveguide_input**
- **pump_light_from** is the starting value of the relative optical pumping input power. The absolute power is given by the statement **light_power**.
- **pump_light_to** is the final value of the relative optical pumping input power. The absolute power is given by the statement **light_power**.
- **photon_scan** To collect information about the xy-segments in the 3D simulation, it is necessary to scan the photon density to “probe” the xy-segment. **photon_scan** may be used to control how much of photon density to scan. The amount of photon linear density variation is given by $photon_scan * 1.e8 / photon_fac1 / m$ for edge emitting laser and $photon_scan * 1.e8 / (photon_fac * fac_vcSEL) 1 / m$ for VCSEL, where **fac_vcsel** is the **photon_fac** given in **vcSEL** statement. The amount of photon to scan determines how accurate the 2D information is retrieved. This may directly affect the smoothness of the L-I curve in the 3D simulation.
- **3d_points** is used to set the number of 3D xy-segment scans from **current_from** to **current_to**. The more 3d_points, the more accurate the simulation and one should expect to see a smoother L-I curve. **3d_points** controls how self-consistent the 3D solver is. If **3d_points** is too small, the self-consistency or the coupling between 2D and 1D modules is poor. If it is too large, the administrative overhead and output data size will be very large. A typical value of 10-30 is recommended for most applications.
- **photon_scan_num** One of the major tasks of the 3D driver is to extract the optical modal gain spectrum data and carrier-current relation for different xy-planes. These data must cover a large enough range of carrier concentrations for each xy-plane. Each photon scan point will produce one gain curve. This parameter gives the number of photon scan points to be extracted from the xy-plane scan. The larger this number, the more accurate the modal optical gain spectrum, but also the more time consuming. A typical value of 4-10 is recommended.
- **electrode** is the electrode index number for the xy-plane. This should match the boundary number in the .geo file. It is the contact number used in the xy-plane, not the electrode number used in the z-direction.
- **zseg_num** defines the segment along the z-direction. The mesh, material and doping may be different for each segment.

- **light_ext** takes 0 or 1 . When it is 1, the xy-segment photon scan is based on extrapolation of light intensity from the previous step. If 0, no such extrapolation is taken place. The user should choose zero if scan3d turns out to be difficult to converge or the LI look extremely noisy. The reason is that extrapolation may be dangerous sometimes.
- **bias_scan_num** is the nominal bias steps in current/voltage bias between two **3d_points**. The actual bias steps may vary depending on the convergence situation.

2.390 scan

parameter	data type	values [defaults]
hot_electron	char	[no]
var	char	
vari(i=2..9)	char	[void]
function_labeli(i=2..9)	char	[void]
infile_label	char	[void]
outfile_label	char	[void]
stop_thermal	char	[no]
value_to	real	
valuei_to(i=2..9)	real	[-9999.]
print_step	real	[-9999.]
init_step	real	[-9999.]
min_step	real	[-9999.]
max_step	real	[-9999.]
mark_3d	intg	[0]
zseg_num	intg	[1]
power3d	intg	[0]
auto_finish	char	[void]
auto_condition	char	[within] above below
auto_until	real	[0.9]
auto_within	real	[0.01]
auto2_finish	char	[void]
auto2_condition	char	[within] above below
solve_rtg	char	[no] yes
auto2_until	real	[0.9]
auto2_within	real	[0.01]
Note for -9999		[-9999]=special treatment

The statement **scan** is used by the main simulator engine to activate the equation solver while one or more control variables are being changed (scanned).

Usually this means the device electrode is biased to a certain voltage to calculate the electrical characteristics. For voltage/current control of Crosslight's device simulators, the convention is that if it is not changed by the scan statement, the voltage of an electrode remains the same. For example, if a device has three electrodes and only one of them (say electrode number 3) is changed by a **scan** command. This means the other two (electrode numbers 1 and 2) have a voltage bias remaining unchanged. Before the simulation, all electrodes are assumed grounded.

On the other hand, the current flow at an electrode can be floating unless explicitly controlled by a **scan** statement. In the previous example of three electrodes, when voltage of electrode 3 is scanned, the current flow at electrode 1 may be varied according to device physics while its voltage is being fixed. In some applications, an electrode may be open completely (with current flow being zero) during the entire simulation (eg., in latchup simulation). For such a case, a simple treatment would be not to declare such an electrode boundary at all so that the boundary there would be regarded as natural semiconductor or metal boundary without current flowing outwards. If the boundary is already declared as an electrode and you still wish it to be open (zero current), you must specify the current there as a scanned variable and control it to be zero for all the scan commands. Without current control by scan, such an electrode would only have fixed voltage while current would be most likely be flowing there.

The convention for the sign of the current flow is that current flowing outwards of the device is defined as positive current while flowing inwards is regarded as negative current. Please note that this convention is different than other programs.

The simulation program uses the **scan** command to vary many other control variables such as external circuit resistance and incident light power. For transient simulation, the main controlling variable is time and usually many other variables change with it and therefore, the scan command is critical for transient simulation.

A variable (such as the voltage of a contact) is scanned (or biased) from an initial solution to a new solution under new bias. As shown in Figure 2.17, the progress of the scanned variable is usually not strictly controlled. It can vary from a minimum step to a maximum step. If the user wishes to have more solutions, a smaller maximum step may be chosen so that more solution data points may be obtained. For example, if a user wishes to see more points in the I-V curve, the maximum step may be set to the desirable interval and the program would be forced to take that step size.

The design of this command is such that fewer Newton iteration would be required to reach the final bias.

- **var** is the name of the main variable to be scanned. It is also the first variable if more than one variable are to be scanned (or varied). For details of settings for this parameter, please see the table at the end of this section.

- **vari**($i=2..9$) is the name of the i th variable to be scanned in addition to the main variable above. For details of settings for this parameter, please see the table at the end of this section. The variation of these variables are by default, linearly proportional to the first scanned variable **var** above.
- **function_labeli**($i=2..9$) is used only if more than one variable is scanned. It is used to declare or label a function relation between the i th variable with the main variable **var**. Such as function relation is further defined in a separate statement called **scan_function**. If this parameter is not used, the program assumes the default linear relation between the i th variable and the main scanned variable **var**.
- **infile_label** is the label of the initial guess data file for this **scan** statement. Without the use of this parameter, the program take the previous solution (.out file) as the initial guess. The label of a previous **scan** is defined by the **outfile_label** parameter.
- **outfile_label** is the label of the final output data file from this statement. The label may be used by a later **scan** statement to be referred to as the initial guess data file. Without the use of this parameter, the final output data file from this statement will be labeled as "void".
- **stop_thermal** instructs the main solver to stop using heat-flow model for the operation of the present statement. This means the temperature distribution will be frozen regardless bias condition.
- **value_to** is the final value of the first scanned variable to be reached. As an example, one may want to ramp up the voltage from zero volt (equilibrium) to 1.6 volts (see Fig. 2.17). There is no default for this parameter and it must be defined for each scan command. Please note that the initial value of a scanned variable is not explicitly defined in the present scan command because the program gets it from a previous **scan** command.
valuei_to($i=2..9$) if set by user, is the final value of the i th variable. If this variable is not defined by user, a separate **scan_function** statement must be used to determine how the i th variable is to be varied with respect to the first or main variable **var**.
- **print_step** is the interval of the first scanned variable **var** at which full structural data are to be printed to the hard disk. Since there may be many bias points for one **scan** statement, it is important to choose this interval so that the proper amount of structural data is printed without taking up too much hard disk space. Please see Fig. 2.17) for the relationship between bias points and **print_step=0.8**. Each printed data set includes all the structural data at the present bias and all the bias data (i.e. I-V and L-I data) between the previous printed data set and the present one. For example the second data set

is printed at 0.8 volts and it may include several I-V and L-I data points from zero bias to 0.8 volts, as well as all the detailed structural data (e.g. electron concentrations at mesh points) at 0.8 volts. If this parameter is not defined by the user, the program sets it to difference between the initial and final value of the scanned variable.

- **init_step** is the initial scanning (or bias) step to be taken, which starts at a known solution (such as the solution found at equilibrium or from a previous scan) and tries to find a new solution at a new bias set by the initial bias step. This step is needed because it is not always possible to go to a high bias (set by **value_to** at one step. Once the solution at this new bias is found the program will decide what the next bias point should be until **value_to** is reached. If this parameter is not defined by user, the program defaults it to 20 percent of the difference between the initial and final value of the scanned variable.
- **max_step** is the maximum bias step allowed. It is useful when many bias points are required between the initial solution and the final solution. If this parameter is not defined by user, the program defaults it to 50 percent of the difference between the initial and final value of the scanned variable.
- **min_step** is the minimum scanning step allowed. When the actual bias step is smaller than this, the program recognizes that it will take too long to reach the final bias and the will stop execution. Or in another word, it is difficult to achieve convergence for such a condition. If this parameter is not defined by user, the program defaults it to small fraction of 1×10^{-5} of difference between the initial and final value of the scanned variable.
- **zseg_num** defines the segment on which bias/variables are to be changed. This is only used when a multiple segment device is solved using decoupled method so that one segment is solved at a time.
- **hot_electron** The program may also solve for the electron energy distribution. The electron energy or electron temperature is the fourth variable to be solved in the basic equations after potential, electron and hole concentrations.
- **mark_3d** is an internal parameter used by PICS3D for the purpose of setting up **scan3d**. The user need not set this parameter.
- **power3d** same as **mark_3d**.
- **auto_finish** is used to terminate the bias condition according to a specific condition. The **scan** statement will terminate if the value of this parameter reaches the value set by parameter **auto_until**. This parameter takes one of the following: If it is set to "rtgain" (PICS3D), the simulator checks the round trip gain of the longitudinal modes so that the action of this statement will be terminated if the round trip gain reaches the value set by parameter

auto_until. Similarly, it may take “current_1” to ”current_5” to use current at one of the electrodes as a condition for termination. “laser_power_total” “laser_power_front” and “laser_power_back” may be used to denote total, front facet or back facet laser power in laser diode simulation (LASTIP). “voltage_k” means the voltage bias of the kth electrode. “incr_current_k” means the rate of current increase for the kth electrode per bias unit. For example, if voltage bias is applied to electrode 1, using incr_current_2 would mean if current at electrode 2 increases at a rate per unit voltage bias at electrode 1, the **scan** command will be terminated. ”step_current_k” or ”step_voltage_k” means using step size of electrode current or voltage as condition for auto-finish. ”lattice_temp” would use maximum lattice temperature as a condition.

Please note that all controlling variables mean the absolute magnitude. For example, -2 volt controls in the same way as 2 volt.

- **auto_condition** is condition used to terminate the scan. For value of within, above and below, the scan will terminate only if the bias is within, above and below the value given in **auto_until**.
- **auto_until** is the value of control parameter (round trip gain for example) that is used as criteria when **auto_finish** function is activated to terminate the operation of the **scan** statement. If the device is biased to satisfy this criteria, the present **scan** will be terminated and the simulator proceed to the next statement.
- **auto_within** is the error tolerance of how the criteria in **auto_finish** is satisfied.
- **auto2_finish auto2_condition auto2_until auto2_within** , the solver supports a 2nd condition for automatically finishing the scan procedure. If the these parameters are defined, both conditions must be satisfied to finish the scan. For details, please refer to the corresponding parameters of **auto_**.
- **solve_rtg** turns on the round-trip-gain (RTG) equations for longitudinal mode solution. That is to say the simulator not only solves the usual drift-diffusion equations, it also solves the longitudinal waveguide optical modes. For this parameter to work, a previous **scan** command should use set **auto_finish=rtgain** so that the RTG equation parameters are initialized.

Example(s)

```
scan var=voltage_1 value_to=5
```

The above simple command instructs the program to solve (similar to the solve command for some other programs) the equations from a previous bias to a bias of 5 volts for electrode number 1. The bias step and printing interval take defaults values.

If the device has more than one electrode, this command states that the voltages for all other electrodes remain the same during the scan.

```
equilibrium
scan var=voltage_2 value_to=10
```

The above two simple commands instruct the program to solve equations at equilibrium (zero bias) first. Then solve equations until the bias of electrode 2 reaches 10 volts.

```
scan var=voltage_1 value_to=1.6 print_step=0.8 &&
  init_step=0.02 min_step=1.e-5 max_step=0.1
```

The above statement biases the voltage at contact number one to a value of 1.6 volts. A small bias of 0.02 volts is tried initially. It may take a dozen or so bias points to get to 1.6 volts. The following statement scans both the time and voltage linearly.

```
scan var=time var2=voltage_1 value_to=1.e-11 &&
  value2_to=0.5 print_step=1.e-12 &&
  init_step=3.e-13 min_step=1.e-13 max_step=1.e-12 &&
```

The following scans the time (transient simulation) to 50 pico seconds as the first variable. The second variable is incident light power (relative) which relates to the first variable time via a Gaussian function defined in a separate statement **scan_function**. Please note the use of label **gs_func** to identify which **scan_function** it is referred to.

```
scan var=time value_to=50.e-12 &&
  var2=light function_label2=gs_func max_step=1.e-12
scan_function label=gs_func type=gaussian gsn_dt=4.e-12
```

The statement **scan_function** can be used to define many types of functions such as square pulse and sin() and cos() functions.

voltage_i (i=1,2,...,9)	Voltage of the ith electrode.
current_i (i=1,2,...,9)	Current of ith electrode. Positive current flows out of the device and negative current flows in. Unit is Ampere/m. For 3D simulation or cylindrical coordinate modeling, unit is Ampere.
light	The light intensity of the incident light. For photo-detectors, the unit is determined by the light_power . For optical amplifiers, the unit is determined by yz_amplifier.
light2	Used by the optical amplifier model for the intensity of the 2nd light source. For optical amplifiers, the unit is determined by yz_amplifier.
new_doping	The peak doping density profile marked by new_doping in the .sol file.
new_dopingi (i=2,3,4,5)	The peak doping density profile marked by new_dopingi in the .sol file.
bandgap_reduction	The relative reduction of overall energy bandgap.
mobility_control	Factor multiplied to the low field mobility affecting all parts of the device. The factor is expressed as $10^{m.c}$ where m.c. is "mobility_control".
surf_charge_expo	Exponent of a factor multiplied to the surface charge density as defined by the interface statement. The factor is expressed as $10^{s.c.e}$ where s.c.e is the "surf_charge_expo".
light_3damp	is the incident light intensity in a 3D waveguide (PICS3D simulation).
xcir_resistance_i(i=1..9)	is the external circuit resistance attached to the ith contact.

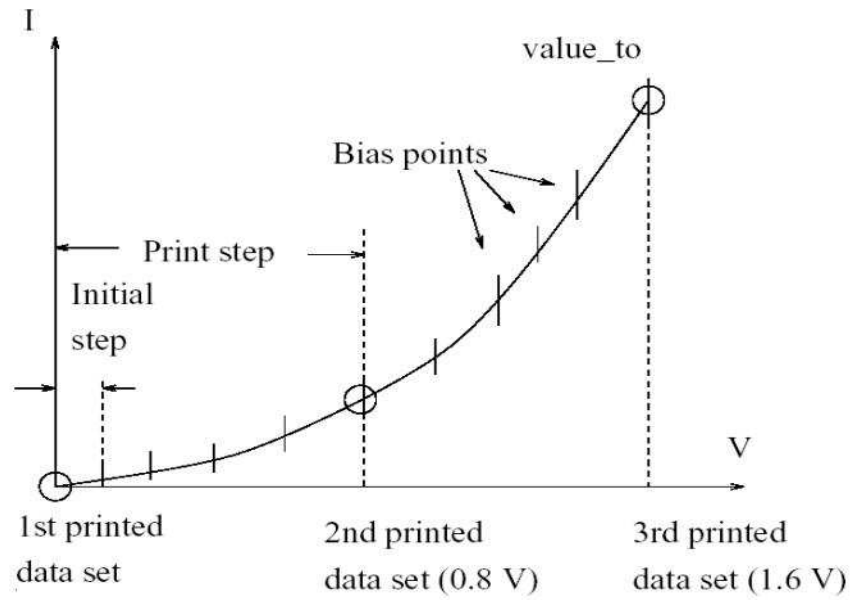


Figure 2.17: Schematics for the `scan` statement.

2.391 scan_function

parameter	data type	values [defaults]
label	char	[mylabel]
type	char	[void]
user_datafile	char	[void]
gsn_t1	real	[0.]
gsn_dt	real	[4.e-12]
gsn_s1	real	[0.]
gsn_s2	real	[1.]
pulse_t1	real	[0.]
pulse_tr	real	[1.e-12]
pulse_dt	real	[4.e-12]
pulse_tf	real	[1.e-12]
pulse_s1	real	[0.]
pulse_s2	real	[1.]
sin_constant	real	[0.]
sin_amp	real	[1.]
sin_period	real	[1.e-9]
sin_phase	real	[0.]
cos_constant	real	[0.]
cos_amp	real	[1.]
cos_period	real	[1.e-9]
cos_phase	real	[0.]

The statement **scan_function** is used to define relation between different variables in the **scan** statement.

- **label** corresponds to the parameter of **function_Labeli**(i=2..9) defining a function between the *i*th variable and the first (main) variable.
- **type** is the type of function. It currently supports the following types: **gaussian**, **pulse**, **sin** and **cos**. Their meanings are as suggested by their names.
- **user_datafile** if defined, is the user data file containing two columns of data to define a function relation.
- **gsn_t1** , **gsn_dt** , **gsn_s1** and **gsn_s2** are parameters t_1 , Δt , s_1 and s_2 respectively, used to describe a Gaussian relation between the 1st and 2nd variable. If we use t as the 1st variable, the Gaussian function is defined as follows:

$$if \quad t_1 \leq t \leq t_1 + 2\Delta t \quad (2.47)$$

$$\begin{aligned}
 c_l &= 4\ln(2) \\
 var2 &= s_1 + \frac{s_2 - s_1}{1 - \exp(-c_l)} \left[\exp\left(-\left(\frac{t - t_1 - \Delta t}{\Delta t}\right)^2 c_l\right) - \exp(-c_l) \right] \\
 &\text{else} \\
 var2 &= s_1
 \end{aligned}$$

- **pulse_t1** , **pulse_tr** , **pulse_dt** , **pulse_tf** , **pulse_s1** and **pulse_s2** are used to define a square pulse of the following shape in the code:

```

if( t>=t_1 && t<=t_1+t_r ) {
    var2=s_1+(s_2-s_1)*(t-t_1)/t_r;
} else if(t.ge.t_1+t_r.and.t.le.t_1+t_r+dt) {
    var2=s_2;
} else if(t.ge.t_1+t_r+dt.and.t.le.t_1+t_r+dt+t_f) {
    var2=s_2+(s_1-s_2)*(t-(t_1+t_r+dt))/t_f;
} else {
    var2=s_1;
}

```

- **sin_constant** , **sin_amp** , **sin_period** , and **sin_phase** are used to define a sines function of the form $\text{constant} + \text{amp} * \sin(2 * \pi / \text{period} * \text{var} + \text{phase})$, where phase is in unit of radians.
- **cos_constant** , **cos_amp** , **cos_period** , and **cos_phase** are used to define a cosines function similar to that of sines above.

Example(s)

The following example is to describe a sines function:

```

scan_function label=my_sine_lable type=sin sin_constant=0 sin_amp=0.2 &&
    sin_period=1.e-9 sin_phase=0

```


2.392 section

parameter	data type	values [defaults]
active	char	[yes] no
grating_model	char	[kappa] klayers (k=1...)
kappa_real	real	(1/ m)
kappa_imag	real	(1/ m)
delta_kreal	real	[0.] (1/ m)
delta_kimag	real	[0.] (1/ m)
length	real	[300.e-6] (m)
phase_shift	real	[0.] (π)
mesh_ratio	real	[1.]
layerj (j=1..k)	real	[0.096e-6] (m)
indexj (j=1..k)	real	
h1_real	real	(1/ m)
h1_imag	real	(1/ m)
surface_p.c	real	
top_refl	real	[0.3]
sec_num	intg	[1]
mesh_points	intg	[8]
pitch	real	m
delta_pitch	real	[0.] (m)
grating_phase	real	[0.] (π)

The statement **section** is used to specify the physical properties of a section of a laser. A laser can be divided into many sections. Each section can be further divided into subsections (or into a mesh) for calculation purposes. For the purpose of symbol definition, we write the z-dependent coupling coefficient as

$$\kappa(z) = \kappa_m(z) \exp(j\psi_k(z)),$$

$$\kappa_m(z) = \kappa_{m0} + (z - z_1)\delta\kappa_m/(z_2 - z_1).$$

$$\psi_k(z) = \psi_{k0} + (z - z_1)\delta\psi_k/(z_2 - z_1).$$

where z_1 and z_2 are the starting and ending points of the section.

Here we define chirp as the spatial variation of the propagation constant βl . We use ζ_{ch} to denote this variation. Then the chirp (in units of π) can be written as

$$\zeta_{ch}(z) = \zeta_{ch0} + (z - z_1)\delta\zeta_{ch}/(z_2 - z_1).$$

- **active** is used to indicates that the section contains active layer.

- **grating_model** is the grating model being used. It may take the following values: 1) **kappa**: kappa model uses kappa to describe the grating structure in an edge laser/waveguide structure. 2) **1layer**, **2layers**, ..., **klayers**: The layer model uses multiple layers of DBR consisting of periods of **klayers** ($k=1\dots$) of different indices. In older versions, **layer0** is equivalent to **1layer**; **layer** is equivalent to **2layers**.
- **kappa_real** and **kappa_imag** are used to represent the real and imaginary parts of index coupling and gain/loss coupling, respectively. They are effective only when the above parameter **grating_model=kappa**. If these are not provided, the program estimates the corresponding values from the statement of **grating_model** (not to be confused with the above parameter **grating_model** in this statement).
- **delta_kreal** and **delta_kimag** are used to represent the change in the real and imaginary parts of index coupling and gain/loss coupling, respectively.
- **length** is the length of the section.
- **phase_shift** is the phase shift (in units of π) at the end of the section. For example, in a standard phase shifted DFB laser, we need to create two sections. The phase shift is placed at the end of the first section.
- **mesh_ratio** is the ratio of the adjacent mesh interval. If it is unity (default), the mesh is uniform. If it is greater than 1, the mesh interval increases with distance. If it is less than 1, the mesh interval decreases with distance. If it is negative, the mesh obeys the absolute value over the first half of the distance and the other half is symmetric to the first half. A schematic diagram illustrating the effect of various ratios is given in Fig. 2.19.
- **layerj** ($j=1..k$) is the thickness of the j th layer that make up one period in the layer model when parameter **grating_model=klayers**.
- **indexj** ($j=1..k$) is the refractive index of the j th layer that make up one period in the layer model when **grating_model=klayers**. We discuss this in several cases. For a single layer, i.e., **grating_model=1layer**, if the index value is specified directly by the user, such a value will be used by the program at all times during the simulation for computation of longitudinal modes. If the single layer index is not specified, the program computes the index from the material macros and updates it at different injection/temperature conditions. For DBR gratings of a VCSEL/RCLED consisting of more than one layer in a period, one must specify all layer indices. There are two ways of doing this. One way is to specify refractive indices of the layers directly using positive numbers for all layers in a period. In this case, the program uses such values at all times during the simulation regardless of the injection/temperature conditions. The

other way is to specify the indices using negative signs (say -3.65, -3.45 for 2layers). Negatively signed indices force the program to use an average value of bulk index from the macro according to injection and temperature conditions while the index difference between layers are kept at a constant as specified by the negatively signed indices (layer 1 and layer 2 index difference will always be set at 0.2 in the above 2layers example). The change in the average value due to injection/temperature effects will directly affect the tuning condition of a VCSEL/RCLED. If we wish to model accurately index/loss change of each layer in a DBR stack, we should specify each layer as a single layer without any user-defined index. This way, the program automatically provides modeling details of optical, electrical and thermal properties of all layers in a DBR stack. However, the drawback for doing so is that a large number of mesh is required.

- **sec_num** is the section number or section index.
- **mesh_points** is the subsection number or mesh points used for the purpose of calculation. All physical quantities are assumed to be constant within the subsection.
- **grating_phase** is the phase of the grating (see Ω in the User's Manual).
- **pitch** and **delta_pitch** define the grating pitch and the change in the pitch within a section. If **pitch** is not specified by user, the reference pitch **ref_pitch** is assumed.
- **h1_real** and **h1_imag** are the real and imaginary part of the h1 parameter describing the 2nd order grating structure.
- **surface_p.c** is the surface power coupling coefficient for 2nd order grating. If not provided, the simulator computes this parameter.
- **top_refl** is the

Example(s)

```
section length=800.e-6 sec_num=1 mesh_points=20
```

2.393 section_air

parameter	data type	values [defaults]
length	real	[300.e-6] (m)
sec_num	intg	[1]
mesh_points	intg	[3]

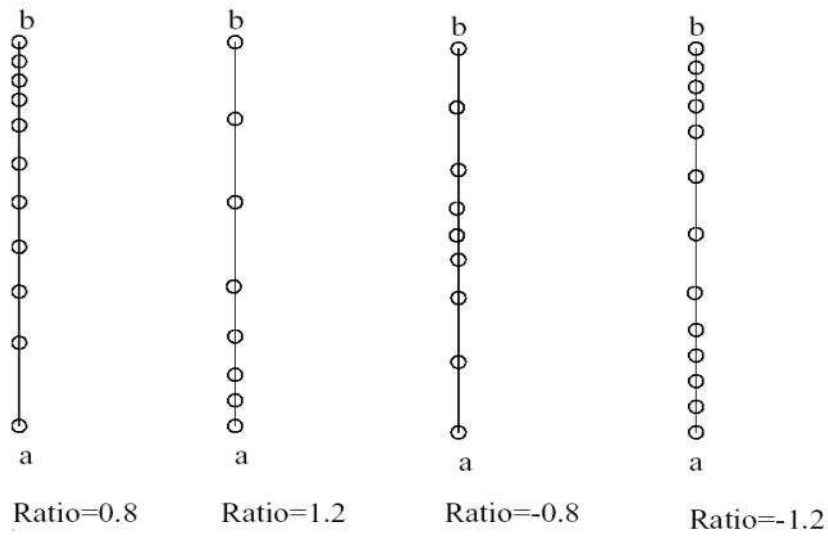


Figure 2.18: Effects of different ratio on putting mesh points on an edge (a b).

The statement **section_air** is a special case of **section**. It is used to describe an air gap within a photonic device.

- **length** is the length of the section.
- **sec_num** is the section number or section index.
- **mesh_points** is the subsection number or mesh points used for the purpose of calculation. All physical quantities are assumed to be constant within the subsection.

Example(s)

```
section_air length=200.e-6 sec_num=2 mesh_points=10
```

2.394 section_location

parameter	data type	values [defaults]
start	real	[0] (um)
end	real	[1.] (um)

- **section_location** is used to specify the starting point of the longitudinal mode cavity. If not specified, it will start from zero. For VCSEL, this statement may be automatically generated by the LAYER program.
- **start** is the starting coordinate of the optical cavity.
- **end** is the ending coordinate of the optical cavity.

Example(s)

```
section_location start=2. end=12.5
```

2.395 section

parameter	data type	values [defaults]
active	char	[yes] no
grating_model	char	[kappa] klayers (k=1...)
kappa_real	real	(1/ <i>m</i>)
kappa_imag	real	(1/ <i>m</i>)
delta_kreal	real	[0.] (1/ <i>m</i>)
delta_kimag	real	[0.] (1/ <i>m</i>)
length	real	[300.e-6] (m)
phase_shift	real	[0.] (π)
mesh_ratio	real	[1.]
layerj (j=1..k)	real	[0.096e-6] (m)
indexj (j=1..k)	real	
h1_real	real	(1/ <i>m</i>)
h1_imag	real	(1/ <i>m</i>)
surface_p.c	real	
top_refl	real	[0.3]
sec_num	intg	[1]
mesh_points	intg	[8]
pitch	real	<i>m</i>
delta_pitch	real	[0.] (<i>m</i>)
grating_phase	real	[0.] (π)

The statement **section** is used to specify the physical properties of a section of a laser. A laser can be divided into many sections. Each section can be further divided into subsections (or into a mesh) for calculation purposes. For the purpose of symbol definition, we write the z-dependent coupling coefficient as

$$\kappa(z) = \kappa_m(z) \exp(j\psi_k(z)),$$

$$\kappa_m(z) = \kappa_{m0} + (z - z_1)\delta\kappa_m/(z_2 - z_1).$$

$$\psi_k(z) = \psi_{k0} + (z - z_1)\delta\psi_k/(z_2 - z_1).$$

where z_1 and z_2 are the starting and ending points of the section.

Here we define chirp as the spatial variation of the propagation constant βl . We use ζ_{ch} to denote this variation. Then the chirp (in units of π) can be written as

$$\zeta_{ch}(z) = \zeta_{ch0} + (z - z_1)\delta\zeta_{ch}/(z_2 - z_1).$$

- **active** is used to indicate that the section contains active layer.
- **grating_model** is the grating model being used. It may take the following values: 1) **kappa**: kappa model uses kappa to describe the grating structure in an edge laser/waveguide structure. 2) **1layer**, **2layers**, ..., **klayers**: The layer model uses multiple layers of DBR consisting of periods of **klayers** ($k=1\dots$) of different indices. In older versions, **layer0** is equivalent to **1layer**; **layer** is equivalent to **2layers**.
- **kappa_real** and **kappa_imag** are used to represent the real and imaginary parts of index coupling and gain/loss coupling, respectively. They are effective only when the above parameter **grating_model=kappa**. If these are not provided, the program estimates the corresponding values from the statement of **grating_model** (not to be confused with the above parameter **grating_model** in this statement).
- **delta_kreal** and **delta_kimag** are used to represent the change in the real and imaginary parts of index coupling and gain/loss coupling, respectively.
- **length** is the length of the section.
- **phase_shift** is the phase shift (in units of π) at the end of the section. For example, in a standard phase shifted DFB laser, we need to create two sections. The phase shift is placed at the end of the first section.
- **mesh_ratio** is the ratio of the adjacent mesh interval. If it is unity (default), the mesh is uniform. If it is greater than 1, the mesh interval increases with distance. If it is less than 1, the mesh interval decreases with distance. If it is negative, the mesh obeys the absolute value over the first half of the distance and the other half is symmetric to the first half. A schematic diagram illustrating the effect of various ratios is given in Fig. 2.19.
- **layerj** ($j=1\dots k$) is the thickness of the j th layer that make up one period in the layer model when parameter **grating_model=klayers**.

- **indexj** ($j=1..k$) is the refractive index of the j th layer that make up one period in the layer model when **grating_model=klayers**. We discuss this in several cases. For a single layer, i.e., **grating_model=1layer**, if the index value is specified directly by the user, such a value will be used by the program at all times during the simulation for computation of longitudinal modes. If the single layer index is not specified, the program computes the index from the material macros and updates it at different injection/temperature conditions. For DBR gratings of a VCSEL/RCLED consisting of more than one layer in a period, one must specify all layer indices. There are two ways of doing this. One way is to specify refractive indices of the layers directly using positive numbers for all layers in a period. In this case, the program uses such values at all times during the simulation regardless of the injection/temperature conditions. The other way is to specify the indices using negative signs (say -3.65, -3.45 for 2layers). Negatively signed indices force the program to use an average value of bulk index from the macro according to injection and temperature conditions while the index difference between layers are kept at a constant as specified by the negatively signed indices (layer 1 and layer 2 index difference will always be set at 0.2 in the above 2layers example). The change in the average value due to injection/temperature effects will directly affect the tuning condition of a VCSEL/RCLED. If we wish to model accurately index/loss change of each layer in a DBR stack, we should specify each layer as a single layer without any user-defined index. This way, the program automatically provides modeling details of optical, electrical and thermal properties of all layers in a DBR stack. However, the drawback for doing so is that a large number of mesh is required.
- **sec_num** is the section number or section index.
- **mesh_points** is the subsection number or mesh points used for the purpose of calculation. All physical quantities are assumed to be constant within the subsection.
- **grating_phase** is the phase of the grating (see Ω in the User's Manual).
- **pitch** and **delta_pitch** define the grating pitch and the change in the pitch within a section. If **pitch** is not specified by user, the reference pitch **ref_pitch** is assumed.
- **h1_real** and **h1_imag** are the real and imaginary part of the h_1 parameter describing the 2nd order grating structure.
- **surface_p.c** is the surface power coupling coefficient for 2nd order grating. If not provided, the simulator computes this parameter.
- **top_refl** is the

Example(s)

```
section length=800.e-6 sec_num=1 mesh_points=20
```

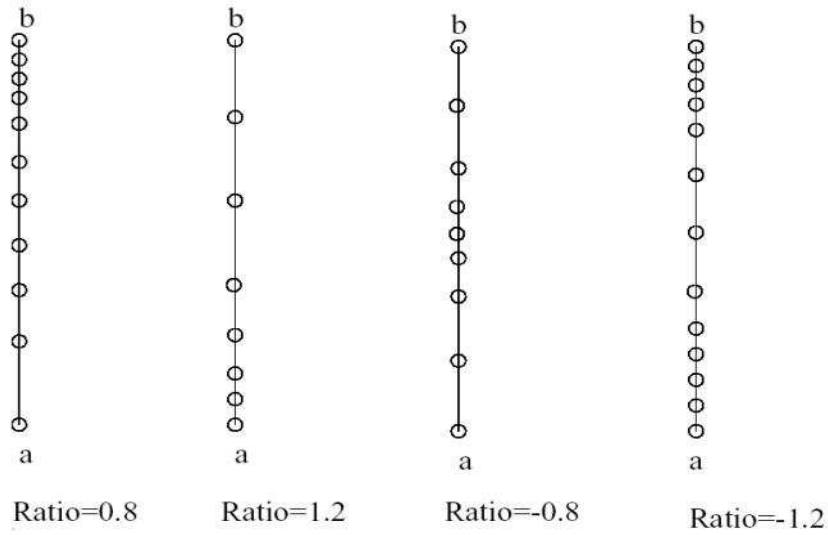


Figure 2.19: Effects of different ratio on putting mesh points on an edge (a b).

2.396 self_consistent

parameter	data type	values [defaults]
conditional_off	char	[no]
impose_limit	char	[no]
loop_num	intg	[9]
wave_range	real	[0.005] (um)
sample_distance	real	(um)
electrode_off	intg	[1]
change_limit	real	[0.1]
voltage_off	real	[-9999.]
current_off	real	[-9999.]
wave_range_bottom	real	(um)
wave_range_top	real	(um)
wave_range_left	real	(um)
wave_range_right	real	(um)

The **self_consistent** statement is used in the .sol file to invoke the self-consistent quantum well carrier density model. By default, flat-band confined states are used in our simulators assuming uniform carrier density profile with modification from local variation of Fermi level and confined quantum level. This statement will enable the use of quantum carrier profile in addition to variation due to Fermi level change. This model is especially important when piezo-electric effects need to be taken into account in Wurtzite material systems.

The quantum mechanical wave equation is solved within all complexes when this statement is used. A complex is defined as a number of layers coupled to each other due to overlap of wave functions.

The self-consistent quantum mechanical solver finds quantized energy states which may appear or disappear when the potential profile is changed. Such appearance or disappearance may cause the global Newton solver to be poorly initialized and thus diverge. In such a case, one may wish to turn off the self-consistent solver conditional upon the bias being beyond a certain value. This means the density of states (DOS) of the QW/QDOT will be frozen at a value before the conditional turn-off.

The parameters **conditional_off** **electrode_off** **change_limit** **voltage_off** and **current_off** are related to convergence of the self-consistent quantum mechanical solver and should not be adjusted unless convergence has become an issue in a self-consistent simulation.

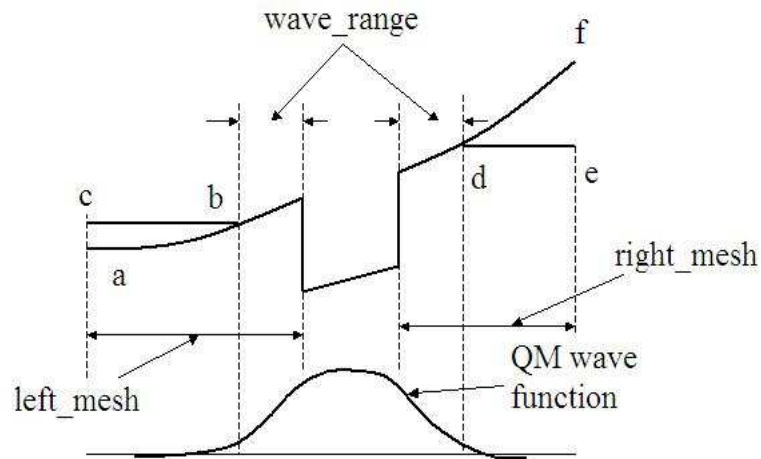


Figure 2.20: Schematic diagram explaining the self_consistent statement.

Consider an MQW system with arbitrary conduction band (similarly for valence band) potential profile. The critical parameters are related to boundary conditions at the first and last quantum wells. For simplicity, we consider a special case of boundary condition for a single quantum well as shown in Fig. 2.20. Usually the range of 1D solution is chosen to be sufficiently large such that the confined quantum mechanical wave function will decay to zero at **left_mesh** and **right_mesh** of the **modify_qw** statement. However, the potential profile we choose to solve must not extend this far since stable confined quantum states may not exist when one side of the potential is too low. Therefore, we choose a reasonable distance **wave_range** to truncate the potential profile (at points b and d) and horizontally (or artificially) extend the potential (to points c and e instead of the original c and f). After such adjustment of potential profile, reasonable confined states can be obtained in most cases. If more confined states are required than from the above solution, the parameter **more_cond_energy** of the **modify_qw** statement may be used to increase the range within which confined levels are searched.

- **wave_range** is used to define a range for the solution of the quantum mechanical wave equation. Please note that the solution of the The range starts from a distance of “wave_range” before the first quantum well of a complex and ends at a distance of “wave_range” after the last quantum well of the complex. If a different material (or another quantum) appears within this distance of “wave_range”, the range will be cut short by the material interface. This range serves two purposes: 1) the conduction/valence band profile within this range is used to solve the Schroedinger wave equation; 2) the wave function is truncated beyond this range, which means the wave function may or may not be zero at this point of range. With the potential at both ends extended horizontally beyond this parameter, the wave function is forced to be zero at a distance defined by parameters of **left_mesh** and **right_mesh** in the **modify_qw** statement for the left and right side, respectively. If a complex MQW system is under large voltage bias, the selection of this parameter is critical since it determines the number of confined states. Any states with energies above the confining potential at both ends of this range are regarded as unconfined states.

sample_distance, if specified, is the absolute position of a data sample slice across a quantum well along a direction perpendicular to the quantum well plane. We use potential profile from such a slice to compute the quantum levels. If this parameter is not supplied by the user, the default location is to use a potential profile sampled at mid-point along the length of the quantum well on a 2D plane.

- **loop_num** is the maximum number of iteration in equilibrium that is need to reach self-consistency for both the potential and quantum confined carrier density profile.

- **conditional_off** is to indicate whether the self-consistent update of quantum mechanical wave solver is to be turned off conditionally for convergence reason.
- **electrode_off** is used only when **conditional_off=yes**. It defines the electrode used as a condition to turn of the self-consistent update solution.
- **impose_limit** indicates if a limit is imposed on relative change of potential for self-consistent calculation. Too large a change is thought to be harmful to convergence. One may turn on this option if non-convergence due to quantum confinement is encountered.
- **change_limit** is to impose a limit on relative change of potential profile in the quantum solution. The simulator calculates the average applied field and will only allow the profile to be changed by a certain amount.
- **voltage_off** is used only when **conditional_off=yes**. It defines the voltage of the electrode used as a condition to turn of the self-consistent update solution. When the voltage bias is beyond this value, the self-consistent update is turned off.
- **current_off** is used only when **conditional_off=yes**. It defines the electrical current (in units of A/m or A for 2 or 3D) of the electrode used as a condition to turn of the self-consistent update solution. When the current bias is beyond this value, the self-consistent update is turned off.

wave_range_bottom, **wave_range_top**, **wave_range_left**, **wave_range_right** are related to **wave_range** above. By default, same **wave_range** boundary is set for all four sides of the quantum region until overridden by one or more of these parameters.

Example(s):

```
self_consistent loop_num=15
```

The following statement updates quantum states for drain voltage below 5 volts only, for convergent reason.

```
self_consistent wave_range=0.05 &&
  electrode_off=1 conditional_off=yes voltage_off=5
```

This means the above 5 volts, the quantum states and density of states (DOS) are the same as at 5 volts and the density of states. This may cause some inaccuracy.

2.397 `set_active_reg`

is used to override all parameters in the **active_reg** previously used regardless of the material number. This statement is especially useful if we wish to override parameters in .mater file which are automatically generated by the Layer program. Please see **active_reg** for detailed explanation of each parameter.

2.398 `set_barrier_width`

parameter	data type	values [defaults]
barrier_width_bot	real	[0.1] (um)
barrier_width_top	real	[0.1] (um)
active_mater	intg	[1]

set_barrier_width is used to let the program take into account the barrier composition grading when modeling the carrier confinement of a quantum well. The program needs to know the distance over which the composition grading takes place. It works together with statement **grade_active_bar**.

- is the bottom barrier width.
- is the top barrier width.
- is the active layer material of the quantum well related to the barrier described in this statement.

Example:

```
set_barrier_width barrier_width_bot=0.01 barrier_width_top=0.01 active_mater=3
```

2.399 `set_minority_carrier`

The command **set_minority_carrier** may be used to artificially reduce the bandgap so that larger minority carrier density can be used to overcome possible convergence problems.

The parameter **virtual_eg_kt** can be used to set a reduced bandgap in unit of kT where k is the Boltzman constant.

Example(s)

```
set_minority_carrier virtual_eg_kt=30
```

would set the bandgap to be 30 kT for the purpose of computing minnority carriers.

2.400 set_polarization

parameter	data type	values [defaults]
xy_plane	char	[no]
ref_column	intg	[1]
screening	real	[0.5]
substrate_latt	real	[3.1989] (Å)

set_polarization is used to work with the LAYER program so that interface charges due to polarization effect for nitride material layers may be produced in the .mater file to be included in electrical simulation.

So far the following material macros are recognized: algan, ingan, alinn, gaalinn, and gan.

The model is based on the following reference: Vincenzo Fiorentini, Fabio Bernardini, Oliver Ambacher "Evidence for nonlinear macroscopic polarization in III-V nitride alloy heterostructures," Appl. Phys. Lett., vol. 80, No. 7, p.1204, 2002.

- **xy_plane** would produce interface charge on the x-y plane for a 3D simulation. The interface charge would be defined using the **doping** command.
- **ref_column** is a reference column of the layer structure which has a complete layer stacks of all layers.
- **screening** is the screening factor. A factor of unity means 100 percent of the theoretical interface charge is used.
- **substrate_latt** is the substrate lattice constant. By default, it is set to that of GaN.

Example(s)

```
set_polarization screening=0.7
```

This would allow 70 percent of the total theoretical value be used on the layer interfaces.

2.401 set_spon_spectrum

parameter	data type	values [defaults]
average_polarization	char	[yes]

set_spon_spectrum is used to set the polarization status for the spontaneous emission spectrum.

- **average_polarization** is used to indicate if spontaneous emission spectrum is averaged over different polarizations. If not, the TE/TM mode setting in the **active_reg** statement will take effect.

Example(s)

```
set_spon_spectrum average_polarization=no
```

2.402 set_wavelength

parameter	data type	values [defaults]
wavelength	real	[0.5] (um)
backg_loss	real	[500] (1/m)

set_wavelength is used to define some critical optical properties of a device. It is used to initialize the optical mode distribution and internal loss.

- **wavelength** is the operating optical wavelength.
- **backg_loss** is the background loss coefficient. By default, regions outside the active region are assumed to have a constant loss background (which can be determined experimentally). Alternatively one can use the material statement **absorption** to set the material loss in different materials. A non-zero **absorption** setting will override the **backg_loss** here.

Example(s)

```
set_wavelength wavelength=0.5 backg_loss=1000.
```

2.403 set_xydata_for_scan

parameter	data type	values [defaults]
scan_var	char	[void]
scan_var_scale	char	[linear]
scan_var_facet	char	[void]
scan_var_scale_lit	real	[2.]
scan_var_scale_curr	real	[2.]
scan_var_scale_horizontal	real	[1.]
scan_var_horizontal_power	real	[-9999.d0]
scan_var_mode_index	intg	[1]
scanline	intg	
scan_num	intg	

set_xydata_for_scan is a post-processor statement used to set plotting options affecting subsequent plots. It is used to convert structure data (xy-data) into bias dependent plots.

- **scanvar** is used to define scan variables to be used in AC analysis plots. The scan variables are the same as used in the **plot_scan** statement. Please refer to **plot_scan** for more details.

scanvar_xxx, all parameters with prefix **scanvar_** are the same as those (without the prefix) defined in **plot_scan** statement. Please refer to **plot_scan** for more details.

- **scanline** is used to denote bias data from a whole **scan** statement. The data from statement **equilibrium** is counted as the scanline=1; those from the next **scan** statement counted as scanline=2, etc. For this parameter to work, the **get_data** command must already include the data sets associated with this parameter.
- **scan_num** is used to denote bias data from a whole **scan** statement. The data from statement **equilibrium** is counted as the scan_num=0; those from the next **scan** statement counted as scanline=1, etc. For this parameter to work, the **get_data** command must already include the data sets associated with this parameter.

```
set_xydata_for_scan scanvar=voltage_1
```

The above statement will cause the program plot small signal quantities (regarded as xy-data or structural data) versus voltage bias at electrode number 1.

2.404 set_3dray_internal_interface

parameter	data type	values [defaults]
model	char	[fixed_index]
import_file1	char	void
import_file2	char	void
import_file3	char	void
import_file4	char	void
import_file5	char	void
import_file6	char	void
import_file7	char	void
import_file8	char	void
import_file9	char	void
thickness1	real	[0.1] (um)
thickness2	real	[0.1] (um)
thickness3	real	[0.1] (um)
thickness4	real	[0.1] (um)
thickness5	real	[0.1] (um)
thickness6	real	[0.1] (um)
thickness7	real	[0.1] (um)
thickness8	real	[0.1] (um)
thickness9	real	[0.1] (um)
refr_index1	realx2	[2.1245, 0.0]
refr_index2	realx2	[2.1245, 0.0]
refr_index3	realx2	[2.1245, 0.0]
refr_index4	realx2	[2.1245, 0.0]
refr_index5	realx2	[2.1245, 0.0]
refr_index6	realx2	[2.1245, 0.0]
refr_index7	realx2	[2.1245, 0.0]
refr_index8	realx2	[2.1245, 0.0]
refr_index9	realx2	[2.1245, 0.0]
inside_ref_point	realx3	[0.0, 0.0, 0.0] (um)
outside_ref_point	realx3	[0.0, 0.0, 0.0] (um)
nlay	intg	[1]

set_3dray_internal_interface is used to specify special internal interfaces for the 3D raytracing program in APSYS. For example, this can be used to represent surface coatings that inside the a structure's geometry. For coatings on the outer surface of a structure, the **surface_model** parameter of **do_raytrace_3d** should be used instead. To define the position of an interface, the user must enter the XYZ coordinates of two points on either side of the interface.

The number of layers in the coating is controlled by **nlay**. The parameters numbered 1 to 9 refer to that particular layer. Layer #1 is defined as the one closest to **inside_ref_point**.

The complex refractive index of each layer is controlled by the **model** parameter. If this is set to **fixed_index**, then the value of the **refr_index** parameter will be used. If **model** is set to **import_data**, then the index will be read from a text file specified by the **import_file** parameter. This file must be formatted so that the first column contains the wavelength (in μm), the second column, the real part of the refractive index and the third column, the imaginary part of the refractive index.

2.405 `shal_acpt_level`

parameter	data type	values [defaults]
group1		

shal_acpt_level is used to define the shallow acceptor level of the p-type dopants. The level is measured from the valence band and in the unit of eV.

Note that the parameter **level** of the **doping** statement may be used to describe the energy level of the shallow or deep impurities over the whole device. If the properties of the impurity in each different material need to be defined, the user must override the **level** parameter with one of the statements **shal_dnr_level**, **shal_acpt_level** or **trap_level.i**.

2.406 `shal_acpt_level.i`

parameter	data type	values [defaults]
group1		

shal_acpt_level.i is used to define the shallow acceptor level of the *i*th p-type dopant. The level is measured from the valence band and in the unit of eV.

Note that the parameter **level** of the **doping** statement may be used to describe the energy level of the shallow or deep impurities over the whole device. If the properties of the impurity in each different material need to be defined, the user must override the **level** parameter with one of the statements **shal_dnr_level**, **shal_dnr_level.i**, **shal_acpt_level**, **shal_acpt_level.i** or **trap_level.i**.

2.407 `shal_dnr_level`

parameter	data type	values [defaults]
group1		

`shal_dnr_level` is used to define the shallow donor level of the n-type dopants. The level is measured from the conduction band and in the unit of eV.

Note that the parameter **level** of the **doping** statement may be used to describe the energy level of the shallow or deep impurities over the whole device. If the properties of the impurity in each different material need to be defined, the user must override the **level** parameter with one of the statements `shal_dnr_level`, `shal_acpt_level` or `trap_level.i`.

2.408 `shal_dnr_level.i`

parameter	data type	values [defaults]
group1		

`shal_dnr_level.i` is used to define the shallow donor level of the *i*th n-type dopant. The level is measured from the conduction band and in the unit of eV.

Note that the parameter **level** of the **doping** statement may be used to describe the energy level of the shallow or deep impurities over the whole device. If the properties of the impurity in each different material need to be defined, the user must override the **level** parameter with one of the statements `shal_dnr_level`, `shal_dnr_level.i`, `shal_acpt_level`, `shal_acpt_level.i` or `trap_level.i`.

2.409 `show_ray_3d`

parameter	data type	values [defaults]
<code>show_model</code>	char	[one_ray]
<code>emit_from_xyz</code>	realx3	[0 0 0] (um)
<code>emit_from_phi</code>	real	[45] (degrees)
<code>emit_from_theta</code>	real	[0] (degrees)
<code>show_secondary</code>	intg	[20]
<code>show_angle_num</code>	intg	[20]

`show_ray_3d` works for 3D ray-tracing and is used to generate traveling ray paths data in a form so that the GNUPLOT script `_sho.gnu` can be used to plot the ray

paths. This statement should be used before the statement **do_raytrace_3d** so that the specified ray paths can be saved in data files.

- **show_model** takes one of “one_ray”, “all_direction”, “all_theta” and “all_phi”, meaning to plot one-ray, many rays in all directions, many rays in all theta angles and many rays in all phi angles, respectively. Here we refer to spherical coordinates theta and phi angles.
- **emit_from_xyz** is the position of the source where the traveling rays will be shown. If a source does not exist at this position, a source nearest to it will be shown.
- **emit_from_phi** may vary from 0 to 360 degrees and is the fixed angle of phi if **show_model=all_theta**.
- **emit_from_theta** may vary from -90 to 90 degrees and is the fixed angle of theta if **show_model=all_phi**.
- **show_secondary** is the number of secondary rays to be shown.
- **show_angle_num** is the number of initial angular rays from one point source to be shown.

Example(s)

```
show_ray_3d emit_from_phi=90 emit_from_theta=0 show_secondary=16520 &&
  emit_from_xyz=(300 4 100) show_model=all_phi &&
  show_angle_num=15
```

The above statement enables the showing of many rays in all the phi angles (on x-y plane). The theta angle is fixed at 0 degrees, meaning the ray directions are exactly on x-y planes. A total of 15 initial rays for such conditions are to be plotted. The figure will show the paths of all the rays originated from these 15 initial rays.

2.410 `shift_affinity`

parameter	data type	values [defaults]
<code>delta_affinity</code>	real	[0.0] (eV)
<code>mater</code>	intg	[1]

shift_affinity is used to shift the electron affinity for the whole material. This is very convenient for fused junction devices where two completely different material systems are forced together. This statement may be used to control the band offset easily instead of defining a new affinity for each material.

- **delta_affinity** is the amount of shift in electron affinity.
- **mater** is the material number being affected.

Example(s)

```
shift_affinity delta_affinity=0.05 mater=3
shift_affinity delta_affinity=0.05 mater=4
shift_affinity delta_affinity=0.05 mater=5
```

The above example shifts the conduction band downwards by 0.05 eV for materials with material numbers of 3, 4, and 5.

2.411 sor_par

parameter	data type	values [defaults]
max_iter	intg	[5000]

sor_par was previously a statement commonly used to solve the lateral modes of a waveguiding structure using the SOR iterative method. This statement appeared in many example devices as the default wave equation solver. Since the availability of the more reliable and more accurate direct sparse eigen solver in new versions, this solver is automatically replaced by the direct eigen solver (statement **direct_eigen**). Therefore, use of this statement with `max_iter>0` means the use of **direct_eigen** with default parameters. Please refer to the statement of **direct_eigen** for more details. To force the simulator to use the older SOR solver (for compatibility reason), please use the statement **use_sor**.

2.412 sparse_eigen_solver

parameter	data type	values [defaults]
use_mf	char	[yes],no
fast_dynamic_search	char	[no],yes

sparse_eigen_solver is used to control the sparse eigen solver used in the .sol file. These include lateral mode solver for wave guide modes and quantum mechanical wave solver for quantum wells.

- **use_mf** is used to indicate whether multi-frontal solver will be used.
- **fast_dynamic_search** indicates whether a fast dynamic mode search technique is used.

Example(s)

```
sparse\_eigen\_solver use_mf=yes
```

2.413 sp.rate_wavel

parameter	data type	values [defaults]
data_file	char	
auto_pn_ratio	char	[no]
field_dep_curves	char	[no]
wavel_range	realx2	(μm)
conc_range	realx2	[1.e23 1.e24] (m^{-3})
pn_ratio	real	[1]
av_index	real	[3.3]
field_range	realx2 [1.e5 1.e7]	
curve_number	intg	
data_point	intg	

sp.rate_wavel plots the spontaneous emission rate versus wavelength.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **auto_pn_ratio** is used to indicate if automatic setting of hole/electron density ratio is used in PICS3D simulation. If positive, averaged hole/electron density from the drift-diffusion solver is used. This parameter, if positive, will override **pn_ratio**.
- **field_dep_curves** indicates whether or not the curves are generated with different applied electrical field intensities. If this is set to no, the curves are plotted with different carrier concentrations while the field is set to zero. If this is set to yes, the carrier concentration is set to the initial value in **conc_range** while the field is varied.
- **av_index** is the estimated average refractive index.
- **wavel_range** is the wavelength range.
- **conc_range** is the electron concentration range in the well.

- **pn_ratio** is the ratio of hole over electron concentrations. Note that this ratio can be set to an arbitrary number in the gain preview. In the main solver, this ratio is automatically determined by the simulator according to the local Fermi levels.
- **field_range** is the range of the applied field for producing the different curves. This parameter is active only if **field_dep_curves** is set to yes.
- **curve_number** is the number of spectral curves to be plotted for different carrier concentrations.
- **data_point** is the number of data points in one curve.

Example(s)

```
sp.rate_wavel wavel_range=(1.0 1.4) &&
conc_range=(5.e23 5.e24) curve_number=5 data_point=100
```

2.414 spec_heat

parameter	data type	values [defaults]
(see) material_par		

The material statement **spec_heat** is used to define the specific heat (in J/kg/K) of the semiconductor material.

The use of this parameter and related examples are given under **material_par** in section [2.272](#).

2.415 splot_xyz

parameter	data type	values [defaults]
variable	char	elec_conc, hole_conc, trap1_conc, trap2_conc, potential, occup_trap1, occup_trap2, donor_conc, acceptor_conc, wave_intensity, local_gain, etc...
data_file	char	[vtttek]
wave_option	char	[right]
xy_from	realx2	
xy_to	realx2	
view_xrot	real	[0.]
view_zrot	real	[0.]
xrange	realx2	
yrange	realx2	
zrange	realx2	
z_min	real	[-1.e9]
z_max	real	[1.e9]
mode_index	intg	1
grid_sizes	intgx2	[20, 20]
trap_index	intg	[1]

splot_xyz is a post-processor statement used to plot surface involving all xyz dimensions. Surface is chosen by cutting a line on the xy-plane and extending the line along the z-direction.

- **variable** is the variable to be plotted.
- **data_file** is the file to which the graphic data is written in ASCII format. Note that plot_device should be set to “data_file” for this parameter to take effect.
- **wave_option** is used to specify which longitudinal mode is being plotted. It takes the following values:
 - **right** for right going wave of the main longitudinal mode.
 - **left** for left going wave of the main longitudinal mode.
 - **total** for total of **right** and **left** added together.
 - **sm_right** for right going wave of the 1st longitudinal side mode.
 - **sm_left** for left going wave of the 1st longitudinal side mode.
 - **sm_total** for total of **sm_right** and **sm_left** added together.

- **xy_from** The surface is chosen by cutting a line on the xy-plane and extending the line along the z-direction. This statement is used to define the beginning point of the cut.
- **xy_to** The surface is chosen by cutting a line on the xy-plane and extending the line along the z-direction. This statement is used to define the ending point of the cut.
- **view_xrot** rotate the plot around the x-axis.
- **view_zrot** rotate the plot around the z-axis.
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.
- **zrange** specifies a range for the z-axis.
- **z_min** is the minimum value of z-range of the plot.
- **z_max** is the maximum value of z-range of the plot.
- **mode_index** is used to label the lateral mode number if wave intensity is being plotted.
- **grid_sizes** are the grid sizes of 3D plot.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.

Example(s)

```
splot_xyz variable=elec_conc xy_from=(0.5, 1.4970) xy_to=(0.5 1.5030) &&  
grid_sizes=(60, 20) view_zrot=20.
```


2.416 splot_xy

parameter	data type	values [defaults]
variable	char	(see list)
data_file	char	
integration	char	[no]yes
point_ll	realx2	(μm)
point_ur	realx2	(μm)
view_xrot	real	[0.]
view_zrot	real	[0.]
xrange	realx2	
yrange	realx2	
zrange	realx2	
z	real	(micron)
integration_xrange	realx2	[-1.e4 1.e4] (μm)
integration_yrange	realx2	[-1.e4 1.e4] (μm)
grid_sizes	intgx2	[20, 20]
mode_index	intg	[1]
trap_index	intg	[1]

splot_xy is a post-processor statement used to plot surface data on the xy-plane at a specific z-location.

- **variable** is the variable to be plotted. The detailed variable listing is given in App. C.
- **data_file** is the file to which the graphic data is written in ASCII format.
- **integration** is used to turn on the numerical integration on x-y plane of the variable being plotted.
- **point_ll** is the lower-left corner point of the 2D window.
- **point_ur** is the upper-right corner point of the 2D window.
- **view_xrot** rotate the plot around the x-axis.
- **view_zrot** rotate the plot around the z-axis.
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.
- **zrange** specifies a range for the axis of the variable being plotted.

- **z** specifies the z-coordinate of the x-y plane on which the variable is being plotted.
- **integration_xrange** specifies a x-range for numerical integration. (see also integration parameter above).
- **integration_yrange** specifies a y-range for numerical integration. (see also integration parameter above).
- **grid_sizes** are the grid sizes of surface plot.
- **mode_index** is the index of the lateral mode in a multi-lateral mode simulation. If only the fundamental mode is concerned, the default value of one should be used.
- The parameter **trap_index** has been added for SRH recombination, so that the user can specify which trap center is being plotted.

Example(s)

```

plot_xy variable=elec_conc z=50. grid_sizes=(35, 35) &&
      xrange=(0. 1.5) yrange=(0.0 3.0) view_xrot=0 view_zrot=30

```

2.417 start_loop

parameter	data type	values [defaults]
symbol	char	[void]
value_from	intg	[1]
value_to	intg	[2]
step	intg	[1]

start_loop is a preprocessor statement used in .sol/.plt file to describe a looped variable.

- **symbol** is the looped variable. **value_from** is the starting value of the looped integer. **value_to** is the ending value of the looped integer. **step** is the step of looped integer variable.

Example(s)

```

start_loop symbol=%j value_from=1 value_to=3 step=1
scan var=voltage_%j value_to=0.5
end_loop

```

The above commands cause the symbol %j to take values of 1,2 and 3 while the **scan** command is repeatedly used to scan the voltage of electrode numbers 1, 2 and 3, in sequence. The above is equivalent to the following three commands:

```
scan var=voltage_1 value_to=0.5
scan var=voltage_2 value_to=0.5
scan var=voltage_3 value_to=0.5
```

2.418 stop

This statement forces the solver to stop. It has no parameters. If this statement is used to force a stop of the simulator at a certain point, the simulation program thinks it is an abnormal stop and will not print the .std data.

2.419 strain_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **strain_bar** is an active layer macro statement used to define the strain in the quantum barrier. Compressive strain is negative. (eg., 3 percent strain is 0.03)

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.420 strain_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **strain_well** is an active layer macro statement used to define the strain in the quantum well. Compressive strain is negative. (eg., 3 percent strain is 0.03)

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.421 strained_mobility

parameter	data type	values [defaults]
carrier_type	char	[n] p
vsat_factor	real	[0.8]
strained_material	char	[yes],no
import_ref_population	char	[void]
export_ref_population	char	[void]
vsat_factor	real	[0.8]
unstrained_110_enh	real	[1.]
angle_from_100	real	[0.] (degree)

The statement **strained_mobility** is used to specify the strain dependent mobility for strained silicon. The basic idea is to evaluate a strain dependent mobility enhancement factor based on valley/subband population. Then this enhancement factor is multiplied to the unstrained silicon mobility to construct the strained mobility. The strained mobility is evaluated according to valley/subband averaged effective conduction masses in different directions, as well as valley splitting.

- **carrier_type** is the carrier type of interests.
- **strained_material** indicates whether the structure is strained. This setting affects reference population data import/export as detailed below.
- **import_ref_population** is the data file to be imported that stores the quantum level population of the unstrained material as a reference for the purpose of computing the strain-induced effective mass and mobility change. This is to be used only when **strained_material=yes**.
- **export_ref_population** is the data file to be exported that stores the quantum level population of the unstrained material as a reference for the purpose of computing the strain-induced effective mass and mobility change. This is to be used only when **strained_material=no**.
- **vsat_factor** is the factor used for the saturation velocity. If this factor is unity, both low field and the saturation velocity are enhanced by the same factor.
- **unstrained_110_enh** is the enhancement of unstrained hole mobility in the 110 direction due to anisotropic effective hole mass.
- **angle_from_100** is the angle of crystal orientation with respect to 100.

Example:

```
strained_mobility carrier_type=p vsat_factor=0.5 &&
  import_ref_population=ref_pmos_population strained_material=yes &&
  unstrained_110_enh=0.93 angle_from_100=45
```

The above command imports quantum level reference data from a file placed in the installation directory and sets up the strained mobility model for angle=45 degrees from 100.

2.422 suprem_contact

parameter	data type	values [defaults]
side	char	[upper],lower,left,right
num	intg	
xrange	realx2	(um)
yrange	realx2	(um)
touch_mater	intg	

suprem_contact is used to define electrodes on the mesh imported from CSuprem, a process simulation program typically used to simulate the dopant profile.

- **side** specifies the which side the contact is on.
- **num** is the contact number.
- **xrange** is x-range of the contact.
- **yrange** is y-range of the contact.
- **touch_mater** indicate the material index to which the contact is in touch with.

Example(s)

```
suprem_contact xrange=(-0.05 0.05) num=2 side=upper touch_mater=1
```

The above defines a contact -0.05 to 0.05 on top of mesh which touches the material number 1 within APSYS simulator.

2.423 **suprem_property**

parameter	data type	values [defaults]
generic_impurity	char	[void],donor,acceptor
donor_type2	char	[void], arsenic ,phosphorus ,anti- mony ,selenium ,silcon ,germanium ,generic
acceptor_type2	char	[void] ,boron ,beryllium ,magne- sium ,tin ,zinc ,carbon ,generic
n_doping_level	real	[0.01] (eV)
p_doping_level	real	[0.01] (eV)
n_doping_level2	real	[0.01] (eV)
p_doping_level2	real	[0.01] (eV)
qw_thick	real	[0.01] (um)
qw_xrange	realx2	[-0.05 0.05] (um)
silicon_mater	intg	
poly_mater	intg	
gaas_mater	intg	
aluminum_mater	intg	
oxide_mater	intg	
nitride_mater	intg	
oxynitride_mater	intg	
photoresist_mater	intg	
qw_silicon_mater	intg	

suprem_property is used to define properties related to data imported from the CSuprem, a process simulation program typically used to simulate the dopant profile. Please note that the coordinate system in APSYS is different than SUPREM in that the positive y coordinate points upwards in the former.

- **generic_impurity** defines the type of dopant assigned to generic impurity.
- **donor_type2** assigns a different type (type 2) of donor so that different ionization energy can be defined.
- **acceptor_type2** assigns a different type (type 2) of acceptor so that different ionization energy can be defined.
- **n_doping_level** is the ionization energy of the donor.
- **p_doping_level** is the ionization energy of the acceptor.
- **n_doping_level2** is the ionization energy of the donor type 2.

- **p_doping_level2** is the ionization energy of the acceptor type 2.
- **qw_thick** is the quantum well (qw, if any) thickness.
- **qw_xrange** is the x-range of the quantum well.
- **silicon_mater** defines the material number for silicon.
- **poly_mater** defines the material number for poly silicon.
- **gaas_mater** defines the material number for GaAs.
- **aluminum_mater** defines the material number for aluminum.
- **oxide_mater** defines the material number for SiO2.
- **nitride_mater** defines the material number for silicon nitride.
- **oxynitride_mater** defines the material number for oxy-nitride.
- **photoresist_mater** defines the material number for photo-resist.
- **qw_silicon_mater** defines the material number for quantum-well silicon. This is actually the same material as silicon but since quantum well region receives special treatment, a different material number is assigned.
-

Example(s)

```
suprem_property silicon_mater=1 oxide_mater=3 qw_silicon_mater=2 &&
poly_mater=4 &&
generic_impurity=acceptor acceptor_type2=generic p_doping_level2=0.156 &&
qw_xrange=(-0.05 0.05)
```

The above defines a QW from -0.05 to 0.05 microns and also defines acceptor type 2 as having ionization energy of 0.156 eV.

2.424 symbol_variable

parameter	data type	values [defaults]
variation	char	[function] table
returned	char	
num	intg	[1]

symbolic_variable is used to define functional relationship between variables in the command system. The supported syntax and mathematical expressions are identical to those in the material macro library file. Please refer to the header information in the crosslight.mac file that comes with the software installation.

- **variation** decides if the relationship is defined by a function or a look up table. For the former, a math function of the form should follow

```
function(var1,var2,...) end_function
```

For a table the following form should be used:

```
table(var1,var2,...) end_table
```

- **returned** is the returned symbol which may be used in any of the states following it.
- **num** is a number labeling the relation definition in case there are more than one such definitions.

Example(s)

```
symbolic_variable returned=%y
function(%x)
s2=%x**2+sin(%x);
s2**2+%x-5.
end_function
```

The above function is used to define a relation between symbolic variable %x and %y so that

```
symbolic_variable variation=table returned=%ai
table(%volt)
0.3  10.
0.8  18.
1.8  28.
end_table
```

This statement uses a table to define the relation. Linear interpolation will be used is the argument does not fall on the tabulated values.

2.425 tau_density

parameter	data type	values [defaults]
hh_or_lh	char	[hh] lh ch
gamma_detail	char	[no] yes
data_file	char	[void]
density_start	real	[2.e23] (m^{-3})
density_end	real	[2.e24] (m^{-3})
pn_ratio	real	[1.]
data_point	intg	[10]
elec_level	intg	[1]
hole_level	intg	[1]

tau_density plots the intraband relaxation time (ie., **tau_sc**) in 10^{-13} sec. versus carrier density. This statement is only used in the gain preview module.

- **hh_or_lh** indicates whether heavy hole or light hole subband is involved.
- **gamma_detail** may be used to plot individual contribution to gain broadening factor (Γ) from different scattering mechanisms such as electron-electron, electron-hole, etc..
- **data_file** is the file to which the graphic data is written in ASCII format.
- **density_start** is the starting point of the carrier density at which the intraband relaxation time is calculated.
- **density_end** is the ending point of the carrier density at which the intraband relaxation time is calculated.
- **pn_ratio** is the ratio of hole over electron concentrations. Note that this ratio can be set to an arbitrary number in the gain preview. In the main solver, this ratio is determined automatically by the simulator according to the local Fermi levels.
- **data_point** is the number of data points in data curve.
- **elec_level** and **hole_level** are the electron and hole subband levels involved in the optical transition.

Example(s)

```
tau_density gamma_detail=yes density_end=5.e24
```

2.426 `tau_energy`

parameter	data type	values [defaults]
(see) <code>material_par</code>		

tau_energy is the energy relaxation time of hot electrons in seconds. It is used only when hot electron model is activated.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.427 `tau_model`

parameter	data type	values [defaults]
<code>phonon_omega_LO</code>	real	[34.E-3] (eV)
<code>epsilon_infinity</code>	real	[11.4]
<code>mater</code>	intg	1

tau_model is used to turn on the intraband scattering model and to set a couple of important material parameters for this model. This statement may be used in both the gain preview file (.gain) and also the main input file (.sol).

- **phonon_omega_LO** is the longitudinal optical phonon energy in eV.
- **epsilon_infinity** is the optical (or at infinite frequency) dielectric constant.
- **mater** is the material number of the active region under consideration.

Example(s)

```
tau_model mater=2 phonon_omega_LO=46.E-3 epsilon_infinity=10.4
```

2.428 tau_temperature

parameter	data type	values [defaults]
hh_or_lh	char	[hh] lh ch
gamma_detail	char	[no] yes
data_file	char	[void]
temp_start	real	[200.]
temp_end	real	[400.]
density	real	[2.e24] (m^{-3})
pn_ratio	real	[1.]
data_point	intg	[10]
elec_level	intg	[1]
hole_level	intg	[1]

tau_temperature plots the intraband relaxation time (ie., **tau_scatt**) in 10^{-13} sec. versus temperature. This statement is only used in the gain preview module.

- **hh_or_lh** indicates whether heavy hole or light hole subband is involved.
- **gamma_detail** may be used to plot individual contribution to gain broadening factor (Γ) from different scattering mechanisms such as electron-electron, electron-hole, etc..
- **data_file** is the file to which the graphic data is written in ASCII format.
- **temp_start** is the starting point of temperature range.
- **temp_end** is the ending point of temperature range.
- **density** is the carrier density at which the intraband relaxation time is calculated.
- **pn_ratio** is the ratio of hole over electron concentrations. Note that this ratio can be set to an arbitrary number in the gain preview. In the main solver, this ratio is determined automatically by the simulator according to the local Fermi levels.
- **data_point** is the number of data points in data curve.
- **elec_level** and **hole_level** are the electron and hole subband levels involved in the optical transition.

Example(s)

```
tau_temperature temp_start=200. temp_end=400. density=2.e24 &&
data_point=10 elec_level=1 hole_level=1 gamma_detail=yes
```

2.429 tax_mass_bar

parameter	data type	values [defaults]
(see) material_par		

The material statement **tax_mass_bar** is an active layer macro statement used to define the anisotropic effective mass of the conduction band for silicon quantum MOS.

The band structure model has a ellipsoidal shaped constant-energy surface of cylindrical symmetry around a symmetry axes. For example if z-axes is along the symmetry axes of the ellipsoid, the energy dispersion becomes:

$$\frac{E(k_x, k_y, k_z) = E_0 + \hbar^2}{2m_0 \left(\frac{k_z^2}{tax_mass} + \frac{k_x^2}{tax_mass} + \frac{k_y^2}{tax_mass} \right)}$$

tax_mass_barr is the mass transverse to the symmetry axes in the quantum barrier region.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.430 tax_mass_well

parameter	data type	values [defaults]
(see) material_par		

The material statement **tax_mass_well** is an active layer macro statement used to define the anisotropic effective mass of the conduction band for silicon quantum MOS.

The band structure model has a ellipsoidal shaped constant-energy surface of cylindrical symmetry around a symmetry axes. For example if z-axes is along the symmetry axes of the ellipsoid, the energy dispersion becomes:

$$\frac{E(k_x, k_y, k_z) = E_0 + \hbar^2}{2m_0 \left(\frac{k_z^2}{tax_mass} + \frac{k_x^2}{tax_mass} + \frac{k_y^2}{tax_mass} \right)}$$

tax_mass_well is the mass transverse to the symmetry axes in the quantum well region.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.431 temperature

parameter	data type	values [defaults]
temp	real	[300] (K)

temperature sets the isothermal temperature in the device.

- **temp** is the temperature in K.

Example(s)

```
temperature temp=350
```

2.432 thermal_interf

parameter	data type	values [defaults]
x_label	char	[void]
y_label	char	[void]
within_x1_label	char	[void]
within_x2_label	char	[void]
within_y1_label	char	[void]
within_y2_label	char	[void]
thm_num	intg	
thm_type	intg	[1]
thm_with_x	realx2	[-1.e5 1.e5] (um)
thm_with_y	realx2	[-1.e5 1.e5] (um)
thm_x	realx	(um)
thm_y	realx	(um)
thm_lat_temp	real	[300.] (Kelvin)
thm_ht_flow	real	[100.] (W/m)
thm_cond	real	[10.] (W/(mK))
thm_ext_temp	real	[300.] (K)

The statement **thermal_interf** defines the thermal properties of a non-electrode thermal boundary.

- **thm_num** is the thermal interface number. The geometric location of the interface is given through parameters **thm_with_x** and **thm_with_y** to specify the range in x - and y - directions. If the line position for the thermal interface is used when the location is given through parameters **thm_x** or **thm_y**.

- **thm_type** is the non-electrode boundary condition used for the thermal equations. For **thm_type**=1, the thermal interface is assumed to have a lattice temperature given by the parameter **thm_lat_temp**. If **thm_type**=2, the thermal interface is assumed to have an outgoing heat power flow given by the parameter **thm_ht_flow**. The lattice temperature of the thermal interface is unknown and is to be determined by the thermal equation solver. If **thm_type**=3, the thermal interface is assumed to be connected to a thermal conductor with its conductance given by parameter **thm_cond**. This thermal conductor is connected to a heat sink with a fixed temperature given by parameter **thm_ext_temp**. Please note that if this thermal interface is also declared as having an external circuit with a resistor, then self-heating in this resistor will be taken into account. Again the lattice temperature of the thermal interface is unknown and is to be determined by the thermal equation solver.
- **thm_within_x** is used to specify the range in x direction within which the thermal interface is located.
- **thm_within_y** is used to specify the range in y direction within which the thermal interface is located.
- **thm_x** , if specified, all mesh points with such x -coordinate value will be regarded as the thermal interface points.
- **thm_y** , if specified, all mesh points with such y -coordinate value will be regarded as the thermal interface points.
- **x_label**, **y_label** use pre-defined labels to specify **thm_x** and **thm_y** parameters above, respectively. The labels must be predefined using **x_position**, **y_position** or **define_vertical_position** command. They may be predefined in the .layer file relative to a specific layer so that there is no need to find out the absolute coordinates.
- **thm_lat_temp** is the lattice temperature for the thermal interface of type 1.
- **thm_ht_flow** is the heating power flow going out off the thermal interface of type 2. The unit of Watt/m for a 2D section of a cubic coordinate system. The unit is Watt for a 2D section of a cylindrical system.
- **thm_cond** is the thermal conductance of conductor which is connected to the thermal interface of type 3. The heat flow is given by $\Sigma(T - T_{ext})$ where Σ is the thermal conductance and T_{ext} is the temperature of external thermal contact. The unit of Σ is Watt/m/K for a 2D section of a cubic coordinate system. It is Watt/K for a 2D section of a cylindrical system. For an ideal uniform device, the thermal conductance of a cubic system can be converted to that of cylindrical system by formula: $\Sigma_{cyl} = \frac{\pi R^2}{2D} \Sigma_{cub}$. R is the radius of the cylindrical device and D is the half-width of the cubic device. If **extern_cir** is

used to specify the resistance of this thermal conductor, the self-heating effect will also be taken into account. A temperature difference will appear between the thermal interface and the external heat sink.

- **thm_ext_temp** is the temperature of the heat sink which is connected to the thermal conductor for thermal interface type 3.

Example(s)

```
thermal_interf thm_num = 1 thm_type = 1 thm_lat_temp = 77. &&
thm_with_x = (2. 2.) thm_with_y = (2.2 3.)
```

```
thermal_interf thm_num = 2 thm_type = 2 thm_ht_flow = 0. &&
thm_y = 3.
```

2.433 thermal_kappa

parameter	data type	values [defaults]
(see) material_par		

The statement **thermal_kappa** is used to work together with the thermal modeling modules. It has units of Watt/m/K. If this statement is missing from the input file or the macro library in a thermal simulation, the program will complain and stop. The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.434 thermal_kappa_xy

parameter	data type	values [defaults]
dir	char	[y],x
factor	real	[1.]
mater	intg	[1]

This statement is created to model anisotropic thermal conductivity (thermal_kappa) on the x-y plane. This may be useful for the effective medium approximation of multiple layers (such as DBR mirrors in a VCSEL).

- **dir** is the direction in which the `thermal_kappa` is modified by a factor.
- **factor** is the factor to be multiplied to the `thermal_kappa` in one direction.
- **mater** is the material number of the material affected by this statement.

Example(s)

```
thermal\_kappa_xy dir=y factor=0.6 mater=3
```

The above statement decreases the `thermal_kappa` in y-direction by a factor of 0.6 in material number 3.

2.435 tmass_gamma_bar

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the barrier conduction band effective mass in direction perpendicular to the c-axis. This statement is applicable for wurtzite structure active region [1]. The mass in c-axis direction is specified by **mass_gamma_bar**.

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.436 tmass_gamma_bulk

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the conduction bane effective mass in direction perpendicular to the c-axis. This statement is applicable for wurtzite structure bulk region [1].

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.437 tmass_gamma_well

parameter	data type	values [defaults]
(see) material_par		

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

It is the well conduction band effective mass in direction perpendicular to the c-axis. This statement is applicable for wurtzite structure active region [1]. The mass in c-axis direction is specified by **mass_gamma_well**.

[1] S. L. Chuang, “Optical Gain of Strained Wurtzite GaN Quantum Well Lasers”, IEEE J. Quantum Electron., VOL. 32, NO. 10, OCTOBER 1996, p. 1791

2.438 top_contact

parameter	data type	values [defaults]
contact_type	char	[ohmic] schottky
use_schottky_xp	char	[yes] no
column_num	intg	[1]
from	real	
to	real	
contact_num	intg	
barrier	real	(volt)
work_function	real	(volt)
intern_xp_size	real	[0.0002] (um)

top_contact defines the placement of the top contact in the “.layer” geometry file.

- **contact_type** indicates the type of contact and can take either **ohmic** or **schottky**.
- **use_schottky_xp** is used to control the use of extra point near the contact in case the contact is of Schottky type.
- **column_num** describes the column where this contact is found, defined in the “column” statement of the “.layer” file.
- **to** and **from** set the x-range where the contact lies. Note that the origin is at the upper left-hand corner of each polygon and that units are in μm .

- **contact_num** defines the index number of the contact.
- **barrier** is the barrier height of the Schottky contact.
- **work_function** is the work function of the Schottky contact.
- **intern_xp_size** is the internal extra point size around a Schottky contact if parameter **use_schottky_xp** is set. It will not be used if the contact type is not Schottky.

Example(s)

```
top_contact column_num=3 from=0 to=1.8 contact_num=1
```

2.439 trap_excitation

parameter	data type	values [defaults]
emit_carrier	char	[electron] hole
cross_section	real	[1.e-19] (m^2)
power_depend_coef	real	[0] ($m^4/Watt$)
trap_index	intg	[1]
mater	intg	[1]

The statement **trap_excitation** is used to specify the photo properties of a deep level trap.

- **emit_carrier** specifies the type of carriers emitted upon light excitation.
- **cross_section** is the cross section of light absorption. When it is multiplied by the density of traps responding to light, an equivalent absorption coefficient may be obtained.
power_depend_coef is the power dependent cross section coefficient. When multiplied by the optical power density in units of $Watt/m^2$, it adds to the light absorption cross section of the trap.
- **trap_index** is the index (or label) of the deep trap.
- **mater** is the index of the material being affected.

Example(s)

```
trap_excitation emit_carrier=electron &&
cross_section=1.e-19 trap_index=1 mater=2
```

2.440 trap_level_i

parameter	data type	values [defaults]
(see) material_par		

trap_level_i is used to define the deep trap level of the recombination centers. The level is measured from the conduction band and in the unit of eV.

Note that the parameter **level** of the **doping** statement may be used to describe the energy level of the shallow or deep impurities over the whole device. If the properties of the impurity in each different material need to be defined, the user must override the **level** parameter with one of the statements **shal_dnr_level**, **shal_acpt_level** or **trap_level_i**.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.441 trap_ncap_i

parameter	data type	values [defaults]
material_par		

The material statement **trap_ncap_i** is the electron capture cross section in m^2 of the trap_i (i=1,2,...).

The material statement **trap_pcap_i** is the hole capture cross section in m^2 of the trap_i (i=1,2,...).

The relationship between capture cross sections and minority carrier lifetime is explained under the statements **lifetime_n** and **lifetime_p**.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.442 trap_pcap_i

parameter	data type	values [defaults]
material_par		

The material statement **trap_ncap_i** is the electron capture cross section in m^2 of the trap_i (i=1,2,...).

The material statement **trap_pcap_i** is the hole capture cross section in m^2 of the trap_i (i=1,2,...).

The relationship between capture cross sections and minority carrier lifetime is explained under the statements **lifetime_n** and **lifetime_p**.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section [2.272](#).

2.443 tunneling

parameter	data type	values [defaults]
direction	char	[y],x
carrier	char	[electron], hole
barrier_type	char	[rectangle]
x1_label	char	[void]
y1_label	char	[void]
x2_label	char	[void]
y2_label	char	[void]
barrier_x1_label	char	[void]
barrier_y1_label	char	[void]
barrier_x2_label	char	[void]
barrier_y2_label	char	[void]
local_correction	char	[yes]no
trans_broad	char	[no], rectangle, lorentzian
point_ll	realx2	[μm]
point_ur	realx2	[μm]
barrier_pnt_ll	realx2	[μm]
barrier_pnt_ur	realx2	[μm]
broad_width	real	[1.] (Kb*T)
scale_tunnel_coef	real	[1.]
adjust_barrier	real	[0.] (eV)
mesh_multiply	intg	[2]
energy_points	intg	[3000]
contact_collection	char	[no]
main_contact	intg	[no]
	char	[no],yes
miniband_period_y1_label	char	[void]
miniband_period_y2_label	char	[void]
save_miniband	char	[miniband_data.txt]
miniband_period_y1	real	(um)
miniband_period_y2	real	(um)
miniband_upper_range	real	[0.5](eV)
miniband_number	intg	[2]
miniband_fd_mesh	intg	[100]
miniband_k_points	intg	[20]

tunneling enables the quantum tunneling transport mechanism.

- **direction** is the direction of the tunneling current.

- **carrier** is the type of carrier concerned.
- **barrier_type** is the type of barrier being tunneled through. It may take one of the following: "rectangle" (default), "triangle", "smooth_wkb" and "propagation_matrix". The type should be chosen such that it matches the actual potential profile in the device. For "rectangle", the actual profile should have steep edges on both sides. For "triangle", it is required that one side of the barrier be much steeper than the other. For "smooth_wkb", the potential barrier profile should be rather smooth. "propagation_matrix" may be used for rather general profile since it uses more versatile 2 by 2 propagation matrix method. This method is rather time consuming but more accurate than the others. More details may be found in the User's manual.

This command can also be used to activate the mini-band conduction model to transport carrier over a far-away distance mesh point. The theory is based on the the following reference: Andreas Wacker, "Semiconductor superlattices: a model system for nonlinear transport," Physics Reports, vol. 357 (2002) 1-111.

Numerical calculation of mini-band is similar to that used by the following paper: Kenji Nakamura, Akira Shimizu, Masanori Koshihara, and Kazuya Hayata, "Finite-Element Analysis of the Miniband Structures of Semiconductor Superlattices with Arbitrary Periodic Potential Profiles," IEEE JOURNAL OF QUANTUM ELECTRONICS. VOL. 27, NO. 8. AUGUST 1991, p 2035.

- **local_correction** is used to indicate whether a mesh-dependent quantum tunneling correction is applied to all local mesh points. If set to no, the tunneling current is calculated at the top fo the barrier and is assumed to be uniform across the whole rectangle region. The local mesh correction method is recommended for relatively smooth potential profiles while non-local correction works better for thin and high tunneling barriers.
- **trans_broad** is used to indicate if and what type of energy level broadening is used when using the "propagation_matrix" method. The profile of the broadened level may in take rectangle or Lorentzian shape.
- **point_ll**, **point_ur** are upper left and lower right corner points of the area being considered for tunneling. The area must be large enough to cover the potential barrier and must be at least a few mean free path of the tunneling carrier. But it should not be so large as to affect carrier transport in areas unrelated to quantum tunneling.
- **x1_label,y1_label,x2_label,y2_label** are used to define a rectangle range for the same purpose as **point_ll**, **point_ur**. The labels must be predefined using **x_position**, **y_position** or **define_vertical_position** command. They may be predefined in the .layer file relative to a specific layer so that there is no need to find out the absolute coordinates.

- **barrier_pnt_ll**, **barrier_pnt_ur** are used to specify a region within the **point_ll** and **point_ur**. Such a region is used as follows: The program assumes incident wave will hit a potential barrier with increasing potential and leaves a barrier with decreasing potential. It is OK to have wells within the complex barrier but incident/transmitting facet must appear as barriers. Due to this assumption we search for the lower points on the and left and right sides of the complex barrier, outside of such a region as specified by these parameters but within **point_ll/ur**. We artificially flattens all potential before and after these lower points. Thus, we require that these points exclude any intentional lower points (such as a well within an RTD) that we wish to include in the tunneling model. By default, if these parameters are not defined by user, they are set to a middle point within the **point_ll** and **point_ur** so that any potential well within the barrier complex will be thrown out of the tunneling calculation.
- **barrier_x1_label**, **barrier_y1_label**, **barrier_x2_label**, **barrier_y2_label** are used to define a rectangle range for the same purpose as **barrier_pnt_ll**, **barrier_pnt_ur**. The labels must be predefined using **x_position**, **y_position** or **define_vertical_position** command. They may be predefined in the .layer file relative to a specific layer so that there is no need to find out the absolute coordinates.
- **broad_width** is the broadening of tunneling energy level in units of $k_b T$, where k_b is the Boltzmann constant and T is the temperature.
- **scale_tunnel_coef** provides a way to artificially scale the tunneling coefficient.
- **adjust_barrier** works when **local_correction=no** and is used to adjust the barrier height used to compute the carrier density at the top of the barrier. Please note that this parameter only affects the carrier concentration at the top of the barrier (thus also the magnitude of tunneling current) but does not affect the potential profile used to calculate the tunneling transparency.
- **mesh_multiply** is the multiplication factor of mesh points along the path of tunneling current. In the propagation matrix method, we first cut a path in the direction of the tunneling. Then, the original mesh points along the path is multiplied by this factor to get more accurate results.
- **energy_points** is the number of energy divisions used in evaluation of tunneling current when using the "propagation_matrix" method.
- **contact_collection** is used when **local_correction=no**. This allows the solver directly collect the current by the contact. Convergence is usually better when this is used.
- **main_contact** is the main contact of the current collector when **contact_collection=yes**. Since current flow directly to the contact, the program would figure out what

proportion of current to assign to each contact if a device has more than two electrodes. This is based on the voltage bias as well as on the proportion of the drift-diffusion current.

- **miniband_model** would enable the mini-band tunneling through a region specified by this statement.
- **miniband_period_y1_label** labels a position which refers to a label defined in the .layer file corresponding to the lower limit of a period.
- **miniband_period_y2_label** labels a position which refers to a label defined in the .layer file corresponding to the upper limit of a period.
- **save_miniband** would save the data of the miniband to such as file.
- **miniband_period_y1** defines a position which corresponds to the upper limit of a period.
- **miniband_period_y2** defines a position which corresponds to the upper limit of a period.
- **miniband_upper_range** defines an upper range within which we search the energy solution of the miniband.
- **miniband_number** is the number of miniband to be solved and used in optical/transport modeling.
- **miniband_fd_mesh** is the finite difference mesh number within a period of the miniband.
- **miniband_k_points** is the number of k-points to be solved for the miniband structure.

Example(s):

```
tunneling point_ll=(0., 2.49) point_ur=(0.5 2.6) &&
  barrier_type=triangle
tunneling point_ll=(0., 4.7) point_ur=(0.5 4.95) &&
  barrier_type=triangle carrier=hole
```

2.444 use_bulk_affinity

parameter	data type	values [defaults]
all_material	char	[yes]
macro_name	char	[void]
mater	intg	[1]

The material statement **use_bulk_affinity** is an active layer macro statement used to define the band offset of the quantum well. It directs the program to use bulk material affinity to define band offsets of the active layers. This statement overrides all statements of **band_offset**, **band_discont**, and **band_discont_right**.

- **all_material** lets all active layer materials to use bulk affinity as band offsets.
- **macro_name** indicates all materials with this macro name are affected.
- **mater** is the material number affected. It is not used if **macro_name** is already set.

Example: The following statement causes all active layers to use bulk material affinities to set their band offsets:

```
use_bulk_affinity all_material=yes
```

2.445 use_bulk_bandgap

This command would force the program to take the bandgap of the bulk macro if there is a conflict between active and bulk macros.

2.446 use_bulk_property

This command would force the program to take the material parameters of the bulk macro if there is a conflict between active and bulk macros.

This command affects bandgap, affinity and effective masses.

2.447 use_macrofile

parameter	data type	values [defaults]
macroj (j=1,2,...)	char	[void]

The statement **use_macrofile** is used to override or to add to the default macro files crosslight.mac and more.mac. The simulator will check macro files loaded by this statement before going to the default macros. More than one macro files can be loaded this way

- **macroj** (j=1,2,...) is the user-defined macro file.

Example(s):

```
use_macrofile macro1=mydata.mac
```

2.448 use_sor

parameter	data type	values [defaults]
print_sor	char	[noprint]
wave_tol	real	[1.e-7]
beta_tol	real	[1.e-7]
omega	real	[1.5]
omegab	real	[1.5]
max_iter	intg	[5000]
update	intg	[15]

use_sor instructs the simulation program to use the SOR iterative method to find the solution of lateral modes. This statement was previously **sor_par**.

- **print_sor** controls whether the iteration process is to be printed during execution.
- **wave_tol** is the tolerance for the wave-amplitude. Note that the wave amplitude is normalized to the order of 1.
- **beta_tol** is the tolerance for the eigenvalue which is also normalized to the order of 1.
- **omega** is the initial relaxation parameter used for the wave amplitude iteration. This parameter is adaptive during iteration.
- **omegab** is the initial relaxation parameter used for the eigenvalue. This parameter is also adaptive during iteration.
- **max_iter** is the maximum number of iterations. If it is zero, the built-in optical field solver is switched off and the SOFTWARE uses a default Gaussian optical field with the field center specified by the **init_wave** statement, or, if the **optical_field** statement is used, SOFTWARE uses the optical field specified therein.
- **update** is the interval of iterations at which the eigenvalue is updated.

Example(s)

```
use_sor max_iter=0000
```

```
use_sor max_iter=5000 print_sor=noprint
```

2.449 valj_mass_para (j=1..3)

parameter	data type	values [defaults]
(see) material_par		

The material statement **valj_mass_para** is an active layer macro statement used to define the relative valence band valley mass in a direction parallel to the quantum well, for the *j*th valley. It is used for general_cx_strain type of macro, or general complex strain macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.450 valj_mass_perp (j=1..3)

parameter	data type	values [defaults]
(see) material_par		

The material statement **valj_mass_perp** is an active layer macro statement used to define the relative valence band valley mass in a direction perpendicular to the quantum well, for the *j*th valley. It is used for general_cx_strain type of macro, or general complex strain macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.451 valj_valley_propk (j=1..3;k=1,2)

parameter	data type	values [defaults]
(see) material_par		

The material statement **valj_valley_propk** is an active layer macro statement used to define the valley dependent property (*k*th property, such as mobility) for the *j*th valence band valley. The simulator will automatically compute the property based on population of the subbands concentrations in the valley. It is used for general_cx_strain type of macro, or general complex strain macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.452 val_bandj_edge (j=2,3)

parameter	data type	values [defaults]
(see) material_par		

The material statement **val_bandj_edge** is an active layer macro statement used to define the 2nd or 3rd valence band valley with respect to the highest valence band valley, in unit of eV. It is used for general_cx_strain type of macro, or general complex strain macro.

The parameters for this statement are the same as for all other material statements. The use of these parameters and related examples are given under **material_par** in section 2.272.

2.453 vcsel_eeim

parameter	data type	values [defaults]
polarization	char	[te],tm,hybri
boundary_radius	real	[6.] (μm)

This statement works together with parameters of "export_veeim" in statement **rt_gain_phase** or "import_veeim" in statement **vcsel_model**. It should be placed in the 2nd part of the .sol file. It is used to specify the data exchange between the VEEIM (VCSEL EEIM) program. VEEIM is a vectorial wave solver for optical modes in a cylindrical multilayer system.

- **polarization** is the polarization in a cylindrical system. In the present release (Release 2000.11), only TE polarization is implemented. **boundary_radius** is the outer boundary of an imaginary metallic cylinder which is used to generate the basic modes of the VEEIM model. Any realistic mode should be regarded as a linear combination of the basic modes. This radius is chosen large enough such that the wave intensity can be regarded as negligible at this radius. It should also be small enough so that it is comparable with the size of lobes of the lateral modes.

Example(s)

```
vcsel_eeim polarization=te boundary_radius=6.
```

2.454 vcsel_model

parameter	data type	values [defaults]
import_veeim	char	[no],yes
use_eim	char	[no],yes
rectangle_system	char	[no]
rect_symetric_x	char	[yes]
index_core	real	[3.5]
index_cladding	real	[2.0]
core_radius	real	
photon_fac	real	[1.e-3]
eim_zero	real	(um)
rect_core_xrange	real	(um)
rect_core_zrange	real	(um)
rect_core_origin	realx2	[0. 0.](um)
bessel_order	intg	[0]
rect_y_order	intg	[1]

vcsel_model is used to activate the Vertical Cavity Surface Emitting Laser (VCSEL) model.

- **import_veeim** works together with statement **vcsel_eeim** to import the results of the VEEIM program. The imported data will affect the round trip gain calculation based on the results of VEEIM module. Please also see statements of **vcsel_eeim**, **rtgain_phase**.
- **use_eim** activates the effective index method model for the VCSEL simulation environment. By default, this parameter is set to "no" and the fiber-EIM model is used.
- **rectangle_system** turns on the rectangle system for VCSEL simulation.
- **rect_symetric_x** indicates whether only half of a symmetric device is simulated to save mesh. If so, the symmetric axis is assumed to be located at $x=0$.
- **index_core** is the refractive index of the core region in the modified effective index method.
- **index_cladding** is the refractive index of the outer region, or the cladding region (not to be confused with cladding in the transverse direction of an edge emitting laser) in the modified effective index method.

- **core_radius** is the radius of the core region in the modified effective index method.
- **photon_fac** is the photon factor used to normalize the cavity photon number for VCSEL. Also see **photon_scan** in **scan3d**.
- **eim_zero** is the radius outside of which the EIM wave function is forced to be zero. It is used only when the EIM model is activated by the parameter **use_eim** above.
- **rect_core_xrange** is the x-range (or size) of the high index core in x-direction.
- **rect_core_zrange** is the z-range (or size) of the high index core in z-direction.
- **rect_core_origin** is origin (lower left corner) of the rectangle high index core on the x-z plane.
- **bessel_order** is the order of the Bessel function used to model the wave function in the modified effective index method. Higher order means higher order lateral modes.

Example(s)

```
vcSEL_model axis=y index_core=3.5 index_cladding=1.5 &&
  core_radius =4.  z1=0.  z2=15.  bessel_order=0
```

2.455 vcsel_section

parameter	data type	values [defaults]
vcSEL_type	char	
grating_model	char	[kappa]layer,layer0
active	char	[no]yes
layerk,k=1,2,...	real	[0.096e-6] (m)
indexj,j=1,2,...	real	
core_radius	real	(um)
mesh_points	intg	[10]
section_index	real	

vcSEL_section is used to describe physical properties and geometrical structure of the VCSEL section. It is only used in **.layer** files to generate **.vcSEL** files.

- **vcSEL_type** is the label of VCSEL section which has to be referred to later (in statement **layer**).

- **active** is used to indicate if a VCSEL section is an active section.
- **grating_model**, please see the same parameter under **section**.
- **layerk**, $k=1,2,\dots$ are the thicknesses of the layers that make up one period in the layer model when **grating_model=1layer, 2layers, etc..**
- **indexj** are the refractive indices of the layers that make up one period in the layer model when **grating_model=1layer, 2layers, etc..** For more details of this parameter, please be referred to the same parameter under statement **section**.
- **core_radius** is the radius of core region, which indicates to the simulator that this VCSEL section is terminated by an insulator layer on the outside.
- **mesh_points** is the subsection number or mesh points used for the purpose of calculation. All physical quantities are assumed to be constant within the subsection.
- **section_index** is identical to **index1** above.

Example(s)

```
vcsl_section vcsl_type=n-dbr &&
  grating_model=2layers active=no &&
  layer1 =0.122e-6 layer2 =0.112e-6 &&
  index1 = 3.17 index2 = 3.46 &&
  mesh\_points=15
```

Example(s)

```
vcsl_section vcsl_type=n-dbr &&
  grating_model=2layers active=no &&
  layer1 =0.122e-6 layer2 =0.112e-6 &&
  index1 = -3.17 index2 = -3.46 &&
  mesh\_points=15
```

When the indices are specified as negative numbers, only the difference is used by the program while the simulator computes and updates the average index from macros and from index models according to injection/thermal conditions. For more explanation of negative index numbers, please see statement **section**.

2.456 vectorial_wave

parameter	data type	values [defaults]
mode	char	[te] tm

The statement **vectorial_wave** is used to turn on the vectorial wave solver. Please note that this statement only affects the wave distribution. It does not affect the optical evaluation. You should also set the polarization in **active_reg** for the TE or TM mode.

- **mode** is the polarization mode and may take te or tm.

Example:

```
vectorial_wave mode=te
```

2.457 view_dipole

parameter	data type	values [defaults]
data_file	char	[void]
mode	char	[te]tm
gamma_subband	intg	[1]

view_dipole plots the dipole moments in a k.p theory.

- **data_file** is the file to which the graphic data is written in ASCII format.
- **mode** denotes the polarization.
- **conc_range** is the electron concentration range in the well.
- **gamma_subband** is the Gamma subband used in the optical transition.

Example(s)

```
view_dipole gamma_subband=1
```

2.458 view_ganvalence

parameter	data type	values [defaults]
data_file	char	[void]
data_file2	char	[void]
region	char	[well] barrier barrier2

view_ganvalence is to be used in .gain file to plot the active region basic bulk band structure (ie., HH, LH, CH) for wurtzite GaN-based structure along with parabolic fit.

- **data_file** is the data file to which the band structure data is to be saved.
- **data_file2** is the data file to which the band structure fitting data is to be saved if plot_device is set to data_file.
- **region** is used to specify if the band structure for well or barrier is to be plotted.

Example(s)

```
view_ganvalence region=well
```

2.459 view_kpsubband

parameter	data type	values [defaults]
data_file	char	[void]
include_data	char	[void]
mqw_complex	intg	[1]

view_kpsubband is used in .gain file to plot the quantum well valence subbands from k.p theory.

- **data_file** is the data file to which the band structure data is to be saved.
- **include_data** includes data files from other subband calculations for the purpose of comparison. The format is a column-wise list of k_t and energy E . Data from different subbands are separated by an empty line.
- **mqw_complex** is used to select the MQW complex (or structure) index if there are more than one type of MQW structure.

2.460 vplot_xy

parameter	data type	values [defaults]
variable	char	(see list)
data_file	char	
mater_boundary	char	[yes]no
point_ll	realx2	(μm)
point_ur	realx2	(μm)
xrange	realx2	
yrange	realx2	
z	real	(micron)
maxvector_scale	real	[1.]
grid_sizes	intgx2	[20, 20]

vplot_xy is a post-processor statement used to generate a vector plot (or flow plot) with flow directions. The size of the vector is proportional to the flux intensity and the direction is the same as the local flux. The plot is taken at a specific xy-plane.

- **variable** is the variable to be plotted. **variable= elec_cur, hole_cur, disp_cur, elec_fiel, or total_cur** produces a diagram of a distribution of arrows representing the intensity and direction of the corresponding vector as a function of location. Please see App. [B](#) for details.
- **data_file** is the file to which the graphic data is written in ASCII format.
- **mater_boundary** is used to indicate if material boundary is plotted.
- **point_ll** is the lower-left corner point of the 2D plotting window.
- **point_ur** is the upper-right corner point of the 2D plotting window.
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.
- **z** is the z-position of the xy-plane.
- **maxvector_scale** is used to scale the maximum length of a 2D vector field. A vector field is plotted by drawing only the vectors of length between the minimum and maximum vector length. Vectors of lengths below the minimum are not plotted. Vectors of lengths above the maximum are plotted with the same vector size. Increasing this parameter generates a plot with many small size vectors. Decreasing this value magnifies the smaller vectors while leaving the large ones with the same size.

- **grid_sizes** are the grid sizes of the 2D plot (not to be confused with mesh used in simulation). To generate the vector plot (or flow plot), the vector of the physical quantity must be evaluated at a given **grid_size**.

Example(s)

```
vplot_xy variable=elec_curr z=50. grid_sizes=(35, 35) &&
  xrange=(0. 1.5) yrange=(1.1 1.9)
```

2.461 vplot_xyz

parameter	data type	values [defaults]
variable	char	(see list)
data_file	char	
xy_from	realx2	(μm)
xy_to	realx2	(μm)
xrange	realx2	
yrange	realx2	
z_min	real	(micron)
z_max	real	(micron)
maxvector_scale	real	[1.]
grid_sizes	intgx2	[35, 35]

vplot_xy is a post-processor statement used to generate a vector plot (or flow plot) on a plane parallel to the z-axis with flow directions. The size of the vector is proportional to the flux intensity and the direction is the same as the local flux.

- **variable** is the variable to be plotted. **variable= elec_cur, hole_cur, disp_cur, elec_fiel, or total_cur** produces a diagram of a distribution of arrows representing the intensity and direction of the corresponding vector as a function of location. Please see App. B for details.
- **data_file** is the file to which the graphic data is written in ASCII format.
- **xy_from** is the starting point of the cut line made by the plotting plane.
- **xy_to** is the ending point of the cut line made by the plotting plane.
- **xrange** specifies a range for the x-axis.
- **yrange** specifies a range for the y-axis.

- **z_min** is the starting z-range for the plot.
- **z_max** is the ending z-range for the plot.
- **maxvector_scale** is used to scale the maximum length of a 2D vector field. A vector field is plotted by drawing only the vectors of length between the minimum and maximum vector length. Vectors of lengths below the minimum are not plotted. Vectors of lengths above the maximum are plotted with the same vector size. Increasing this parameter generates a plot with many small size vectors. Decreasing this value magnifies the smaller vectors while leaving the large ones with the same size.
- **grid_sizes** are the grid sizes of the 2D plot (not to be confused with mesh used in simulation). To generate the vector plot (or flow plot), the vector of the physical quantity must be evaluated at a given **grid_size**.

Example(s)

```
vplot_xyz variable=elec_curr  &&
  xy_from=(0.5 0.) xy_to=(0.5 3.)
```

2.462 wave_boundary

parameter	data type	values [defaults]
x1_label	char	[void]
y1_label	char	[void]
x2_label	char	[void]
y2_label	char	[void]
point_ll	realx2	[0. 0.](μm)
point_ur	realx2	[1. 1.](μm)
fld_center	realx2	(μm)
n_ubdata	realxn	
n_lbdata	realxn	
ubdata_num	intg	[0]
lbdata_num	intg	[0]

The statement **wave_boundary** sets the boundary condition for the optical wave equations. Use of this statement overrides the corresponding parameters in in statement **init_wave**. This statement can be generated by the graphic user interface and appears in the .doping file.

- **x1_label y1_label x2_label y2_label** user pre-defined labels to specify **point_ll**, **point_ur** parameters, respectively. The labels must be predefined using **x_position**, **y_position** or **define_vertical_position** command. They may be predefined in the .layer file relative to a specific layer so that there is no need to find out the absolute coordinates.
- **point_ll** and **point_ur** are lower-left and upper-right points of the window within which the wave equation is solved (see also Fig. 2.21). This window must be less than or equal to the full device size defined in the .geo input file. In some cases, this window must be reduced in size because only a limited number of lateral modes are needed. In gain guided devices, the lasing mode is not the highest index mode. Since PICS3D searches the optical mode starting from the highest mode index, there may be many non-lasing modes within the window. In the current version, PICS3D can only solve up to 10 modes. Since the number of non-lasing, high index modes increases with size of the window, the user should carefully adjust the window size for gain guided structures so that the lasing mode is included within the 10 modes.
- **fld_center** is the estimated optical mode center. Note that this information is only used to initialize the wave function and may or may not coincide with the optical field center in the final solution.
- **n_ubdata** and **n_lbdata** are window specifications. By default, the window for the wave equation is specified by "point_ll" and "point_ur" to be a rectangular window. Now it is possible to override the upper and lower borders with the parameters **n_ubdata**, **n_lbdata**, **ubdata_num** and **lbdata_num** where **n_ubdata** (for upper boundary data) is used to specify the upper boundary points (xi, yi) in the format

$$n_ubdata=(x1 \ y1 \ x2 \ y2 \ x3 \ y3 \ \dots)$$
 where the prefix **n_** should be equal to the number of data values in the bracket (or half of the points in 2D space). The related parameter for this statement, **ubdata_num**, must match the number of data value in **n_ubdata**.

```
init_wave ... &&
    ubdata_num=8 8_ubdata=(0. 3.16 0.773 3.16 1.5 2.16 6. 2.16)
```

is used to describe a non-planar metal boundary for the wave equation. Similarly, **n_lbdata** and **lbdata_num** are used to describe the lower boundary.

Example(s)

- **binary_search** indicates whether binary search is used to narrow the solution.
- **wavelength** is the input light wavelength in μm .
- **input_watt** is the input light power in Watt. Please also see **init_factor** parameter below.
- **init_factor** is used to multiply the **input_watt** to obtain the total initial input power. In case the input power needs to be varied, parameters "wg_light_from" and "wg_light_to" of **scan3d** statement can be used to define a factor which is added to this parameter.
- **weak_refl_iter** is the number of weak-reflection iterations. More iteration results in more self-consistent solution.

Example(s)

```
waveguide_input wavelength=1.3 input_watt=1.e-4
```

2.464 wurtzite_offset_model

parameter	data type	values [defaults]
use_strained_bandgap	char	[no]
partition_hydro_term	char	[no]
bulk_strain_exist	char	no

The command **wurtzite_offset_model** is used to control the band offset of quantum well layers of wurtzite types.

To understand this command, please note the following introduction of band Offsets and alignments in Crosslight device simulators:

For bulk macro, the band alignment is solely determined by the affinity parameter. For active_layer macro, the band alignment is determined by band_offset, band_discont, or band_discont_right. band_discont and band_discont_right override band_offset. Then band alignment settings in active_layer macro override the affinity setting in bulk macro.

For quantum well with strain in well and/or barrier, it may be confusing to use band_offset since hydrostatic strain changes the bandgap. Such change may be partitioned between the conduction and valence band with a certain ratio.

The shear strain may split the HH/LH/CH valence bands, making the bandgap multi-valued. Different use of band_offset is controlled by the command zincblende_offset_model or wurtzite_offset_model, for zincblende or wurtzite material, respectively.

For wurtzite material, the default is to use the unstrained bandgap to calculate the offset and let the hydrostatic term to appear in the valence band only.

- **use_strained_bandgap** would allow the use of strained bandgap in offset calculation.
 - **partition_hydro_term** would allow the partition of hydrostatic strain term between valence and conduction band according to `av_over_a` in `active_reg`. Not using this partition means setting the hydro term to the valence band alone.
- bulk_strain_exist** would turn on the bulk material strain model which affects the material bandgap calculation.

```
wurtzite_offset_model use_strained_bandgap=yes
```

The above command would use strained bandgap to calculate band offset and this may introduce some inconsistency with band offset estimated from unstrained bandgap (`eg0` parameter).

2.465 x_position

parameter	data type	values [defaults]
label	char	[void]
x	real	

x_position is used by the program to refer to a y-coordinate using a label.

- **label** is a position label at a specific x-coordinate in microns.
- **x** is the x-coordinate of the point.

Example(s)

```
x_position label=mid_mqw x=0.0051
```

This example labels a x-position 0.0051 micron as “`id_mqw`” so that other statements can refer to such as position using the label of “`mid_mqw`”.

2.466 xpoint_size

parameter	data type	values [defaults]
xpoint	real	[0.0001 μm]

xpoint_size is the statement used in the “.layer” file to define the placement of an extra point. It is used by the LAYER program. It will be effective only when the **version2002** parameter is set for the statement **internal_xpoint**.

- **xpoint** sets the distance of the point from the beginning or end of the segment.

Example(s)

```
xpoint_size xpoint=0.01
```

2.467 y_position

parameter	data type	values [defaults]
label	char	[void]
y	real	

y_position is used by the program to refer to a y-coordinate using a label.

- **label** is a position label at a specific y-coordinate in microns.
- **y** is the y-coordinate of the point.

Example(s)

```
y_position label=mid_mqw y=0.0051
```

This example labels a y-position 0.0051 micron as “id_mqw” so that other statements can refer to such as position using the label of “mid_mqw”.

2.468 yz_amplifier_model

parameter	data type	values [defaults]
2nd_light	char	[no] yes (μm)
wavelength	real	[1.55] (μm)
wave1d_yrange	realx2	[-1.e5 1.e5] (μm)
x_width	real	[1.] (μm)
input_watt	real	[0.001] (watt)
wavelength2	real	[1.58] (μm)
z_segment	intg	[50]

The statement **yz_amplifier_model** is used to activate the optical amplifier model for APSYS.

- **2nd_light** is used to indicate whether the 2nd light input is present. **wavelength** is the wavelength of the 1st light source.
- **wave1d_yrange** is the optical window in the y-direction of the LD amplifier within which the guide wave equation is solved. It is assumed that at the ends of the y-range, the wave intensity is zero.
- **x_width** is the device width in the x-direction. The simulation is performed on the y-z plane. Thus, x width is the third dimension.
- **input_watt** is the unit of the input light. In the **scan** statement, the light is usually varied between 0 to 1. The unit is given by this parameter.
- **wavelength2** is the wavelength of the 2nd light input source. The symbol **light2** is used in **scan** to set the intensity of this 2nd light source.

Example(s)

```
yz_amplifier wavelength=1.55 z_segment=10
```

2.469 zero_doping

parameter	data type	values [defaults]
mater	intg	[1]
macro_name	char	[void]

The statement **zero_doping** is a statement which forces the doping to be zero in a particular material.

- **mater** is the material number affected by this statement. This parameter is ignored if **macro_name** is used.
- **macro_name** indicates all materials using this macro name be set to zero doping impurities.

2.470 zener

parameter	data type	values [defaults]
balance_zero_bias	char	[yes]
tun_mass	real	
xrange	realx2	
yrange	realx2	

zener is used to activate the interband tunneling (or Zener tunneling as in Zener diodes). Since this statement is useful for heavily doped reverse junctions, it may be useful to use this statement together with the impact ionization model.

- **balance_zero_bias** balances out the tunneling current with thermal current at zero bias. This may be set to "no" in some cases for the purpose of achieving convergence at low bias voltages.
- **tun_mass** is the Zener interband tunneling mass to be set by the user. If it is not specified, the combined effective mass is used as in the following formula: $2m_em_h/(m_e + m_h)$.
- **xrange** specifies a range for the abscissa.
- **yrange** specifies a range for the ordinate.

Example(s)

```
$ Just one word:
zener
$ This will enable the interband tunneling model to
$   use the standard combined effective mass.
```

2.471 zincblende_offset_model

parameter	data type	values [defaults]
use_strained_bandgap	char	[yes]
partition_hydro_term	char	[yes]

The command **zincblende_offset_model** is used to control the band offset of quantum well layers of zincblende types.

To understand this command, please note the following introduction of band Offsets and alignments in Crosslight device simulators:

For bulk macro, the band alignment is solely determined by the affinity parameter. For active_layer macro, the band alignment is determined by band_offset, band_discont, or band_discont_right. band_discont and band_discont_right override band_offset. Then band alignment settings in active_layer macro override the affinity setting in bulk macro.

For quantum well with strain in well and/or barrier, it may be confusing to use band_offset since hydrostatic strain changes the bandgap. Such change may be partitioned between the conduction and valence band with a certain ratio.

The shear strain may split the HH/LH/CH valence bands, making the bandgap multi-valued. Different use of band_offset is controlled by the command zincblende_offset_model or wurtzite_offset_model, for zincblende or wurtzite material, respectively.

For zincblende material, the default is to use the strained bangap to calculate the offset and let the hydrostatic term partitioned between conduction and valence bands.

- **use_strained_bandgap** would allow the use of strained bandgap in offset calculation.
- **partition_hydro_term** would allow the partition of hydrostatic strain term between valence and conduction band according to av_over_a in active_reg. Not using this partition means setting the hydro term to the valence band alone.

`zincblende_offset_model use_strained_bandgap=no`

The above command would calculate band discontinuity using the unstrained bandgap.

2.472 zsegment_setting

parameter	data type	values [defaults]
limit_gain	char	[no]
zseg_num	intg	[1]

zsegment_setting is used to set some model parameters for a specific z-segment.

- **limit_gain** indicates whether optical gain is limited by the gain interpolation table and disallow any extrapolation. This parameter affects package PICS3D only.
- **zseg_num** is the z-segment affected by this command.

2.473 **z_structure**

parameter	data type	values [defaults]
link_taper_contact	char	[yes] (no)
uniform_zseg_from	real	(um)
uniform_zseg_to	real	(um)
taper_pointsj (j=1,2,...)	realx2	[0. 0.] (um)
y_taper_pointsj	realx2	[0. 0.] (um)
uniform_length	real	(um)
taper_length	real	[0.] (um)
xp1_size	real	[0.] (um)
xp2_size	real	[0.] (um)
mesh_ratio	real	[1.]
shift_center	real	[0.] (um)
cylindrical_origin	real	[0.](um)
zseg_num	intg	[1]
zplanes	intg	[1]
cylindrical	char	[no] yes

The statement **z_structure** is used to define a structure in the z-direction consisting of many z-segments. Each z-segment is regarded as having uniform material property but may have variations of solution variables such as electron and hole concentrations. A z-segment is further divided into many sections (see the **section** statement) for the purpose of longitudinal mode modeling of a waveguiding device (PICS3D).

Taper lines are guiding lines on a segment. These lines may connect to similar ones on the next segment to form bounding planes to force mesh connection between mesh points on the same side of the bounding planes. For example, if segment 1 has a taper line at x=1 and segment 2 has one at x=1.5, then all mesh points on the left side or right side of the taper lines are bound by a plane connecting x=1 on segment 1 to x=1.5 on segment 2. In modeling flow of physical variables, the mesh points on the same side of the taper line will be forced to connect to similar points on the side of the taper line on adjacent segment.

- **link_taper_contact** indicates whether contacts in different segments are joined together if there is tapering between segments. This statement takes effect only

when two electrodes have the same contact number. This parameter works together with the statement of **3d_solution_method**. This statement is effective when the method of “3d_flow=no”, meaning current is not allowed to flow between xy-planes.

- **uniform_zseg_from** and **uniform_zseg_to** are used to specify the positions of the uniform segments along the longitudinal direction. We assume tapering occurs between the uniform segments. **uniform_zseg_from** and **uniform_zseg_to** can be the same point, meaning that no uniform segment at such location but there may be tapering connecting to the next segment. Also note that the two ends of the device must be specified as uniform segments although the length of such segments can be zero.
- **zseg_num** is the segment number. Please note that only a uniform segment counts as a segment.
- **taper_pointsj** ($j=1,2,\dots$), are used to define the j th taper line. A taper line is a line (usually a material boundary) on a segment that linearly extends to a corresponding line on the adjacent segment. For example, a taper line located at $x=1$ on segment 1 can linearly extend to a taper line at $x=5$ on segment 2. This can be used to define a tapered structure with width changing from 1 to 5. There can be more than one taper line on each segment. The taper line does not have to be vertical but can intersect the upper and lower horizontal boundaries on a segment with points of this parameter. For example, **taper_pointsj**=(0.5 0.8) means the taper line is not vertical but intersects the lower boundary line at $x=0.5$ and the upper boundary at $x=0.8$.
- **y_taper_pointsj** is the same as **taper_pointsj** except it is arranged in the y-direction.
- **uniform_length** is the length of the uniform segment. This will override the parameters of **uniform_zseg_from** and **uniform_zseg_to**.
- **taper_length** is the taper length or length between two adjacent uniform segments.
- **xp1_size** is the relative position of an extra point in the 1D mesh along the z-direction. It is used when an extra point is needed relative to **uniform_zseg_from**.
- **xp2_size** is the relative position of an extra point in the 1D mesh along the z-direction. It is used when an extra point is needed relative to **uniform_zseg_to**.
- **mesh_ratio** is the ratio of the adjacent mesh interval. If it is greater than 1, the mesh interval increases with distance. If it is less than 1, the mesh interval

decreases with distance. If it is negative, the mesh obeys the absolute value over the first half of the distance and the other half is symmetric to the first half. Its use is similar to the **ratio** parameter in statement of **put_mesh**.

- **shift_center** is used in the case of negative **mesh_ratio**. The negative ratio will cause the mesh distribution to vary using the center point of the distance as a symmetric point. This parameter causes the center to be shifted in either direction.
- **cylindrical_origin** is origin of the cylindrical system if applicable.
- **zplanes** is the number of mesh planes used for solving variables of the PDE. For each uniform segment, we can use only one mesh plane to describe the doping and material distribution. But for variables such as potential and electron densities, there must be at least one mesh plane. This parameter works together with the statement of **3d_solution_method**. For the method of “3d_flow”, meaning current is allowed to flow between xy-planes, there should be at least two zplanes.
- **cylindrical** would set the current segment to be in cylindrical system.

Example(s):

```
z_structure uniform_zseg_from=0. uniform_zseg_to=20. &&
  zseg_num=1 zplanes=2 taper_points1=(1. 1.)
z_structure uniform_zseg_from=110. uniform_zseg_to=110. &&
  zseg_num=2 zplanes=1 taper_points1=(3.5 3.5)
z_structure uniform_zseg_from=200. uniform_zseg_to=400. &&
  zseg_num=3 zplanes=2 taper_points1=(6. 6.)
```

Comments: The first uniform segment extends from 0 to 20 um with two mesh planes to describe the variables located at 0 and 20 um, respectively. The vertical taper line is located at x=1. There is a single mesh plane in segment 2 at z=110. The taper line is at x=3.5. There are 2 mesh planes in segment 3, located at z=200 and z=400, respectively. The taper line is located at x=6.

Appendix A

List of Graphic Devices

The graphical devices supported by PICS3D are listed in Table [A.1](#). For most workstations with x11-based graphic capability, a good choice is to use the option “x11” in the parameter **plot_device**. If the user still has difficulty in generating graphics, Appendix A of the User’s Manual describes the steps to produce a tecktronic window.

Table A.1: Graphic devices supported by PICS3D. The device names in the first column can be used in the parameter of **plot_device**.

device	description
aed512	AED 512 Terminal
aed767	AED 767 Terminal
aifm	Adobe Illustrator 3.0 Format
bitgraph	BBN Bitgraph Terminal
corel	EPS format for CorelDRAW
dxg	dxg-file for AutoCad (default size 120x80)
dumb	printer or glass dumb terminal
dxg800a	Roland DXY800A plotter
eeepic	EEPIC – extended LaTeX picture environment
emtex	LaTeX picture environment with emTeX specials
epson_180dpi	Epson LQ-style 180-dot per inch (24 pin) printers
epson_60dpi	Epson-style 60-dot per inch printers
epson_lx800	Epson LX-800, Star NL-10, NX-1000, PROPRINTER ...
excl	Talaris EXCL Laser printer (also Talaris 1590 and others)
gpig	GPIC – Produce graphs in groff using the gpig preprocessor
hp2623A	HP2623A and maybe others
hp2648	HP2648 and HP2647
hp7580B	HP7580, and probably other HPs (4 pens)
hp500c	HP DeskJet 500c, [75 100 150 300] [rle tiff]
hpgl	HP7475 and (hopefully) lots of others (6 pens)
hpljii	HP Laserjet series II, [75 100 150 300]
hpdj	HP DeskJet 500, [75 100 150 300]
hppj	HP PaintJet and HP3630 [FNT5X9 FNT9X17 FNT13X25]
imagen	Imagen laser printer
iris4d	Silicon Graphics IRIS 4D Series Computer
kc_tek40xx	MS-DOS Kermit Tek4010 terminal emulator - color
km_tek40xx	MS-DOS Kermit Tek4010 terminal emulator - monochrome
kyo	Kyocera Laser Printer with Courier font

Table A.2: Continued from the previous table

device	description
latex	LaTeX picture environment
mf	Metafont plotting standard
mif	Frame maker MIF 3.00 format
nec_cp6	NEC printer CP6, Epson LQ-800 [monochrome color draft]
pbm	Portable bitmap [small medium large] [monochrome gray color]
pcl5	HP LaserJet III [mode] [font] [point]
postscript	PostScript graphics language [mode "fontname" font_size]
prescribe	Prescribe - for the Kyocera Laser Printer
pstlatex	LaTeX picture environment with PostScript
specials	
pstricks	LaTeX picture environment with PSTricks macros
qms	QMS/QUIC Laser printer (also Talaris 1200 and others)
regis	REGIS graphics language
selanar	Selanar
starc	Star Color Printer
table	Dump ASCII table of X Y [Z] values to output
tandy_60dpi	Tandy DMP-130 series 60-dot per inch graphics
tek410x	Tektronix 4106, 4107, 4109 and 420X terminals
tek40xx	Tektronix 4010 and others; most TEK emulators
texdraw	LaTeX texdraw environment
tgif	TGIF X-Window draw tool (file version 10)
tpic	TPIC - LaTeX picture environment with tpic
specials	
vx384	Vectrix 384 and Tandy color printer
vttek	VT-like tek40xx terminal emulator
x11	X11 Window System

Appendix B

Vectorial Variables For Plotting

In this appendix, we list all the vectorial variables that can be plotted using statements such as **plot_2d**.

variable	definition
elec_curr	electron current
hole_curr	hole current
disp_curr	displacement current
elec_field	electric field (static)
total_curr	total current
flowline_curr	flowline plot of total current

Appendix C

Scalar Variables For Plotting

In this appendix, we list all the scalar variables that can be plotted using statements such as **plot_1d**.

variable	definition
absorption	background optical absorption not including gain/loss from active layers.
acceptor_conc	acceptor concentration.
acceptor_conc2	acceptor concentration (level2).
all_conc	both electron and hole.
all_heat	all heat sources.
band	energy band. (for plot_1d and lplot_xy)
bulk_elec_conc	bulk elec. concentration in QW system.
bulk_hole_conc	bulk hole concentration in QW system.
ch_val_band	cristal field hole valence band edge for wurtzite structures.
cond_band	conduction band.
disp_curr_mag	magnitude of displacement current.
disp_curr_x	displacement current x-component.
disp_curr_y	displacement current y-component.
disp_curr_z	displacement current z-compoenent.
donor_conc	donor concentration.
donor_conc2	donor concentration (level2).
elec_conc	electron concentration.
elec_curr_mag	magnitude of electron current.
elec_curr_x	electron current x-component.
elec_curr_y	electron current y-component.
elec_curr_z	electron current z-compoenent.
elec_diff_gain	electron differential gain (cm^2).
elec_imref	electron IMREF.
elec_mobility	average nodal electron mobility.
	[table to be continued]
exciton_dipole_source	exciton dipole source density.
exciton_singlet	singlet exciton density.
exciton_triplet	triplet exciton density.

field_mag	electrical field maginitude.
field_x	electric field x-component.
field_y	electric field y-component.
hole_conc	hole concentration.
hole_curr_mag	magnitude of hole current.
hole_curr_x	hole current x-component.
hole_curr_y	hole current y-component.
hole_curr_z	hole current z-compoenent.
hole_diff_gain	hole differential gain (cm^2).
hole_imref	hole IMREF.
hole_mobility	average nodal hole mobility.
impact_alpha_n	electron impact ionization coefficient.
impact_alpha_p	hole impact ionization coefficient.
impact_ionization	total impact ionization rate.
incident_power	same as optical_energy.
index_change	refractive index change.
interband_pumping	opitcally pumped interband absorption.
joule_heat	Joule heat source.
kpref_val_band	reference valence band edge for wurtzite structures.
	[table to be continued]

lattice_temp	latttice temperature.
lh_val_band	light hole valence band edge.
linear_bulk_elec	bulk elec. concentration in QW system (linear scale).
linear_bulk_hole	bulk hole concentration in QW system (linear scale).
linear_elec_conc	electron concentraton in linear scale.
linear_heat	Heat source linear in current.
linear_hole_conc	hole concentraton in linear scale.
linear_rec_elec	same as rec_elec_conc but in linear scale.
linear_rec_hole	same as rec_hole_conc but in linear scale.
linear_subb_elec	QW subband elec. concentration (linear scale).
linear_subb_hole	QW subband hole concentration (linear scale).
local_gain	local optical gain.
material_num	internal material number.
negative_fix_charge	negative fixed space charge.
occup_trap	occupancy of trap.
optic_heat	optical heat source.
optical_energy	relative energy distribution scaled to incident light energy density.
optical_gen	optical generation rate induced by incident light.
peltier_heat	Peltier heat source.
positive_fix_charge	positive fixed space charge.
potential	potential.
	[table to be continued]

radiation_heat	radiation heat source term.
real_index	real index.
rec_elec_conc	electron concentration participated in recombination only as opposed to that in transport.
rec_hole_conc	hole concentration participated in recombination only as opposed to that in transport.
recomb_aug	Auger recombination.
recomb_heat	recombination heat source.
recomb_rad	radiative recombination.
recomb_srh	SRH recombination.
recomb_st	stimulated recombination.
space_charge	net space charge.
subb_elec_conc	QW subband elec. concentration.
subb_hole_conc	QW subband hole concentration.
thomson_heat	Thomson heat source.
total_curr_mag	magnitude of total current.
total_curr_x	total current x-component.
total_curr_y	total current y-component.
total_curr_z	total current z-component.
trap_conc	trap concentration.
trap_level	trap level.
val_band	valence (also heavy hole) band edge.
wave_2ndwavel	SOA Traveling Wave (2nd wavelength).
wave_intensity	relative wave intensity.
wave_intensity_all_modes	wave intensity of all modes.

Appendix D

Bias Dependent Variables For Plotting

In this appendix, we list all the bias dependent variables that can be plotted using statements such as **plot_scan** **fourier_power**.

variable	definition
all_mode_power	Emitted laser power for all lateral modes in units of mW. See also scale_lit and facet .
amplifier_power	Traveling-wave semiconductor amplifier (2D model) power at the primary wavelength.
amplifier_power	Traveling-wave semiconductor amplifier (2D model) power at the secondary wavelength.
bottom_led_power	LED power emitted from the bottom of the device.
current_i	Total current (sum of electron, hole and displacement current) at electrode i; for laser simulation this is related to scale_curr .
disp_curr_i	Displacement current (time derivative of the displacement vector, non-zero only for transient simulation).
effective_index	Real effective refractive index for a waveguide device.
efficiency	Laser emission power efficiency (or slope efficiency).
elec_curr_i	Electron current at electrode i
elec_curr_i	Electron current at electrode i.
hole_curr_i	Hole current at electrode i.
current_i	total injection current into a laser diode (applicable for LASTIP).
laser_power	Emitted laser power for a single lateral mode. See also scale_lit and facet .
led_effi	LED internal quantum efficiency.
led_effx1	Similar to led_effz1.
led_effx2	Similar to led_effz1.
led_effy1	Similar to led_effz1.
led_effy2	Similar to led_effz1.
led_effz1	LED power extraction efficiency from facet 1 (i.e., left or bottom facet) facing the z-direction based on the (cubic) broad-area LED model. This result may be different from that calculated from the ray tracing model which may be plotted using the rayplot_bias or 3drayplot_bias statement.
led_effz2	Similar to led_effz1 above.
	[table to be continued]

led_power	Total amount of emitted optical power from all facets of an LED (APSYS) in the (cubic) broad-area LED model. This may be different from the optical power calculated from the ray tracing model which may be plotted using the ray-plot_bias or 3drayplot_bias statement.
led_spon_power	Total amount of integrated spontaneous recombination in unit of power.
light	Incident light power for a photo-sensitive device.
peak_gain	Peak model gain in a waveguide device with lateral mode and active layer(s). It is the modal gain at peak wavelength.
rtg_left_power	Left power emission in RTG model.
rtg_left_power_allmode	All-mode left power emission in RTG model.
rtg_photon_num	photon number in RTG model.
rtg_photon_num_allmode	All-mode photon number in RTG model.
rtg_right_power	Left facet power emission in RTG model.
rtg_right_power_allmode	Right facet power emission in RTG model.
rtg_surf_power	Surface emitting power (for 2nd order grating) in RTG model.
rtg_surf_power_allmode	All-mode surface emitting power (for 2nd order grating) in RTG model.
rtg_wavelength	Wavelength in RTG model.
rtg_2facet_power	2-facet power for a particular mode.
rtg_2facet_power_allmode	All-mode power for a particular mode.
sum_space_charge	Sum of space charge over a region.
temp_max	Maximum temperature in device.
time	Time in pico seconds used in a transient simulation.
	[table to be continued]

top_led_power	LED power emitted from the top of the device .
total_joule_heat	Integrated total Joule heat.
total_optic_heat	Total heat due to optical absorption.
total_peltier_heat	Integrated total Peltier heat.
total_recomb_heat	Integrated total recombination heat.
total_thomson_heat	Integrated total Thomson heat.
tunneling_enhancement	Maximum tunneling enhancement / increase as used to modify the classical drift-diffusion current.
volt_res_i	Voltage at boundary number i. connected to an external resistor. It is the voltage excluding the resistor.
voltage_i	Voltage at electrode i.
wavelength	Laser emission wavelength at peak modal gain for LASTIP.
	[end of table]

Index

data_file, 2-206
do_multilayer_optics , 2-145
export_opt_property , 2-163
file_name1 , 2-213
file_name2 , 2-213
file_name3 , 2-213
file_name4 , 2-213
file_name5 , 2-213
file_name6 , 2-213
file_name7 , 2-213
file_name8 , 2-213
file_name9 , 2-213
ind_file_use , 2-213
mater_num1 , 2-213
mater_num2 , 2-213
mater_num3 , 2-213
mater_num4 , 2-213
mater_num5 , 2-213
mater_num6 , 2-213
mater_num7 , 2-213
mater_num8 , 2-213
mater_num9 , 2-213
segment_num, 2-206
2nd_harmonic, 2-41
2nd_light, 2-474
2ndmode_phase, 2-172 2-173
3_points, 2-365
3d_amplifier_model, 2-42
3d_angle_store , 2-124
3d_emission_model , 2-124
3d_flow, 2-51
3d_plane_control, 2-50
3d_points, 2-396
3d_solution_method, 2-51
3ddata_set, 2-49

3dget_data, 2-49
3drayplot_angle, 2-42
3drayplot_bias, 2-44
3drayplot_phi, 2-46
3drayplot_project, 2-46
3drayplot_surfpower, 2-48
3drayplot_theta, 2-48
4_points, 2-365

A

a_bar, 2-52
a_bulk, 2-52
a_scatt, 2-60
a_well, 2-52
above_previous , 2-112
above_reference , 2-112
abs_spectrum_file , 2-251
absorption_coef , 2-251
absorption_spec, 2-53
absorption, 2-53
AC analysis, 2-54 2-55 2-57
ac_bar , 2-53
ac_bulk , 2-54
ac_data , 2-296
ac_light_input, 2-296
ac_light, 2-54
ac_parameters, 2-55
ac_sparse_solver, 2-57
ac_voltage, 2-57
ac_well , 2-59
acceptor_type2 , 2-438
active region, 2-60
active_and_embedded, 2-204
active_layer , 2-254
active_layer_depth, 2-251
active_loss , 2-69

- active_loss, 2-60
- active_macro , 2-369
- active_macro, 2-204 2-232 2-289
- active_mater, 2-420 2-420
- active_reg_zdim, 2-65
- active_reg, 2-60
- active_regnum, 2-277
- active_temper, 2-66
- active, 2-409 2-413 2-462
- add_all_modes, 2-172
- add_all_wells, 2-376
- add_hori_intercept, 2-326
- add_overlap, 2-260
- add_oxd_memory , 2-317
- add_to_value, 2-286
- add_variable , 2-385
- add_vert_intercept, 2-326
- adjust_active_reg , 2-69
- adjust_barrier, 2-453
- adjust_current, 2-69
- adjust_doping, 2-70
- adjust_range, 2-282
- adjust_resistor, 2-71
- affinity, 2-71
- agrade_from, 2-232
- agrade_to, 2-232
- agrade_var, 2-232
- ai_bar,i=1,2,... , 2-72
- ai_bulk,i=1,2,... , 2-72
- ai_well,i=1,2,... , 2-72
- alias, 2-110
- all_active_mater, 2-204 2-289
- all_interface, 2-196
- all_interfaces, 2-222
- all_material, 2-456
- all_mode_power, D-494
- all_newdoping, 2-159
- allowed, 2-80
- alpha_n, 2-73
- alpha_p, 2-73
- alpha_wavel, 2-74
- aluminum_mater , 2-438
- amplifier_power, D-494
- amplifier_power2, D-494
- amplitude, 2-175
- an_active , 2-209
- an_passive, 2-209
- analytical_recomb, 2-60
- angle , 2-387
- angle_from_100 , 2-436
- angle, 2-229 2-246 2-365
- anisotropic PML, 2-364
- ap_active , 2-209
- ap_passive, 2-209
- append_data , 2-326
- append_data, 2-124
- apply_e_field , 2-180
- arc_height1, 2-130 2-302
- arc_height2, 2-130 2-302
- arc_height3, 2-130 2-302
- arc_height4, 2-130 2-302
- arnoldi_base, 2-116
- at_mesh_near_x , 2-385
- auger coefficient, 2-74 2-74
- auger_n, 2-108 2-254 2-74
- auger_p, 2-108 2-254 2-74
- auto_condition , 2-399
- auto_finish , 2-399
- auto_pn_ratio, 2-108 2-184 2-429
- auto_until , 2-399
- auto_within , 2-399
- auto2_condition , 2-399
- auto2_finish , 2-399
- auto2_until , 2-399
- auto2_within , 2-399
- autodef_entry_point, 2-145
- autodef_wavelength, 2-145
- av_index, 2-108 2-179 2-183 2-184 2-254 2-429
- av_over_a, 2-60
- avar_symboli, 2-369
- avar_symbolk, 2-232
- avari, 2-232 2-369
- average_index , 2-251
- average_polarization, 2-421
- axial_approx, 2-60

axis, 2-109

B

b_bar, 2-74

b_well, 2-75

back_index, 2-75

back_reflection, 2-75

backg_loss, 2-215 2-422

backward , 2-89 2-91

backward , 2-90 2-92

backward , 2-168 2-89

balance_zero_bias, 2-475

band_diagram_file , 2-203

band_discont_right, 2-76

band_discont, 2-76

band_distance, 2-77

band_edge_file, 2-77

band_gap, 2-79

band_offset, 2-79

band_range , 2-77

band_valleys, 2-269

bandgap_gen_iqe, 2-80

bandgap_narrow, 2-79

bandgap_optical_gen, 2-80

bandgap_reduction, 2-159 2-405

barrier_pnt_ll, 2-453

barrier_pnt_ur, 2-453

barrier_type, 2-453

barrier_width_bot, 2-420 2-420

barrier_width_top, 2-420 2-420

barrier_x1_label, 2-453

barrier_x2_label, 2-453

barrier_y1_label, 2-453

barrier_y2_label, 2-453

barrier, 2-102 2-243 2-449

beam_deviation , 2-118

beam_model , 2-118

begin_cavity, 2-80

begin_complex, 2-81

begin_zmater, 2-82

below_previous , 2-112

below_reference , 2-112

bend_extern, 2-82

bend_xy_plane, 2-82

bessel_order , 2-330

bessel_order, 2-461

beta_n, 2-83

beta_p, 2-84

beta_tol, 2-458

between_mater, 2-313

between_materials, 2-112

bgrade_from, 2-232

bgrade_to, 2-232

bgrade_var, 2-232

bias data, 1-38

bias variables, D-493

bias_electrode, 2-44

bias_scan_num, 2-396

bias_variable , 2-44

bias3d, 2-84

binary_search, 2-42 2-470

blank_area, 2-85

bn_active , 2-209

bn_passive, 2-209

bottom_contact, 2-85

bottom_exit_index , 2-382

bottom_led_power, D-494

boundary_i, 2-365

boundary_radius, 2-460

boundary_smooth, 2-86

boundary_type , 2-254

boundary_type, 2-174 2-215

boundary_type1, 2-298

boundary_type2, 2-298

boundary_xpoint, 2-86

boundary, 2-86

bp_active , 2-209

bp_passive, 2-209

bpm_grating, 2-87

bpm_multimode, 2-88

bpm, 2-93

bpmintegr_xy, 2-89

bpmplot_xy1d, 2-89

bpmplot_xyz1d, 2-90

bpmsplot_xy, 2-91

bpmsplot_xz, 2-92

bpmplot_yz, 2-92
 broad-area, 2-316
 broad_width, 2-453
 broaden , 2-387
 broaden , 2-387
 broadening, 2-60
 buffer_x, 2-174
 buffer_y, 2-174
 buffer_z, 2-174
 bulk_strain_exist, 2-471
 bulk_treatment, 2-94
 bulk_xfunck, 2-94

C

c11_bar, 2-95
 c11_well, 2-95
 c12_bar, 2-95
 c12_well, 2-96
 c13_bar , 2-96
 c13_bulk , 2-96
 c13_well , 2-97
 c33_bar , 2-97
 c33_bulk , 2-97
 c33_well , 2-98
 capacitance, 2-170
 capture cross section, 2-451 2-451
 carr_dep_life, 2-244
 carrier number, 2-309
 carrier_conc, 2-329
 carrier_type , 2-436
 carrier, 2-453
 cavity_num, 2-80
 center, 2-175
 change_limit , 2-416
 change_var_type, 2-304
 change_variable, 2-304
 char_impedance, 2-348
 charge_type, 2-130 2-302
 clad_bot , 2-254
 clad_top , 2-254
 column_num , 2-188
 column_num, 2-232 2-369 2-449 2-81 2-98
 column, 2-98

complex MQW, 1-30
 complex_index1 , 2-384
 complex_index1, 2-380
 complex_index2 , 2-384
 complex_index2, 2-380
 complex_index3 , 2-384
 complex_index4 , 2-384
 complex_index5 , 2-384
 complex_region, 2-99
 complex_start_label, 2-289
 complex, 2-417
 component, 2-175
 conc , 2-77
 conc_log_scale, 2-108 2-184 2-254
 conc_logscale , 2-77
 conc_points , 2-375
 conc_range, 2-108 2-179 2-183 2-184 2-254
 2-375 2-429
 cond_bandj_edge, 2-100
 cond_bandj_valley, 2-101
 cond_subband, 2-332
 cond_valley , 2-77
 cond_valley_prop, 2-296 2-77
 condition, 2-352 2-359 2-360 2-362
 conditional_off , 2-416
 condj_mass_para, 2-101
 condj_mass_perp, 2-101
 condj_valley_propk, 2-102
 confine_left, 2-289
 connect_planes , 2-339
 connect_planes, 2-338
 const_index, 2-246
 contact_collection , 2-453
 contact_num, 2-243 2-449 2-57
 contact_refl , 2-164
 contact_refl1, 2-164
 contact_refl2, 2-164
 contact_refl3, 2-164
 contact_type, 2-243 2-449
 contact, 2-102 2-170
 contact1_compindex, 2-164
 contact1_layer2, 2-164
 contact1_layer3, 2-164

contact1_thick , 2-164
 contact2_compindex, 2-164
 contact2_layer2, 2-164
 contact2_layer3, 2-164
 contact2_thick , 2-164
 contact3_compindex, 2-164
 contact3_layer2, 2-164
 contact3_layer3, 2-164
 contact3_thick , 2-164
 contour , 2-91
 contour , 2-92
 convention, 2-105
 converge_test, 2-304
 core_radius , 2-330
 core_radius, 2-318 2-461 2-462
 cos_amp , 2-407
 cos_constant , 2-407
 cos_period , 2-407
 cos_phase , 2-407
 coulomb_dim, 2-60
 coulomb_int_acc, 2-60
 coulomb_screen, 2-60
 coupled_dots , 2-371
 cplot_xy, 2-106
 cplot_xyz, 2-105
 cross_section, 2-450
 current_conc, 2-108
 current_dep_life, 2-244
 current_distr, 2-54 2-57
 current_from, 2-396
 current_i, 2-405 D-494
 current_off , 2-416
 current_scale, 2-282
 current_to real, 2-366
 current_to, 2-396
 curve_math, 2-346
 curve_number, 2-184 2-429
 cut_at_x , 2-112
 cylindrical_origin, 2-477
 cylindrical, 2-109 2-477

D

d_fall , 2-188

d_fall, 2-189
 d_high , 2-188
 d_high, 2-189
 d_low , 2-188
 d_low, 2-189
 d_rise , 2-188
 d_rise, 2-189
 d, 2-229
 damping_step, 2-193 2-304
 data_file , 2-387 2-387
 data_file , 2-385
 data_file , 2-339
 data_file , 2-265 2-441 2-443 2-465
 data_file , 2-464
 data_file , 2-176
 data_file, 2-105 2-106 2-108 2-155 2-168
 2-172 2-179 2-182 2-183 2-184 2-224
 2-240 2-241 2-241 2-254 2-263 2-332
 2-334 2-335 2-336 2-338 2-340 2-343
 2-344 2-345 2-346 2-348 2-350 2-352
 2-355 2-359 2-360 2-361 2-362 2-388
 2-390 2-390 2-42 2-429 2-431 2-433
 2-44 2-44 2-46 2-464 2-466 2-467
 2-48 2-48 2-77
 data_file2 , 2-464
 data_num, 2-192 2-268 2-273
 data_point , 2-77
 data_point , 2-441 2-443
 data_point, 2-108 2-155 2-179 2-183 2-184
 2-224 2-254 2-429
 data_set, 2-214 2-352 2-359 2-360 2-362
 2-394
 datafile_number, 2-145
 datafile_range , 2-124
 datafile_range , 2-118
 datafile_unit, 2-130
 dataset_end, 2-346
 dataset_start, 2-346
 dbr_truncate, 2-110
 dbr1_box1 , 2-164
 dbr1_box2 , 2-164
 dbr1_box3 , 2-164
 dbr1_index1 , 2-164

dbr1_index2 , 2-164
dbr1_layer1 , 2-164
dbr1_layer2 , 2-164
dbr2_box1 , 2-164
dbr2_box2 , 2-164
dbr2_box3 , 2-164
dbr2_index1 , 2-164
dbr2_index2 , 2-164
dbr2_layer1 , 2-164
dbr2_layer2 , 2-164
decade, 2-178
deep traps, 1-29
define_alias, 2-110
define_material, 2-111
define_symbol, 2-111
define_vertical_position, 2-112
delta_active_loss, 2-66
delta_affinity, 2-427
delta_in, 2-282
delta_index , 2-254
delta_index, 2-215
delta_kimag, 2-409 2-413
delta_kreal, 2-409 2-413
delta_ncarr_loss, 2-66
delta_pcarr_loss, 2-66
delta_pitch, 2-409 2-413
delta_so_bar, 2-113
delta_so_well, 2-113
delta_tau_scat, 2-66
delta_time, 2-396
delta_wavel, 2-238 2-282
deltai_bar,i=1,2,3, 2-114
deltai_bulk,i=1,2,3,4, 2-114
deltai_well,i=1,2,3, 2-114
density , 2-443
density_end , 2-441
density_start , 2-441
density_tol, 2-304
density, 2-395
detect_ratio, 2-299
device_kind , 2-118
device_kind, 2-124
DFB, 2-216 2-217 2-255
dgd_n_type2, 2-271
dgdw_type2, 2-271
di_bar,i=1,2,3,4, 2-115
di_bulk,i=1,2,... , 2-115
di_well,i=1,2,3,4, 2-116
diel_av, 2-60
dielectric_constant, 2-116
dip_factor , 2-69
dip_factor_pump, 2-289
dip_factor, 2-60
dir , 2-279 2-447
dir, 2-110 2-374
direct_eigen, 2-116
direction , 2-89
direction, 2-130 2-136 2-453
directory, 2-277
disp_curr_i, D-494
distance_range , 2-385
distance, 2-46
distance1 , 2-310
distance2 , 2-310
dkdn_imag, 2-271
dkdn_nref, 2-271
dkdn_real, 2-271
dndn_type2, 2-271
do_raytrace_3d, 2-124
do_raytrace, 2-118
dome_base_thickness , 2-124
dome_bottom_mirror, 2-124
dome_cyl_height , 2-124
dome_cyl_radius , 2-124
dome_direction , 2-118
dome_height , 2-118
dome_opindex , 2-118
dome_radius , 2-118
dome_sph_radius , 2-124
dome_width , 2-118
donor_type2 , 2-438
dopant_index_from, 2-320
dopant_index_to , 2-320
doped_organic , 2-235
doping profile, 2-130
doping, 2-130

dos_reduction_factor, 2-371
 double_mesh, 2-136
 dox_efield0_pf_elec, 2-138
 dox_efield0_pf_hole, 2-138
 dox_el_weight, 2-139
 dox_exciton_eg, 2-139
 dox_extern_spectrum, 2-139
 dox_gaussian_divj, 2-140
 dox_gaussian_sdj, 2-140
 dox_hopping_energy, 2-140
 dox_level_req, 2-235
 dox_max_phonon, 2-235
 dox_peak_abs, 2-140
 dox_phonon_cloud, 2-235
 dox_triplet, 2-235
 dox_vib_quanta, 2-140
 dox_xp_coupling, 2-140
 dox2_el_weight, 2-140
 dox2_extern_spectrum, 2-141
 dox3_el_weight, 2-142
 dox3_extern_spectrum, 2-142
 dox4_el_weight, 2-143
 dox4_extern_spectrum, 2-143
 dox5_el_weight, 2-144
 dox5_extern_spectrum, 2-145
 doxk($k_z=2$)_triplet, 2-235
 drift_away, 2-376

E

EAM, 2-470
 edge_curve, 2-147
 edge, 2-147 2-368
 edge1_prof, 2-130 2-302
 edge2_prof, 2-130 2-302
 edge3_prof, 2-130 2-302
 edge4_prof, 2-130 2-302
 eeim_optic, 2-148 2-151
 effective_exit_index, 2-124
 effective_index, D-494
 efficiency_model, 2-238
 efficiency, 2-238 D-494
 efield_ref, 2-320
 efield0_pf, 2-324
 efield0_pf_elec, 2-153
 efield0_pf_hole, 2-154
 eg_fac1, 2-209
 eg_fac2, 2-209
 eg0_bar, 2-150
 eg0_bulk, 2-150
 eg0_well, 2-151
 eim_zero, 2-461
 el_frac_non_active, 2-317
 el_vel_model, 2-269
 elec_capture, 2-196
 elec_carr_loss, 2-154
 elec_conc_subb, 2-296
 elec_conc, 2-431
 elec_curr_i, D-494 D-494
 elec_dos_energy, 2-155
 elec_eifactor, 2-199
 elec_escape, 2-376
 elec_field, 2-77
 elec_field_model, 2-279
 elec_fly_over, 2-376
 elec_hv_critical_field, 2-279
 elec_hv_expo_field, 2-279
 elec_hv_minfac, 2-279
 elec_intel1_beta, 2-279
 elec_intel1_e_crit, 2-279
 elec_intel2_alpha, 2-279
 elec_intel2_e_univ, 2-279
 elec_lambda0, 2-199
 elec_level, 2-441 2-443
 elec_lombardi_alpha, 2-279
 elec_lombardi_b, 2-279
 elec_lombardi_beta, 2-279
 elec_lombardi_delta, 2-279
 elec_mobility, 2-296
 elec_phonon, 2-199
 elec_set1, 2-201
 elec_set2, 2-201
 elec_set3, 2-201
 elec_setnum, 2-201
 elec_temp, 2-102
 elec_vel_factor, 2-202
 electro-absorption modulators, 2-470

electrode_off , 2-416
 electrode, 2-102 2-173 2-396
 electron affinity, 2-71
 electron_mass, 2-155
 electron_mobility, 2-156
 electron_ref_dens, 2-156
 electron_sat_vel, 2-156
 embedded_material, 2-157
 embedded_structure, 2-232 2-60
 emission_model , 2-118
 emit_carrier, 2-450
 emit_from_phi , 2-426
 emit_from_theta, 2-426
 emit_from_xyz , 2-426
 emit_points_number , 2-124
 emit_sector , 2-118
 end , 2-412
 end_cavity, 2-157
 end_complex, 2-157
 end_loop, 2-158
 end_zmater, 2-158
 end, 2-346 2-352 2-359 2-360 2-361 2-362
 energy_points, 2-453
 energy_range, 2-155 2-224
 entry_point_x, 2-145
 entry_point_y, 2-145
 entry_point_z, 2-145
 epsilon_infinity , 2-442
 equilibrium, 2-159
 equiv_mobility, 2-160
 equiv_tunnel_junc, 2-160
 every_bias , 2-164
 exch_coef , 2-69
 exch_coef, 2-60
 exciton, 2-60
 expo_mix, 2-60
 export_data, 2-174
 export_eeimmsg, 2-148 2-151
 export_gain_data, 2-161
 export_grid, 2-174
 export_kp_data, 2-162
 export_kp_para, 2-162
 export_raytrace, 2-164
 export_ref_population, 2-436
 export_super_struc, 2-167
 export_var, 2-174
 export_veeim, 2-395
 export_wave, 2-168
 ext_funck, 2-169
 extern_cir, 2-104 2-447
 extern_parallel_cap, 2-170
 extern_parallel_cir, 2-170
 extern_parallel_ind, 2-170
 extern_parallel_res, 2-170
 extern_parallel_voltage, 2-170
 extern_temp, 2-102
 external_cir, 2-170
 external_opindex, 2-145
 external_package , 2-124
 extra_point_1, 2-368
 extra_point_2, 2-368
 extra_time, 2-174
 extract_data, 2-124
 extract_to_file, 2-124
 extrap_conc , 2-204 2-375
 extrap_energy , 2-204
 extrap_energy, 2-375
 extrap_pn_ratio , 2-204 2-375
 extrap_temper , 2-204 2-375
 extrapolate, 2-304

F

facet , 2-176
 facet, 2-343 2-355
 factor , 2-447
 factor_elec , 2-279
 factor_file, 2-287
 factor_hole , 2-279
 factor_vertical, 2-340
 farfield_couple, 2-173
 farfield, 2-172
 fast_dynamic_search, 2-428
 fd_mesh, 2-60
 fdtd_model, 2-174
 fdtd_source, 2-175
 fermi_n , 2-77

fermi_p , 2-77
field_dep_curves, 2-429
field_range, 2-429
file , 2-161 2-162 2-162 2-167 2-203 2-204
file , 2-326
file, 2-207 2-366
filter_name, 2-355
filter_value, 2-355
fit_hori_from , 2-326
fit_hori_to , 2-326
fit_mass , 2-294
fit_outfile, 2-108
fit_power, 2-193
fit_range, 2-193
fit_vert_from , 2-326
fit_vert_to , 2-326
fitting_model, 2-181
fix_charge, 2-220
fld_center, 2-215 2-468
fld_range_n1, 2-201
fld_range_n2, 2-201
fld_range_p1, 2-201
fld_range_p2, 2-201
flux_area , 2-118
flux_plot, 2-178
flux_represent , 2-118
flux_size_auto, 2-118
force_yrange, 2-282
fourier_power, 2-176
fphase_input, 2-260
fraction , 2-381
fraction_mode2, 2-246
fraction_mode3, 2-246
fraction_mode4, 2-246
fraction_mode5, 2-246
fraction_TE, 2-246
franz_keldysh, 2-60
free-carrier, 2-209
free_carr_passive, 2-209
freeze_both_mater, 2-309
freeze_carrier, 2-304 2-309
freeze_electron_mater, 2-309
freeze_hole_mater, 2-309

freq_control, 2-178
freq_end , 2-176
freq_point, 2-54 2-55 2-57
freq_shd_num, 2-41
freq_shd1, 2-41
freq_shd2, 2-41
freq_shd3, 2-41
freq_shd4, 2-41
freq_shd5, 2-41
freq_start, 2-176 2-178
fresnel_reflections, 2-246
from_bottom , 2-112
from_top , 2-112
from, 2-243 2-332 2-334 2-449
front_back , 2-254
front_back, 2-215 2-301
front_index, 2-178
front_reflection, 2-178
full_ionization, 2-177
func_num, 2-82
function, user-defined, 1-18
function_labeli, 2-399

G

gaas_mater , 2-438
gain preview, 2-108 2-155 2-179 2-179 2-183
2-183 2-184 2-184 2-212 2-225 2-441
2-442 2-443 2-74 2-77
gain.sat, 2-215
gain_correction , 2-363
gain_correction, 2-148 2-151
gain_coulomb, 2-60
gain_density, 2-179
gain_fitting, 2-181
gain_module, 2-180
gain_num_integr, 2-289
gain_spectrum, 2-182
gain_spon, 2-183
gain_wavel, 2-184
gainsearch_num, 2-304
gamma_detail , 2-441 2-443
gamma_subband, 2-464
gammak_bar, 2-185

- gammak_well, 2-185
 - gaussian_tail, 2-229
 - gen_datafile , 2-251
 - gen_datafile, 2-246
 - gen_datatype , 2-251
 - gen_datatype, 2-246
 - generic_impurity , 2-438
 - get_active_layer, 2-186
 - get_data, 2-187
 - global_newton, 2-282
 - global, 2-271
 - go_backward, 2-93
 - gpeak_model, 2-181
 - grade_active_mater, 2-191
 - grade_from, 2-157 2-191 2-232
 - grade_num, 2-157
 - grade_points, 2-191
 - grade_to, 2-157 2-191 2-232
 - grade_var, 2-157 2-232
 - grading_angle, 2-130 2-302
 - grading_anglej, 2-229
 - grading_from, 2-130 2-302
 - grading_fromj, 2-229
 - graphic devices, A-481
 - graphical devices, 2-350
 - grating_compos, 2-188
 - grating_dia_ratio, 2-331
 - grating_height, 2-331
 - grating_model, 2-189 2-409 2-410 2-413 2-414 2-462
 - grating_order , 2-188
 - grating_order, 2-189 2-87
 - grating_phase, 2-409 2-413
 - grating_xrange, 2-189
 - grating_yrange, 2-189
 - grid_sizes, 2-105 2-106 2-335 2-336 2-431 2-433 2-466 2-467
 - group_index , 2-382
 - group_index , 2-330
 - group_index, 2-238 2-269
 - group1, 2-192 2-273 2-425 2-425 2-426 2-426
 - gsn_dt , 2-407
 - gsn_s1 , 2-407
 - gsn_s2 , 2-407
 - gsn_t1 , 2-407
 - guide_bot , 2-254
 - guide_top , 2-254
- ## H
- hl_imag, 2-409 2-413
 - hl_real, 2-409 2-413
 - half_mesh, 2-192
 - heat_flow, 2-102 2-193
 - height_range, 2-331
 - height, 2-147 2-369
 - heteroj_capture, 2-196
 - heteroj_scrn_factor, 2-317
 - hh_or_lh , 2-441 2-443
 - hi_active_macro , 2-188
 - hi_aavar_symbol1-5, 2-188
 - hi_aavar1 , 2-188
 - hi_aavar2 , 2-188
 - hi_aavar3 , 2-188
 - hi_aavar4 , 2-188
 - hi_aavar5 , 2-188
 - hi_macro_name , 2-188
 - hi_var_symbol1-5, 2-188
 - hi_var1 , 2-188
 - hi_var2 , 2-188
 - hi_var3 , 2-188
 - hi_var4 , 2-188
 - hi_var5 , 2-188
 - hole_capture, 2-196
 - hole_carr_loss, 2-197
 - hole_conc_subb, 2-296
 - hole_curr_i, D-494
 - hole_eifactor, 2-199
 - hole_escape , 2-376
 - hole_field_model , 2-279
 - hole_fly_over, 2-376
 - hole_hv_critical_field, 2-279
 - hole_hv_expo_field, 2-279
 - hole_hv_minfac, 2-279
 - hole_intel1_beta , 2-279
 - hole_intel1_e_crit, 2-279
 - hole_intel2_alpha , 2-279

- hole_intel2_e_univ, 2-279
- hole_lambda0, 2-199
- hole_level, 2-441 2-443
- hole_lombardi_alpha, 2-279
- hole_lombardi_b, 2-279
- hole_lombardi_beta, 2-279
- hole_lombardi_delta, 2-279
- hole_mass, 2-198
- hole_mobility, 2-198 2-296
- hole_phonon, 2-199
- hole_ref_dens, 2-199
- hole_sat_vel, 2-199
- hole_set1, 2-201
- hole_set2, 2-201
- hole_set3, 2-201
- hole_setnum, 2-201
- hole_temp, 2-102
- hole_vel_factor, 2-202
- hole_vel_model, 2-269
- hori_intercept, 2-326
- hot_electron, 2-399
- huge_elem, 2-304
- I**
- ignore_cband, 2-77
- ignore_vband, 2-77
- imag_index_high, 2-189
- imag_index_low, 2-189
- imag_index, 2-178 2-314 2-75
- imag_indexj, 2-242
- imag_part, 2-334
- impact_baraff, 2-199
- impact_chynoweth, 2-201
- impact_damp, 2-304
- impact_ionization, 2-296
- impact_model, 2-202
- import_dir, 2-203 2-371
- import_dir2-19, 2-371
- import_file1, 2-424
- import_file2, 2-424
- import_file3, 2-424
- import_file4, 2-424
- import_file5, 2-424
- import_file6, 2-424
- import_file7, 2-424
- import_file8, 2-424
- import_file9, 2-424
- import_gain_data, 2-204
- import_kp_data, 2-206
- import_qdot_data, 2-371
- import_qdot_data2-19, 2-371
- import_ref_population, 2-436
- import_rto, 2-238 2-246
- import_super_struct, 2-203
- import_veeim, 2-461
- import_wave, 2-206
- impose_limit, 2-416
- impurity, 2-130 2-302 2-70
- inc_curve, 2-355
- incident light, 2-246 2-251
- incident_light_side, 2-118
- incident_light_side, 2-124
- incident_power, 2-251
- incident_power, 2-246
- include_data, 2-179 2-183 2-184 2-355 2-465
- include, 2-207
- increase, 2-324
- indep_xmqw, 2-98
- independent MQW, 1-30
- independent_mqw, 2-207
- index change model, 2-209
- index_cladding, 2-330
- index_cladding, 2-461
- index_core, 2-330
- index_core, 2-461
- index_field_dependence, 2-208
- index_model, 2-209
- index_newdoping, 2-159
- index_spectrum, 2-211
- index_varj, 2-87
- index_wavel, 2-212
- index, 2-329
- indexj, 2-409 2-413 2-462
- inductance, 2-170
- infile_label, 2-399
- InGaAlAs/InP, 2-186

InGaAlP/GaAs, 2-186
 InGaAs/AlGaAs, 2-186
 InGaAsP/InP, 2-186
 InGaAsP, 2-186
 InGaN, 2-186
 init_factor, 2-42 2-470
 init_sol, 2-214
 init_step, 2-399
 init_wave, 2-215
 init_wavel, 2-254
 init_wavel, 2-215
 initial_amplitude, 2-88
 initial_phase, 2-88
 initial_rays, 2-124
 initial_rays, 2-118
 initialize_bpm, 2-298
 inner_bar_gain, 2-219
 input_contact, 2-55
 input_f2, 2-340
 input_file, 2-334 2-340 2-343 2-344 2-348
 input_var, 2-176
 input_watt, 2-42 2-470 2-474
 inside_ref_point, 2-424
 integer_func, 2-219 2-222
 integration, 2-265
 integration_length, 2-265
 integration_length, 2-263 2-332
 integration_start, 2-265
 integration_start, 2-263 2-332
 integration_xrange, 2-336 2-433
 integration_yrange, 2-336 2-433
 integration, 2-263 2-332 2-336 2-433
 interband, 2-209
 interface_mater, 2-196 2-222
 interface, 2-220
 intern_loss, 2-330
 intern_loss, 2-329
 intern_parallel_cap, 2-170
 intern_parallel_cir, 2-170
 intern_parallel_ind, 2-170
 intern_parallel_res, 2-170
 intern_parallel_voltage, 2-170
 intern_spon_par, 2-271

intern_xp_size, 2-243 2-449
 internal extra point, 2-222
 internal_xpoint, 2-222
 intersubband_model, 2-224
 intervalence band absorption, 2-63
 iter_flag, 2-304
 iter_num, 2-282
 iter_print, 2-282
 iter_step, 2-304

J

j.e_model, 2-193
 jdos_energy, 2-224
 joule_mobil_flag, 2-193
 joule, 2-193
 jtotal_model, 2-193
 junc_coefs, 2-102
 junction_type, 2-375

K

k_range_wurt, 2-60
 k_range, 2-60
 kane_para_f_bar, 2-225
 kane_para_f_well, 2-225
 kappa_imag, 2-409 2-413
 kappa_l, 2-110
 kappa_real, 2-409 2-413
 kp_bands, 2-289
 kp_points, 2-289
 kp_pot_sym, 2-289
 kp88_cond_parab, 2-289
 kp8x8_angle_points, 2-184

L

l.w.e.factor, 2-209
 label, 2-112
 label, 2-407
 label, 2-364 2-472 2-473
 laser_current_i, D-494
 laser_power, D-494
 lateral_mode3d, 2-226
 lattice_bar, 2-226
 lattice_base, 2-227
 lattice_bulk, 2-227

- lattice_constant, 2-227
- lattice_temp, 2-102
- lattice_well, 2-228
- lax_mass_bar, 2-228
- lax_mass_well, 2-228
- layer_conf, 2-231
- layer_mater, 2-232
- layer_num, 2-314 2-81
- layer_type, 2-235
- layer_xrange, 2-231
- layer_yrange, 2-231
- layer, 2-229
- layer1 , 2-384
- layer1 , 2-380
- layer2 , 2-384
- layer2 , 2-380
- layer3 , 2-384
- layer4 , 2-384
- layer5 , 2-384
- layerj, 2-409 2-413
- layerk, 2-462
- lband_bar, 2-237
- lband_well, 2-237
- lbandata_num, 2-215 2-468
- LED extraction, 2-45
- led_control, 2-238
- led_eff_distr, 2-240
- led_effi, D-494
- led_effx1, D-494
- led_effx2, D-494
- led_effy1, D-494
- led_effy2, D-494
- led_effz1, D-494
- led_effz2, D-494
- led_farfield, 2-241
- led_power, D-495
- led_simple, 2-242
- led_spectrum, 2-241
- led_spon_power, D-495
- led_top_coating, 2-242
- led_xrange, 2-238
- led_yrange, 2-238
- left_contact, 2-243
- left_f_refl, 2-260
- left_mesh, 2-289
- left_phase, 2-260
- length , 2-254
- length, 2-215 2-409 2-411 2-413
- level_srch, 2-60
- level, 2-105 2-106 2-130 2-172 2-302 2-335 2-70
- life_n, 2-108 2-254
- life_p, 2-108 2-254
- lifetime_model, 2-244
- lifetime_n, 2-244
- lifetime_p, 2-245
- lifetime, 1-29
- light_3damp, 2-405
- light_current, 2-254
- light_dir , 2-251
- light_dir, 2-246
- light_ext, 2-396
- light_power_qwip, 2-251
- light_power, 2-246
- light, 2-405 D-495
- light2, 2-405
- limit_elem, 2-304
- limit_gain, 2-476
- limits_i, 2-365
- linear_active, 2-209
- linear_heat, 2-257
- linear_passive, 2-209
- linear_source, 2-193
- linear_term, 2-208
- link_taper_contact, 2-477
- lo_active_macro , 2-188
- lo_aavar_symbol1-5, 2-188
- lo_aavar1 , 2-188
- lo_aavar2 , 2-188
- lo_aavar3 , 2-188
- lo_aavar4 , 2-188
- lo_aavar5 , 2-188
- lo_macro_name , 2-188
- lo_var_symbol1-5, 2-188
- lo_var1 , 2-188
- lo_var2 , 2-188

lo_var3 , 2-188
 lo_var4 , 2-188
 lo_var5 , 2-188
 load_macro, 2-258
 load_mesh, 2-259
 local_correction, 2-453
 log_freq , 2-176
 log_freq1, 2-54 2-55 2-57
 log_freq2, 2-54 2-55 2-57
 log_scale, 2-130
 log_scan, 2-55 2-57
 longitudinal, 2-260
 loop_integer, 2-262
 loop_num, 2-416
 loop_real, 2-262
 loopback, 2-304
 loss, 2-53 2-53
 lower_bound_label, 2-289
 lower_level, 2-224
 lower_wj, 2-229
 lplot_xy, 2-263
 lplot_xyz, 2-265
 Luttinger-Kohn, 2-61

M

macro library, 2-258 2-268
 macro_name , 2-369
 macro_name, 2-232 2-456 2-474
 macro1, 2-393
 macro2, 2-393
 macroj, 2-457
 main_contact, 2-453
 main_input, 2-187 2-49
 mark_3d , 2-399
 mass_density, 2-266
 mass_gamma_bar, 2-266
 mass_gamma_bulk, 2-267
 mass_gamma_well, 2-267
 mass_l_bar, 2-267
 mass_l_well, 2-268
 mass_model, 2-294
 mater , 2-203 2-324
 mater , 2-99

mater , 2-279 2-447
 mater , 2-442
 mater_active , 2-371
 mater_active , 2-204
 mater_boundary, 2-106 2-335 2-466
 mater_from , 2-320
 mater_to , 2-320
 mater_var, 2-268
 mater, 2-157 2-177 2-186 2-192 2-199 2-201
 2-211 2-235 2-258 2-269 2-273 2-276
 2-286 2-289 2-329 2-367 2-367 2-394
 2-427 2-450 2-456 2-474 2-60 2-66
 material_macro, 1-18
 material_3d, 2-271
 material_label, 2-232
 material_label1, 2-222
 material_label2, 2-222
 material_par, 2-156 2-156 2-198 2-273 2-275
 2-275 2-277 2-277 2-389 2-391 2-391
 2-451 2-451 2-53 2-53
 material, 2-269 2-365
 math_oper, 2-332
 mathematical functions, 1-18
 max_conc, 2-130 2-302
 max_curr, 2-254
 max_double_times, 2-392
 max_electron_mob, 2-275 2-275 2-277 2-277
 max_element , 2-116
 max_emit_angle , 2-382
 max_hole_mod, 2-275 2-275 2-277 2-277
 max_index , 2-116
 max_iter, 2-193 2-304 2-428 2-458
 max_kp8x8_angle, 2-289
 max_level, 2-282
 max_mb_screen_length, 2-289
 max_primary , 2-124
 max_secondary , 2-124
 max_secondary , 2-118
 max_step , 2-399
 max_temp_incr, 2-193
 maxvector_scale, 2-335 2-466 2-467
 mb_damping_factor, 2-289
 mb_pf_expo_factor, 2-289

- mb_pf_model, 2-289
- megbyte , 2-67 2-67 2-68 2-68
- merge_next, 2-355
- mesh generation, layer, 2-229 2-232 2-243
2-369 2-395 2-449 2-473 2-85 2-98
- mesh generation, 2-147 2-259 2-276 2-352
2-364 2-365 2-368 2-86
- mesh refinement, 2-136 2-192 2-392
- mesh_inf, 2-136 2-259 2-86
- mesh_infile, 2-352
- mesh_multiply, 2-453
- mesh_num, 2-98
- mesh_outf, 2-136 2-276 2-392 2-86
- mesh_output, 2-276
- mesh_points, 2-409 2-411 2-413 2-462
- mesh_ratio , 2-477
- mesh_ratio, 2-409 2-413
- message file, 1-38
- metal_conductivity, 2-276
- metal, 2-276
- method_line_base, 2-392
- method, 2-373
- mf_cpu, 2-304
- mf_md_flag, 2-304
- mf_pivot_update, 2-304
- mf_solver, 2-304
- mf2_flag, 2-193
- min_cyl_plane_distance, 2-50
- min_density, 2-304 2-392
- min_electron_mob, 2-275 2-275 2-277 2-277
- min_flux_points , 2-118
- min_hole_mob, 2-275 2-275 2-277 2-277
- min_lit, 2-304
- min_step , 2-399
- miniband_fd_mesh, 2-453
- miniband_k_points, 2-453
- miniband_model miniband_model, 2-453
- miniband_number, 2-453
- miniband_period_y1_label, 2-453
- miniband_period_y1, 2-453
- miniband_period_y2_label, 2-453
- miniband_period_y2, 2-453
- miniband_upper_range, 2-453
- minimum, 2-296
- mirror_ref , 2-254
- mirror_ref, 2-215
- mj, 2-229
- mmb_gaintable, 2-277
- mobility_low field, 2-156 2-198
- mobility_control, 2-405
- mobility_order, 2-159
- mobility_xy, 2-279
- mobility, 2-156 2-156 2-156 2-198 2-199
2-199 2-275 2-275 2-277 2-277 2-311
2-73 2-73 2-83 2-84
- modal_base, 2-148 2-151
- modal_gain_method, 2-215
- mode_index , 2-265
- mode_index , 2-88
- mode_index , 2-176
- mode_index, 2-105 2-106 2-172 2-173 2-189
2-231 2-263 2-332 2-335 2-336 2-338
2-355 2-379 2-431 2-433
- mode_num , 2-226
- mode_num, 2-298 2-299 2-301
- mode_srch, 2-282
- mode, 2-463 2-464 2-60
- model_points, 2-331
- model, 2-220 2-424
- modify_bulk_macro, 2-286
- modify_opt_gen_rate, 2-287
- modify_plot, 2-288
- modify_qw, 2-289
- modify_wurtzite, 2-294
- modu_bias, 2-295
- modu_mode_index, 2-57
- monitor_emission, 2-295
- more_cond_energy, 2-289
- more_output, 2-296
- more_val_energy, 2-289
- mqw_complex , 2-465
- muller_functol, 2-148 2-151
- muller_maxit, 2-148 2-151
- muller_vartol, 2-148 2-151
- multi-frontal, 2-196
- multi-lateral mode, 2-298

multimode_detect, 2-299
 multimode_mirror, 2-301
 multimode, 2-298

N

n_doping_level , 2-438
 n_doping_level2 , 2-438
 n_doping, 2-232
 n_dopingj real, 2-229
 n_index_newdoping, 2-159
 n_layer , 2-124
 n_lbdata, 2-215 2-468
 n_newdoping_index, 2-232
 n_sections, 2-295
 n_side_down , 2-376
 n_ubdata, 2-215 2-468
 n_user_values, 2-181
 n_values, 2-192 2-268 2-273
 n_variables , 2-265
 n_variables, 2-263 2-332 2-340 2-346 2-352
 2-355 2-359 2-360 2-361 2-362
 n, 2-229
 name , 2-110
 name, 2-186 2-258 2-268 2-365
 nb_mesh, 2-174
 nb_wavel, 2-174
 ncarr_loss , 2-69
 ncarr_loss, 2-329 2-60
 negative_stimulate, 2-304
 new_doping, 2-302 2-405
 new_dopingi, 2-405
 newdoping_index, 2-130
 newdoping_order, 2-159
 Newton's method, 2-304
 newton_freeze_carrier, 2-309
 newton_par, 2-304
 nitride_mater , 2-438
 nlay, 2-424
 nonlocal_transp_region, 2-310
 norm_field, 2-311
 nstep, 2-159
 num , 2-437
 num_emit_angle , 2-382

num_newdoping, 2-159
 num_optic_y_grid , 2-382
 num_points, 2-85
 num_spec_source , 2-382
 num, 2-102 2-439
 number_of_layers, 2-242
 number_showray , 2-118
 number, 2-368

O

ohmic_junction, 2-313
 oled_control, 2-313
 omega_xrange, 2-282
 omega_xshift, 2-282
 omega_ydamp, 2-282
 omega_ymin, 2-282
 omega_yrange, 2-282
 omega, 2-458
 omegab, 2-458
 one_point, 2-125
 opindex_incident , 2-118
 opt_absorption, 2-193
 opt_iter, 2-304
 optic_coating, 2-314
 optic_pumped_escape, 2-376
 optic_pumped_fly, 2-376
 optic_range_x1_label , 2-382
 optic_range_x2_label , 2-382
 optic_range_y1_label , 2-382
 optic_range_y2_label , 2-382
 optical_field, 2-315
 optical_generation, 2-246
 option, 2-172
 order, 2-136 2-276 2-86
 organic_exciton_diff, 2-317
 organic_exciton, 2-296
 origin, 2-130 2-302
 original_joule_term, 2-193
 outer_section, 2-318
 outfile , 2-89
 outfile_label , 2-399
 outfile_label, 2-159
 output data format, 1-37

output_contact, 2-55
output_file, 2-379 2-54 2-55 2-57
output_scale_factor, 2-118
output_var, 2-176
output, 2-317
outside_ref_point, 2-424
override_barrier, 2-60
ox_dopant_el_transfer, 2-320
ox_el_weight, 2-321
ox_exciton_eff, 2-322
ox_exciton_eg, 2-321
ox_extern_spectrum, 2-322
ox_gaussian_divj, 2-323
ox_gaussian_sdj, 2-323
ox_hopping_energy, 2-324
ox_level_req, 2-235
ox_life_field_dependence, 2-324
ox_max_phonon, 2-235
ox_peak_abs, 2-325
ox_phonon_cloud, 2-235
ox_pump_only, 2-246
ox_use_exciton_diff, 2-235
ox_use_extern_spec, 2-235
ox_vib_quanta, 2-325
ox_xp_coupling, 2-326
oxd_diff_length, 2-318
oxd_lifetime, 2-319
oxd2_diff_length, 2-319
oxd2_lifetime, 2-319
oxide_mater, 2-438
oxynitride_mater, 2-438

P

p_doping_level, 2-438
p_doping_level2, 2-438
p_doping, 2-232
p_dopingj, 2-229
p_newdoping_index, 2-232
package_refr_index, 2-124
para_extract, 2-326
parameter_type, 2-348
parameter, 1-16
parameterize, 2-375

partition_hydro_term, 2-471 2-475
passive_3d, 2-329
passive_carr_loss, 2-329
passive_fiber, 2-330
pc_led_model, 2-331
pcarr_loss, 2-69
pcarr_loss, 2-329 2-60
peak_gain, D-495
peltier, 2-193
penetration_depth, 2-246
period, 2-87
period_number, 2-380
permittivity_imag, 2-363
permittivity_real, 2-363
pf_cont_shift, 2-130 2-302
pf_lifetime_expo, 2-324
pf_model, 2-130 2-302 2-70
phase_shift, 2-409 2-413
phi_range, 2-42
phi, 2-48
phonon_omega_LO, 2-442
photon_fac, 2-215 2-461
photon_scan_num, 2-396
photon_scan, 2-396
photoresist_mater, 2-438
pick_xmode, 2-298
pick_ymode, 2-298
pitch, 2-409 2-413
plane, 2-125
plasma_coeff, 2-60
plate_thickness, 2-124
plate_width, 2-124
plot_1d, 2-332
plot_2d, 2-335
plot_3d, 2-336
plot_3dcolor, 2-338
plot_3dmesh, 2-339
plot_ac_laser, 2-343
plot_ac_modal_gain, 2-344
plot_ac_parameters, 2-348
plot_amp, 2-345
plot_as_abs, 2-184
plot_bias, 2-346

- plot_data, 2-350
- plot_device, 2-350 2-352 2-353
- plot_distri, 2-350
- plot_mesh, 2-352
- plot_modu, 2-352
- plot_phase , 2-89 2-91
- plot_phase , 2-90 2-92
- plot_qcl_3levels, 2-77
- plot_rtgain, 2-353
- plot_scan, 2-355
- plot_shd, 2-359
- plot_slope, 2-355
- plot_spec, 2-360
- plot_structure, 2-361
- plot_z, 2-362
- pml_length , 2-363
- pml_mesh , 2-363
- PML_thickness, 2-174
- pml, 2-363
- pn_ratio , 2-77
- pn_ratio , 2-441 2-443
- pn_ratio_points , 2-375
- pn_ratio_points , 2-161
- pn_ratio_range , 2-161
- pn_ratio_range , 2-375
- pn_ratio, 2-108 2-179 2-183 2-184 2-254 2-429
- pnt_num, 2-130 2-302
- point_from, 2-368
- point_ll, 2-106 2-215 2-335 2-336 2-433 2-453 2-466 2-468
- point_num, 2-263
- point_to, 2-368
- point_ur, 2-106 2-215 2-335 2-336 2-433 2-453 2-466 2-468
- point_x, 2-395
- point, 2-364
- point1 , 2-85
- point1_near , 2-310
- point2 , 2-85
- point2_near , 2-310
- point3 , 2-85
- point4 , 2-85
- pointj , 2-87
- points, 2-172 2-173
- polar_num, 2-298
- polar, 2-46 2-48
- polarization , 2-460
- polarization_flag, 2-288
- polarization, 2-145 2-365
- poly_mater , 2-438
- polygon_name , 2-99
- polygon, 2-136 2-365 2-368
- position label, 2-472 2-473
- position_from, 2-157
- position_phi, 2-46
- position_theta, 2-46
- position_to, 2-157
- positive_current_flow, 2-105
- post-processing, 2-182 2-187 2-288 2-332 2-335 2-336 2-338 2-339 2-355
- power_couple , 2-251
- power_depend_coef, 2-450
- power_in, 2-282
- power_loss , 2-386
- power_refl , 2-386
- power_transmission, 2-178
- power3, 2-366
- power3d , 2-399
- precalculated_gain, 2-396
- precision , 2-124
- precision, 2-118
- previous_layer, 2-366
- print_active_layer, 2-367
- print_coefs, 2-366
- print_flag, 2-193 2-304
- print_macro, 2-367
- print_output, 2-159
- print_point, 2-168
- print_points , 2-87
- print_rtgain, 2-282
- print_sor, 2-458
- print_step , 2-399
- prn_gain_num, 2-215
- prn_gain_range, 2-215
- profile , 2-251

profile, 2-246 2-315
prop_file , 2-77
pulse_dt , 2-407
pulse_s1 , 2-407
pulse_s2 , 2-407
pulse_t1 , 2-407
pulse_tf , 2-407
pulse_tr , 2-407
pump_light_from, 2-396
pump_light_to, 2-396
pure_index_loss , 2-363
pure_index_loss, 2-148 2-151
put_mesh, 2-368

Q

q_transport, 2-376
qdot_layer_mater, 2-369
qdot_material, 2-224 2-371
quantum escape, 2-376
quantum fly over, 2-376
quantum well model, 2-289
quasi_3d_transient, 2-373
qw_optics_control, 2-375
qw_plane, 2-60
qw_print , 2-69
qw_print, 2-60
qw_silicon_mater , 2-438
qw_states , 2-296
qw_thick , 2-438
qw_wave_ht , 2-77
qw_wave_ht, 2-332
qw_wave, 2-332
qw_xrange , 2-438
qwell_normal, 2-374
qwip_period , 2-251
QWIP, 2-251

R

r, 2-229 2-98
radiation_source, 2-193
radiative_boundary, 2-389
radiative_recomb, 2-389
radius, 2-46
range_n , 2-69

range_p , 2-69
range1, 2-136
range2, 2-136
rate_damp, 2-304
ratio, 2-368
raw_output, 2-379
ray-tracing, 2-239 2-247
ray3d_convert, 2-164
rayplot_angle, 2-390
rayplot_bias, 2-390
rayplot_showray, 2-390
rcled_dbr, 2-380
rcled_information, 2-296
rcled_mix_lambertian, 2-381
rcled_model, 2-382
rcled_optic_layer, 2-384
rcled_plot_y, 2-385
rcled_power_angle, 2-386
rcled_refl_coating, 2-386
rcled_spectrum_angle, 2-387
rcled_spectrum, 2-387
rcled_surface_plot_xy, 2-388
RCLED, 2-297
real_func, 2-391
real_index_high, 2-189
real_index_led, 2-242
real_index_low, 2-189
real_index_out, 2-242
real_index_spec, 2-391
real_index, 2-178 2-314 2-391 2-75
real_indexj, 2-242
recombination electron, 2-70
recombination hole, 2-70
recombination, 2-193
recover_prev_mqw, 2-304
rect_core_origin, 2-461
rect_core_xrange, 2-461
rect_core_zrange, 2-461
rect_symetric_x, 2-461
rect_x_order, 2-461
rectangle_system, 2-461
rectangle, 2-331
ref_column, 2-421

ref_density , 2-382
ref_density, 2-260
ref_elec_current, 2-244
ref_elec_density, 2-244
ref_energy, 2-224
ref_hole_current, 2-244
ref_hole_density, 2-244
ref_index, 2-260
ref_interface_mater , 2-310
ref_midpoint, 2-260
ref_pitch, 2-260
ref_pn_ratio , 2-382
ref_point , 2-93
ref_temper, 2-66
ref_wavel, 2-260
reference_point, 2-268
reference_to , 2-112
refine_mesh, 2-392
refl_phase_in_pi, 2-386
refl_x1, 2-238
refl_x2, 2-238
refl_y1, 2-238
refl_y2, 2-238
reflection_phase, 2-178
refr_index1 , 2-124
refr_index1, 2-424
refr_index2 , 2-124
refr_index2, 2-424
refr_index3, 2-424
refr_index4, 2-424
refr_index5, 2-424
refr_index6, 2-424
refr_index7, 2-424
refr_index8, 2-424
refr_index9, 2-424
refractive_index, 2-391 2-391
region , 2-464
region_label , 2-310
region_num , 2-87
reject_edges, 2-124
rel_cutpoint , 2-89
rel_x , 2-90
rel_y , 2-90 2-92
relative_dist, 2-361 2-362
relative, 2-44
reload_mater, 2-393
remove_above_line, 2-289
replace_macrofile, 2-393
res_tol, 2-193 2-304
rescale_poisson, 2-304
resistance, 2-170
resistor_region, 2-394
restart, 2-394
returned, 2-439
right_contact, 2-395
right_f_refl, 2-260
right_mesh, 2-289
right_phase, 2-260
rmesh_fd, 2-60
rotation_angle , 2-268
rtg_2facet_power_allmode, D-495
rtg_2facet_power, D-495
rtg_left_power_allmode, D-495
rtg_left_power, D-495
rtg_photon_num_allmode, D-495
rtg_photon_num, D-495
rtg_right_power_allmode, D-495
rtg_right_power, D-495
rtg_start, 2-282
rtg_surf_power_allmode, D-495
rtg_surf_power, D-495
rtg_wavelength, D-495
rtgain_data, 2-353
rtgain_phase, 2-395
rtgain_scan, 2-296

S

sample_distance, 2-416
sample_mesh, 2-209
sample_point, 2-209
sample_x, 2-282
sample_y, 2-282
save_miniband, 2-453
scale_1, 2-345 2-350 2-355
scale_2, 2-340 2-345 2-350 2-355
scale_abs_spec , 2-251

- scale_add_variable, 2-385
- scale_curr, 2-173 2-176 2-254 2-355 2-55 2-57
- scale_eqn_n, 2-304
- scale_eqn_p, 2-304
- scale_fit_inv_slope, 2-326
- scale_fit_slope, 2-326
- scale_horizontal, 2-176 2-355
- scale_lit , 2-176
- scale_lit, 2-173 2-254 2-355 2-55 2-57
- scale_mb_screen_length, 2-289
- scale_power, 2-346
- scale_refl, 2-124
- scale_rtg_var, 2-215
- scale_tran, 2-124
- scale_tunnel_coef, 2-453
- scale_value, 2-286
- scale_var, 2-362
- scale_variable , 2-385
- scale_vertical , 2-176
- scale_vertical, 2-348 2-355
- scale_wavelength, 2-241
- scale_x, 2-346
- scale_y, 2-346
- scale_z_coupling, 2-50
- scale, 2-182 2-241
- scan variables, D-493
- scan_data, 1-38 2-187
- scan_function, 2-407
- scan_num, 2-355 2-423 2-54 2-55 2-57
- scan_var, 2-345 2-355
- scan_voltage, 2-396
- scan, 2-399
- scan3d, 2-396
- scanline, 2-355 2-423 2-54 2-55 2-57
- scanvar , 2-423
- scanvar_facet , 2-423
- scanvar_horizontal_power , 2-423
- scanvar_mode_index , 2-423
- scanvar_scale , 2-423
- scanvar_scale_curr , 2-423
- scanvar_scale_horizontal , 2-423
- scanvar_scale_lit , 2-423
- Schottky contact, 2-86
- screening, 2-421
- search_gainpeak, 2-304
- search_subband, 2-224
- sec_num , 2-330
- sec_num, 2-110 2-271 2-295 2-318 2-329 2-409 2-411 2-413
- section_air, 2-411
- section_index, 2-462
- section_location, 2-412
- section, 2-409 2-413
- section1, 2-295
- section2, 2-295
- section3, 2-295
- section4, 2-295
- section5, 2-295
- segment_from, 2-314
- segment_num, 2-314
- segment_to, 2-314
- segment, 1-33
- select_index , 2-116
- select_modes , 2-116
- self_consistent, 2-416
- sequential_model , 2-376
- set_3dray_internal_interface, 2-424
- set_active_reg , 2-420
- set_barrier_width, 2-420
- set_ch_level1 , 2-371
- set_ch_level2 , 2-371
- set_ch_level3 , 2-371
- set_cond_level1 , 2-371
- set_cond_level2 , 2-371
- set_cond_level3 , 2-371
- set_domed , 2-118
- set_grid, 2-355
- set_hh_level1 , 2-371
- set_hh_level2 , 2-371
- set_hh_level3 , 2-371
- set_kp8x8_angle, 2-289
- set_lh_level1 , 2-371
- set_lh_level2 , 2-371
- set_lh_level3 , 2-371
- set_low_temperature, 2-193

- set_minority_carrier, 2-420
- set_mirror1_side, 2-118
- set_mirror2_side, 2-118
- set_n_side , 2-160
- set_polarization, 2-421
- set_spon_spectrum, 2-421
- set_wavelength, 2-422
- set_xydata_for_scan, 2-422
- shal_acpt_level_i, 2-425
- shal_acpt_level, 2-425
- shal_dnr_level_i, 2-426
- shal_dnr_level, 2-426
- shape, 2-130 2-147 2-302
- sheet_density, 2-130
- sheet_location, 2-130
- shift_affinity, 2-427
- shift_barrier, 2-71
- shift_center , 2-477
- shift_center, 2-229 2-368 2-98
- show_angle_num , 2-426
- show_model , 2-426
- show_points, 2-355
- show_ray_3d, 2-426
- show_secondary , 2-426
- side , 2-437
- side, 2-240 2-241 2-48
- sigma_factor, 2-116
- silicon_mater , 2-438
- sin_amp , 2-407
- sin_constant , 2-407
- sin_period , 2-407
- sin_phase , 2-407
- size, 2-175
- slit_diff, 2-246
- smith_chart, 2-348
- smooth_step , 2-93
- smooth_width, 2-390 2-42 2-46 2-48 2-48
- smooth, 2-390 2-42 2-46 2-48 2-48
- sol_inf, 2-187 2-214 2-49
- sol_outf, 2-317
- solve_rtg, 2-399
- solve_wave, 2-232
- sommerfeld_fac, 2-60
- sor_par, 2-428
- sort_imag, 2-148 2-151
- sort_mode, 2-282
- sort_modepeak, 2-298
- sort_modes , 2-226
- sort_modespan, 2-298
- sp_rate_wavel, 2-429
- space_charge, 2-296
- sparse_eigen_solver, 2-428
- sparse_fill, 2-67
- spec_heat, 2-430
- spectral dep. index, 2-391
- spectral dep. loss, 2-53
- spectrum_all_bias, 2-238
- spectrum_field_file, 2-246
- spectrum_file, 2-178 2-211 2-246 2-314
- spectrum_loop_num, 2-124
- spectrum_loop_range, 2-124
- spectrum_num, 2-238
- spectrum_range , 2-387
- splot_xy, 2-433
- splot_xyz, 2-431
- spon_all_points , 2-382
- spon_all_points, 2-238
- spon_mode, 2-215
- spon_out , 2-254
- spon_out, 2-215
- spontaneous emission, 2-429
- spontaneous recombination coefficient, 2-389
- square_term, 2-208
- srch_sidemode, 2-282
- srch_start, 2-282
- SRH recombination, 1-29 2-244 2-245
- stack_refl, 2-387 2-387
- stack_view_point , 2-382
- standing_wave, 2-315 2-353
- start , 2-412
- start_loop, 2-434
- start, 2-346 2-352 2-359 2-360 2-361 2-362
- statement, 1-16
- std_wave_range, 2-353
- std_wave, 2-246
- step_decrease, 2-304

step_increase, 2-304
step_limit, 2-282
step_num, 2-84
step, 2-434
stop_iter, 2-304
stop_thermal , 2-399
stop, 2-435
strain_bar, 2-435
strain_well, 2-435
strained_material , 2-436
strained_mobility, 2-436
structural data, 1-38
subband_valley, 2-155
subhh_levelk, 2-289
subhh_mxfack, 2-289
sublh_levelk, 2-289
sublh_mxfack, 2-289
substrate_latt, 2-421
sum_2ndmode, 2-172 2-173
sum_space_charge_mater, 2-296
sum_space_charge, D-495
suprem_contact, 2-437
suprem_import, 2-259
suprem_property, 2-438
surf_charge_expo, 2-159 2-405
surf_elec_dens , 2-251
surface_bottom, 2-288
surface_density , 2-371
surface_mater, 2-331
surface_model_side, 2-118 2-124
surface_model, 2-118 2-124
surface_p.c, 2-409 2-413
surface_power_grid, 2-124
surface_y_label, 2-331
surface_y, 2-331
surftrap_num, 2-220
symbol_variable, 2-439
symbol, 2-111 2-262 2-262 2-434
symbolic, 2-67
symmetric, 2-172 2-173
syntax of statements, 1-16

T

tail_bulk_ref, 2-289
tail_dos_n, 2-289
tail_dos_p, 2-289
tail_energy, 2-289
tail_state_scale, 2-289
taper_length , 2-477
taper_pointsj (j=1,2,...), 2-477
tau_bkg, 2-60
tau_broaden , 2-371
tau_broaden, 2-224
tau_broaden2-19 , 2-371
tau_density, 2-441
tau_energy, 2-442
tau_model, 2-442
tau_scat , 2-69
tau_scat, 2-60
tau_temperature, 2-443
tax_mass_bar, 2-444
tax_mass_well, 2-444
TE mode, 2-61 2-63
te_portion , 2-124
te_portion , 2-118
TE, 2-464
temp_end , 2-443
temp_max, D-495
temp_start , 2-443
temp, 2-445
temper_points , 2-375
temper_points , 2-161
temper_range , 2-161
temper_range , 2-375
temperature dependence, 2-66
temperature, 2-445
thermal_cond, 2-102
thermal_interf, 2-445
thermal_kappa_xy, 2-447
thermal_kappa, 2-447
thermal_type, 2-102
theta_range , 2-387
theta_range, 2-42
theta_x, 2-172
theta_y, 2-172
theta, 2-173 2-46

- thickness , 2-99
- thickness, 2-314 2-60
- thickness1, 2-124 2-424
- thickness2, 2-124 2-424
- thickness3, 2-424
- thickness4, 2-424
- thickness5, 2-424
- thickness6, 2-424
- thickness7, 2-424
- thickness8, 2-424
- thickness9, 2-424
- thicknessj, 2-242
- thm_cond, 2-445
- thm_ext_temp, 2-445
- thm_ht_flow, 2-445
- thm_lat_temp, 2-445
- thm_num, 2-445
- thm_transient, 2-193
- thm_type, 2-445
- thm_within_x, 2-445
- thm_within_y, 2-445
- thm_x, 2-445
- thm_y, 2-445
- thomson, 2-193
- threshhold_srch, 2-282
- tilt_imref , 2-180
- time, D-495
- TM mode, 2-61 2-63
- tm_portion , 2-124
- tm_portion , 2-118
- TM, 2-464
- tmass_gamma_bar , 2-448
- tmass_gamma_bulk , 2-448
- tmass_gamma_well , 2-449
- to, 2-243 2-332 2-334 2-449
- tol_eqn, 2-282
- tolerance, 2-282
- top_contact, 2-449
- top_emission , 2-382
- top_emission , 2-387 2-387
- top_emission, 2-331
- top_exit_index , 2-382
- top_led_power, D-496
- top_refl, 2-409 2-413
- total_active_layer , 2-382
- total_amount, 2-241
- total_incident_power , 2-118
- total_joule_heat, D-496
- total_optic_heat, D-496
- total_peltier_heat, D-496
- total_recomb_heat, D-496
- total_thomson_heat, D-496
- touch_macro , 2-102
- touch_mater , 2-102
- touch_mater, 2-437
- touch_var_symbolk(k=1..9), 2-102
- touch_vark(k=1..9), 2-102
- touchstone_export, 2-348
- touchstone_format, 2-348
- touchstone_freq_units, 2-348
- trans_broad, 2-453
- transfer_fraction, 2-320
- transfer_from_host, 2-320
- transp_contact, 2-164
- transp_contact1, 2-164
- transp_contact2, 2-164
- transp_contact3, 2-164
- transport_file , 2-203
- trap_density, 2-220
- trap_excitation, 2-450
- trap_index , 2-265
- trap_index, 2-105 2-106 2-263 2-332 2-335
2-336 2-338 2-355 2-379 2-431 2-433
2-450
- trap_information, 2-296
- trap_level_i, 2-451
- trap_level, 2-220
- trap_life_n, 2-220
- trap_life_p, 2-220
- trap_ncap_i, 2-451
- trap_pcap_i, 2-451
- trap_type, 2-220
- traplevel_model, 2-130
- traplevel_stddev, 2-130
- traplevel_tail_side, 2-130
- traplevel_tail, 2-130

triplet , 2-324
tun_mass, 2-160 2-475
tunneling_enhancement, D-496
tunneling, 2-453
type , 2-407
type, 2-102 2-235 2-269 2-326 2-94

U

ubdata_num, 2-215 2-468
unconfined, 2-290
unfreeze_electron_mater, 2-309
unfreeze_hole_mater, 2-309
uniform_extraction, 2-251
uniform_length , 2-477
uniform_stdwave, 2-315
uniform_zseg_from, 2-477
uniform_zseg_to, 2-477
units, 1-39
unstrained_110_enh , 2-436
update_lateral_mode, 2-304
update_mater, 2-193
update_mqw, 2-304
update_rc_extraction, 2-304
update_spec_source , 2-382
update_tunneling, 2-304
update, 2-315 2-458
upper_bound_label, 2-289
upper_wj, 2-229
use_active_mater, 2-189
use_av_pmass, 2-289
use_bulk_affinity, 2-456
use_bulk_bandgap, 2-457
use_bulk_property, 2-457
use_bulkmass, 2-289
use_eim, 2-461
use_elec_hv_expo_factor, 2-279
use_envelop_func, 2-69
use_fermi_level , 2-77
use_gain_spec, 2-277
use_hole_hv_expo_factor, 2-279
use_index_spec, 2-277
use_intern_refl, 2-238
use_l.w.e.factor, 2-209

use_macro, 2-108 2-254
use_macrofile, 2-457
use_mf , 2-428 2-57
use_physical_model, 2-160
use_schottky_xp, 2-243 2-449
use_sheet_density , 2-180
use_sor, 2-458
use_spon_spec, 2-277
use_strained_bandgap, 2-471 2-475
use_xy_range , 2-99
use_y_coord , 2-112
user_datafile , 2-407
user_label1, 2-355
user_label2, 2-355
user_label3, 2-355
user_title, 2-355
user_values_num, 2-181
user_xlabel, 2-355
user_ylabel, 2-355

V

val_bandj_edge, 2-460
val_subband, 2-332
val_valley , 2-77
val_valley_prop , 2-77
val_valley_prop, 2-296
valence_mixing , 2-69
valence_mixing, 2-60
valj_mass_para, 2-459
valj_mass_perp, 2-459
valj_valley_propk, 2-459
valley_ch, 2-235
valley_gamma, 2-235
valley_hh, 2-235
valley_l, 2-235
valley_lh, 2-235
value , 2-111
value_from, 2-262 2-262 2-434
value_to , 2-399
value_to, 2-262 2-262 2-434
value, 1-16 2-192 2-273 2-82 2-82
valuei_to, 2-399
var , 2-399

- var_diff, 2-392
 - var_fac, 2-304
 - var_name, 2-191
 - var_name1-9, 2-186
 - var_namei, 2-192
 - var_namej, 2-258 2-273
 - var_num , 2-265
 - var_num, 2-192 2-263 2-273 2-332 2-340 2-346 2-352 2-355 2-359 2-360 2-361 2-362
 - var_symbol1-9, 2-186
 - var_symboli, 2-192 2-369 2-82 2-82
 - var_symbolj, 2-258 2-273
 - var_symbolk, 2-232
 - var_tol, 2-193 2-304
 - var1-9, 2-186
 - vari, 2-192 2-232 2-369 2-399 2-82 2-82
 - variable , 2-385
 - variable , 2-265
 - variable_range , 2-385
 - variable, 2-105 2-106 2-182 2-263 2-286 2-332 2-334 2-335 2-336 2-338 2-340 2-345 2-346 2-350 2-352 2-355 2-359 2-360 2-361 2-362 2-379 2-388 2-390 2-392 2-431 2-433 2-44 2-466 2-467
 - variables, B-485 C-487
 - variation, 2-191 2-192 2-268 2-273 2-439 2-82 2-82
 - varj, 2-258 2-273
 - vcsel_eeim, 2-460
 - vcsel_model, 2-461
 - vcsel_section, 2-462
 - vcsel_type , 2-229
 - vcsel_type, 2-462
 - VCSEL, 2-279 2-397 2-447
 - vectorial variables, B-485
 - vectorial_wave, 2-463
 - velocity_n, 2-220
 - velocity_p, 2-220
 - version2002, 2-222 2-259
 - versus_bias, 2-54 2-55 2-57
 - vert_intercept, 2-326
 - view_dipole, 2-464
 - view_ganvalence, 2-464
 - view_kpsubband, 2-465
 - view_xrot , 2-387
 - view_xrot , 2-339
 - view_xrot , 2-91
 - view_xrot , 2-92
 - view_xrot, 2-336 2-338 2-42 2-431 2-433 2-46
 - view_zrot , 2-387
 - view_zrot , 2-339
 - view_zrot , 2-91
 - view_zrot , 2-92
 - view_zrot, 2-336 2-338 2-42 2-431 2-433 2-46
 - volt_res_i, D-496
 - voltage_control, 2-396
 - voltage_from, 2-396
 - voltage_i, 2-405 D-496
 - voltage_off , 2-416
 - voltage_to, 2-396
 - vplot_xy, 2-466
 - vplot_xyz, 2-467
 - vsat_factor , 2-436
 - vsat_factor , 2-436
- W**
- w, 2-98
 - w0_model, 2-181
 - wave equation, 2-215 2-428 2-458
 - wave_boundary, 2-468
 - wave_option, 2-431
 - wave_range_bottom, 2-416
 - wave_range_left, 2-416
 - wave_range_right, 2-416
 - wave_range_top, 2-416
 - wave_range, 2-416
 - wave_tol, 2-458
 - waveId_yrange, 2-474
 - waveguide photo-detectors, 2-470
 - waveguide_input, 2-470
 - waveguide_mode, 2-246
 - wavel_points , 2-375
 - wavel_points_number, 2-145

- wavel_range , 2-254
 - wavel_range , 2-375
 - wavel_range_factor, 2-145
 - wavel_range, 2-174 2-179 2-183 2-184 2-215 2-429
 - wavel_xrange, 2-282
 - wavelength , 2-382
 - wavelength , 2-251
 - wavelength_range , 2-387 2-387
 - wavelength_range, 2-182
 - wavelength, 2-145 2-238 2-246 2-313 2-42 2-422 2-470 2-474 2-474 D-496
 - weak_refl_iter, 2-470
 - weak_reflection, 2-470
 - weight , 2-371
 - weight2-19 , 2-371
 - well_num, 2-108 2-254
 - wetting_area_ratio , 2-371
 - wg_light_from, 2-396
 - wg_light_to, 2-396
 - wi , 2-148 2-151
 - width , 2-254
 - within_x, 2-220
 - within_x1_label, 2-220 2-445
 - within_x2_label, 2-220 2-445
 - within_y, 2-220
 - within_y1_label, 2-220 2-445
 - within_y2_label, 2-220 2-445
 - wj (j=1,2,...,9), 2-229
 - work_function, 2-102 2-243 2-449
 - working_direction , 2-124
 - wurtzite_offset_model, 2-471
- X**
- x-taper, 1-33
 - x_datafile, 2-130
 - x_end_label , 2-384
 - x_end_label , 2-380 2-386
 - x_end_label, 2-167
 - x_from_label, 2-332
 - x_label , 2-445
 - x_label, 2-220
 - x_position, 2-472
 - x_prof, 2-130 2-302 2-315
 - x_range, 2-331 2-48
 - x_segment, 2-238 2-315
 - x_start_complex , 2-99
 - x_start_label , 2-384
 - x_start_label , 2-380 2-386
 - x_start_label, 2-167
 - x_to_label, 2-332
 - x_width , 2-99
 - x_width, 2-474
 - x, 2-220 2-472
 - x1_label, 2-453 2-468
 - x2_label, 2-453 2-468
 - xcir_resistance_i, 2-405
 - xp_size, 2-222 2-86
 - xp1_size , 2-477
 - xp1, 2-98
 - xp2_size , 2-477
 - xp2, 2-98
 - xpoint_size, 2-473
 - xpoint, 2-473
 - xrange , 2-339
 - xrange , 2-160 2-89 2-91
 - xrange , 2-265
 - xrange , 2-90 2-92
 - xrange , 2-89
 - xrange, 2-105 2-106 2-263 2-332 2-335 2-336 2-338 2-340 2-343 2-344 2-345 2-350 2-352 2-355 2-388 2-431 2-433 2-437 2-466 2-467 2-475
 - xrange1, 2-392
 - xrange2, 2-392
 - xsearch_imag, 2-148 2-151
 - xsearch_range, 2-148 2-151
 - xsearch_real, 2-148 2-151
 - xsize_power2 , 2-93
 - xy_data_column, 2-130
 - xy_data_factor, 2-130
 - xy_data, 1-38 2-187
 - xy_datafile, 2-130
 - xy_from, 2-105 2-263 2-431 2-467
 - xy_hori_from, 2-326
 - xy_hori_to, 2-326

xy_label1, 2-355
 xy_label2, 2-355
 xy_label3, 2-355
 xy_linedata, 2-130
 xy_plane, 2-421
 xy_point , 2-265
 xy_product_scale, 2-326
 xy_skipline, 2-130
 xy_to, 2-105 2-263 2-431 2-467
 xy_vert_from, 2-326
 xy_vert_to, 2-326
 xy_x_transform, 2-130
 xy_y_transform, 2-130
 xy, 2-364

Y

y-taper, 1-33
 y.data, 2-315
 y_at_bottom , 2-380
 y_datafile, 2-130
 y_end_label , 2-99
 y_end_label , 2-160
 y_end_label, 2-167
 y_from_label, 2-332
 y_label , 2-445
 y_label, 2-220
 y_on_top , 2-384
 y_on_top , 2-380 2-386
 y_position, 2-473
 y_prof, 2-130 2-302 2-315
 y_range, 2-48
 y_start_complex , 2-99
 y_start_label , 2-99
 y_start_label , 2-380
 y_start_label, 2-160 2-167
 y_taper_pointsj, 2-477
 y_to_label, 2-332
 y, 2-220 2-473
 y1_label, 2-453 2-468
 y2_label, 2-453 2-468
 ymode_num, 2-298
 yrange , 2-339
 yrange , 2-160 2-89 2-91

yrange , 2-265
 yrange , 2-90 2-92
 yrange , 2-89
 yrange, 2-105 2-106 2-263 2-332 2-335 2-336
 2-338 2-340 2-343 2-344 2-345 2-350
 2-352 2-355 2-388 2-431 2-433 2-437
 2-466 2-467 2-475
 yrange1, 2-392
 yrange2, 2-392
 ysize_power2 , 2-93
 yz_amplifier_model, 2-474

Z

z-bending, 1-33
 z , 2-89 2-91
 z , 2-89
 z_connect_method, 2-51
 z_connect, 2-51
 z_dim, 2-170
 z_from , 2-265
 z_link_same_mater_only, 2-50
 z_max, 2-105 2-431 2-467
 z_min, 2-105 2-431 2-467
 z_segment, 2-474
 z_structure, 2-477
 z_to , 2-265
 z, 2-106 2-263 2-433 2-466
 zdim, 2-348 2-65
 zener, 2-475
 zero_doping, 2-474
 zincblende_offset_model, 2-475
 zplanes, 2-477
 zrange , 2-339
 zrange , 2-91
 zrange , 2-92
 zrange, 2-336 2-338 2-431 2-433
 zseg_num , 2-399
 zseg_num , 2-160
 zseg_num, 2-108 2-159 2-184 2-259 2-317
 2-392 2-393 2-394 2-395 2-396 2-476
 2-477 2-82 2-82
 zsegment_setting, 2-476
 zstep_size , 2-93